TS5000 Robot Controller

Robot Language Manual (SCOL2)

Rev.8

February 29, 2024

IMPORTANT

- Ensure that this instruction manual is provided to the actual final user of the industrial robot.
- Be sure to read this instruction manual before operating the industrial robot.
- After reading through this manual, keep it nearby for future reference.

Shibaura Machine

The controller supported by this instruction manual is the TS5000.

Copyright 2019 by SHIBAURA MACHINE Co., Ltd.
No part of this document may be reproduced in any form without obtaining prior written permission from SHIBAURA MACHINE Co., Ltd.
The information contained in this manual is subject to change without prior notice to effect improvements.

Introduction

Thank you for purchasing this SHIBAURA MACHINE robot system.
This manual describes the commands for the SCOL2 language, which is the robot language used in the robot controller TS5000, and the way of programming.
The SCOL2 (Symbolic Code Language for robot version 2) language consists of commands that support a wide range of robot functions. These commands can be used to enable simple programming of robot movements.
This manual is intended for all users, from people who are writing robot programs for the first time to experienced programmers. However, because the content is limited to robot language, for information about how to operate the robot controller, please refer to the respective manuals.

This manual has the following chapters.

Chapter 1: An Outline of Robot Language
This describes the relationship between robot movements and robot language and the main functions of commands in robot language. Be sure to always read this section for gaining an overview of robot language.

Chapter 2: Writing Programs in Robot Language
This describes the various rules when writing programs using robot language. Be sure to always read this section before writing programs.

Chapter 3: Explanation of Robot Commands
This provides detailed descriptions of the individual commands in robot language. The commands are presented in alphabetical order, and so refer to this when writing programs.

Chapter 4: Program Examples
This presents examples of programs using robot language. Refer to this when writing programs using robot language.

Chapter 5: Programming Hints and Warnings
This describes when a program should actually be written and the restrictions and hints for programs. Be sure to read this section before writing programs. Also review this chapter if your written program does not work properly.

Manufacturer

Shibaura Machine Co., Ltd.
2068-3, Ooka, Numazu-shi, Shizuoka-ken 410-8510, Japan
Tel: +81-55-926-5032
Fax: +81-55-925-6527

If you have purchased this controller outside of Japan, please contact the company that you purchased it from.

Contents

Scope of Use 13

Usage Procedures 14

Warranty 22

Standards, laws, and regulations pertaining to the safety of industrial robots 23

Safety Information 24

Service Network 25

1. An Outline of Robot Language 26

1.1 Robot Movement 26

1.2 Robot Language 27

1.3 Types of Commands 29

2. Writing Programs in Robot Language 31

2.1 Program Configuration 31

2.1.1 Files 31

2.1.2 Global section 31

2.1.3 Program section 32

2.1.4 Data section 32

2.1.5 Commonization of program and data sections 32

2.2 Character Set 34

2.3 Identifiers 35

2.4 Programs 36

2.4.1 Program Declaration 36

2.4.2 Statements 36

2.5 Constants 37

2.5.1 Scalar type constants 37

2.5.2 Vector type constants 39

2.5.3 Character string type constants 41

2.5.4 System constants 42

2.6 Variables 43

2.6.1 Scalar type variables 43
2.6.2 Vector type variables 44
2.6.3 Character string type variables 46
2.6.4 Global variables and auto variables 48
2.6.5 Array variable 50
2.6.6 System Variables 53
2.7 Expressions 55
2.7.1 Computational Expressions 55
2.7.2 Logical expressions 56
2.7.3 Priority Order of calculation operations 57
2.7.4 Substitution into scalar type variables 58
2.7.5 Computation of scalar type data 58
2.7.6 Substitution into vector type data 59
2.7.7 Computation of vector type data 61
2.7.8 Substitution of character string type data 62
2.7.9 Character string concatenation 63
2.7.10 Comparison calculation for character string type data 64
2.8 Commands and functions 65
2.8.1 Program control commands 65
2.8.2 Movement control commands 66
2.8.3 Calculator commands (Calculation functions) 66
2.8.4 Conversion functions 67
2.8.5 I/O (Input/output) control commands 67
2.8.6 Character string functions 68
2.8.7 Vector variable calculation functions 69
2.8.8 System switches 69
2.8.9 Library functions 70
2.9 Labels 71
2.10 Remarks and Comments 71
2.11 Subprograms 72
2.11.1 Writing subprograms 72
2.11.2 Argument of subprogram 72
2.11.3 Subprogram example 73
2.12 Library 74
2.13 Multitask processing 76
3. Explanation of Robot Commands 80
3.1 Command Explanations 80

3.2 Command description 81

ABS( ) 82
ACCEL 83
ACCUR 85
ACOS( ) 87
ALARM( ) 88
AND 90
AREAIN() 91
ASC( ) 92
ASIN( ) 93
ATAN( ) , ATAN2( ) 94
BASE 96
BCDIN( ) 98
BCDOUT( ) 100
BITAND( ) 102
BITNOT( ) 103
BITOR( ) 104
BITXOR( ) 105
CHR( ) 106
CINT( ) 108
CLOSE1 , CLOSE2 , CLOSEI1 , CLOSEI2 109
COARSE 111
CONFIG 112
CONT 114
COS( ) 115
CPACCEL 116
CPDECEL 117
CPSPEED 118
CREAL( ) 119
CROSS( ) 120
CSTR( ) 121
CTRLCODE 122
CUROVRD 123
CYCLE 124
DATA.....END 125
DECEL 126
DELAY 128
DEST 129
DIM.....AS..... 130
DIN( ) 132
DISABLE 133

DOUT( ) 135
DOUT( ) 136
ENABLE 137
EXIT 139
EXP( ) 140
EXTPLT 141
FGAIN 142
FINE 143
FOR...TO...STEP...NEXT 144
FORCE3LIM 147
FORMAT( ) 149
FREE 152
FREELOAD 153
GAIN 155
GLOBAL.....END 157
GOTO 158
GOTO( ) 159
HERE 161
HEXIN( ) 162
HEXOUT( ) 164
IF.....THEN.....ELSE 166
IF...THEN...ELSE...ENDIF 167
IGNORE 169
INERTIA 170
INNER( ) 171
INPSTSTP,INPSTSIP1~8 172
INPTMOTP,INPTMOIP1~8 174
INPUT 176
INSTR( ) 178
INSTRREV( ) 180
INT( ) 182
INV( ) 183
IP1STATUS~IP8STATUS 184
IP1~8 185
IPCLOSE( ) 186
IPOPEN( ) 188
JOINT( ) 190
JPSN() 191
KILL( ) 192
LCASE( ) 194
LEFTY 195
LN( ) 196

LOADCELL 197
LOADLIB 198
LOCUS 200
LOG10( ) 201
MAXTASK 202
MEMORYI 204
MEMORYJ 205
MEMORYL 206
MEMORYP 207
MEMORYR 208
MEMORYT 209
MINUSLIM 210
MOD 211
MODE 212
MOTION 213
MOTIONT 215
MOVE 217
MOVEA 219
MOVEC 221
MOVECV 223
MOVEI 225
MOVEJ 227
MOVES 230
MOVESYNC 232
MOVET 234
MOVEW 235
MOVEWI 236
NOT 237
NOWAIT 238
OFF 239
OFFGAIN( ) 241
ON 242
ON.....DO..... 244
ONGAIN( ) 249
OPEN1 , OPEN2 , OPENI1 , OPENI2 250
OR 252
OVERRIDE 253
PAI 255
PALLET 256
PALLETZ 262
PASS 264
PASS 266

PASSMM 268
PAUSE 269
PAYLOAD 270
PLCDATAR1~8 272
PLCDATAW1~8 274
PLCDREGDW 276
PLCDREGF 277
PLCDREGUW 278
PLCDREGW 279
PLCSLR01~08 280
PLCSLW01~08 282
PLCSSR01~08 284
PLCSSW01~08 286
PLUSLIM 288
POINT( ) 289
PRESS 290
PRINT.....CR 291
PROGRAM.....END 293
PSNCMD 294
PSNCMDJ 296
PSNCMDW 297
PSNFBK 299
PSNFBKJ 300
PSNFBKW 301
PULOUT( ) 302
PULTIME 304
RCYCLE: 305
REACH 307
READY 308
REAL( ) 309
REMAIN 310
REMAINT 312
REMARK 314
REPLACE( ) 315
RESET 316
RESUME 317
RETURN 319
RIGHTY 320
SAVEF1~8 321
SAVEI1~8 322
SEGMENT 323
SELECT.....CASE..... ENDSELECT 324

SETGAIN( ) 326
SGN( ) 327
SHIFT( ) 328
SIN( ) 329
SLOWDOWN 330
SLOWDOWN 331
SLOWSTART 332
SLWSPD 333
SMOOTH (Option function) 335
SPEED 339
SPURT 340
SQRT( ) 342
STOP 343
STR( ) 344
STRCAT( ) 345
STRCOMP( ) 347
STRLEFT( ) 348
STRLEN( ) 350
STRMID( ) 351
STRREAD( ) 353
STRRIGHT( ) 354
STRSPLIT( ) 356
SWITCH 358
SWITCH 360
TAN( ) 361
TASK( ) 362
TID 363
TIMER,TIMER1 to 4 364
TOOL 365
TORQUE 367
TP 369
TRANS( ) 370
TRIM( ) 371
UCASE( ) 373
WAIT 374
WHILE...ENDWHILE 375
WITH 377
WORK 379
XOR 381

4. Program Examples 382

4.1 Program to move robot back to its mechanical origins 382

4.2 Warm-up program 382
4.3 Robot motion 383
4.4 I/O signals 383
4.5 Interlock 384
4.6 Pick and place 385
4.7 Palletize 387
4.8 Creating a program for monitoring an insertion error 389
4.9 Program example of short-cut (PASS) movement 391

5. Programming Hints and Warnings 394

5.1 Program Execution Timing 394
5.1.1 Arm Movement and Signal I/O Timing 394
5.1.2 Synchronization of Arm Movement and Program Execution 396
5.1.3 DELAY Command and WAIT Command 398
5.2 Things Not to Do When Programming 401
5.2.1 Variables 401
5.3 Things to Watch Out for When Writing a Program 402
5.3.1 Types of Commands 402
5.3.2 Robot Coordinate Systems 404
5.3.3 Short-Cut Movement 410
5.3.4 Robot Configuration 415
5.3.5 Data section 416
5.3.6 Global section 419
5.3.7 Robot Movement Speed 421
5.3.8 Robot Acceleration 422

Appendix 423

Appendix A List of Commands 423
Appendix B List of Reserved Words 431
Appendix C Contents of Library File (SCOL.LIB) 433
Appendix D How to Read Symbols 436

Scope of Use

The product has been developed for use in a wide range of automation tasks including transport work such as the movement and alignment of workpieces and assembly work such as insertion of parts and screw tightening.
The customer should also use this product for the purpose of automating these tasks and for incorporating into automation equipment.

This product is not intended for use in tasks involving human contact (such as tasks for medical care and nursing care) or to be incorporated in equipment that sustains human life (such as life support devices and incubators), equipment on which people ride (including transportation equipment such as cars and aerospace equipment such as airplanes, and recreational equipment such as roller coasters), and equipment that protects human life or the human body (such as safety devices).
This product is not intended to be incorporated in equipment for controlling large heat quantities (such as nuclear power controllers and incinerator controllers).
This product is prohibited from being used for tasks or incorporated in equipment where a failure or malfunction could likely pose a direct threat to human life or harm to the human body. The customer is responsible for providing any warranties if this product is used in this way.

Usage Procedures

This describes the operating conditions for the TS5000. Usage under different conditions could expose the user to unexpected risks. Be sure to follow the required conditions.

1. Main Specifications of Product: Refer to the robot manual for your robot.
2. Usage Restrictions: Tables 1 to 6 provide the “Restricted Specifications Sheet” indicating the usage restrictions.

Table 1. Restricted Specifications Sheet

1. Purpose and application for using product (usage restrictions)	
1) Intended use	To make the robot perform operations for specific purposes (item transport, assembly, inspection, machining, etc.) (1) Transportation [1] Clear a transportation route to the unpacking site. [2] Use a forklift or other equipment to lift the packing box. [3] Move the packing box to the unpacking site. (2) Unpacking [1] Clear the space required for unpacking. [2] Cut the packing straps and unpack. [3] Cut the packing straps securing the robot. [4] Peel off the tape from the controller packing box, and unpack. [5] Check that the robot and controller are not damaged. [6] Check the accessories and their quantities. - When transferring the robot to the installation site [7] Dispose of, or store and manage the packing materials. - When storing the robot [7] Secure the robot in place in the same way as the state at delivery, and repack. [8] Move the packing box to the storage site. (3) Transfer - Robot [1] Check the orientation of the robot during transfer. [2] Check that the robot fixing jigs are mounted. [3] If using a crane, mount the supplied eyebolts. [4] Check the tightness of the fixing bolts of the robot fixing jig. [5] Use a crane or cart to transfer the robot to the installation site. - Controller [1] Use a cart to transfer the controller to the installation site.

Table 1. Restricted Specifications Sheet (continued)

1. Purpose and application for using product (usage restrictions)	
1) Intended use (continued)	(4) Installation [1] Use bolts to secure the robot to the frame and other parts. [2] Fabricate any needed cables. [3] Remove the robot fixing jigs and eyebolts. [4] Install the controller and connect it to the robot. [5] Press the robot by hand to move it to a location where it is easy to perform work. [6] Mount the end effector. [7] Connect the required external devices. (5) Teaching [1] Move the robot to the desired position or orientation by guiding it with a teach pendant or pushing it by hand. [2] Teach the position where the robot will operate. [3] Create the operation sequence (program, etc.). [4] Make sure that the signals and data are transferred to and from the external equipment. [5] Check operation at low speed (250 mm/sec or less). [6] While gradually increasing the operating speed, make sure that the operation matches the specific purpose. (6) Operation [1] Start robot operation. [2] Make sure that operation is being performed to match the specific purpose at the specified speed. [3] Stop robot operation. (7) Repacking [1] Remove the end effector. [2] Move the robot to the position and orientation where the fixing jig can be mounted by guiding it with a teach pendant or pushing it by hand. [3] Mount the fixing jig. [4] Turn off the power, and remove the wiring. [5] Obtain the space needed for packing. [6] If using a crane to transfer the robot, mount the supplied eyebolts. [7] Transfer the robot and controller to the packing site. [8] Pack the robot and controller so that they are in the original delivery state. [9] Clear a transportation route to the transportation equipment (truck, etc.). [10] Use a forklift to lift the packing box. [11] Load the packing box onto the transportation equipment.

Table 1. Restricted Specifications Sheet (continued)

1. Purpose and application for using product (usage restrictions)	
1) Intended use (continued)	(8) Maintenance Work performed when the primary power is off: - Robot [1] Check the bolts for any looseness and backlash and re-tighten them. [2] Make sure that the cable clamp is not loose. [3] Check for any damage. [4] Clean off any dust. [5] Inspect and apply corrosion inhibitor. [6] Check for any wear on the cable and air tube. [7] Check that there is no misalignment between the motion of each joint and the origin position. - Controller [1] Replace the memory backup batteries in the controller. Work performed when the primary power is on: [1] Check the greasing state and apply grease if needed (when the servo power is off). [2] Check for any abnormal vibrations and sounds during operation (when the servo power is on). [3] Check the teaching point for any misalignment (when the servo power is on). [4] Replace the backup batteries for the robot position detector (when the servo power is off). (9) Disposal [1] Remove the end effector. [2] Move the robot to the position and orientation where the fixing jig can be mounted by guiding it with a teach pendant or pushing it by hand. [3] Mount the fixing jig. [4] Turn off the power, and remove the wiring. [5] Use a crane to transfer the robot, and mount the supplied eyebolts. [6] Transfer the robot and controller to the disposal site. [7] Dispose of the robot and controller using a method compliant with your country's laws and regulations.

Table 1. Restricted Specifications Sheet (continued)

1. Purpose and application for using product (usage restrictions)	
2) Foreseeable misuse	- Use for work which applies excessive impact to the robot or where excessive impact is applied by the robot - Modification of robot, controller, and optional parts, or supplying power while these covers are removed - Operation in other than the specified installation state (floor, suspended, and wall-mounted) - Operation on a frame or other structure that does not satisfy the specified strength - Operation under conditions not satisfying the specified installation requirements - Entry into the hazard zone of the robot by mistakenly assuming that a stop to wait for an operation signal during operation was an operation stop - Usage under conditions where the specification values are exceeded or usage under improper settings - Stop of the robot caused by power off during operation or when the servo power is on - Unintended operation caused by pressing a wrong button - Operating without checking operation in MANUAL mode - Operating without checking the tightness of the robot and hand set bolts - Unintended operation due to improper wiring - Unintended operation due to incorrect program selection - Riding on or leaning against the robot or hanging on the arm
3) Unexpected start	- Malfunction due to excessive noise - Robot start due to start signal unexpectedly sent from peripheral equipment to the robot - Unintended start due to abnormal communication data - Malfunction due to voltage fluctuation - Power on due to powering after power outage - Malfunction due to failure or the like

Table 2. Restricted Specifications Sheet

2. Replacement or other modification of program components (time restrictions)	
1) Mechanical restrictions	- Before operating the robot, perform operation at a low speed (20% or less of maximum speed) for five (5) minutes - Replace consumable parts as required or periodically by conducting daily inspection, regular inspection (every 3 and 6 months and every year) and overhauls (every 5 years).
2) Electrical restrictions	

Table 3. Restricted Specifications Sheet

3 Movable range of product (restriction on space)		
1) Work envelope	1st axis 2nd axis 3rd axis 4th axis Additional axis — —	Compliant with 1.4 Specifications Table Compliant with 1.4 Specifications Table Compliant with 1.4 Specifications Table Compliant with 1.4 Specifications Table Compliant with individual specifications. — —
2) Interface	Transportation: Unpacking: Transfer: Installation: Teaching: Operation: Repacking: Maintenance: Disposal: 	The packing box shall be able to be transferred by a transportation worker. The packing box shall be able to be carried in. Space of at least 2 m^3 shall be provided. The robot shall be able to be hoisted by a crane. The robot shall be able to be lifted by two or more people and loaded onto a cart or similar device. There shall be no obstacles on the route from the unpacking site to the installation site. The product shall be installed in a location suitable for the robot's shape and range of motion The product shall be installed at a location suitable for the outline and that provides adequate ventilation of the controller. The cables shall be installed at a location where they will not be stepped on by operators. Teaching work shall be able to be performed from outside the hazard zone of the robot. Space shall be provided so that, if a person enters the hazard zone, the person will not get entrapped by the robot or peripheral equipment. A fence or enclosure shall be installed to prevent any operator from approaching the hazard zone of the robot. The robot shall be able to be operated from outside the hazard zone of the robot. An operator in the fence or enclosure shall be visible from the operation location. Work shall be able to be performed to remove the robot hand, mount the fixing jig, separate the robot and controller, and remove the robot and controller from the frame. The robot shall be able to be moved from the installation site to the packing site. The repacking work shall be able to be performed. A forklift or other equipment shall be able to be used to lift the packing box. Space required for maintenance shall be provided. After maintenance, operation shall be able to be checked from outside the hazard zone of the robot. Space shall be provided so that, in the hazard zone, a person will not get entrapped by the robot or peripheral equipment. Work shall be able to be performed to remove the robot hand, mount the fixing jig, separate the robot and controller, and remove the robot and controller from the frame. The robot shall be able to be moved from the installation site to the disposal location.

Table 3. Restricted Specifications Sheet (continued)

3. Movable range of product (restrictions on space)	
3) Work environment	- A fence, enclosure, or other protection shall be provided that surrounds the hazard zone of the robot to prevent people from entering the area during operation. - A display indicating the robot operation status shall be provided to indicate whether the robot is operating or stopped. - Lighting shall be provided for enabling visual checks by operators. - To prevent operators from slipping, the floor shall not be covered with oil or similar substances. - No noises that impair attention shall be generated.

Table 4. Restricted Specifications Sheet

4. Life cycle of product
In the product life cycle of assembly, shipment testing, transportation, unpacking, transfer, installation, teaching, operation, repacking, maintenance, and disposal, the risk assessment is carried out for the life cycle of “transfer and installation” (transportation, unpacking, transfer, installation, and repacking), “use” (teaching, operation, and maintenance), and “disposal”.

Table 5. Restricted Specifications Sheet

5. Person who approaches the machine		
Applicable person	Knowledge and requirements	
1) Transportation worker	Knowledge	Contents of the instruction manual
	Requirement	Person who has received training for forklifts and other transfer equipment work as defined by the respective country's laws and regulations and who has been permitted to perform the work by the employer
2) Unpacking worker	Knowledge	Contents of the instruction manual
	Requirement	—
3) Transfer worker	Knowledge	Contents of the instruction manual
	Requirement	Person who has received training for crane work as defined by the respective country's laws and regulations and who has been permitted to perform the work by the employer
4) Installation worker	Knowledge	Contents of the instruction manual, operating procedures for target robot
	Requirement	When guiding a robot, a person who has received training for robot teaching work as defined by the respective country's laws and regulations and who has been permitted to perform the work by the employer
5) Teaching worker Work supervisor	Knowledge	Contents of the instruction manual, operating procedures for target robot, work regulations, and robot operations
	Requirement	Person who has received training for robot teaching work as defined by the respective country's laws and regulations and who has been permitted to perform the work by the employer
6) Operator	Knowledge	Contents of the instruction manual, operating procedures for target robot, work regulations, and robot operations
	Requirement	—
7) Repacking worker	Knowledge	Contents of the instruction manual, operating procedures for target robot
	Requirement	Person who has received training for crane work as defined by the respective country's laws and regulations and who has been permitted to perform the work by the employer When guiding a robot, a person who has received training for robot teaching work as defined by the respective country's laws and regulations and who has been permitted to perform the work by the employer

Table 5. Restricted Specifications Sheet (continued)

5. Person who approaches the machine		
Applicable person	Knowledge and requirements	
8) Maintenance worker Work supervisor	Knowledge	Contents of the instruction manual, operating procedures for target robot, work regulations, and robot operations Storage procedures for target robot
	Requirement	Person who has received training for robot inspection work as defined by the respective country's laws and regulations and who has been permitted to perform the work by the employer
9) Disposal worker	Knowledge	Contents of instruction manual, operating procedures for target robot, and waste disposal methods
	Requirement	Person who has received training for crane work as defined by the respective country's laws and regulations and who has been permitted to perform the work by the employer When guiding a robot, a person who has received training for robot teaching work as defined by the respective country's laws and regulations and who has been permitted to perform the work by the employer
10) People passing nearby Other workers	- Measures shall be taken to prevent entry into the fence, enclosure, or other protection surrounding the hazard zone of the robot. - Measures shall be taken to make the robot stop operation when someone approaches the hazard zone. - If a person enters the hazard zone of the robot to perform work, robot operation shall be stopped, the drive power turned off, and a teaching worker or maintenance worker shall supervise the situation to ensure safety.	

Table 6. Restricted Specifications Sheet

6. Other

Warranty

We conduct strict testing and inspection of this product before delivery to ensure that it satisfies our performance standards.
We warrant this product to be free from defects under the following terms and conditions.

1. Warranty period

The warranty period of the product is either of the following, whichever is earlier.

(1) 18 months after shipping from our factory
(2) 12 months after installation at customer site

2. Warranty scope

(1) The warranty is provided for this product. The warranty covers only the specifications and functions included in the product specifications, catalog, and instruction manual. Any secondary or incidental damages caused by failure of this product are not covered by the warranty under any circumstances.
(2) We will repair a faulty product free of charge only if this product has been operated and used in accordance with the instruction manual supplied with the product.
A fee will be charged for any repairs after the warranty period has expired.

3. Disclaimer

The warranty shall not apply to the following cases.

(1) Any failure or damage arising from careless use or misuse, or usage that is not described in the instruction manual.
(2) Any failure caused by aging or normal wear and tear (natural fading of paints, deterioration of consumable parts[1]).
(3) Irregularities confined to sensory quality such as noise not recognized as affecting functions
(4) When any modification or disassembly without our consent was performed
(5) Any failure or damage arising from inadequate maintenance and inspection or improper repair
(6) Any failure or damage due to an act of God, fire, and other force majeure
(7) Internal data such as programs and points created or modified by customers
(8) When a product purchased in Japan is taken abroad

*1): Consumable parts refer to the replacement parts for maintenance as listed in the manual of the controller.

4. Safety Information

(1) Shibaura Machine shall provide no guarantees for the standard performance of this product when used outside the specifications.
(2) If the customer does not observe the safety information appearing for DANGER, WARNING, CAUTION, and NOTES contained in this instruction manual, Shibaura Machine will not be liable for any resulting injuries, death, or other accidents, damage, product failure, or any other problems.
(3) Note that the safety information appearing for DANGER, WARNING, CAUTION, NOTES, and other explanations in this instruction manual are within the scope of usage assumed by Shibaura Machine, and they do not foresee all possible incidents for all situations that could occur.

Standards, laws, and regulations pertaining to the safety of industrial robots

The international industrial standard ISO/DIS 12100 "Safety of machinery" specifies risk reduction measures for machinery and equipment.
Step 1: Inherently safe design measures - Limitations on force, speed, energy, etc.
Step 2: Safeguarding - Installation of safety fences, etc.
Complementary protective measures - Installation of emergency stop devices, etc.
Step 3: Information for use - Warning signs, alarms, instruction manuals, etc.
Based on the above, the ISO/IEC has defined "Guide 51", which classifies the various standards in a hierarchal structure.

Safety standards for industrial robots are standardized as separate safety standards for machinery in Type C.

- ISO 10218, ISO 10218-1

Be sure to always follow the standards, laws, and regulations set forth in the country where the industrial robot is used.

Safety Information

This instruction manual contains important information to ensure the safe and correct use of the controller and to prevent injury to the operator and others and damage to property.
Be sure that you fully understand the following information (signal words and symbols) before reading the main text, and always follow the provided safety instructions.

Display of Signal Words and Their Meanings

Signal Word and Symbol	Meaning
⚠ DANGER	Indicates an imminently hazardous situation which, if not avoided, will result in death or serious injury.
⚠ WARNING	Indicates a potentially hazardous situation which, if not avoided, will result in death or serious injury.
⚠ CAUTION	Indicates the possibility that incorrect operation could cause injury[1)] to the operator.
NOTICE	Indicates a situation which could result in damage to property[2)] only.

1) Injury refer to wounds, burns, electric shocks, and similar cases that do not require hospitalization or prolonged outpatient treatment.
2) Damage to property refers to extensive damage to property or materials.

[Teaching and Operation]

Be sure to carefully follow the safety information below to ensure the safe use of this product.

⚠ DANGER

- Before executing a program that you created, check that the created program was input correctly.

The robot that has made an unexpected motion may collide with the peripheral equipment.

⚠ WARNING

- Before executing a program that you created, check the emergency stop circuit.

The robot that has made an unexpected motion may collide with the peripheral equipment.

Service Network

Please refer to our distributor's handouts

1. An Outline of Robot Language

This section describes the connection between robot language and robot movement, and presents a rough outline of commands used in robot language.

1.1 Robot Movement

Robots do work in place of people. For example, let's say that somebody has to attach a part to a workpiece coming down a conveyor. The employee takes a part from a parts bin and attaches the part to a workpiece transported to his or her station by a conveyor. If we were to set up a robot to do this work instead, we would have an arrangement something like that shown in Figure 1.1.

In Fig. 1.1, the robot grabs a part from the parts feeder and attaches the part to a prescribed position of the workpiece coming down the conveyor. Considering this work from the point of view of the robot, we would come up with a diagram like that of Figure 1.2. In this figure, the robot first moves straight down from Point B, and at Point A it grabs a part. After grabbing the part, the robot moves back up from Point A to Point B. From Point B, the robot moves to Point C, which is directly above the part attachment location Point D. The robot then drops down from Point C to Point D, and attaches the part to the workpiece. When the robot is finished attaching the part, it moves back up to Point C, and then finally back to Point B. This completes one work cycle.

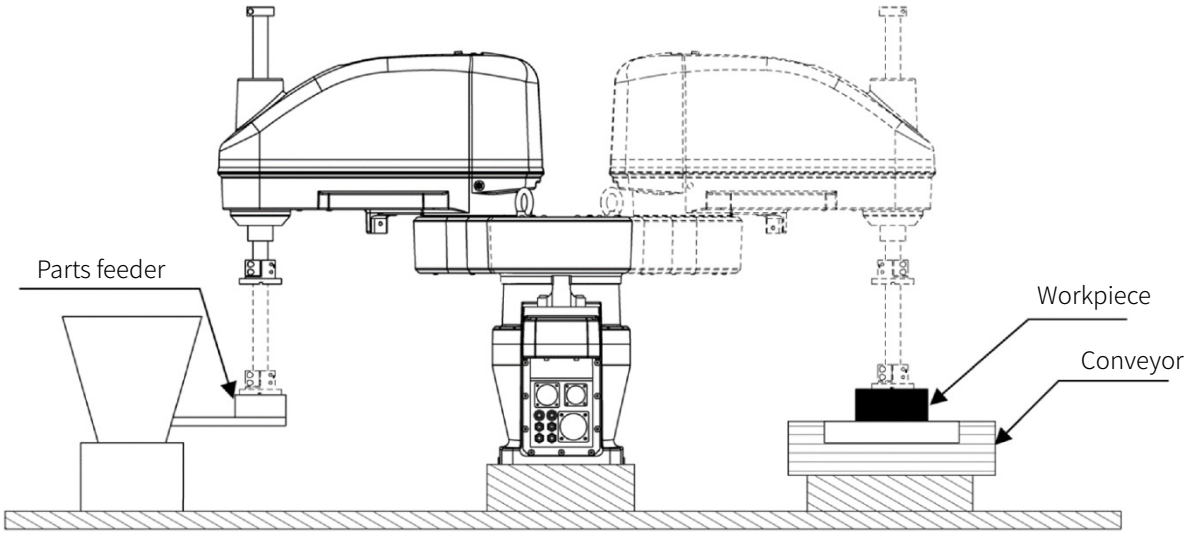


Fig. 1.1 Assembly work

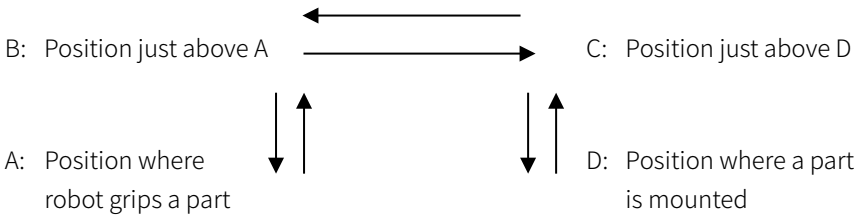


Fig. 1.2 Robot movement

1.2 Robot Language

Robots do assembly work and other tasks in place of people. However, someone still has to teach the robot what to do.

Robots will only do what you tell them to do, and it's important to tell them exactly what you want it to do. Telling a robot what to do is called “teaching.” Making a robot do what you taught it to do is called “playback.” Of course, this only applies to what are called “playback robots,” which repeat (or play back) the movements you instructed the robot when teaching.

There are various ways to teach a robot what to do. One is the method of making a robot do a job in order. For example, when carrying out painting and spot welding work, someone teaches a robot as he does it. The robot does the work as it is moving along a taught motion.

In order to achieve more complicated work, however, we need change robot motion according to the states of peripheral equipment. As understood by the assembling example, there may be problems with change in attachment parts in response to types of workpieces carried by a conveyor, or with repeated attachment and detachment of the parts when mis-attachment occurs. The robot needs response to such circumstances that occur around the robot. Thus we must provide the method how the robot responds to various circumstances.

A language used for writing robot movement programs is called “Robot Language”. A robot task expressed in the robot language is referred to as “Program”, and a task to create the program is called “Programming”. TS5000 employ the SCOL2 Language (which stands for Symbolic Code Language for Robot version 2) as our unique robot language. The example of the assembly work described in the previous section can be expressed in the SCOL2 Language as shown below.

```
PROGRAM ASSEMBLY
  MOVE B                    Move to Point B.
  OPEN1                     Open Hand 1.
  MOVE A                    Move to Point A.
  CLOSE1                    Close Hand 1.
  DELAY 0.5                 Wait 0.5 second before grabbing the part.
  MOVE B                    Move to Point B.
  MOVE C                    Move to Point C.
  MOVE D                    Move to Point D.
  OPEN1                     Open Hand 1.
  DELAY 0.5                 Wait 0.5 second before letting go off the part.
  MOVE C                    Move to Point C.
  MOVE B                    Move to Point B.
END
```

The word PROGRAM marks the beginning of a program and the word END marks the end of a program. The name of this particular program is ASSEMBLY. MOVE A means to move to Point A. OPEN 1 and CLOSE 1 mean to, respectively, open and close Hand 1 of the robot. DELAY 0.5 means not to do anything for 0.5

second. Furthermore, the locations of Points A, B, C and D are defined (taught) beforehand by physically guiding the robot (in the teaching mode) to these points.
Thus we can express robot tasks in SCOL2 by arranging movement commands that are given to the robot in sequence for achieving the taught tasks.

1.3 Types of Commands

In the previous section, we saw how SCOL2 is used to express the action of the robot. Here, we explain a little bit more about SCOL2 commands themselves.

In addition to commands like "MOVE A" which actually move the robot, there are many other commands which do such things as sending signals to external equipment or directing the robot to do the same thing over and over again. Table 1.1 presents a list of SCOL2 commands.

The SCOL2 commands are generally divided into 6 types.

(1) Program control commands

Program control commands control the execution of the program by doing such things as executing certain parts of the program in accordance with external signals or causing portions of the program to be carried out repeatedly.

(2) Movement control commands

These commands move the robot. Commands which temporarily stop the robot are also included in this category. Commands which actually move the robot are called movement commands.

(3) Calculator commands

These commands are used to invoke (use) mathematical functions such as the trigonometric functions and the square root function.

(4) I/O (Input/output) control commands

These commands are used to read in (input) or send out (output) signals to and from external equipment. Data input/output of communication channel are included in the I/O control command.

(5) Movement condition commands

These commands are used to specify the configuration and speed of various joints of the robot while it is moving.

(6) Movement reference commands

These commands are used to reference and check the movement of the robot. For example, these commands could be used to determine what percentage of a certain motion has been completed at a certain time. They also include referencing to robot operating mode.

These commands, in combination with other commands, are used, for example, to output signals to the external equipment when the robot has completed up to 70% of the given task, or to branch the program when movement of the robot is recognized as an error if the movement is not finished within a specified time.

By combining these commands, tasks suitable for the robot can be programmed.

Table 1.1 Functions of the SCOL2 language

Type	Purpose	Command
Program control commands	(1) Monitor external signals, timers, etc	ON~(BREAK) DO~, IGNORE, IF~THEN~ (ELSE~), WAIT
	(2) Control program execution.	PROGRAM.....END, GOTO, RCYCLE:, RETURN, FOR~TO~(STEP~).....NEXT, STOP, TASK, KILL, SWITCH
	(3) Remarks and comments on program	REMARK
Movement control commands	(1) Move the robot.	MOVE, MOVES, MOVEC, MOVEA, MOVEI, MOVEJ, MOVET, MOVEW, MOVEWI
	(2) Temporarily stop the robot.	DELAY
Calculator functions	(1) Perform calculations for real numbers.	SIN, COS, TAN, ASIN, ACOS, ATAN, ATAN2, SQRT, ABS, SGN, LN, MOD, EXP, LOG10, AND, OR, NOT
I/O control commands	(1) Input and output of external signals.	ALARM, DIN, DOUT, PULOUT, BCDIN, BCDOUT, HEXIN, HEXOUT
	(2) Input and output of communication data.	INPUT, PRINT~CR
Movement condition commands	(1) Specify conditions for controlling robot movement.	CONFIG, ACCUR, ACCEL, DECEL, SPEED, PASS, TORQUE, GAIN, SETGAIN, PAYLOAD
Movement reference commands	(1) Check robot movement.	MOTION, MOTIONT, REMAIN, REMAINT CUROVRD, PSNCMD, PSNFBK
	(2) Check system movement.	MODE
	(3) Assign a coordinate system.	TOOL, BASE, WORK

2. Writing Programs in Robot Language

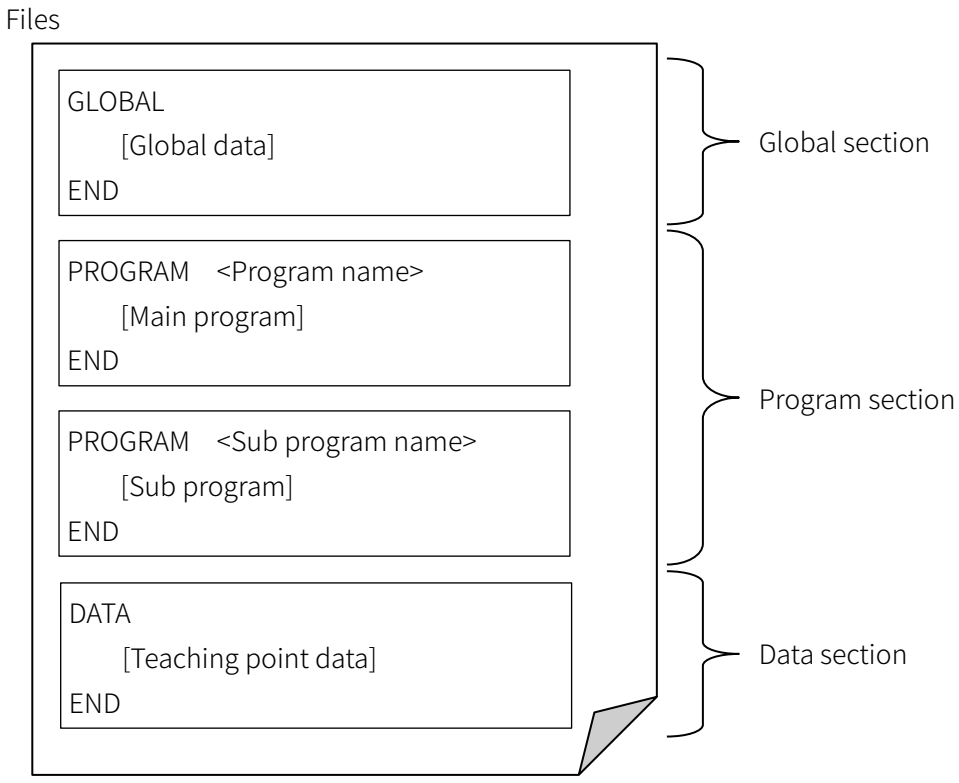
Chapter 1 presented an overview of robot language; this chapter describes how to write programs in robot language.

2.1 Program Configuration

Below we present a general outline of program configuration with the SCOL2 language.

2.1.1 Files

In order to get the robot to perform a task, you need both a program written in robot language and positional data for use by the program.
That is, for the TS series, you have to have a matched set of a program (or programs) and teaching point data. This matched set is called a file. Program execution and editing are on the file basis. Multiple files can be saved to the controller.



2.1.2 Global section

Variables that can be referenced and changed from any location in the main program or subprogram are called global variables. Global variables are specified in the global section at the start of the file.
For details on global variables, see section 2.6.4.

2.1.3 Program section

The instructions in robot language for the tasks performed by a robot are called a program.
A program can also be called and used by another program. It is possible preparing frequently used operations and preset operations as a single program and calling these programs when needed.
Programs that are called in this way are called subprograms, and the program that calls the subprogram is called a main program.
A single file can include multiple programs. When a program is executed, unless specified otherwise, the program at the start of the file is executed as the main program. When a subprogram is called, the subprogram must be located in the same file as the main program. Also, even if multiple programs are contained in a file, all of these programs are not executed in order. Instead, the programs are executed until the end of the main program as a single unit.
For details on programs, see section 2.4 Programs.
Programs are edited from the teach pendant using the program editor function of the controller. For details on how to use the program editor, see the “TP5000 Operation Manual”.

2.1.4 Data section

The teaching point data is created in the data section of the same file as the program. Teaching point data created in the file is shared for all programs contained in that file. The teaching data from different files cannot be used.
The data editor function of the controller can be used for teaching to automatically add data to the file. For details on how to use the data editor, see the “TP5000 Operation Manual”.

2.1.5 Commonization of program and data sections

When duplicated operations or point data exist in multiple program files, the program section or data section can be commonized, and the commonized SCOL (SCOL2) language program or data can be referenced no matter which program file is selected.

Data section commonization allows a program file (DATA.TBL) with one data section to be commonly used by multiple programs with program sections. The name of the program file with the common data section must be DATA.TBL.

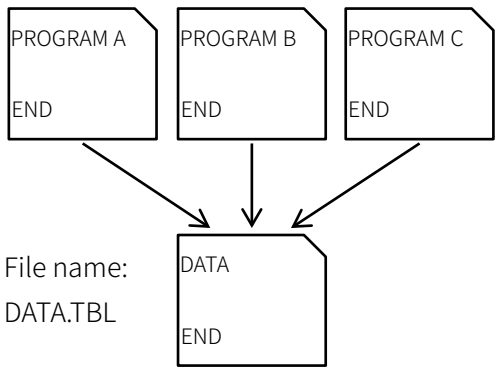


Fig.2.1 Commonization of data section

Program section commonization allows a program file (SCOL.PRG) with one program section to be commonly used by multiple programs with data sections. The name of the program file with the common program section must be SCOL.PRG.

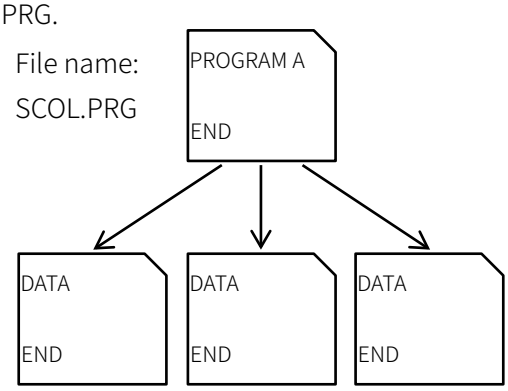


Fig.2.2Commonization of program section

Parameter setting of "USER.PAR" is required for commonization of program and data sections.

Parameter key	COMMON-FILE-FUNC	
Parameter data	= (function selection)	
Data type	Integer type	
Number of data	1	
Data range	0 to 2	
Description.	=0	Disable
	=1	Commonizes the data section. It is necessary to create “DATA.TBL”.
	=2	Commonizes the program section. It is necessary to create a "SCOL.PRG" file.
Default	=0	

Please refer to the "ST85464 TSAssist User's Manual" for details on usage.

2.2 Character Set

The SCOL2 character set is made up of alphanumeric characters and the following special symbols.

Alphanumeric characters

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
a b c d e f g h i j k l m n o p q r s t u v w x y z
1 2 3 4 5 6 7 8 9 0

Special symbols

“ ’ ( ) + - * / , . < > =
! [ ] { } % ^ & ? _
spaces

With the exception of the small letters, these characters and symbols can all be input from the teach pendant. When executing a program, the robot makes no distinction between capital letters and small letters. For reading method of symbols, see “Appendix D.”

2.3 Identifiers

In the SCOL2 language, commands, program names, variable names, and label names that specify the branch destination of a program are all represented by identifiers. Identifiers must be represented by letters, numbers, and underscores “_”. The maximum length is 20 characters. They shall not start with a number. Also, it is impossible to use only one underscore as a variable name.

Example)

OK:

L1

_Z_OFFSET

_123

NG :

1ABC	Character strings that start with a number cannot be used as an identifier.
%123	Symbols other than underscores cannot be used.

The robot makes no distinction between capital letters and small letters. All three identifiers below are considered “SHIBAURAROBOT”.

Example)

SHIBAURAROBOT

shibaurarobot

ShibauraRobot

Identifiers already being used in SCOL2 language cannot be used for another purpose. In addition to SCOL2 language commands, some of these identifiers have special usages in the system or are reserved for future expansion. For details on the reserved SCOL2 language commands, see Appendix B. Also, identifiers that start with two underscores (__A, etc.) are used in libraries provided by the manufacturer; therefore, it is possible to compile, but doing so may cause unexpected behavior, so do not use them .

Do not use identifiers with the same name for different meanings. For example, when creating a program, do not use the same identifier for both the program name and variable name.

2.4 Programs

This paragraph describes SCOL2 programs.

2.4.1 Program Declaration

Declaration of a combination program of SCOL2 commands is written in the following format.

```
PROGRAM    <name of your program>
    Contents of your program
END
```

A program is made up of everything from the PROGRAM statement to the END statement. Write a program name after the PROGRAM statement. The program name is expressed using an identifier. Put the content of the program between the PROGRAM statement and the END statement.

Example)

PROGRAM SAMPLE	Program name “SAMPLE”
REMARK SAMPLE	Comment
SPEED = 20	Set the movement speed to 20% of the maximum speed.
MOVE A1	Move the robot to position A1.
DELAY 0.5	Wait for 0.5 sec.
MOVE A2	Move the robot to position A2.
DELAY 0.5	Wait for 0.5 sec.
END	End of program

2.4.2 Statements

As shown in the example, the body of the program is composed of statements made up of an arrangement of SCOL2 commands. Normally, a statement is a line until a carriage return. Up to 130 characters can be contained in a single line. You may add spaces as you wish in order to make the program neater and easier to read. You can write a remark and comment using the symbol (') in a statement.

No spaces can be placed between characters structuring a word of a command and identifier.

2.5 Constants

The values and character strings contained in a program are called constants. Constants can be scalar type (integer type, real number type), vector type (PAYLOAD type, TRANS type, POINT type, and JOINT type), or character strings, and these are called scalar type constants, vector type constants, or character string type constants, respectively.

2.5.1 Scalar type constants

[1] INT type (integer type) constants

SCOL2 can handle integer values (“whole numbers”) in the range of -2147483648 to +2147483647. When an integer is used as a constant in a program, if it is positive, directly describe the value; if it is negative, describe the value following the “-” symbol.

Example)

```
0
234
-39208
5963
```

Constants in hexadecimal format such as 0x01 and 0xFFFF can also be used. The values are handled as signed 32-bit integers, so 0x01 indicates 1, 0xFFFF indicates 65535, and 0xFFFFFFFF indicates -1. Either uppercase or lowercase can be used for hexadecimal constants, but add the prefix “0x” or “0X”.

Example)

```
0x123
0x1
0X00FF
```

[2] REAL type (real number type)constants

SCOL2 can handle real numbers having an absolute value in the range of approximately 5.87×10^{-39} (2^{-127}) to 6.80×10^{38} $\{(2^{23}-1)\times2^{106}\}$. The number significant digits for the mantissa [the mantissa is the part of the number to the right of the decimal point) is approximately 7 in Base 10. (The precision is 2^{23}). The number of allowable digits is 9 for the integer and 3 for the decimal. If a value consisting of more than these digits is entered by the INPUT command, an error occurs.

When a real number is used in the program, if it is positive, directly describe the value; if it is negative, describe the value following the "-" symbol. When the decimal part is 0, it is omissible. However, when the decimal point is omitted, the data are treated as INT type data. In addition, since the integer part cannot be omitted, even if the absolute value of a numeric value is less than 1, it is necessary to designate 0 to the integer part.

Example)

```
      1234.567
        -28.16
           0.00985
           1.0
1234567.
     -369.
```

When setting values after the decimal place, try to keep the minimum setting units shown below because the precision of the execution results is limited.

Distance (X, Y, and Z data)	0.001	mm
Angles (C)	0.001	deg.
Time	0.01	sec.
Rates (Speed, torque, etc.)	1	%
Mass	0.01	kg
Inertia	0.01	kg·m

2.5.2 Vector type constants

Vector type constants are written by enclosing values in { } such as in the format {<Value>, <Value>, ...}
The constant is determined to be PAYLOAD, TRANS, or POINT type based on the number of numerical values it contains (number of elements).

[1] PAYLOAD type (load type) constants

A PAYLOAD type constant is a vector type constant with two elements.
It is written in the format {<Weight>,<Center of gravity offset>}.
The weight and center of gravity offset are real numbers.

Example)

```
PL = { 10.0 , 10.0 }
```

[2] TRANS type (coordinate type) constants

A TRANS type constant is a vector type constant with four elements.
It is written in the format {<X-coordinate value>, <Y-coordinate value>, <Z-coordinate value>, <C-coordinate value>}.
Each coordinate value is a real number.

Example)

```
T = { 10.0 , 20.0 , 30.0 , 40.0 }
```

[3] POINT type (position type) constants

A POINT type constant is a vector type constant with six elements.
It is written in the format {<X-coordinate value>,<Y-coordinate value>,<Z-coordinate value>, <C-coordinate value>,<T-coordinate value>, <CONFIG>}.
Each coordinate value is a real number, and CONFIG is an integer value.
CONFIG can also be expressed by the system constants (FREE, LEFTY, RIGHTY).

Example)

```
P1 = { 10.0 , 20.0 , 30.0 , 40.0 , 50.0 , 1 }
P2 = { 10.0 , 20.0 , 30.0 , 40.0 , 50.0 , LEFTY }
```

[4] Omitting of elements in vector type constants

For vector type constants, if the type is determined by the statement location, the inclusion of values in the { } can be omitted. The omitted values are treated as 0.

Example)

```
P = { 10 , 20 , 30 }
MOVE { 400 , 20 }
```

If P is a POINT type variable, the vector type constant for the right side is also a POINT type variable.
Because MOVE is followed by POINT type values, the values are POINT type constants.

If the constant type at a certain location is unclear, the type is determined based on the number of numerical values as shown below.

{<Value>}	•••• PAYLOAD type
{<Value>,<Value>}	•••• PAYLOAD type
{<Value>,<Value>,<Value>}	•••• TRANS type
{<Value>,<Value>,<Value>,<Value>}	•••• TRANS type
{<Value>,<Value>,<Value>,<Value>,<Value }	•••• POINT type
{<Value>,<Value>,<Value>,<Value>,<Value>,<Value>}	•••• POINT type

2.5.3 Character string type constants

Character string literals are strings where alphanumeric characters and delimiters are enclosed by double quotations, and letters are case-sensitive.
The strings enclosed in double quotations can also include the characters and special characters shown in section 2.2.
Also, the symbol (\), double quotation (“), line feed (LF), carriage return (CR), or tab can be specified within double quotations by using the escape character (\). The escape character is not included in the character count for the character string.

Writing special characters using escape characters
\ → \ \
“ → \ “
Line feed (LF) → \n or \N
Carriage return (CR) → \r or \R
Tab → \t or \T

Example)
“SHIBAURA MACHINE”
“ROBOT123 \ R “

2.5.4 System constants

In order to make programs easier to read, SCOL2 provides the system constants shown in Table 2.1. These names can be substituted into the program in place of numbers in order to make it easier to see what you are doing. System constants can be referenced from any location in the program.

Table 2.1 List of system constants

Name	Value	Comments (Locations for use)
FREE LEFTY RIGHTY	0 1 2	In the system variable CONFIG CONFIG of POINT type variable
COARSE FINE	0 1	Reference or setting of system variable ACCUR
OFF ON	0 1	In the system variable GAIN In the SETGAIN command
PAI	3.141593	Pi value
CONT CYCLE SEGMENT	0 1 2	In the MODE command
TP IP1 IP2 IP3 IP4 IP5 IP6 IP7 IP8	0 3 4 5 6 7 8 9 10	Communication channel used by PRINT/INPUT command

Example)

```
IF (P1. CONFIG = = LEFTY)
A = 2 * PAI
MODE = CONT
```

2.6 Variables

A variable is an area where a numerical value is temporarily stored in SCOL2 language. Multiple variables are separated by adding identifiers (variable names).
Constants are objects where identifiers were added to frequently used values in SCOL2 language. They are pre-installed in the system and are also called system constants.

Variables and constants are considered to be divided based on the data handled. These divisions are called data types, and in SCOL2 language, scalar type (integer type, real number type), vector type (POINT type, TRANS type, PAYLOAD type, and JOINT type), and character string type can be used. The variable type is determined by an explicit declaration using a DIM command or when a value is substituted for the first time. After the variable type is determined, the type does not change. A variable whose type has not been determined yet is called an undefined variable.

Variables are divided into global variable and auto variable (local variable) according to the definition method. All taught data and variable defined in the area between the reserved words GLOBAL and END are called the global variable. Such variables can be referred and changed from any part of the program. For all data types of global variables, the array can be declared. For descriptions of global variable and array, see Paragraphs 2.6.4 and 2.6.5.

The work area in the controller is used for all data. The defined value is substituted for the global variables other than the array without a specific initial value at the start of the program. If the value is entered for the variable during program execution, only the work area is changed. If the power of controller is turned off, execution file is reselected or the file is edited, work area is reset by the variable's initial value saved in the file and the changed value is lost accordingly. This is also applicable for change of the taught data. Backup variables (MEMORYI, MEMORYJ, MEMORYL, MEMORYP, MEMORYR, MEMORYT) can be used to retain the value for variables even after the power is turned off or to share use of a variable with another program. For a detailed explanation of backup variables, see 3.2.

2.6.1 Scalar type variables

There are two types of scalar variables -- integer type and real number type. Each type can store one value.

[1] INT type (integer type) variables

SCOL2 can handle integer values in the range of - 2147483648 to + 2147483647.
When a real number is substituted in an integer type variable, the real number portion after the decimal point is dropped, and the value is substituted as an integer value.

[2] REAL type (real number type) variables

SCOL2 can handle real numbers having an absolute value in the range of approximately 5.87×10^{-39} (2^{-127}) to 6.80×10^{38} $\{(2^{23}-1)\times2^{106}\}$. The number significant digits for the mantissa [the mantissa is the part of the number to the right of the decimal point) is approximately 7 in Base 10.

2.6.2 Vector type variables

As opposed to scalar type data which only holds one data element, vector type data holds multiple data elements. Vector data includes four types -- POINT type, TRANS type, PAYLOAD type, and JOINT type. Position data that was taught in the data editor (data contained in the global section) is handled as vector type global variables.

To access elements of vector type data, .write <Element Number> or .<Element Name> after Variable Name.
The elements of vector type data can be calculated as scalar type data.

[1] POINT type (position type) variables

POINT type data is used as robot position data.
The component elements of POINT type data are as follows.

Element number	Element name	Element type	Value
1	X	REAL type (real number type)	X value [mm]
2	Y	REAL type (real number type)	Y value [mm]
3	Z	REAL type (real number type)	Z value [mm]
4	C	REAL type (real number type)	C value [deg]
5	T	REAL type (real number type)	T value [mm] or [deg]
6	CONFIG	INT type (integer type)	0 • • • Free (CONFIG undefined) 1 • • • Left-handed system 2 • • • Right-handed system

Example)

P = POINT(300 , 300 , 100 , 0 , 0 , 1)	Substitutes into POINT type variable P
P. X = 100.0	Substitutes into P element using element name
R1 = P. 1	Obtains value where element number is used
IF (P. CONFIG = = 1)	Determines element using IF statement

[2] TRANS type (coordinate type) variables

The TRANS type data is used to specify the robot coordinate system. The component elements of TRANS type data are as follows.

Element number	Element name	Element type	Value
1	X	REAL type (real number type)	X value [mm]
2	Y	REAL type (real number type)	Y value [mm]
3	Z	REAL type (real number type)	Z value [mm]
4	C	REAL type (real number type)	C value [deg]

Example)

T = TRANS(300 , 300 , 100 , 0)	Substitutes into TRANS type variable T
T. Y = 100.0	Substitutes into T element using element name
TT = T. 1	Obtains value where element number is used
IF (T. C > = 0)	Determines element using IF statement

[3] PAYLOAD type (load type) variables

The PAYLOAD type data defines the load applied to the robot hand including the hand section (end effector section). The component elements of PAYLOAD type data are as follows.

Element number	Element name	Element type	Value
1	M	REAL type (real number type)	Weight of hand load [kg]
2	L	REAL type (real number type)	Center of gravity offset [mm]

Example)

PL = PAYLOAD(1.5 , 30)	Substitutes into PAYLOAD type variable PL
PL. L = 100.0	Substitutes into PL element using element name
W = PL. 1	Obtains value where element number is used
IF (PL. L > = 50)	Determines element using IF statement

[4] JOINT type variables

Joint type variables are used for storing values for robot joint 1st to 5th axes.

Element number	Element name	Element type	Value
1	J1	REAL type (real number type)	1st axis value [deg]
2	J2	REAL type (real number type)	2nd axis value [deg]
3	J3	REAL type (real number type)	3rd axis value [mm]
4	J4	REAL type (real number type)	4th axis value [deg]
5	J5	REAL type (real number type)	5th axis value [mm] or [deg]

Example)

```
J = JOINT(0 , 300 , 100 , 0 , 0)      Substitutes into JOINT type variable J
J. J1 = 100.0                         Substitutes into J element using element name
JJ = J. 1                             Obtains value where element number is used
IF (J. J1 > = 0)                      Determines element using IF statement
```

2.6.3 Character string type variables

STRING type variables are used for storing strings.
Character string type variables are global variables only. Auto variables that are character string type variables cannot be used.
In contrast to other data types, character string type arrays can use one-dimensional arrays only.
The data type for character string type variables can be either explicitly declared as character string type by a DIM command, or it can be defined by substituting a character string constant into an undefined data type variable.
The length of the area for an undefined variable where a character string constant was declared is automatically calculated from the number of characters that were substituted. Character string type variables that were explicitly declared by a DIM command have an area of the size that was specified in the DIM command. The size can be specified from 1 to 512. Character string type variables with 0 characters cannot be defined. However, a character string constant with 0 characters can be substituted into a character string type variable that was already defined.

Example)

```
GLOBAL
   DIM A AS STRING(10)          Defines a string type variable of size 10.
   B = "ABCDE"                  The size will be 5 of the string length.
   DIM C(2) AS STRING(8)        Defines a string type array of size 8 and number
   C(1) = "123"                 of elements 2.
   C(2) = "456"
END

PROGRAM MAIN
   A = B + C(1)                 Strings can be combined with the + operator.
   PRINT TP, A, CR              ABCDE123 is output.
END
```

String expressions cannot be assigned to variables with undefined data types.

Example)

```
GLOBAL
   A = "ABCDE"
   B = A                        A compilation error will occur.
END
```

Only data of string type can be assigned to a variable of string type. To convert a numerical value to a string, the FORMAT command must be used.

The areas where the character string type variables are actually determined are as follows.
Area length (2 bytes) + Character string (selectable 1 to 512 bytes) + Terminator code (0x00 1 byte)
A value indicating the number of characters in the character string is stored in the area length. The terminator code is not included in the number of characters.
The specified size is contained in the area length of the character string type variable that was defined by the DIM code. The character string section is initialized by 0x00.

2.6.4 Global variables and auto variables

Variables include global variables and auto variables (local variables).
Global variables can be referenced and changed from any location in the main program or subprogram.
Auto variables, on the other hand, can be used in declared programs only. For this reason, the content of variables may be different even if a variable with the same name is used in the main program and subprogram.
When data is passed from one program to another, either the variable is defined as a global variable, or the value is passed as an argument of a subprogram.

[1] Global variable declaration

A scalar type variable is declared as a global variable using a DIM command in the global section.

```
GLOBAL
  DIM A AS INT                    INT type global variable A declaration
  DIM B AS REAL                   REAL type global variable B declaration
END
```

Variables with an initial value are declared as shown below in the global section.

```
GLOBAL
  A = 0                           INT type global variable A (initial value 0)
  B = 10.5                        REAL type global variable B (initial value 10.5)
END
```

Vector type global variables are declared in the data section. In other words, all position data that was created in the data editor is handled as global variables.
Vector type variables cannot be defined in the global section.
(Vector type variables are contained in the data section.)

[2] Auto variable declaration

Auto variables are valid within the program only.

In programs where you want to use variables, a DIM command is used to make declarations as shown below.

```
PROGRAM ABC
  DIM A AS INT                    INT type auto variable A
  DIM B AS REAL                   REAL type auto variable B
  DIM C AS POINT                  POINT type auto variable C
  DIM D AS TRANS                  TRANS type auto variable D
  •••••
END
```

Also, auto variables that have an initial value can be declared by substituting the initial values into the identifier. Data type variables are determined based on the data type of the value that is first substituted into the variable. In other words, if integer type data is first substituted, the variable becomes an integer type variable.

```
PROGRAM ABC
  A = 10                          INT type auto variable A (initial value 10)
  B = 0.5                         REAL type auto variable B (initial value 0.5)
  C = INT(10.0)                   INT type auto variable C (initial value 10)
  D = POINT(300 , 300 , 100 , 0 , 0 , 1)   POINT type auto variable D
  •••••
END
```

Auto variables contained in a program can be used in the declared program only. For this reason, even if an auto variable with the same name is used in another program, it is treated as a separate variable, so its content is different.

2.6.5 Array variable

In scalar type variables and vector type variables, one identifier is used to express one data entry. However, a program can be simplified by adding numbers to a single identifier for expressing multiple data entries. A variable where numbers were added to a single identifier is called an array variable.
In SCOL2 language, array variables are global variables. (Array variables cannot be auto variables.)

[1] Array variable declaration

To handle array variables, the type and total number of elements must be declared using a DIM command in the global section. All array variables are global variables. Scalar type and vector type array variables can be a maximum of 5 dimensions. Arrays for character string type variables can be 1 dimension only.

DIM <Variable name > (<number of elements 1> [,<number of elements 2>] … [,<number of elements 5>])

Example of array declaration

```
GLOBAL
  DIM A(10) AS INT                  Defines ten integer type arrays
  DIM B(10 , 3) AS REAL             Defines 10×3 real number type arrays
  DIM C(5) AS POINT                 Defines five POINT type arrays
  DIM G(5) AS STRING(10)            Defines five character string type variables with
END                                 a maximum of ten characters
```

Variables where an array was declared by a DIM command in the global section are written in the program in the following format.

Variable name (<element number> [,<element number>] … [,<element number>])

For the element number following the variable name, any value ranging from 1 to a number specified by the DIM command can be selected. However, if the total number of “variable name + “(“ + element + “)” exceeds twenty (20) characters, an execution error is generated.
When the SCOL2 program is executed, specific one (1) data is selected from multiple data based on the variable name and element number.

In the following example, the values are stored to a 3-element array variable called “A”.

```
GLOBAL                                  Global section
  DIM A(3) AS INT                       Declares 3-element integer type array variable A
END
PROGRAM SAMPLE
  A(1) = 123 + 5
  A(2) = 520
  A(3) = -200
END
```

[2] Initial value setting of array variable

In the same way as non-array global variables, the initial values of scalar type array variables should be written in the global section, and the initial values of vector type array variables should be written in the data section.

The initial value of the array type global variable, which is not set clearly, is indefinite.
The variable is required to be initialized by the user program.

The following example is a program that totals the values that were set to a 5-element array variable with the identifier “A”.
The array declaration by the DIM command and the initial value are set in the global section.

```
GLOBAL                                  Global section
  DIM A(5) AS INT                       Declares 5-element integer type array variable A
  A(1) = 1                              Sets initial value
  A(2) = 5
  A(3) = 100
  A(4) = 2
  A(5) = 30
END
PROGRAM SAMPLE
  TOTAL = 0
  FOR I = 1 TO 5
    TOTAL = TOTAL + A(I)
  NEXT I
END
```

In vector type variable arrays, an array declaration by a DIM command is written in the global section, and the initial value is written in the data section. Specify the initial value of the vector type variable in the data section that is enclosed by the reserved words from DATA to END and edited by the data editor.

When 25 positions (5 x 5) are processed as the array variable named “P” and each axis is moved to respective positions in turn, the program is as follows. (Write the initial value of the POINT type array variable P(5 , 5) in the data section.)

```
GLOBAL                                          Global section
  DIM P(5 , 5) AS POINT                         Declares 5×5 POINT type array variable P
END
PROGRAM SAMPLE                                  Program section
  FOR I = 1 TO 5
    FOR J = 1 TO 5
      MOVE P(I , J)
    NEXT J
  NEXT I
END
-------------------------------------------------------------------------------------------
DATA                                            Data section
  POINT P(1 , 1) = 650 , 0 , 100 , 0 , 0 / LEFTY
  POINT P(1 , 2) = 650 , 0 , 110 , 0 , 0 / LEFTY
                          •
                          •
                          •
  POINT P(5 , 5) = 650 , 0 , 150 , 0 , 0 / LEFTY
END
```

In the above example, if the movement to position P (I, J) is commanded by mistake while I = 5 and J = 6, an execution error is generated because access to or substitution of an element other than specified by the DIM command is not possible. Array index types are allowed for integer types only. If real number type or vector type data is used, an error occurs when they are executed.

2.6.6 System Variables

The SCOL2 language provides special variables that are used in the programs to specify and referent robot operating conditions and system conditions. These variables are called system variables. Just like other variables, you can refer to these variables in the program, change their value, etc. However, you have to be careful when setting or substituting values into system variables since doing this will directly effect robot operating conditions. Furthermore, be warned that SCOL2 does not check to see whether a value substituted into a system variable is within the permissible range. When the value is not in the permissible range at program execution, the system sets a value according to the following rules.

- When you try to insert a value less than the minimum permissible value, the minimum permissible value will be entered in its place.
- When you try to insert a value greater than the maximum permissible value, the maximum permissible value will be entered in its place.

System variables pertaining to robot arm movement control in particular are called movement condition system variables.

[1] Movement condition system variables

When the content of a movement condition system variable is updated, the result will affect the movements which follow it, and the movement being executed is not affected. System variable results are retained until the system variable is set again. However, a WITH clause can be used to temporarily set a system variable value for a single movement command only.

Example)

```
MOVE A1 WITH SPEED = 50
```

Table 2.2 Movement condition system variables

Name	Description	Valid values	Initial value	Data type
ACCEL	Specifies acceleration (during acceleration).	0 to 100%	100	INT type
ACCUR	Specifies positioning accuracy.	0 , 1	1	INT type
BASE	Base coordinate system			TRANS type
CONFIG	Robot configuration	0 , 1 , 2		INT type
CPACCEL	CP movement (MOVES, MOVEC) hand acceleration			
CPDECEL	CP movement (MOVES, MOVEC) hand deceleration			
CPSPEED	CP movement (MOVES, MOVEC) hand speed			
DECEL	Specifies acceleration (during deceleration).	0 to 100%	100	INT type
GAIN	Servo gain for selected joint axis	0 , 1		
LOCUS	Path tracking precision	0 , 1		INT type
OVERRIDE	Override for movement speed	0 to 100%	100	INT type
PASS	Shortcut movement parameter (%)	50 to 100%	100	INT type
PASSMM	Shortcut movement parameter (mm)			
PAYLOAD	Robot hand load data			PAYLOAD type
SLOWDOWN	Slowdown movement parameter			
SLOWSTART	Slow start movement parameter			
SLWSPD	Slow down/Start movement slow speed			
SPEED	Movement speed	0 to 100%	100	INT type
SPURT	Acceleration automatic adjustment function			
TOOL	Tool coordinate system			TRANS type
TORQUE	Torque limit value for selected joint axis	0 to max %	300	
WORK	Work coordinate system			TRANS type

*The maximum value is set for each system.

2.7 Expressions

This paragraph describes expressions provided by SCOL2 for substitution, calculation and judgment.

2.7.1 Computational Expressions

In the SCOL2 language, the results of computations on the right side of an equal sign are placed on the left. Variables and constants may be used in the expressions.

In SCOL2 language, expressions can be not only used independently to perform substitution and calculation, but also used within commands. Expressions include computational expressions where the calculation result is substituted into a variable and logical expressions that determine greater than/less than or true/false results.
The following operators can be used. Note that the execution results of “0 / 0” and “0^0” are not returned as errors but returned as 0 and 1, respectively.

Table 2.3 Operands

Type	Operand	Function	Example
Calculation functions	^	Exponentiation	A ^ B (A to the B power)
	-	Minus sign	-A
	* , /	Multiplication, division	A * B , A / B
	+ , -	Addition, subtraction	A + B , A - B
	MOD	Remainder	A MOD B (The remainder when A is divided by B.)
	=	Substitution	A = B (Puts the value of B into A.)
Relational function	= =	Equal	A = = B
	< > , > <	Not equal	A < > B , A > < B
	<	Less than	A < B
	>	Greater than	A > B
	< = , = <	Less than or equal	A < = B , A = < B
	> = , = >	Greater than or equal	A > = B , A = > B
Logical operands	AND	Logical product	A AND B
	OR	Logical sum	A OR B
	NOT	Negation	NOT A
Character string operands	+	Character string concatenation	TEXT + “ABC”

Parentheses ( ) may be used inside the expressions.

2.7.2 Logical expressions

With SCOL2, logical expressions can be used in combination with the commands IF, WAIT and ON.
Also, six relational operands are available (<, >, < = (or = <), > = (or = >), < > (or > <), and = =). Also, logical expressions may be combined using the logical operands AND, OR and NOT. Scalar constants, scalar variables and the results of calculations may be used as data in logical expressions.
When evaluating equivalence, use the "= =" sign and not the "=" sign. When comparing real numbers, differences of 0.001 or less will be ignored.
Logical expressions will return an integer value of 1 if true and 0 if false. The calculation results will be an integer value.

Example 1)

```
IF K = = K2 * K3 THEN K = K2
ON MOTION > = 50 DO DOUT(1 , 2)
```

Example 2)

When "IF J1 THEN GOTO TRUE1 ELSE GOTO FALSE1" is executed, if J1 is recognized as an integer value of 0 or a real number value where $|J1| \leq 0.001$, the result is FALSE, and this branches to FALSE1.
If J1 is recognized as a non-zero integer value or is a real number value where $|J1| > 0.001$, the result is TRUE, and this branches to TRUE1.

[1] Logical values

Logical values are used in the program when making conditional judgments. Logical expressions and commands such as DIN (which check input signals) return logical values.
A logical value may have one of two values: TRUE or FALSE. Internally, this is handled as integer data, and a TRUE result is 1, and a FALSE result is 0.
Strictly speaking, 0 is considered as FALSE and everything else is considered as TRUE.

2.7.3 Priority Order of calculation operations

In SCOL2 language, calculation is performed in the same order of priority as regular computational operations. Specifically, operations are performed based on the rules below.
If the expression contains brackets, the operations inside the brackets are performed first.
Operations are performed in the order of assignment of negative signs, calculation of exponents, multiplication and division (*, /), and addition and subtraction (+, -).
If two operations have the same priority, the operations are performed from the left to the right of the expression.
Table 2.4 presents the order of computational priority for various operations.

Table 2.4 Order of computational priority

Priority	Operation	Operand	Grouping convention
High	Parenthesis	( )	Left to right
	Specification of vector type elements	.	Left to right
	Assignment of negative signs and negations	- , NOT	Right to left
	Multiplication, division, remainder and exponentiation	* , / , MOD , ^	Left to right
	Addition, subtraction	+ , -	Left to right
	Comparison	< , > , < = , > = , = < , = >	Left to right
	Equality, inequality	= = , < > , > <	Left to right
	Logical product, logical sum	AND , OR	Left to right
Low	Substitution	=	Right to left

Explanation of grouping convention

Left to right • • • 1 + 2 – 3 is interpreted as (1 + 2) – 3.

Right to left • • • NOT–3 is interpreted as NOT (–3).

Example)

a = b + c * d / (e - f) - g

The order of computation for the above expression is

1. Calculate e – f. e - f
2. Calculate c * d. c * d
3. Divide c * d by e – f. (c * d) / (e - f)
4. Add the above result to b. b + (c * d / (e - f))
5. Subtract g from the above result. (b + c * d / (e - f)) - g

2.7.4 Substitution into scalar type variables

Type variables automatically substitute values into scalar type variables.
If the left side variable is an integer value, the right side calculation result is automatically converted to an integer value and substituted. When converting from a real number value to an integer value, the portion after the decimal point is discarded.
If the left side variable is a real number value, the right side calculation result is automatically converted to a real number value and substituted. When converting from an integer value to a real number value, be careful because the number of significant figures is limited.
Vector type variables cannot be directly substituted into scalar type variables. Scalar type variables can store only one value, and so for vector type variables, take out the elements, and then substitute them.

Example)

```
A = P1. X
```

2.7.5 Computation of scalar type data

Scalar type data can be used in calculations in combination with computational operands. Vector type data elements can also be calculated as scalar type data. If any real number data is contained in an expression, the calculation result will be a real number. Also, the following functions will all return a real number.
SIN , COS , TAN , ASIN , ACOS , ATAN , ATAN2 , SQRT , REAL , LN , LOG10 , EXP

To explicitly clarify the data type, use the INT and REAL commands.

Example)

```
A = P1. X / 25
P1 = POINT(P2. X , P2. Y , 0 , 0 , 0)
```

2.7.6 Substitution into vector type data

Data can be substituted only if both the right and left sides are the same type of vector variables.
In other words, POINT type data can be substituted to POINT type variables only.
To substitute constants, variables, and calculation results into vector type data, commands must be used to convert to scalar type data and vector type data. Data conversion commands include POINT(), TRANS(), PAYLOAD(), and JOINT().
These commands can be used to substitute into vector type data. The POINT command generates POINT type data, and the TRANS command generates TRANS type data. If an element is not included in the expression, that element is treated as 0 within the expression. For details on how to use these commands, see "Section 3."

Example)

```
P1 = POINT(P2. X , P2. Y , P2. Z + 50 , 0 , LEFTY)
T1 = TRANS(100 , 100 , 0 , 0)
PL = PAYLOAD(100 , 100)
J1 = JOINT(0 , 0 , 100 , 0 , 0)
```

When declaring POINT type and TRANS type data, use the POINT command and TRANS command for explicitly indicating the data type.

When position data which have not been taught are used in a program of the robot language, the position data are temporarily stored in the controller memory. Thus, when the program is reset, the position data are cleared. The position data are only valid in the program which uses data. Therefore, to use the position data in a subprogram, it is necessary to pass it as an argument. For details of arguments, see "2.11.2 Subprogram arguments."

The substitution and reference to the array type data (type of variable name [index number]) are dealt in the same manner as the original data type (scalar type and vector type) of the array type data.

When substituting values into vector type data, the elements of the vector type data can be indicated by enclosing by { } on the right side.

Example)

```
P1 = { 100.0 , 100.0 , 100.0 , 0.0 , 0.0 , LEFTY }
```

If the elements in { } are not enough for the number of elements for the variable type on the left side, the omitted element is assumed to be 0, and the value is substituted. On the other hand, if there are too many elements, the excess elements are ignored.

If the type of variable on the left side is not determined, the variable type on the left side is determined based on the number of elements in { }.

Example) Substitution into undefined variables

V1 = { 100.0 }	Because there is one element, V1 becomes a PAYLOAD type variable.
V2 = { 100.0 , 200.0 }	Because there are two elements, V2 becomes a PAYLOAD type variable.
V3 = { 100.0 , 200.0 , 300.0 }	Because there are three elements, V3 becomes a TRANS type variable.
V4 = { 100.0 , 200.0 , 300.0 , 400.0 }	Because there are four elements, V4 becomes a TRANS type variable.
V5 = { 100.0 , 200.0 , 300.0 , 400.0 , 500.0 }	Because there are five elements, V5 becomes a POINT type variable.
V6 = { 100.0 , 200.0 , 300.0 , 400.0 , 500.0 , 1 }	Because there are six elements, V6 becomes a POINT type variable.

However, because it can be difficult to know the type of variable, it is recommended that the following declarations be made when substituting values into undefined variables.

```
V2 = PAYLOAD(100.0 , 200)
V4 = TRANS(100.0 , 200.0 , 300.0 , 400.0)
V6 = POINT(100.0 , 200.0 , 300.0 , 400.0 , 500.0 , LEFTY)
```

2.7.7 Computation of vector type data

Calculations can be performed using operands (+, -, *) for vector type data.

[1] Addition and subtraction of vector type data elements

You can add and subtract corresponding elements of two vectors. Computation is a possible only between the same type variables. The <CONFIG> element is not involved in the calculations but rather takes the value of the variable substituted into it.

Example)

Given the following two position vectors and two coordinate vectors;

P1 : POINT(10 , 20 , 30 , 40 , 50 , RIGHTY)

P2 : POINT(-5 , 10 , -15 , 20 , -25 , LEFTY)

H1 : TRANS(100 , 50 , -50 , 0)

H2 : TRANS(12 , 34 , 56 , 78)

and performing the following operations,

P3 = P1 - P2

H3 = H1 - H2

we obtain:

P3 : POINT(15 , 10 , 45 , 20 , 75 , RIGHTY)

H3 : TRANS(88 , 16 , -106 , -78)

In the above example, the term of <CONFIG> of P3 is <CONFIG> of P1.

[2] Matrix calculation of vector type data (parallel movement and rotation)

Matrix calculation of vector type data (parallel movement and rotation) is written using the operand "*".

Matrix calculation is possible using the following types of vector type variables.

<POINT type variable> = <POINT type variable> * <POINT type variable>

<POINT type variable> = <POINT type variable> * <TRANS type variable>

<POINT type variable> = <TRANS type variable> * <POINT type variable>

<TRANS type variable> = <TRANS type variable> * <TRANS type variable>

2.7.8 Substitution of character string type data

Character string type data can be substituted from character string type data to character string type variables only.
If the area length of the character string type variable for the substitution target is larger than the number of characters in the character string type data of the substitution source, the excess area for the character string type variables of the substitution target is filled with 0x00.
If the area length of the character string type variable for the substitution target is smaller than the number of characters in the character string type data of the substitution source, the stored character string is left-justified, and the excess length is cut off.

Example)

```
DIM TXT AS STRING(5)
TXT = “ABC”
```

In this case, the character string type data “ABC” is stored in the character string type variable TXT, and the area for the 4th to 5th characters is filled with 0x00.

```
DIM TXT AS STRING(5)
TXT = “ABCDEFGHIJK”
```

In this case, the character string type data “ABCDE” is stored in the character string type variable TXT, and “FGHIJK” is cut off.

```
DIM TXT AS STRING(10)
TXT = “ABC” + FORMAT(255 , “XXXX”)
```

In this case, the character string type data “ABC00FF” is stored in the character string type variable TXT. The code 0x00 is stored for the 8th to 10th characters.

2.7.9 Character string concatenation

Only the “+” operand can be used for calculation of character string type data. The “+” operand joins character string type data.
It is a character string function that returns character string type variables, character string constants, and character strings that can be joined.
Parentheses ( ) cannot be used for changing the order that character strings are joined in character string concatenation.
Example)

TXT = A + “ABC”

The character string formed by joining the character string stored to the character string type variable A and the character string constant “ABC” is stored to the character string type variable TXT.

TXT = “ABC” + (“DEF” + “GHI”)

Parentheses ( ) cannot be used as shown in the example above for changing the order that character strings are joined.

Instead, the parentheses that can be used in character string concatenation are as follows.
Parentheses indicating argument of character string function
Example)

TXT = “ABC” + CHR(13)

Parentheses specifying the order of calculation operations in character string functions where the argument is specified
Example)

TXT = “ABC” + CHR(10 + (20 + 30))

When the concatenation order does not change even if parentheses are added
Example)

TXT = (“ABC” + “DEF”) + “GHI”

2.7.10 Comparison calculation for character string type data

Comparison calculations for character string type data are performed as follows. Character strings comparisons are case-sensitive.

Example)

```
IF ABC = = "abc" THEN
   TXT = "SAME"
ENDIF
```

In this case, if the content of the character string type variable ABC is “abc”, the condition is satisfied.

```
IF ABC <> "abc" THEN
   TXT = "DIFF"
ENDIF
```

In this case, if the content of the character string type variable ABC is not “abc”, the condition is satisfied.

To compare a part of a character string, use the string function.

Example)

```
IF STRLEFT(ABC, 3) = = "abc" THEN
   TXT = "SAME"
ENDIF
```

In this case, if the first three characters of the character string type variable ABC are “abc” , the condition is satisfied.

* The system X8KCS-02H or older cannot compare character string type data using comparison operators directly. Use the STRCOMP command to compare character strings.

2.8 Commands and functions

This section shows the commands by function. For details on a specific command, see the explanations in chapter 3, Commands.

2.8.1 Program control commands

Commands	Purpose
DATA ..... END	Specifies data section
DIM~AS~	Defines variables and array variables
FOR~TO~(STEP~)..... NEXT	Specifies repeating of program
GLOBAL.....END	Specifies global section
GOTO	Moves to statement having label that specified program execution
GOTO ( )	Branches program execution based on expression in ( )
IF~THEN~(ELSE~)	Conditional statement
IF~THEN~(ELSEIF~ THEN~)(ELSE~)..... ENDIF	Conditional statement (Can be written over multiple lines)
IGNORE	Cancels monitoring of conditions specified by ON command
KILL( )	Stops multi-task operation
LOADLIB	Reads dynamic link library
ON~ (BREAK) DO~	Monitors conditions (BREAK aborts movement and executes the command following DO)
PAUSE	Waits for the end of robot movement and executes the command following DO
PROGRAM ..... END	Specifies program section
RCYCLE:	Label for resetting the cycle
REMARK	Adds comments
RESET	Resets the controller status
RESUME	Restarts robot movement
RETURN	Returns from subprogram to main program
SELECT~CASE~ (CASEELSE~)..... ENDSELECT	Branches program to be executed based on conditional expression value
STOP	Stops execution of program
SWITCH	During multi-task operation, forces switching to another task
TASK( )	Executes multi-task operation
WAIT	Waits until condition is met
WHILE ..... ENDWHILE	Repeats execution while condition is met
WITH	Specifies movement conditions for individual movement commands

2.8.2 Movement control commands

Commands	Purpose
DELAY	Stops robot arm movement for specified time
MOVE	Moves robot to specified position (PTP movement)
MOVEA	Moves specified robot joint axis to specified position (Absolute single axis movement)
MOVEC	Moves robot to specified position (Circular interpolation movement)
MOVECV	Passes robot through specified position (Spline interpolation movement)
MOVEI	Moves specified robot joint axis from current position by specified amount only (Relative single axis movement)
MOVEJ	Moves robot to specified position (Arch movement)
MOVES	Moves robot to specified position (Linear interpolation movement)
MOVET	Moves robot to specified tool direction
MOVEW	Moves robot to specified world coordinates position
MOVEWI	Moves robot to specified world coordinates direction

2.8.3 Calculator commands (Calculation functions)

Commands	Purpose
ABS(x)	Absolute value
ACOS(x)	Arccosine
ASIN(x)	Arcsine
ATAN(x)	Arctangent
ATAN2(x , y)	x / y arctangent (Arctangent)
BITAND(x, y)	Bitwise product
BITNOT(x)	Bitwise NOT
BITOR(x, y)	Bitwise addition
BITXOR(x, y)	Bitwise exclusive OR
COS(x)	Cosine
EXP(x)	Exponential base e
LN(x)	Natural logarithm
LOG10(x)	Common logarithm
SGN(x)	Extracts and returns the sign.
SIN(x)	Sine
SQRT(x)	Square root
TAN(x)	Tangent

2.8.4 Conversion functions

Commands	Purpose
INT( )	Converts numerical values to integers
JOINT( )	Generates JOINT type data
PALLET( )	Creates pallet type variables
PALLETZ( )	Creates pallet type variable (zigzag pallet)
POINT( )	Generates POINT type data
REAL( )	Converts numerical values to real numbers
TRANS( )	Generates TRANS type data

2.8.5 I/O (Input/output) control commands

Commands	Purpose
ALARM ( )	Generates a user alarm or obtains the current status
BCDIN( )	Reads input signals as BCD code
BCDOUT( )	Outputs signals by converting to BCD code
DIN( )	Reads input signal status
DOUT( )	Outputs external signal
HEXIN( )	Reads input signals as HEX (hexadecimal) code
HEXOUT( )	Outputs signals by converting to HEX (hexadecimal) code
INPUT	Reads data from communication channel
IPCLOSE( )	Executes close process for selected communication channel
IPOPEN( )	Executes open process for selected communication channel
PRINT~CR	Outputs data to communication channel and outputs CR code
PULOUT( )	Performs pulse output of external signals

2.8.6 Character string functions

Commands	Purpose
ASC(t)	Returns a character code for first character of a character string
CHR(x)	Returns a character string (1 character) for a specified character code
CINT(t)	Converts a character string to an integer value
CREAL(t)	Converts a character string to a real number value
CSTR(x)	Returns a character string indicating a specified numerical value
FORMAT(x , t)	Converts a numerical value to a character string based on format-specified character string
INSTR(t1 , t2)	Searches for a specified character string from the beginning
INSTRREV(t1 , t2)	Searches for a specified character string from the end
LCASE(t)	Returns a character string after converting to lowercase
REPLACE(t1 , t2 , t3)	Returns a character string where a part of a character string is replaced by a specified partial character string
STR(x)	Returns a character string indicating a specified numerical value (single-byte space in + sign position)
STRCAT(t1 , t2)	Returns a character string formed by joining two character strings
STRCOMP(t1 , t2)	Compares two character strings
STRLEFT(t , x)	Returns a character string having a specified number of characters from the left end of the character string
STRLEN(t)	Returns the number of characters in a specified character string
STRMID(t , x1 , x2)	Returns a character string having a specified number of characters from a specified position
STRREAD(t , x)	Takes out a character string having the specified number of characters from the character string
STRRIGHT(t , x)	Returns a character string having the specified number of characters from the right end of the character string
STRSPLIT(t1 , t2)	Takes out a character string up to the specified delimiter character
TRIM(t)	Returns a character string that stores a copy excluding spaces at the beginning and end
UCASE(t)	Returns a character string after converting to uppercase

2.8.7 Vector variable calculation functions

Commands	Purpose
CROSS( )	Cross product
INNER( )	Inner product
INV( )	Inverse matrix
JPSN( )	Obtains joint position (Obtains position in world coordinates of joint for specified axis number)
REACH( )	Determines whether movement is possible (Determines whether movement to the specified position data is possible using the MOVE command)
SHIFT( )	Moves parallel

2.8.8 System switches

Commands	Purpose
DISABLE	Disables system switch
ENABLE	Enables system switch
EXTPLT	Switches enable/disable for movement outside pallet range
MOVESYNC	Switches movement command synchronization mode/movement command asynchronization mode
NOWAIT	Switches enable/disable for pre-analysis execution
PASS	Switches enable/disable for shortcut movement
SLOWDOWN	Switches enable/disable for slowdown movement
SLOWSTART	Switches enable/disable for slow start movement
SMOOTH	Switches enable/disable for smooth movement
SWITCH	Prohibits/permits task switching

2.8.9 Library functions

SCOL.LIB

Commands	Purpose
CLOSE1, CLOSE2	Closes robot hand after previous movement is completed
CLOSEI1, CLOSEI2	Closes robot hand
FREELOAD	Sets the robot hand load data to 0
OFFGAIN( )	Turns off the gain (servo control) of the selected joint axis
ONGAIN( )	Turns on the gain (servo control) of the selected joint axis
OPEN1, OPEN2	Opens robot hand after previous movement is completed
OPENI1, OPENI2	Opens robot hand
READY	Moves robot to machine origin
SETGAIN( )	Turns on/off the gain (servo control) of the selected joint axis

2.9 Labels

With the SCOL2 language, program branches are specified by labels placed at the beginning of the branch destination. When labeling a statement as a branch, put a colon at the end of the identifier.
When directing the program to branch to another location with the GOTO command, do not put a colon at the end of the label.
Program branching may only be carried out within a single program. You cannot branch from one program to another. Also, you may use the same labels in different programs, but you cannot use the same label in a single program.

Example)

```
LOOP1:
MOVE P1
GOTO LOOP1
```

2.10 Remarks and Comments

The SCOL2 language allows you add comments to your program in order to make it easier to understand. Comments can be entered by using the teach pendant to type in whatever you want to say. Remarks and comments are written in any one of the following formats.

(1) REMARK command
You can write what you want to say after a REMARK command. In the REMARK command, everything written until the end of the line (until the [EXE] key is pressed) is treated as a comment, and it is not executed as a program.
Because the REMARK command is an independent command, it cannot be written after another command.

Example)

```
REMARK SCOL SAMPLE PROGRAM
```

(2) Write content after the apostrophe symbol (').
Text written after a single quotation mark (') is treated as a comment. This method enables you to insert comments after other commands. Everything written after the single quotation mark (') until the end of the line (until the [EXE] key is pressed) is treated as a comment, and it is not executed as a program.

Example)

```
MOVE P1   'MOVES THE ROBOT TO
'*** SCOL COMMENT SAMPLE ***
```

2.11 Subprograms

2.11.1 Writing subprograms

You can call up a subprogram by just writing its name in the main program.

Example)

```
PROGRAM MAIN                    Main program
 REMARK *** SAMPLE 1 ***
 SUB1                           Calls subprogram
END                             End of main program

PROGRAM SUB1                    Sub program
 MOVE HOMEPOS                   Body of subprogram
 RETURN                         Returns to main program
END                             End of subprogram
```

A RETURN command should be inserted in subprograms to send control back to the main program. If you forget to write RETURN, SCOL2 will forgive you and pretend that there is a RETURN command in front of the END statement.

2.11.2 Argument of subprogram

When wishing to pass data between subprograms and the main program, you have to first specify arguments for the subprogram. When an argument is specified in a subprogram, the program statement should be written like this:

PROGRAM <program name> (<Variable name AS Variable type>)

The argument is specified within parentheses ( ) following the program name of the subprogram. The argument values are inserted to the specified variable names in the subprogram. When using multiple arguments, write commas (,) to separate the variable names in parentheses ( ). The maximum number of arguments is 10.

PROGRAM <program name> (<variable name>)

As shown above, the variable type can be omitted when writing the subprogram. When the variable type is omitted, it is assumed to be scalar type, and an integer value or real number value can be set as the argument.

In the main program, specify the data to be transferred when the subprogram is called in parentheses ( ) after the subprogram name. The data specified in parentheses ( ) is transferred in the specified order to the variable names of the subprogram. The argument data from the main program is substituted within

the subprogram, and when the data is changed, the data corresponding to the variables in the main program is also changed at the same time.。

When the data type passed in the main program and the variable type of the subprogram do not match, an error occurs.
When a constant is used as an argument, it cannot be substituted into a variable according to a subprogram.
For a variable which is an argument to a subprogram, a value should be substituted into the variable before the subprogram is executed.

2.11.3 Subprogram example

Example)

```
PROGRAM MAIN                                        Main program
  REMARK *** SAMPLE 2 ***
  K1 = 15
  K2 = 28
  K = 0
  SUB2 (K1 , K2 , K)
  PRINT K
END

PROGRAM SUB2 (N1 AS INT , N2 AS INT , N3 AS INT)    Sub program
  REMARK *** SUBPROGRAM NO.2 ***
  N3 = N1 + N2
  RETURN
END
```

In the above example, three arguments are being passed off between the main program and subprogram. Specifically, K1 of the main program is passed over as N1 of the subprogram. Similarly, K2 of the main program is passed over as N2 of the subprogram. The subprogram adds N1 and N2, and puts the result in a variable called N3. When this happens, the value of K in the main program also changes.
When this program is executed, the values for K1=15 and K2=28 are added in the subprogram, and the result K=43 is displayed on the teach pendant using the PRINT command in the main program.

Note that subprograms cannot be used to call themselves as a subprogram. Also, when you call a subprogram which is in another file, the controller will not understand you and instead will treat the name of that subprogram as an error.

2.12 Library

The SCOL2 language does not allow you to use subprograms which are not in the same file as the main program. However, by putting especially useful subprograms in the library file, you can access the subprogram from the main program.
When writing your own subprogram to add to the library file, enter the program in that file just like you would enter any other subprogram. For information on how to enter a program into a file, refer to the "TP5000 Operation Manual".

The library comes in the following two (2) kinds.

(1) System library

This library is always loaded at program execution.
The file name is "SCOL.LIB" which cannot be changed, but the contents can be added or changed when necessary.
The OPEN1 and CLOSE1 commands are the subprograms which are included in this library file. The contents of the library file which is the standardly provided in the robot controller system disk are shown in Appendix C.

When you create a new subprogram in the system library, add it to the end of the current library file (SCOL.LIB).

(2) Dynamic link library

This is the library file that the user can load when necessary.
The name of library file is ○○○○.LIB.
The dynamic link library is the user program and you should declare its loading.
You can declare the loading of a library file in the GLOBAL area of the user program in the following manner.

```
GLOBAL
 LOADLIB PALLET.LIB                    Declaration of loading library
END
PROGRAM SAMPLE1
 ~~~~~~~~~~~~
     Omitted
 ~~~~~~~~~~~~
END
```

Up to five (5) dynamic link libraries can be loaded at the same time.
Some SCOL2 commands use this dynamic link library.
Even when such a command is used, declaration of loading is necessary with the command of "LOADLIB + file name."

A global section and a data section can be created in library files.
However, if subprograms, teaching points, or global variables with the same name exist in the user program or other library files, a compilation error will occur.
For libraries, refer to files on the controller and SD card.

2.13 Multitask processing

This paragraph describes how to use the multitask function of the SCOL2 language together with the relevant commands and system variables.

Program execution of single task and multitask operation is shown in Fig. 2.1 and Fig. 2.2. The number in the figure designates the order of the program execution. Specific timing of change-over from program to program (task change) is described later.

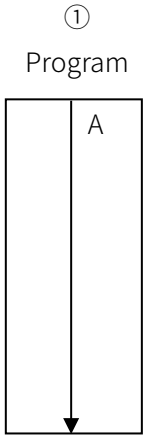


Fig. 2.1 Single task operation

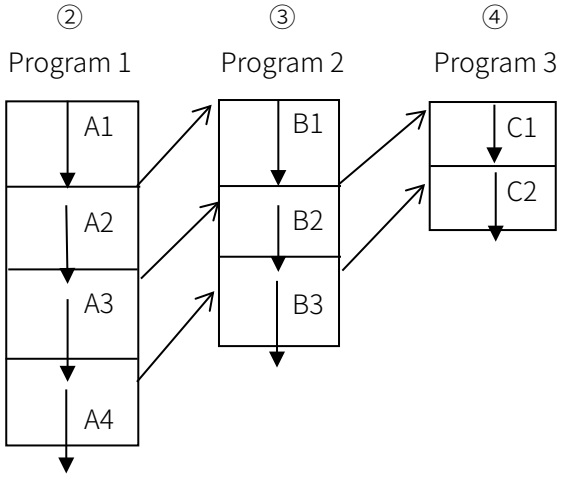


Fig. 2.2 Multitask operation

In Fig. 2.1, program A is executed continuously from the start to the end (single task operation and no subroutine call).

A program which uses no multitask command is executed in the manner as shown in Fig. 2.1 (no subroutine call).

Execution of a program which uses the multitask command is shown in Fig. 2.2.

As shown in Fig. 2.2, the multitask operation is realized, changing over a plural number of individual programs by time sharing, as if the programs were executed in parallel. The order of program execution is shown in the following table.

Order	Program to be executed	
1	A1	Program 1 start
2	B1	Program 2 start
3	C1	Program 3 start
4	A2	
5	B2	
6	C2	1-cycle end of program 3
7	A3	
8	B3	1-cycle end of program 2
9	C1	Program 3 start
10	A4	1-cycle end of program 1
11	B1	Program 2 start
12	C2	
13	A1	Program 1 start
:	:	

Next, the start of multitask is described.
Programs that can handle multi-tasks have a program section without arguments. The program section refers to the program content in SCOL2 language enclosed by PROGRAM and END. The subroutine without argument can be dealt with as a task. The argument cannot be kept in the task.
To deal with a program as task, use the TASK command. The TASK command executes a program specified in the argument as a task. Unless the program starts by the TASK command, the program is not performed as a task.
However, the program section (command content from PROGRAM to END) at the start of the program file is an exception, and it operates as a task even without the TASK command.
To execute the program 2 as a task in the Fig. 2.2, the TASK (“PROG2”) is required to be executed in the program 1. (The program 1 is described at the head of the file, and the program starts as a task without TASK command.)
To execute the program 3 as a task, a new task (“PROG 3”) is required to be executed in the task (in the program 1 or 2 in this case) which has been already started.
If the task and program which have been started are reset or the task operation is released by the SCOL2 language, the task is kept active.

The task ID (the number assigned to the task) is described.
The characteristic numbers (task ID) are assigned to the tasks which have been started by the TASK command respectively. In the example of Fig. 2.2, "1" is assigned to the program 1, "2" is assigned to the program 2 and "3" is assigned to the program 3. This task ID starts from 1 in sequence and this ID increases one by one every time the task starts (every time the task executes). If the task is managed by the SCOL2 language, this task ID is used.
To get the task ID, see the following examples.
Example 1: I1 = TASK("PROG2")
I1 is a user-selected integer variable. The task ID of PROG 2 can be obtained. This command is executed in the program 1. The task ID of its own cannot be referred to in the program 2 in this example.
Example 2: I2 = TID
I2 is a user-selected integer variable. If the system variable TID is referred to, the task ID of its own can be acquired. If this command is executed in the program 2, the task ID of its own can be seen in the program 2 ("2" in this occasion).
If this command is executed in the program 1, the task ID of program 1 ("1" in this occasion) is substituted for "I2".
If the task ID other than the own task is referred to from other tasks, variables of examples 1 and 2 are required to be defined as the global variable.

Change-over of task is described.
As shown in the Fig. 2.2, the system executes the program 1 ~ 3 by time sharing. When this happens, timing of program change-over depends on the following four conditions.

(1) When the program change-over is specified clearly by the SWITCH command of the SCOL2
The SWITCH command is used if the task is changed over clearly by the SCOL2 language. Even if the task change-over conditions specified in the system are not satisfied while the SWITCH command is used, the task can be changed over.
(2) When a new task starts by the TASK command of the SCOL2
If a new task starts by the TASK command, the program control is changed over to the newly started task.
(3) When the task terminates by the KILL command of the SCOL2
If the task of its own terminates by the KILL command, the program control is changed over to the next task.
(4) When the predetermined conditions specified in the system are satisfied and the program is changed over by the system

The task change-over conditions specified in the system are as follows:

(1) When a program in a task is executed for more than 100 msec.

(2) When the data area for movement command becomes full.
Up to four data items can be read beforehand by the movement command. If this internal area for prior reading becomes full, the task is changed over.

(3) When a command required for communication with the external device has been executed.
The INPUT and PRINT commands are not executed by the active SCOL2 program alone. They take time to execute the TP operation and RAM file operation by an operator. The active program waits for the reply of processing finish from the operator and changes over the task.
To cancel the task changeover by the system, specify DISABLE for system variable SWITCH.

During step execution or when task changeover has been cancelled, only the program executed currently continues and the program of another task already started will not run. (The single task operation becomes effective.)

Cautions on creating a multitask program

(1) Motion commands can be used in the main task only. If they are used in the subtask, an error is generated.
The motion commands are MOVEI , MOVEA , MOVE , MOVES , MOVEC , MOVEJ , DELAY
As each task has system variables TIMER, TID and NOWAIT separately, they can be set arbitrarily in each task, or referred to.
For NOWAIT, separate or common use of it can be selected by means of the user parameter].
TIMER , TID , NOWAIT
It is recommended to use NOWAIT separately and set ENABLE NOWAIT in the subtask.
The following system variables are commonly used among the tasks. A value set in one task remains effective in another task. Careful precautions should be taken, therefore, when setting a plural number of tasks.
CONFIG , ACCUR , ACCEL , DECEL , SPEED , PASS , TORQUE , GAIN , TOOL , BASE , WORK , PAYLOAD , NOWAIT

(2) When a command for waiting for the finish of a movement as shown below has been executed while the motion command was currently executed in the main task, the subtask waits for the finish of the active motion command in the main task.
1) Execution of input/output commands in DISABLE NOWAIT mode
The input/output commands includes INPUT , PRINT , DIN , DOUT , BCDIN , BCDOUT , PULOUT
2) Execution of WAIT MOTION command

(3) If execution of an INPUT or PRINT command has been commanded in another task while the same command was executed in one task for different communication channel, the INPUT or PRINT command started first is executed to the last and the another task should wait until the command has been executed.

3. Explanation of Robot Commands

Here we describe in detail what each SCOL2 command means and does.
These commands are listed in alphabetical order.

3.1 Command Explanations

Commands are explained as follows.

(1) Purpose

This paragraph gives a simple explanation of what the command does.

(2) Type

This describes the type of command.

(3) Format

This paragraph describes how to write down the command. The symbols used in the paragraph have the meaning that follows:

[	]	Indicates that the content therein can be omitted. The commands are specified as necessary.
<	>	Indicates the content of the data to be described.
{	}	Indicates that one among the data in this bracket should be selected.
...		Indicates that a plurality of data elements can be specified.

The above symbols are used here for purposes of explanation, and they are not actually written in the program. If these symbols need to be used in a particular case, this will be explained.

(4) Examples

This paragraph presents samples showing how to use the command.

(5) Analysis and advice

This paragraph presents an analysis of the command and describes warnings and restrictions for its use.

(6) Sample

This paragraph presents a short program example using the command.

The meaning of the data in the command format is shown below. An expression can also be used as data.

<Position>	Specifies POINT type data.
<Axis>	Specifies a joint-controlled axis. Data is an integer value from 1 to 5.
<Absolute position>	Specifies the absolute position of each axis. Data is in units of 0.001 mm or 0.001 degrees. Variables or expressions may also be used as data.
<Relative position>	Specifies the travel of each axis in terms of relative position. Data is in units of 0.001 mm or 0.001 degrees. Variables or expressions may also be used as data.
<Time>	Specifies time in units of 0.01 seconds. Variables or expressions may also be used.
<Logical expression>	Specifies a logical expression.
<Statement>	Specifies a statement to be executed. As long as it is a normal SCOL2 statement, you can specify anything you want.
<Monitoring condition>	With an ON statement, specifies condition(s) to be monitored.
<Label>	Specifies a label for branching the program.
<Comment>	Shows comments written in the program.
<Variable>	Indicates a variable.
<Expression>	Indicates a calculation. Individual variables may also be substituted for a calculation.
<Signal name>	Specifies the name of an I/O (Input/Output) signal. The signal name is to be given as an integer. A positive value shows that the signal is ON and a negative value shows that the signal is OFF. Variables and expressions may also be used.
<Mass>	Specifies the mass of the load acting on the robot hand.
<Center of gravity offset>	Specifies the distance between the center of gravity of the load applied to the tip of robot hand and the center of the tool of the hand.
<Configuration>	Specifies the robot configuration with an integer. “0” means the configuration is undefined (not fixed), “1” specifies a left hand configuration, and “2” specifies a right hand configuration.
<Switch>	Specifies the system switch. There are two system switches available.
PASS	This system switch specifies shortcut movement.
NOWAIT	This system switch directs signal I/o to be performed without waiting for the completion of a previous movement command.
<State>	Specifies what is to be reset by the RESET command.

3.2 Command description

In the following pages, descriptions are provided for the commands in alphabetical order.

ABS( )

Purpose

The ABS command returns the absolute value of a number.

Type

Calculator command (Calculation function)

Format

ABS(<expression>)

Examples

AK = ABS(-20.345)

K = ABS(K1)

J1 = K - ABS(N - 28.5)

Analysis and advice

This function returns the absolute value of the <expression>.

You may use a constant, variable or result of calculation for the <expression> term.

However, you may not use vector data.

This command must be used in an expression.

Sample

```
PROGRAM MAIN
 K = 0                                  (“DIM K AS INT” is also possible.)
 ABSSAMPLE(3 , 5 , K)
 PRINT TP , K , CR
END
PROGRAM ABSSAMPLE(K1 , K2 , K)          This program takes the arguments K1 and K2,
 K = ABS(K1 - K2)                       subtracts K2 from K1, finds the absolute value of
 RETURN                                 the result, calls it K, and sends program
END                                     execution back to the main program.
```

ACCEL

Purpose

This command sets the fraction of full acceleration for the robot.

Type

System variable (movement condition command)

Format

ACCEL = <expression>

Examples

ACCEL = 80

ACCEL = 0.8 * ACCEL

MOVE A1 WITH ACCEL = 90

Analysis and advice

ACCEL is a system variable used to specify the acceleration of the robot when the robot is accelerating.

Acceleration is expressed as a percentage of the standard (full) acceleration.

In the SCOL2 language, acceleration during acceleration and acceleration during deceleration are set separately. In order to set the acceleration during deceleration, use the DECEL commands.

This is used to lower the acceleration as needed when the robot is carrying a heavy load. In this case, change the acceleration during both acceleration and deceleration.

You may use a constant, variable or result of calculation for the <expression> term. However, you may not use vector data.

This command must be used in an expression.

An upper limit on acceleration is set in the controller to protect the robot. The robot will not exceed this limit even if you enter a value larger than the upper limit.

Values of 0 or less are taken as 1.

The current acceleration during acceleration can be viewed by viewing this system variable.

The initial value for acceleration is 100%.

Sample

```
PROGRAM ACCELSAMPL
  FOR K = 1 TO 100
  ACCEL = K
  DECEL = K
  MOVE A1
  MOVE A2
  NEXT K
END
```

This program increases the acceleration from 1% to 100% in steps of 1%.

ACCUR

Purpose

This command specifies the positioning accuracy of the robot.

Type

System variable (movement condition command)

Format

ACCUR = <expression>

Examples

ACCUR = 1

N = ACCUR

MOVE A1 WITH ACCUR = COARSE

Analysis and advice

ACCUR is a system variable used to specify the positioning accuracy of the robot. A coarse positioning accuracy is marked by 0 and a fine positioning accuracy is marked by 1.

When the positioning accuracy is set to COARSE, the robot executes the subsequent command before the positioning of the robot is completed. The robot tact time can be reduced by setting the positioning accuracy to COARSE for operations where waiting for high positioning accuracy is unnecessary.

The system constants FINE and COARSE can be used to specify the positioning accuracy. The positioning accuracy is set to fine by ACCUR=FINE and set to coarse by ACCUR=COARSE.

You may use a constant, variable or result of calculation for the <expression> term. However, you may not use vector data.

This command must be used in an expression.

When specifying the positioning accuracy, anything entered that is less than 0 will be taken as 0 and anything greater than 1 will be taken as 1.

You can find the positioning accuracy under which the system currently is operating by referring to ACCUR. An ACCUR value of 0 means coarse, and a value of 1 means fine.

The initial setting for the positioning accuracy is FINE.

Sample

```
PROGRAM ACCURSMPL
  ACCUR = COARSE
  MOVE A1
  MOVE A2
  MOVE A3 WITH ACCUR = FINE
  MOVE A4
END
```

The robot will move with fine positioning accuracy only for movement to point A3. It will move with coarse positioning accuracy for other movements increasing cycle time.

ACOS( )

Purpose

This command returns the arccosine of an entered value.

Type

Calculator command (Calculation function)

Format

ACOS(<expression>)

Examples

K = ACOS(0.577)

J1 = 90 - ACOS(X / L)

Analysis and advice

This function returns the arccosine of the value in the brackets ( ).

The returned value is in units of degrees.

You may use a constant, variable or result of calculation for the <expression> term.

However, you may not use vector data.

This command must be used in an expression.

Sample

```
PROGRAM MAIN
 K = 0.0                          ("DIM K AS REAL" is also possible.)
 ACOS2(2.0, 1.0, K)
 PRINT TP, K, CR
END
PROGRAM ACOS2(L, X, K)            This program takes the arguments L and X,
 K = ACOS( X / L )                divides X by L, takes the arccosine of the result,
 RETURN                           calls it K, and returns to the main program.
END
```

ALARM( )

Purpose

This command generates a user alarm or obtains the current status.

Type

I/O (Input/output) control command

Format

ALARM(<Alarm level>, <Alarm number>)

Examples

ALARM(1, 1)

IF ALARM(1, 7) THEN DOUT(2)

Analysis and advice

This generates the specified user alarm.

<Alarm level> is specified as 1, 2, 4, or 8 only.

<Alarm number> is specified from 1 to 8. Eight user alarms are specified for each level.

Alarm level	Alarm number: User alarm code	
8	1: 008-129	5: 008-133
	2: 008-130	6: 008-134
	3: 008-131	7: 008-135
	4: 008-132	8: 008-136
4	1: 004-129	5: 004-133
	2: 004-130	6: 004-134
	3: 004-131	7: 004-135
	4: 004-132	8: 004-136
2	1: 002-129	5: 002-133
	2: 002-130	6: 002-134
	3: 002-131	7: 002-135
	4: 002-132	8: 002-136
1	1: 001-129	5: 001-133
	2: 001-130	6: 001-134
	3: 001-131	7: 001-135
	4: 001-132	8: 001-136

The alarm message can be set in the user parameter [U026] USER ALARM.

The ALARM status is obtained when the ALARM ( ) command is written on the right side of the expression or in an IF expression. If the status of the user alarm in ( ) is ON, the condition is met, and TRUE (1) is returned. If the condition is not met, FALSE (0) is returned.
In this case, the user alarm status does not change.

Sample

```
PROGRAM MAIN
 ALARM(2, 4)                          Generates user alarm 4 (002-132) at level 2.
END
```

```
PROGRAM MAIN
 IF ALARM(1, 2) THEN                  If user alarm 2 (001-130) at level 1 is turned on,
    DOUT(1)                           output signal 1 is turned on.
 ELSE
    DOUT(-1)
 ENDIF
END
```

Alarm levels are classified as follows.

Level 8	The robot makes an emergency stop and turns off the servo.
Level 4	The robot stops on the spot.
Level 2	The robot stops just before the problem location in the SCOL2 program.
Level 1	The robot only displays alarm messages.

AND

Purpose

The AND command calculates the logical product of expressions.

Type

Calculator command (operand)

Format

<logical expression> AND <logical expression>

Examples

IF DIN(1) AND K <= 3 THEN J = 0

WAIT DIN(5) AND TIMER == 0

Analysis and advice

The AND statement is used to find the logical product of two logical expressions.

If both logical expressions are true, a TRUE value will be returned.

This command must be used in a logical expression.

Sample

```
PROGRAM ANDSAMPLE
 FOR K = 1 TO 50
 IF K == 50 AND DIN(1) THEN J = 1 ELSE J = 0
 PRINT TP,J,CR
 NEXT K
END
```

AREAIN()

Purpose

This command determines whether the area is inside or outside.

Type

System variable (Other)

Format

AREAIN(<Area number>)

Examples

IF AREAIN(1) THEN GOTO L1

WHILE AREAIN(N)

Analysis and advice

This returns whether the robot's TCP is contained in the area defined by AREA.PAR.

It returns 1 if inside the area: otherwise, it returns 0.

It always returns 0 when an area that is not enabled is referenced.

Sample

```
PROGRAM MAIN
  IF AREAIN(1) THEN GOTO L1
  MOVE P1
L1:
  MOVE P2
END
```

If TCP is inside Area 1 at the start of the program, the move to P1 is skipped.

ASC( )

Purpose

This calculates the character code for the first character of the specified character string in the argument.

Type

Character string function

Format

ASC(<character string expression>)

Examples

A = ASC(“Apple”)
B = ASC(TXT) . . . . . TXT is a character string variable

Analysis and advice

This returns the character code for the first character of the specified character string in ( ).
The result is given as an integer value.
If the character string length of the argument is 0, a value of 0 is returned.
The returned character code is in the range from 0 to 255 (0x00 to 0xFF).

Sample

```
PROGRAM MAIN
 DIM A AS INT                    (“A = 0” is also possible.)
 A = ASC(“Apple”)                65, which is the character code for “A”, is
 PRINT TP, A, CR                 returned.
END
```

```
GLOBAL
 DIM TXT AS STRING(10)
END
PROGRAM MAIN
 DIM B AS INT                    (“B = 0” is also possible.)
 TXT = “sample”
 B = ASC(TXT)                    115, which is the character code for “s”, is
 PRINT TP, B, CR                 returned.
END
```

ASIN( )

Purpose

This function finds the arcsine of the value entered.

Type

Calculator command (Calculation function)

Format

ASIN(<expression>)

Examples

K = ASIN(0.577)

J1 = 90 - ASIN(Y / L)

Analysis and advice

This function returns the arcsine of the value in the brackets ( ).

The returned value is in units of degrees.

You may use a constant, variable or result of calculation for the <expression> term.

However, you may not use vector data.

This command must be used in an expression.

Sample

```
PROGRAM MAIN
 K = 0.0                          ("DIM K AS REAL" is also possible.)
 ASIN2(5.0, 2.0, K)
 PRINT TP, K, CR
END
PROGRAM ASIN2(L, Y, K)            This program takes the arguments L and Y,
 K = ASIN(Y / L)                  divides Y by L, takes the arcsine of the result,
 RETURN                           calls it K, and returns to the main program.
END
```

ATAN( ) , ATAN2( )

Purpose

This function returns the arctangent for the value(s) entered.

Type

Calculator command (Calculation function)

Format

ATAN(<expression>)

ATAN2(<expression>, <expression>)

Examples

K = ATAN(0.577)

J1 = 90 - ATAN(Y / X)

N = ATAN2(0.3, 0.5)

D = ATAN2(100 / K, 50 / J)

L3 = ABS(180 - ATAN2(A1.Y, A1.X))

Analysis and advice

This function returns the arctangent of the value(s) in the brackets ( ).

The ATAN2 takes the first expression in the brackets, divides it by the second expression in the brackets, and finds the arctangent of the result.

For both functions, the returned value is in units of degrees.

(Be warned that ATAN2 (0, 0) will return a 0 instead of an error.)

You may use a constant, variable or result of calculation for the <expression> term.

However, you may not use vector data.

This command must be used in an expression.

Sample

```
PROGRAM MAIN
  K = 0.0                                  ("DIM K AS REAL" is also possible.)
  ATANSAMPLE(5.0, 3.0, K)
  PRINT TP, K, CR
END
PROGRAM ATANSAMPLE(X, Y, K)                This program takes the arguments X and Y,
  K = ATAN(Y / X)                          divides Y by X, takes the arctangent of the result,
  RETURN                                   calls it K, and returns to the main program.
END
```

```
PROGRAM MAIN
  K = ATAN2(A1.Y, A1.X)                    The arctangent of Y/X from the X and Y
  PRINT TP, K, CR                          components of the teaching point A1 is
END                                        calculated and shown on the teach pendant
                                           (TP).
```

BASE

Purpose

BASE is a system variable used to specify the base coordinate system.

Type

System variable (movement reference command)

Format

BASE

Examples

```
BASE = TRANS(0, 0, 0, 0)
BASE1 = BASE
MOVE A1 WITH BASE = BASE + TRANS( , , 100)
```

Analysis and advice

BASE is a system variable used to specify the base coordinate system.
It can be handled as TRANS type data.
By referring to the BASE, you can find the values (location) of the present base coordinate system.

You can directly designate values for the base coordinates with one of the following two methods:

```
BASE = TRANS(X, Y, Z, C)
BASE = {X, Y, Z, C}
```

(Although { } can also be used to write as shown above, use of the TRANS command is recommended for explicitly indicating the data type.)

X, Y, Z, C: X, Y, Z and C are real numbers representing the position of the base coordinate system. Units are of millimeters or degrees.

The BASE coordinate system is created by “sliding” a distance of X, Y and Z along the respective axes of the WORLD coordinate system and then twisting the new Z axis by an amount C.
BASE must be used in an expression.
Be aware that if you change base coordinate systems within a program, there may be some misalignment between the positions as taught and the positions where the robot actually moves.

Sample

```
PROGRAM BASESAMPLE
   MOVE A1
   MOVE A2
  BASE = BASE + TRANS( , , 50)
   MOVE A1
   MOVE A2
  BASE = TRANS( )
END
```

The BASE command shifts the BASE coordinate system 50 mm along the Z axis and after that, the robot moves to a point below 50 mm from the taught position.

BCDIN( )

Purpose

This command is used to read in signals as BCD (Binary Coded Decimal) notation.

Type

I/O (Input/output) control command

Format

BCDIN(<signal name>, <signal length>)

Examples

K = BCDIN(1, 2)

J2 = BCDIN(N, N + 2)

GOTO(BCDIN(20, 1)) L1, L2, L3

Analysis and advice

The command causes an input signal to be read in as BCD notation. The signal will start from the signal name and continue to a place 4 times the value of the specified signal length. For example, "K = BCDIN (1, 2)" tells the controller to read in, as BCD notation, an eight unit signal starting from the 1st unit (bit) and continuing to the 8th unit (bit). Higher signal numbers correspond to bits with a higher digit.
Input signals are divided into units of 4 bits (in order of low to high place value (signal number)) and converted into Base 10.
Signals are coded as 1 for ON and 0 for OFF.

Example) In the case where input signals 1 to 12 indicate the state shown below, the value of BCDIN(1,3) is 329.

Input signal number	12	11	10	9	8	7	6	5	4	3	2	1
Input signal state	OFF	OFF	ON	ON	OFF	OFF	ON	OFF	ON	OFF	OFF	ON
Base 2 expression	0	0	1	1	0	0	1	0	1	0	0	1
Base 10 expression				3				2				9

You may use constants, variables, expressions or calculations for <signal name> and <signal length>. However, you may not use vector type data.
This command must be used in an equation.

When an input signal is coded based on 4 bits and if it exceeds 9 (1001 in binary notation), the signal is coded in the binary notation. In other words, if signal "1111" is input in the binary notation, it is read as "15".

Sample

```
PROGRAM BCDINSAMPL
  K = BCDIN(1, 2)
  SPEED = K
  MOVE A1
  MOVE A2
END
```

This program reads in a single made up of eight bits, sets the movement speed in accordance with that signal.

BCDOUT( )

Purpose

This command will put a signal into BCD notation and output the result.

Type

I/O (Input/output) control command

Format

BCDOUT(<signal name>, <signal length>, <expression>)

Examples

BCDOUT(1, 4, 3)

BCDOUT(N, N + 4, K)

Analysis and advice

This command will take the value of the expression and change it into a BCD signal having the name <signal name> and a signal length four times the value of <signal length>.

For example, the command BCDOUT(1, 1, 3) will produce an output signal which is four bits long and starts from signal position 1. The output signal will express the value 3 as a binary number. Therefore, the first two bits (output signal numbers 1 and 2) will be ON.

Higher signal numbers correspond to bits with a higher digit.

Each digit of <expression> is broken down one at a time and converted into a 4 bit code.

The 4 bit code itself is built up in order of smaller to larger output signal number.

When there are not enough room to hold all the bit code corresponding to the Base 10 <expression>, the excess digits of the Base 10 <expression> will be ignored.

Signals are considered as 1 when ON and 0 when OFF.

Example) The command BCDOUT(1, 3, 952) will create an output signal like that shown below.

Base 10 expression				9				5				2
Base 2 expression	1	0	0	1	0	1	0	1	0	0	1	0
Output signal number	12	11	10	9	8	7	6	5	4	3	2	1
Output signal state	ON	OFF	OFF	ON	OFF	ON	OFF	ON	OFF	OFF	ON	OFF

You may use constants, variables, expressions or calculations for <signal name>, <signal length> and <expression>. However, you may not use vector type data.

If the same signals are output consecutively, the signal output last becomes valid.

Be careful of agreement's there being in the signal range by which it is possible to guarantee simultaneousness.
The simultaneousness of BCDOUT to the range in units of 16 bits from DOUT1, DOUT101, and DOUT301 can be guaranteed but the simultaneousness of BCDOUT which strides the boundary cannot be guaranteed.

Sample

```
PROGRAM BCDOUTSMPL
  J = 0
  FOR K = 1 TO 4
    J = 2 ^ (K - 1)
    BCDOUT(1, 1, J)
    TIMER = 0.5
    WAIT TIMER == 0
    BCDOUT(1, 1, 0)
  NEXT K
END
```

This program will output a number from 1 to 4 in steps of 1 at intervals of 0.5 second.

BITAND( )

Purpose

This command calculates the value of the bitwise product.

Type

Calculation operation command (calculation function)

Format

BITAND(<expression>,<expression>)

Examples

K = BITAND(1, 2)

K = BITAND(0x1000, 0x0200)

Analysis and advice

This calculation operation command calculates the value of the bitwise product of two values.

<expression> can be a constant, variable, or calculation expression. However, data other than integer type cannot be used.

This command is used in the expression.

Sample

```
PROGRAM MAIN
 A = 0x1111
 B = 0x1234
 C = BITAND(A, B)                    Calculates the bitwise product.
 PRINT TP, FORMAT(C, "XXXX"), CR     1010 is output.
END
```

System version

X8KCS-05C and later

BITNOT( )

Purpose

This command calculates the value of the bitwise product.

Type

Calculation operation command (calculation function)

Format

BITNOT(<expression>)

Examples

K = BITNOT(1)

K = BITNOT(0x1000)

Analysis and advice

This calculation operation command calculates the value of bitwise NOT. Numerical values are treated as 4-byte data.

<expression> can be a constant, variable, or calculation expression. However, data other than integer type cannot be used.

This command is used in the expression.

Sample

```
PROGRAM MAIN
  A = 0x12345678
  B = BITNOT(A)                          Calculates bitwise NOT.
  PRINT TP, FORMAT(B, "XXXXXXXX"), CR    EDCBA987 is output.
END
```

System version

X8KCS-05C and later

BITOR( )

Purpose

This command calculates the value of bitwise AND.

Type

Calculation operation command (calculation function)

Format

BITOR(<expression>, <expression>)

Examples

K = BITOR(1, 2)

K = BITOR(0x1000, 0x0200)

Analysis and advice

This command calculates the value of the bitwise AND of two values.

<expression> can be a constant, variable, or calculation expression. However, data other than integer type cannot be used.

This command is used in the expression.

Sample

```
PROGRAM MAIN
  A = 0x1111
  B = 0x1234
  C = BITOR(A, B)                    Calculates bitwise AND.
  PRINT TP, FORMAT(C, "XXXX"), CR    1335 is output.
END
```

System version

X8KCS-05C and later

BITXOR( )

Purpose

This command calculates the value of bitwise exclusive OR.

Type

Calculation operation command (calculation function)

Format

BITXOR(<expression>, <expression>)

Examples

K = BITXOR(1, 2)

K = BITXOR(0x1000, 0x0200)

Analysis and advice

This command calculates the value of bitwise exclusive OR of two values.

<expression> can be a constant, variable, or calculation expression. However, data other than integer type cannot be used.

This command is used in the expression.

Sample

```
PROGRAM MAIN
  A = 0x1111
  B = 0x1234
  C = BITXOR(A, B)                    Calculates bitwise exclusive OR.
  PRINT TP, FORMAT(C, "XXXX"), CR     0325 is output.
END
```

System version

X8KCS-05C and later

CHR( )

Purpose

This returns a character (character string of length 1) for specified character code.

Type

Character string function

Format

CHR(<expression>)

Examples

A = CHR(97)

B = CHR(0x42)

C = CHR(NUM) . . . . . A, B, and C are character string variables

Analysis and advice

The value specified by the argument is from 1 to 255.

0x00 (null character) is not handled.

This returns a character corresponding to the character code specified in ( ).

Character codes in the range from 0 to 255 (0x00 to 0xFF) can be specified.

If a value outside the range is specified (value less than 0 or above 255), a character string of length 0 (" ")is returned.

If no argument, or a real number, character string, or undefined variable is specified in ( ), a compile error occurs.

Sample

```
GLOBAL
  DIM A AS STRING(10)
END
PROGRAM MAIN
  A = CHR(97)
  PRINT TP, A, CR
END
```

Returns an “a” corresponding to the character code 97.

```
GLOBAL
  DIM C AS STRING(10)
END
PROGRAM MAIN
  NUM = 90
  C = CHR(NUM)
  PRINT TP, C, CR
END
```

Returns a “Z” corresponding to the character code 90.

CINT( )

Purpose

This converts the specified character string to an integer value.

Type

Character string function

Format

CINT(<character string expression>)

Examples

A = CINT(“1234”)

B = CINT(TXT) . . . . . TXT is a character string variable

Analysis and advice

This converts the character string specified in ( ) to an integer value.

If the argument contains any digits after the decimal point, they are discarded in the 0 direction. (1.5 becomes 1, and -0.5 becomes 0.)

The conversion range is the SCOL2 integer value range (- 2147483648 to + 2147483647).

If a character string expression that cannot be evaluated as a numerical value is specified in the argument, 0 is returned.

Sample

```
PROGRAM MAIN
 A = CINT(“1234”)                    Returns the integer value “1234”
 PRINT TP, A, CR
END
```

```
GLOBAL
  DIM TXT AS STRING(10)
END
PROGRAM MAIN
  TXT = “123.5”
  B = CINT(TXT)                      Returns the integer value “123”
  PRINT TP, B, CR
END
```

CLOSE1 , CLOSE2 , CLOSEI1 , CLOSEI2

Purpose

These commands close the robot hand.

Type

Library function (SCOL.LIB)

Format

CLOSE1
CLOSE2
CLOSEI1
CLOSEI2

Examples

CLOSE1
CLOSEI2

Analysis and advice

These commands are used to close the robot hand. The numbers 1 and 2 refer to Hand1 and Hand2. These commands close the hand by changing the state of the output signal which controls the robot hand. In the initial setting of SCOL.LIB, the execution of DOUT(203,-204) is executed when CLOSE1 is triggered.
The CLOSE command directs the robot to close its hand after it completes the motion in progress.
The CLOSEI command directs the robot to close its hand immediately.
Note that these commands will not work if the file SCOL.LIB is not in the controller ROM drive.
Also, keep in mind that there is a slight delay from when a CLOSE command is executed until the robot actually closes its hand.
Corresponding commands OPEN1, OPEN2, OPENI1 and OPENI2 are provided in order to open the hand.

These commands execute a program written in the system library (SCOL. LIB). The data of SCOL. LIB should be changed according to the robot hand specifications.

Sample

```
PROGRAM CLOSESMPL
 OPENI1                    This program closes the hand1 after the robot
 MOVE A1                   has finished moving to point A1.  The robot
 CLOSE1                    waits 0.5 second until the hand is closed
 DELAY 0.5                 completely after the CLOSE1 command has
 MOVE A2                   been executed.
END
```

```
PROGRAM CLOSEISMPL
 ENABLE NOWAIT
 OPENI1
 DELAY 0.5
 MOVE A1
 CLOSEI1                   Here, the robot will close the hand1 while
 DELAY 0.5                 moving to point A1.
 MOVE A2
END
```

COARSE

Purpose

COARSE is a system constant used to set the positioning accuracy to coarse.

Type

System constant

Format

COARSE

Examples

ACCUR = COARSE

MOVE A1 WITH ACCUR = COARSE

Analysis and advice

The COARSE statement sets positioning accuracy to COARSE, using with the ACCUR command.

As a system constant, COARSE has a value of 0. If you wanted to, you could use it in your program as a constant having the value 0. However, you should not do things like that since it makes your program extremely hard to read.

You cannot substitute into system constants such as COARSE.

For information on positioning accuracy, refer to the ACCUR command.

Sample

```
PROGRAM COARSESMPL
  MOVE A1
  ACCUR = COARSE          This program sets the positioning accuracy to
  MOVE A2                 coarse before moving around the robot.
  MOVE A3
END
```

CONFIG

Purpose

This command is used to specify the robot configuration.

Type

System variable (movement condition command)

Format

CONFIG = <expression>

Examples

CONFIG = 1

MOVE A1 WITH CONFIG = RIGHTY

Analysis and advice

CONFIG is a system variable used to express the configuration of the robot. You should specify the robot configuration when there is a chance of peripheral equipment interfering with the robot motion.

The robot configuration is undefined at 0, left handed at 1 and right handed at 2. In order to specify the system configuration, you may use these numbers or the system constants FREE, LEFTY and RIGHTY.

As you would probably guess, the configuration is undefined at CONFIG = FREE, left handed at CONFIG = LEFTY, and right handed at CONFIG = RIGHTY.

The configuration is included in positional data fed into the robot while teaching. Therefore, when CONFIG = FREE, the robot will move with the same configuration it had when it was being taught.

Unless there is a good reason otherwise, you should leave the configuration undefined, i.e. CONFIG = FREE. This is the initial value that the robot will assume.

The robot configuration may change upon executing a movement command. When conducting linear or circular interpolation (with the MOVES or MOVEC command), the robot configuration cannot be changed and an error will result if you try.

Designation of the configuration for an orthogonal coordinate robot is ignored.

A constant, a variable or a calculation can be used for the <expression> term. However, you may not use vector type data.

Even if it refers to this system variable, the posture of the present robot cannot be referred to.

It is possible for the posture of the present robot to be acquired by the HERE command.

N=HERE.6

The value of the present posture is stored in N.

For SCARA type robots, the value of HERE.6 is undefined at 0, left handed at 1, and right handed at 2. When the starting point position of the robot isn't correctly set, it works in the position where the robot was shifted with the position to have instructed in when changing the posture of the robot. Therefore, when instructing a robot in the position, specify the posture in which the robot actually moves.

Sample

```
PROGRAM CONFIGSMPL
  CONFIG = RIGHTY
  MOVE A1
  MOVE A2
  MOVE A3 WITH CONFIG = LEFTY
  MOVE A4
END
```

The robot moves with a left hand configuration only when moving to A3, and moves with a right hand configuration for other movements.

CONT

Purpose

CONT is a system constant which is used to refer to the system operating mode.

Type

System constant

Format

CONT

Examples

IF MODE <> CONT THEN STOP

Analysis and advice

CONT is used along with the MODE command to refer to the system operating mode.
When MODE = = CONT, the system is operating in the continuous operation mode.
As a system constant, CONT has a value of 0. If you wanted to, you could use it in your program as a constant having the value 0.
However, you should not do it since it will make your program hard to understand.
You cannot substitute into system constants.
For information on operating modes, refer to the MODE command.

Sample

```
PROGRAM CONTSAMPLE
  IF MODE <> CONT THEN STOP
  MOVE A1
  MOVE A2
  MOVE A3
END
```

If the system changes out of the continuous operation mode, program execution will stop and the robot will not move.

COS( )

Purpose

This function returns the cosine of an entered value.

Type

Calculator command (Calculation function)

Format

COS(<expression>)

Examples

K = COS(60)

J1 = 1 - COS(180 - D)

Analysis and advice

This function returns the cosine of the value in the brackets ( ).

Calculations are handled in units of degrees.

You may use a constant, variable or result of calculation for the <expression> term.

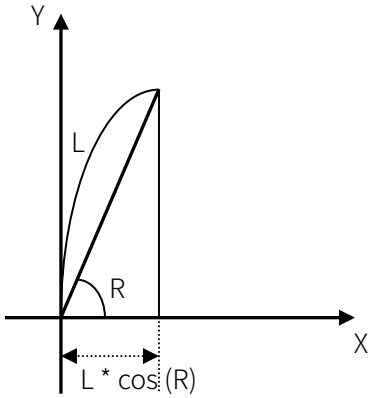
However, you may not use vector data.

This command must be used in an expression.

Sample

```
PROGRAM MAIN
 X = 0.0
 COSSAMPLE(2, 30, X)
 PRINT TP, X, CR
END
PROGRAM COSSAMPLE(L, R, X)
 LOOP:
  IF R > 180 THEN R = R - 360
  IF R < -180 THEN R = R + 360
  IF R > 180 OR R < -180 THEN GOTO LOOP
 X = L * COS(R)
 RETURN
END
```

("DIM X AS REAL" is also possible.)



Given (as arguments) a line segment L with a length L and forming an angle R with the X-axis, this program finds the length of the X-axis component of the line segment L and sends it back to the main program as argument X.

CPACCEL

Purpose

This sets the CP acceleration.

Type

System variable (movement condition command)

Format

CPACCEL = <expression>

Examples

CPACCEL = 1000

A = CPACCEL

MOVES P1 WITH CPACCEL = 2000

Analysis and advice

The hand acceleration at an OVERRIDE of 100% for CP movement (MOVES, MOVEC) is set in mm/sec^2 units. If this system variable is not set, the acceleration for CP movement also changes based on the ACCEL setting.

If this is referenced when this system variable is not set, the hand acceleration for CP movement based on the ACCEL setting can be confirmed.

Sample

PROGRAM MAIN	
ACCEL = 80	
MOVES P1	A linear interpolation movement at ACCEL = 80 is performed.
CPACCEL = 1000	The hand acceleration is set to 1000 mm/s^2.
MOVES P2	Linear interpolation movement is performed at the preset hand acceleration. The hand acceleration setting is disabled.
CPACCEL = OFF	
MOVES P3	A linear interpolation movement at ACCEL = 80 is performed.
END	

CPDECEL

Purpose

This sets the CP deceleration.

Type

System variable (movement condition command)

Format

CPDECEL = <expression>

Examples

CPDECEL = 2000.0

B = CPDECEL

MOVES P1 WITH CPDECEL = 500

Analysis and advice

The hand deceleration at an OVERRIDE of 100% for CP movement (MOVES, MOVEC) is set in mm/sec^2 units. If this system variable is not set, the deceleration for CP movement also changes based on the DECEL setting.

If this is referenced when this system variable is not set, the hand deceleration for CP movement based on the DECEL setting can be confirmed.

Sample

```
PROGRAM MAIN
  DECEL = 100
  CPDECEL = OFF
  PRINT CPDECEL, CR          This confirms the maximum deceleration of the
END                          CP movement.
```

CPSPEED

Purpose

This sets the CP movement speed.

Type

System variable (movement condition command)

Format

CPSPEED = <expression>

Examples

CPSPEED = 1000

MOVES P1 WITH CPSPEED = 500

Analysis and advice

The hand speed at an OVERRIDE of 100% for CP movement (MOVES, MOVEC) is set in mm/sec units.

If this system variable is not set, the speed for CP movement also changes based on the SPEED setting.

Sample

PROGRAM MAIN	
SPEED = 80	
MOVES P1	A linear interpolation movement at SPEED = 80 is performed.
CPSPEED = 1000	The hand speed is set to 1000 mm/s.
MOVES P2	Linear interpolation movement is performed at a hand speed of 1000 mm/s. The hand speed setting is disabled.
CPSPEED = OFF	
MOVES P3	A linear interpolation movement at SPEED = 80 is performed.
END	

CREAL( )

Purpose

This converts the specified character string to a real number value.

Type

Character string function

Format

CREAL(<character string expression>)

Examples

A = CREAL(“123.45”)

B = CREAL(TXT) ..... TXT is a character string variable

Analysis and advice

This converts the character string specified in ( ) to a real number value.

It is displayed as a number to three decimal places.

If a character string expression that cannot be evaluated as a numerical value is specified in the argument, 0.000 is returned.

Sample

```
PROGRAM MAIN
 A = CREAL(“123.45”)                    Returns the real number value “123.450”
 PRINT TP, A, CR
END
```

```
GLOBAL
  DIM TXT AS STRING(10)
END
PROGRAM MAIN
  TXT = “123”
  B = CREAL(TXT)                        Returns the real number value “123.000”
  PRINT TP, B, CR
END
```

CROSS( )

Purpose

This finds the cross product of two TRANS type variables.

Type

Vector variable calculation function

Format

CROSS(<TRANS type>, <TRANS type>)

Examples

A = CROSS(T1, T2)

B = CROSS(TRANS(100, 100, 100, 0), T2)

Analysis and advice

This finds the cross product calculation result of the vectors indicated by the two TRANS type variables in ( ).

The cross product calculation result is stored in the TRANS type variable, and the cross product for the X component, Y component, and Z component is a real number value. The element C is 0.000.

Sample

```
PROGRAM MAIN
  T1 = TRANS(100, 100, 100, 0)
  T2 = TRANS(100, 50, 50, 60)
  T = CROSS(T1, T2)
  PRINT “CROSS PRODUCT TX =“, T.X, CR
  PRINT “CROSS PRODUCT TY =“, T.Y, CR
  PRINT “CROSS PRODUCT TZ =“, T.Z, CR
END
```

This finds the cross product calculation result (X component, Y component, Z component) of the vectors indicated by the TRANS type variables T1 and T2.

CSTR( )

Purpose

This converts the numerical value specified by the argument to a character string.

Type

Character string function

Format

CSTR(<expression>)

Examples

A = CSTR(1234 ..... A is a character string variable

B = CSTR(0.1234) ..... B is a character string variable

Analysis and advice

This converts the numerical value specified in ( ) to a character string.

If the specified numerical value is a real number, it is expressed as a number to 6 decimal places.

In contrast to STR( ), a single-byte space is not included before positive numerical values.

Sample

```
GLOBAL
  DIM A AS STRING(10)
END
PROGRAM MAIN
  A = CSTR(1234)                    Returns the character string “1234”
  PRINT TP, A, CR
END
```

```
GLOBAL
  DIM B AS STRING(10)
END
PROGRAM MAIN
  B = CSTR(0.1234)                  Returns the character string “0.123400”
  PRINT TP, B, CR
END
```

CTRLCODE

Purpose

This system switch allows conversion of control characters by the PRINT or INPUT command.

Type

System switch

Format

{ ENABLE | DISABLE } CTRLCODE

Examples

ENABLE CTRLCODE
DISABLE CTRLCODE

Analysis and advice

This system switch allows conversion of two-digit hexadecimal numbers enclosed with a control character and angle brackets (<>) when the PRINT or INPUT command is executed.
The initial state is determined by [U029] PRINT/INPUT CONTROL CODE CONVERSION.
If conversion is allowed, the PRINT command converts <XX> (where XX is a two-digit hexadecimal number) in the string to the corresponding ASCII code character and outputs it. However, no conversion is performed to output to TP. When the INPUT command receives data in character codes 0x00 to 0x1F and 0x7F or later, it converts that data into a string in the format <XX> and stores it in a string type variable.

Sample

```
GLOBAL
   DIM BUF AS STRING(256)
END
PROGRAM MAIN
   ENABLE CTRLCODE                          Allows conversion of control characters.
   PRINT IP1, "123<00><07>", CR                 Outputs 123NUL BEL CR.
   INPUT IP1, BUF
   IF BUF == "123<00><07>" THEN DOUT(1)     The received control characters are converted to
END                                         a string in the format <XX>.
```

System version

X8KCS-05B and later

CUROVRD

Purpose

This references the currently set override value.

Type

System variable (movement reference command)

Format

CUROVRD

Examples

IF CUROVRD == 10 THEN OVERRIDE = 20

Analysis and advice

When this system variable is referenced, the currently set override value can be referenced. This system variable is referenced only, and the value cannot be substituted.

Sample

```
PROGRAM CUROVRDSAMPLE
  IF CUROVRD <> 20 THEN OVERRIDE = 20
  MOVEA 1, 90
END
```

When the currently set override value is not 20, the override value is set to 20, and the 1st axis is moved to the 90° position.

CYCLE

Purpose

CYCLE is a system constant which is used to refer to the system operating mode.

Type

System constant

Format

CYCLE

Examples

IF MODE <> CYCLE THEN GOTO LOOP

Analysis and advice

CYCLE is used along with the MODE command to refer to the system operating mode.
When MODE = = CYCLE, the system is operating in the cycle operation mode.
As a system constant, CYCLE has a value of 1. If you wanted to, you could use it in your program as a constant having the value 1. However, this is not a good idea since it makes your program unnecessarily hard to understand.
You cannot substitute into system constants such as CYCLE.
For information on the system operating mode, refer to the MODE command.

Sample

```
PROGRAM CYCLESMPL
 LOOP:
   MOVE A1
   MOVE A2
   MOVE A3
 IF MODE <> CYCLE THEN GOTO LOOP
END
```

If the system is not operating in the cycle operation mode, execution of the program will keep on returning to the beginning of the loop and the robot will move over and over again.

DATA.....END

Purpose

This shows the data section which defines the position and coordinate data for the teaching points.

For details on the data section, see “5.3.5 Data section”.

Type

Program control command

Format

DATA

Examples

DATA

Analysis and advice

When editing the program using a teach pendant or the SHIBAURA MACHINE programming support software, the data section is edited using a data editor, not a program editor. In this case, DATA ... END is automatically added.

Sample

```
PROGRAM MAIN
  MOVE HOME
END
```

```
DATA
  POINT HOME = 600, 0, 100, 0, 0 / RIGHTY
END
```

When editing the program using a teach pendant or the SHIBAURA MACHINE programming support software, DATA ... END is automatically added.

DECEL

Purpose

This variable sets the fraction of full deceleration for the robot.

Type

System variable (movement condition command)

Format

DECEL = <expression>

Examples

DECEL = 80

DECEL = 0.8 * DECEL

MOVE A1 WITH DECEL = 90

Analysis and advice

DECEL is a system variable used to specify the acceleration of the robot when the robot is decelerating.

Acceleration is expressed as a percentage of the standard (full) acceleration.

In the SCOL2 language, acceleration during acceleration and acceleration during deceleration are set separately. In order to set the acceleration during acceleration, use the ACCEL command.

This is used to lower the acceleration as needed when the robot is carrying a heavy load. In this case, change the acceleration during both acceleration and deceleration. You may use a constant, variable or result of calculation for the <expression> term. However, you may not use vector data. This command must be used in an expression.

An upper limit on acceleration is set in the controller to protect the robot. The robot will not exceed this limit even if you enter a value larger than the upper limit. Values of 0 or less are taken as 1.

The current acceleration during acceleration can be viewed by viewing the system variables.

The initial value for acceleration is 100%.

Sample

```
PROGRAM DECELSMPL
  FOR K = 1 TO 100
    DECEL = K
    MOVE A1
    MOVE A2
  NEXT K
END
```

This program increases the acceleration from 1% to 100% in steps of 1 %.

DELAY

Purpose

The DELAY command stops the movement of the robot arm for a specified time.

Type

Movement control command

Format

DELAY <time>

Examples

DELAY 0.5

DELAY T * 0.2

Analysis and advice

The DELAY command stops the movement of the robot arm for a specified length of time. The <time> designation is specified in units of seconds. Since the execution precision is limited, try to keep your time designation in units of 0.01 second or more. A constant, variable or calculation may be used for the <time> designation. However, you may not use vector type data.

When the program stop operation is conducted while the DELAY command is being executed, after the specified time elapsed, the automatic operation is stopped. On the other hand, when the automatic operation is cancelled with the servo off operation or the emergency stop operation while the DELAY command is being executed, the DELAY command is executed again when the program is restarted.

DELAY is a movement control command in that it stops the movement of the robot arm for a specified period of time. Keep in mind that the DELAY command stops movement of the robot and does not stop execution of the program itself. When you want to delay the program itself, use the TIMER or WAIT command.

Sample

```
PROGRAM DELAYSMPL
  MOVE A1
  DELAY 2
  MOVE A2
  DELAY 2
  MOVE A3
  DELAY 2
END
```

The robot will stop moving for 2 seconds after it completes each move.

DEST

Purpose

The DEST command returns the destination of the present robot command.

Type

System variable (movement reference command)

Format

DEST

Examples

A1 = DEST

X = DEST.X

Analysis and advice

The DEST command is used to refer to the destination of the movement being executed on the world coordinate system at that time.

DEST can be used in the same way as regular POINT type data, but it can be used for reference only, and it does not allow substitution.

When the robot have come to rest after having positioned itself, DEST will return the location of that position.

Sample

```
PROGRAM DESTSAMPLE
  AA = A
  MOVE A
  ON DIN(1) DO AA = DEST
  MOVE A1
  MOVE A2
  MOVE A3
  MOVE A4
  IGNORE DIN(1)
  PRINT "POSITION DATA =", AA.X, AA.Y, AA.Z
END
```

This program moves the robot from point A1 to point A4 while monitoring Input Signal 1. When the signal turn on, the target position (at that time) will be displayed on the teach pendant.

DIM.....AS.....

Purpose

This defines variables and array variables.

Type

Program control command

Format

DIM <variable> AS <type>

DIM <array variable> (<number of elements>, <number of elements>, ...) AS <type>

Examples

DIM A AS INT

DIM P(5) AS POINT

Analysis and advice

This defines the types of variables and array variables and the number of elements.
Variables include global variables that can be referenced from any location in the defined file and where substitution is possible and include auto variables where substitution is possible and the content of the defined program only can be referenced. (See "2.6.4 Global variables and auto variables".)
The array variable can be defined only as the global variable which can be accessed and modified from any position of the defined program. The value of index of the array is 1 to No. of elements. That is, in this example, it is 1 to 5.
If the initial value set is outside the index range, an error occurs at the execution of SELECT command. Also, access and substitution outside the index range cause an error at program execution.
Type names can be declared as INT (integer type), REAL (real type), POINT (position type), TRANS (coordinate type), PAYLOAD (load type), and STRING (string type).
Only global variables can be defined for the STRING type. The size must also be specified at the time of definition.

Specify the initial value settings for the integer value and real number value arrays in the global section, and specify the initial value settings for the POINT, TRANS, and PAYLOAD type arrays in the data section.

Sample

```
GLOBAL
  DIM ICHI(3) AS POINT          One dimensional array of POINT type is declared.
  DIM MOJI AS STRING(20)        20-character STRING type variable is declared.
END
PROGRAM DIMSAMPLE
  ICHI(1) = P0
  ICHI(2) = POINT(500.0, 0.0, 100.0, 0.0, 0.0, 0)
  ICHI(3) = {500.0, -200.0, 100.0, 0.0, 0.0, 0}
  FOR I = 1 TO 3
    MOVE ICHI(I)
  NEXT I
END
```

```
DATA
  POINT P0 = 500.0, 200.0, 100.0, 0.0, 0.0 / LEFTY
  POINT ICHI(1) = 600.0, 0.0, 50.0, 0.0, 0.0 /
LEFTY
END
```

DIN( )

Purpose

The DIN command reads in the state (ON or OFF) of an input signal (or signals).

Type

I/O (Input/output) control command

Format

DIN(<signal name> [, <signal name>] ...)

Examples

IF DIN(1) THEN GOTO LOOP

WAIT DIN(1, -2, 3)

ON DIN(J, J + 1, J + 2) DO RETURN

Analysis and advice

The DIN command reads in the state (ON or OFF) of an input signal.

DIN is used in conjunction with an IF, WAIT or ON command to judge external signals.

<signal name> specifies the signal number of a signal to be read into the controller.

A positive signal is considered to be ON and a negative signal is considered to be OFF.

Up to 10 signal names can be specified. (Extra signal names exceeding 10 signal names are ignored.)

When the state of all the signals becomes as specified, DIN will return a value of TRUE (1). If even one of the signals is not as specified, DIN will return FALSE (0).

A constant, variable or calculation may be used for the <signal name> specification. However, you may not use vector type data.

Sample

```
PROGRAM DINSAMPLE
  WAIT DIN(1)          The robot will wait until Input Signal 1 turns on
  MOVE A1              before starting to move.
  MOVE A2
  MOVE A3
END
```

DISABLE

Purpose

The DISABLE command is used to disable system switches.

Type

System switch

Format

DISABLE <switch> [, <switch>] …

Examples

DISABLE PASS

DISABLE PASS, NOWAIT

Analysis and advice

The DISABLE command is used to disable system switches related to robot movement. There are six system switches.

(1) PASS

PASS is used to specify short-cut movement. Short-cut movement is an operating mode in which the robot is directed to begin its next move before completing the positioning of its previous move. The timing for switching over from the present movement to the next movement is specified with the system variable PASS command.

Short-cut movement allows you to reduce the time it takes the robot to get from one place to another. For more information, refer to Section 5.

A DISABLE PASS statement will cancel short-cut movement. The initial setting for the controller is DISABLE PASS.

(2) NOWAIT

NOWAIT specifies whether the controller should wait for the robot to finish positioning itself before sending out (or taking in) external signals.

Signal output timing is explained in Section 5.

A DISABLE NOWAIT statement directs the controller to wait for the robot to finish positioning itself before send out (or taking in) an external signal.

The initial setting for the controller is DISABLE NOWAIT.

(3) SWITCH

SWITCH determines whether the task change-over is performed or not in the multitask operation.

The task change-over is prohibited by the DISABLE SWITCH.

In the initial setting, ENABLE SWITCH is effective.

(4) MOVESYNC

Specifies the motion command synchronous mode or motion command asynchronous mode. In the DISABLE MOVESYNC state (i.e., motion command asynchronous mode), the system pre-executes commands all the way to just before four (4) (max.) motion commands ahead and waits for the finish of positioning. If the system variable PASS is set to "ENABLE", short-cut (pass) motion is allowed. The initial status is specified by the user parameter.

(5) SMOOTH

Specify smooth or normal movement. The motion command with SMOOTH specified continues movement without deceleration. The next motion command starts movement without acceleration regardless of whether SMOOTH is specified. ENABLE SMOOTH starts smooth operation, and DISABLE SMOOTH cancels the smooth operation.

The SMOOTH function is an option. DISABLE SMOOTH is specified by default.

Only interpolation motion commands (MOVES, MOVEC) are controlled by the smooth operation

(6) SLOWDOWN

Specify slowdown movement mode or normal movement mode. In slowdown operation, the speed can be changed (decreased) along the current movement. Subsequent motion commands after ENABLE SLOWDOWN change the speed along the movement according to the specified parameter.

DISABLE SLOWDOWN cancels the slowdown operation.

DISABLE SLOWDOWN is specified by default.

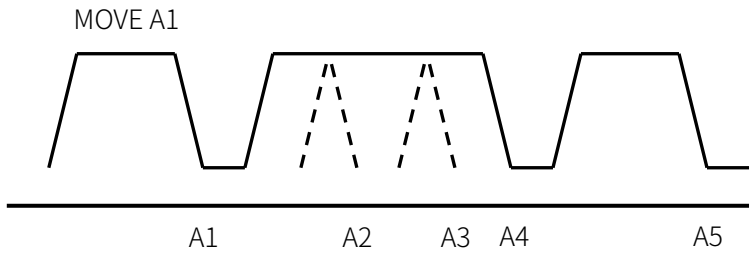
To make the system switch effective, use the ENABLE command.

Sample

```
PROGRAM DISABLESPL
  MOVE A1
  PASS = 80
  ENABLE PASS
  MOVE A2
  MOVE A3
  DISABLE PASS
  MOVE A4
  MOVE A5
END
```

The robot will move from point A1 to point A4 with short-cut movement.

From point A4 onward, the robot will move normally.



DOUT( )

Purpose

DOUT is used to output external signals.

Type

I/O (Input/output) control command

Format

DOUT(<signal name> [, <signal name>] ...)

Examples

DOUT(1, 2, -3)

DOUT(J, J + 1, J + 2)

Analysis and advice

DOUT is used to output external signals. <signal name> specifies the number (name) of a signal to be output from the controller.

A positive signal is considered to be ON and a negative signal is considered to be OFF.

Up to ten signal names may be specified in a signal DOUT command.

A constant, variable or calculation may be used for the <signal name> specification. However, you may not use vector type data.

When the same signal is output consecutively after execution of the DO command, signal output is not guaranteed. When output of multiple signals is specified, the simultaneousness of signal output timing is not guaranteed.

The robot program called DOUT (-4,3,-2,1) is executed by dividing into the robot programs DOUT(-4), DOUT(3), DOUT(-2), and DOUT(1), in that order, from the beginning. In other words, when the robot program DOUT (-4,3,-2,1) after DOUT(4,-3,2,-1) is executed, the signal changes from “1010” → “0010” → “0011” → “0001” → “0101” at approximately an interval of several μsec to several msec until the target value is finally reached.

Sample

```
PROGRAM DOUTSAMPLE
  FOR K = 1 TO 16
    DOUT(K)
    TIMER = 0.5
    WAIT TIMER == 0
    DOUT(-K)
  NEXT K
END
```

This program will send out (turn on) output signals 1 to 16 in order and in 0.5 second intervals.

DOUT( )

Purpose

This obtains the current status of the specified DOUT signal.

Type

I/O (Input/output) control command

Format

DOUT(<signal name> [, <signal name>] ...)

Examples

A = DOUT(1)

IF DOUT(J) THEN DOUT(J + 1)

Analysis and advice

The DOUT status is obtained when the DOUT ( ) command is written on the right side of the expression or in an IF expression. If the status of the signal in ( ) completely matches the specified status and the condition is met, TRUE (1) is returned. If the condition is not met, FALSE (0) is returned.

In this case, the DOUT status does not change.

To output an external signal, specify DOUT ( ) independently in a statement.

Sample

```
PROGRAM DOUTSAMPLE
 RESET DOUT
 IF DIN(1, 2) THEN
  DOUT(1, 2)
 ELSEIF DIN(1) THEN
  DOUT(1)
 ELSEIF DIN(2) THEN
  DOUT(2)
 ENDIF
 IF DOUT(1, 2) THEN DOUT(3)        If both of the output signals 1 and 2 are turned
 DELAY 1.0                         on, output signal 3 is turned on. (for 1 second)
 DOUT(-3)
END
```

ENABLE

Purpose

The ENABLE command is used to put system switches into effect.

Type

System switch

Format

ENABLE <switch> [, <switch>] ...

Examples

ENABLE PASS

ENABLE PASS, NOWAIT

Analysis and advice

The ENABLE command is used to put system switches related to robot movement into effect. There are six (6) system switches.

(1) PASS

PASS is used to specify short-cut movement. Short-cut movement is an operating mode in which the robot is directed to begin its next move before completing its previous move. The timing for switching over from the present movement to the next movement is specified with the PASS command.

Short-cut movement allows you to reduce the time it takes the robot to get from one place to another. For more information, see Section 5.

The shortcut movement is canceled by DISABLE PASS. The initial setting for the controller is DISABLE PASS.

(2) NOWAIT

NOWAIT specifies whether the controller should wait for the robot to finish positioning itself before sending out (or taking in) external signals. Signal output timing is explained in Section 5.

External signal input/output by DISABLE NOWAIT is performed after waiting for robot positioning completion.

The initial setting for the controller is DISABLE NOWAIT.

(3) SWITCH

SWITCH determines whether the task change-over is performed or not in the multitask operation.

The task change-over is prohibited by the DISABLE SWITCH.

In the initial setting, ENABLE SWITCH is effective.

(4) MOVESYNC

Specifies the motion command synchronous mode or motion command asynchronous mode. In the ENABLE MOVESYNC state (i.e., motion command synchronous mode), the system executes all the way to just before the next motion command and waits for the finish of positioning. The initial status is specified by the user parameter. In the MOVESYNC mode, short-cut (pass) motion is not allowed, irrespective of the status of system variable PASS. If the system variable PASS is set to "DISABLE" in the SCOL2 program, short-cut (pass) motion is allowed.

(5) SMOOTH

Specify smooth or normal movement. The motion command with SMOOTH specified continues movement without deceleration. The next motion command starts movement without acceleration regardless of whether SMOOTH is specified.
ENABLE SMOOTH starts smooth operation, and DISABLE SMOOTH cancels the smooth operation.
The SMOOTH function is an option. DISABLE SMOOTH is specified by default.
Only interpolation motion commands (MOVES, MOVEC) are controlled by the smooth operation.

(6) SLOWDOWN

Specify slowdown movement mode or normal movement mode. In slowdown operation, the speed can be changed (decreased) along the movement.
Subsequent motion commands after ENABLE SLOWDOWN change the speed along the movement according to the specified parameter.
DISABLE SLOWDOWN cancels the slowdown operation.
DISABLE SLOWDOWN is specified by default.

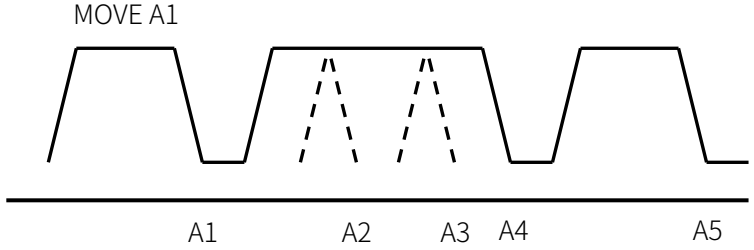
To make the system switch ineffective, use the DISABLE command.

Sample

```
PROGRAM DISABLESPL
  MOVE A1
  PASS = 80
  ENABLE PASS
  MOVE A2
  MOVE A3
  DISABLE PASS
  MOVE A4
  MOVE A5
END
```

This program directs the robot to move A1 to point A4 with short-cut movement.

From point A4 onward, the robot moves without short-cut movement.



EXIT

Purpose

This command aborts iterative processing.

Type

Program control command

Format

EXIT

Examples

```
FOR K = 1 TO 10
   IF DIN(1) THEN EXIT
NEXT
WHILE DIN(1)
   IF DIN(2) THEN EXIT
ENDWHILE
```

Analysis and advice

This command aborts iterative processing between FOR and NEXT, or between WHILE and ENDWHILE.

If iterative processing is aborted, the program skips to the next line of NEXT or ENDWHILE to be processed.

EXIT can only be used in iterative processing. In addition, execution using the combination of the ON and DO commands cannot be performed.

Sample

```
PROGRAM MAIN
  FOR K = 1 TO 64
    IF DIN(1) THEN EXIT          If DIN(1) is ON, iterative processing is aborted.
    DOUT(K)
    DELAY 1.0
    DOUT(-K)
  NEXT
END
```

EXP( )

Purpose

The EXP function returns the exponent of a number to the power e.

Type

Calculator command (Calculation function)

Format

EXP(<expression>)

Examples

K = EXP(3.5)

J1 = N * EXP(L - K)

Analysis and advice

This function is used to calculate the exponent of an <expression> to the power e. (e = 2.71828...)

You may use a constant, variable or result of calculation for the <expression> term. However, you may not use vector data. This command must be used in an expression.

Sample

```
PROGRAM MAIN
  K = 0.0                          ("DIM K AS REAL" is also possible.)
  EXPSAMPLE(5, K)
  PRINT TP, K, CR
END
PROGRAM EXPSAMPLE(N, K)
  K = EXP(N)                       This subprogram takes an argument N, finds the
  RETURN                           exponent of that argument to the base e, calls
END                                the result K, and sends control back to the main
                                   program.
```

EXTPLT

Purpose

This permits movement outside the pallet setting range.

Type

System switch

Format

{ ENABLE | DISABLE } EXTPLT

Examples

ENABLE EXTPLT

DISABLE EXTPLT

Analysis and advice

This permits movement outside the range that was specified when the pallet was created.

DISABLE is specified by default.

Access outside the range requires access at the same dimension as the pallet dimension.

Access using a single element is not possible.

Sample

```
PROGRAM MAIN
  PL1 = PALLET(3, 3, P1, P2, P3, P4)    Sets a 3×3 pallet.
  ENABLE EXTPLT                         Permits movement outside the setting range.
  MOVE PL1(3, 5)                        Moves to a position passing to the P3 direction.
  MOVE PL1(-2, 0)                       Moves to a position in the P4 reverse direction.
END
```

FGAIN

Purpose

This changes the gain of press control.

Type

System variable (motion command condition)

Format

FGAIN = <formula>

Examples

FGAIN = 2.0

PRESS(50, 0, 0) WITH FGAIN = 0.8

Analysis and advice

This changes the gain of press control.

The higher the gain, the faster the tracking speed, but the greater the stress during contact.

Also, too high a gain may cause oscillation during pressing.

The default value is 1.0, and the settable range is from 0.01 to 100.

System version

X8KCS-02 I and later

FINE

Purpose

FINE is a system constant used to set the positioning accuracy to fine.

Type

System constant

Format

FINE

Examples

ACCUR = FINE

MOVE A1 WITH ACCUR = FINE

Analysis and advice

The FINE statement is used with ACCUR command to set positioning accuracy to FINE.

The system constant has a value of 1. This value can be used as constant 1 in the expression. However, do not do it since it makes your program unnecessarily complicated.

You cannot substitute into system constants.

For information on positioning accuracy, refer to the ACCUR command.

Sample

```
PROGRAM FINESAMPLE
  ACCUR = FINE                    This program sets the positioning accuracy to
  MOVE A1                         FINE before moving around the robot.
  MOVE A2
  MOVE A3
END
```

FOR...TO...STEP...NEXT

Purpose

This specifies repeating of program.

Type

Program control command

Format

FOR <variable> = <expression 1> TO <expression 2> [STEP <expression 3>]
••••
[IF <logical expression> THEN EXIT]
NEXT [<variable>]

Examples

```
FOR K = 1 TO 4
  ••••
  INPUT I
  IF I == 3 THEN EXIT
NEXT K

FOR N = K1 TO K1 + K2 STEP K3
  ••••
NEXT N
```

Analysis and advice

The program block between the FOR command and the NEXT command is executed repeatedly. The block will keep on repeating itself until the condition specified by the FOR statement is satisfied.

When a FOR statement is executed, the value of <expression 1> is substituted into the <variable>. When the NEXT statement is executed, the value of <expression 3> specified by the STEP statement is added on to the <variable>. Should the value of the <variable> become greater than the value of <expression 2> at this time, the execution of the program will shift to the statement following the NEXT command. If <variable> is not greater than <expression 2>, the program execution will branch (go back) to the statement following the FOR statement.

The values of <expression 1>, <expression 2> and <expression 3> used in the FOR construct are those in effect when the FOR statement was first executed. Therefore, even should these values be changed while executing the loop, the number of times the loop is repeated will not change.

The <variable> should be used only to control the number of times the loop is repeated. Therefore, do not change the value of the <variable> while executing the loop.
If the value of <expression 3> is 1, you may omit statements after the STEP statement.
A constant, variable or calculation may be used for <expression 1>, <expression 2> or <expression 3>. However, you may not use vector type data.
For the corresponding NEXT statement, you should specify the variable specified by the FOR statement.
If you do not specify <variable> in the NEXT statement, a loop is made between the nearest FOR statement (executed finally) and the NEXT statement.
When another FOR command (being nested) is used in a FOR ~ NEXT loop, the number of nesting levels should be 127 levels or less. When the number of nesting levels exceeds 127 levels, an error occurs.
If EXIT is used, the loop can be exited when the conditional expression is met even during repeating of the FOR statement.

A FOR loop is ended by the NEXT command. Therefore, no matter what, the loop will be executed at least once.

Real numbers may be used for the <variable>, <expression 1>, <expression 2> or <expression 3>. However, since there is a certain imprecision when handling real numbers, try to use integers when telling the FOR statement how many times to repeat itself. Furthermore, values substituted into the <variable>, <expression 1>, <expression 2> or (expression 3> are converted into the data type when the loop is executed. Data types are converted as shown below.

(1) When the <variable> data type is undefined when a FOR statement is executed:
The data type of a variable will be undefined should the identifier used for the <variable> appear for the first time in the program in the FOR command. In such a case, all data will be converted to and processed as the data type of <expression 1>.

(2) When the <variable> data type has been defined before a FOR statement is executed:
When the identifier (used for the variable) has been used in the program beforehand, the data type of the <variable> will be the same as the data type of the data which was first entered into that <variable>.

Example) PROGRAM SAMPLE

```
    J1 = 5
    FOR J1 = 0.1 TO 0.9 STEP 0.01
      MOVE A1
      MOVE A2
    NEXT J1
  END
```

When the above program is executed, the data type of J1 is defined as the INT type before the FOR command is executed. Therefore, all variables used in the FOR command are converted into the INT type. Therefore, the controller will interpret the above FOR statement as:

FOR J1 = 0 TO 0 STEP 0

Consequently, the FOR loop will only be executed once.

When the FOR statement is used in the following manner, an error will occur.

(1)
```
FOR J1 = • • •
 FOR J2 = • • •
 • • • •
NEXT J1
```
Here, you are missing a NEXT statement corresponding to the FOR J2 statement.

(2)
```
FOR K = • • •
 • • •
 IF DIN(1) THEN GOTO L1
 • • •
NEXT K
• • •
L1:
```
A branch command from the inside to the outside of the loop or vice versa is not allowed.

Sample

```
PROGRAM FORSAMPLE
 FOR K = 1 TO 100
  MOVE A1
  MOVE A2
 NEXT K
END
```
The robot will make 100 trips back and forth between A1 and A2.

```
PROGRAM FORSAMPLE2
 FOR K = 1 TO 100
  MOVE A1
  IF DIN(1) THEN EXIT
  MOVE A2
 NEXT K
 MOVE A3
END
```
The robot will make 100 trips back and forth between A1 and A2.
If the conditional expression in the IF statement is met, repeating is stopped, and the loop is exited.

FORCE3LIM

Purpose

This specifies the torque limit in the downward direction of the 3rd axis.

Type

System variable (movement reference command)

Format

FORCE3LIM = <expression>

Examples

FORCE3LIM = 50

MOVE A1 WITH FORCE3LIM = 70

Analysis and advice

This specifies the torque limit of the 3rd axis in units of N.

This command cannot be used depending on models. Executing this command in such models results in an alarm "002-025 : Invalid command". For details on whether this command can be used or not and the range of values to be specified, refer to the "Push Force [N] Unit Designation Function" in the robot manual for each model.

The value specified by this function is only a guide. Variations may occur depending on the operating environment, etc. and the robot configuration. Also, when there is a load on the robot hand, use the PAYLOAD command to set the load information.

When both this command and the TORQUE command specify a torque limit, the limit value specified by the command executed later takes effect. If this command is executed later, the torque limit values other than the torque limit value in the downward direction of the 3rd axis remain the set value specified by the TORQUE command. The torque limit specified by this command can be released by the RESET TORQUE command.

You may use a constant, variable or result of calculation for the <expression> term. However, you may not use vector data. If a value not more than 0 is specified as the limit value of torque, 0 is assumed to have been specified.

Sample

```
PROGRAM MAIN
   PAYLOAD = {2, 50}
   MOVE P1
   FORCE3LIM = 100                          Specifies the torque limit in the downward
   MOVEA 3, 0 WITH ACCUR = COARSE           direction to 100 N.
   DELAY 1.0
   MOVE P1
   RESET TORQUE                             Releases the torque limit.
END
```

FORMAT( )

Purpose

This converts a given number to a character string in the specified format.

Type

Character string function

Format

FORMAT(<expression>, <specified format>)

Examples

TXT = FORMAT(X, “0000.00”)
TXT = FORMAT(1.2, “?0.09”)
TXT = FORMAT(1234, “0.00E + 00”)
TXT = FORMAT(255, “XXXX”) . . . . . TXT is a character string variable

Analysis and advice

This converts a given numerical value to a character string in the specified format.
The specific format characters include “0”, “9”, “?”, “E(e)”, and “X(x)”.

“0” ..... This is a digit placeholder. It indicates a number or zero.
If the value has fewer digits than the specified number of digits, zeros are added for the required digits.
If the value has more digits than the specified number of digits, the integer section of the value is displayed unchanged, and the decimal section is rounded to the specified number of digits.
Example: FORMAT(12.34, “000.000”)
→ “012.340” is returned.

“9” ..... This is a digit placeholder. This either displays a number or nothing at all.
Basically, it functions the same as “0”, but it does not add zeros when the value has fewer digits than the specified number of digits.
Example: FORMAT(12.34, “999.999”)
→ “12.34” is returned.

“?” ..... This is a digit placeholder. This either displays a number or adds a space.
Basically, it functions the same as “0”, but it adds the required number of spaces when the value has fewer digits than the specified number of digits.
Example: FORMAT(12.34, “???.???”)
→ “□12.34□” is returned. (where □ indicates a space)

"E+" … Numerical value is output in exponential form.
"E-" For example, if the specified format is "0.00E + 00", the value of 12,200,000 is
"e+" displayed as 1.22E + 07.
"e-" If the specified format is changed to "90.0e + 0", the value is displayed as 12.2e + 6.

When a "+" after the E is specified in the format, the value is returned by adding a "+" or "-" sign after E.

Example: FORMAT(1234, "0.00E + 0")
→ "1.23E + 3" is returned.
FORMAT(0.1234, "0.00E + 0")
→ "1.23E - 1" is returned.

When a "-" after the E is specified in the format, the value is returned by adding blank or a "-" sign after E.

Example: FORMAT(1234, "0.00E - 0")
→ "1.23E 3" is returned.
FORMAT(0.1234, "0.00E - 0")
→ "1.23E - 1" is returned.

When nothing is specified after the E, the value is returned by adding no sign or a "-" sign after E.

Example: FORMAT(1234, "0.00E0")
→ "1.23E3" is returned.
FORMAT(0.1234, "0.00E0")
→ "1.23E - 1" is returned.

"X" …… This outputs an integer value in hexadecimal format.
"x" If the numerical value has fewer digits than the number of digits in "X" of the display format, zeros are added up to the specified number of digits in the format.
Negative values are 8-digit hexadecimal numbers.
This specified format character cannot be used together with other specified format characters. Also, "X" and "x" cannot be used together.

Example: FORMAT(100, "X")
→ "64" is returned.
FORMAT(100, "xxxx")
→ "0064" is returned.
FORMAT(-1, "XXXX")
→ "FFFFFFFF" is returned.

Sample

```
GLOBAL
  DIM TXT AS STRING(10)
END
PROGRAM MAIN
  X = 123.4
  TXT = FORMAT(X, “0000.00”)          Returns the character string “0123.40”.
  PRINT TP, TXT, CR
END
```

```
GLOBAL
  DIM TXT AS STRING(10)
END
PROGRAM MAIN
  TXT = FORMAT(1.2, “?0.09”)          Returns the character string “␣1.2”. (where ␣
  PRINT TP, TXT, CR                   indicates a space)
END
```

```
GLOBAL
  DIM TXT AS STRING(10)
END
PROGRAM MAIN
  TXT = FORMAT(1234, “0.00E + 00”)    Returns the character string “1.23E + 03”.
  PRINT TP, TXT, CR
END
```

```
GLOBAL
  DIM TXT AS STRING(10)
END
PROGRAM MAIN
  TXT = FORMAT(255, “XXXX”)           Returns the character string “00FF”.
  PRINT TP, TXT, CR
END
```

FREE

Purpose

FREE is a system constant used to make the robot configuration as undefined.

Type

System constant

Format

FREE

Examples

CONFIG = FREE

MOVE A1 WITH CONFIG = FREE

Analysis and advice

The FREE system constant is used along with the CONFIG command in order to specify the robot configuration as undefined.

FREE has the value of 0. If you wanted to, you could use FREE in your program as a constant having the value 0. However, do not do this since it makes your program hard to read.

You cannot change (substitute into) system constants including FREE. For information on robot configuration, see the “CONFIG command.”

Sample

```
PROGRAM FREESAMPLE
  CONFIG = FREE
  MOVE A1
  MOVE A2
END
```

This program will make the robot configuration undefined before beginning to move the robot.

FREELOAD

Purpose

The FREELOAD command will zero the data for the load acting on the end of the robot hand.

Type

Library function (SCOL.LIB)

Format

FREELOAD

Examples

FREELOAD

Analysis and advice

The FREELOAD command will zero the data for the load acting on the end of the robot hand.

The SCOL2 language allows load data to be set for the load applied to the robot end effector so that the robot can operate optimally for any load within the allowable load range of the robot.

Loads acting on the robot hand are set with the system variable PAYLOAD. The controller uses these values to calculate control constants for robot acceleration and deceleration that are appropriate for the load. The load data include the weight of the load on the end effector and the value of the distance from the mechanical interface to the center of gravity. The FREELOAD command is a command for zeroing the load data.

When a FREELOAD command is executed, all load data will become 0. This is true for both mass load data and inertia load data.

This command will not work if you do not have the file SCOL.LIB in the controller.

Sample

```
PROGRAM FREELOADSL
  PAYLOAD = HAND
  MOVE A1
  CLOSE1
  DELAY 0.5
  MOVE A2 WITH PAYLOAD = HAND + MOTOR
  OPEN1
  DELAY 0.5
  MOVE A3
  FREELOAD                          This program sets the load data to zero before
  MOVE A2                           moving around the robot.
END
```

GAIN

Purpose

The GAIN command is used to specify whether the gain (for servo control) is ON or OFF for each axis.

Type

System variable (movement condition command)

Format

GAIN = { <expression>, <expression>, <expression>, <expression>, <expression> }

Examples

GAIN = { 0, 0, 1, 0, 0 }
MOVE A1 WITH GAIN = { , , ON }

Analysis and advice

The GAIN command is used to specify whether the gain (servo control) of each axis is to be ON or OFF.
Should the GAIN be specified as OFF, servo control for that axis will stop the next time a movement command is executed. Axes for which servo control has been stopped are in the servo free state (in which positioning control is not carried out) and can be moved around freely by external forces. Positioning checks are not performed for axes for which the gain is OFF. Furthermore, should you give a movement command to one of those axes, the axis will not move.
This command is used when, for example, one is fitting a part into a hole in a workpiece. By turning off all the gains except for the Z-axis (which must be on so you can move the hand up and down), all the axes (except for the Z-axis) are free (i.e., not locked up) and the robot hand can move freely in the horizontal direction. This allows the robot to move horizontally along with the workpiece and fit in the part in the hole even should the workpiece and part not be aligned exactly.
The GAIN command is treated as a system variable having five values (that correspond to the five axes). The programmer specifies which gains are to be on and off for each axis in the { } brackets following the GAIN commands. Specifications are divided by commas with the first specification corresponding to Axis 1, the second to Axis 2, and so on. A “0” means that the gain is to be OFF and a “1” means that the gain is to be ON. Should specifications be abbreviated (for example, {,, ON}), the controller will assume all non-specified gains to be OFF.

You may use a constant, variable or result of calculation for the <expression> term. However, you may not use vector data. You may also use the system variables ON and OFF. (ON sets the gain on, and OFF sets the gain off.) You must always synchronize the execution of a GAIN command with the movement of the robot. That is, be sure that the GAIN command is executed after the previous movement command is completed (as seen in the sample program).
Should you execute a GAIN command while the robot is still moving, the robot may move incorrectly. For more information on setting gains in synchronization with the robot movement, see the "SETGAIN command."
Even should you complete automatic operation and switch over to the manual mode, the GAIN settings will not change. In order to turn on gains which have been turned off in the manual mode, push the corresponding guide keys for those axes while in the guide mode.
When a selected gain value is not more than 0 for ON and not less than 1 for OFF, 0 and 1 are considered to be specified.

Sample

```
PROGRAM GAINSAMPLE
  MOVE A1
  WAIT MOTION >= 100
  GAIN = { OFF, OFF, ON, OFF, OFF }
  MOVE A2
  OPEN1
  DELAY 0.5
  MOVE A1
  WAIT MOTION >= 100
  GAIN = { ON, ON, ON, ON, ON }
  READY
END
```

This program turns off all gains except that for the Z-axis (Axis 3) before moving the robot to point A2.

```
PROGRAM GMOVE
  G = GAIN
  GAIN = { , , ON }
  MOVE P1
  WAIT MOTION >= 100
  GAIN = G
  MOVE P2
END
```

This program turns off the gains on all axes except Axis 3 (the Z-axis), and later restores the gains to their original state after moving.

GLOBAL.....END

Purpose

This specifies the global section (global variable area).

Type

Program control command

Format

GLOBAL

Examples

```
GLOBAL
A = 10
END
```

Analysis and advice

The GLOBAL ... END command specifies the global section in the SCOL2 program. In the global section, referencing can be performed from any location in the program, and global variables, which allow substitution, are defined. To identify the type of a variable other than the array, setting of an initial value in the substitution format is required.
As the array variable defines the type and number of elements by DIM command and sets the initial value separately, some may not have an initial value.
The following types can be used for the global variable area.
INT type, REAL type, POINT type, TRANS type and PAYLOAD type.
Combination of the above variables with the array.
However, specify the integer type and real number type variable definitions and the initial value settings for the integer type arrays and real number value arrays in the global section, and specify the POINT type, TRANS type, and PAYLOAD type variable definitions and the initial value settings for the POINT type arrays, TRANS type arrays, and PAYLOAD type arrays in the data section.

Sample

```
GLOBAL
 A = 1
END
PROGRAM TEST
 A = A + 1
 PRINT “A =“, A, CR
END
```

GOTO

Purpose

The GOTO command specifies that the execution of the program is to be branched to the location marked by the label of the GOTO command.

Type

Program control command

Format

GOTO <label>

Examples

GOTO LOOP
GOTO RESTART
IF DIN(1) THEN GOTO L1

Analysis and advice

The GOTO command specifies that the execution of the program is to be branched to the location marked by the label of the GOTO command.
Branching locations for the GOTO command are limited to statements in the same program. Should there be no location in the program corresponding to the GOTO label, you will get an error. Also, a syntax error will be generated if there are multiple statements with the same name label in the same program.
In order to label a branching location, put the label name (identifier) at the beginning of the statements you wish to execute. Be sure to put a colon after the identifier.

Sample

```
PROGRAM GOTOSAMPLE
 MOVE A1
 LOOP:
  MOVE A2
  MOVE A3
 GOTO LOOP                    When the program is executed finally, the
END                           program goes back to the label LOOP.
```

GOTO( ) .....

Purpose

This command will cause the execution of the robot to branch off depending on the value of the expression in the brackets.

Type

Program control command

Format

GOTO(<expression>) <label> [, <label>]•••

Examples

GOTO(K) LABEL1, LABEL2, LABEL3
GOTO(N1 - N2) L1, L2, L3, L4, L5

Analysis and advice

The GOTO ( ) command will cause the execution of the program to be branched off in accordance with the value in the brackets ( ).
When the value in the brackets (i.e., the <expression>) is 1, the program will branch off to the <label> furthest to the left. When the value in the brackets is 2, the program will branch off to the <label> second furthest to the left, and so on. Up to 10 labels can be specified. Extra labels exceeding 10 labels are ignored.
If the value in the brackets is greater than the number of labels or should the value be zero or less, program execution will proceed to the statement following the GOTO statement. If the value in the brackets is a real number, all decimal points will be cut off and what remains will be treated as an integer.
You may use a constant, variable or result of calculation for the <expression> term. However, you may not use vector data.
Branching locations for the GOTO ( ) command are limited to statements in the same program. Should there be no location in the program corresponding to the specified GOTO ( ) label, you will get an error. Also, a syntax error will be generated if there are multiple statements with the same name label in the same program.
In order to label a branching location, put the label name (identifier) at the beginning of the statements you wish to execute. Be sure to put a colon after the identifier.

Sample

```
PROGRAM MAIN
  INPUT N
  GOTOSAMPL2(N)
END
PROGRAM GOTOSAMPL2(N)
  GOTO(N) L1, L2, L3
  RETURN
  L1:
    MOVE A1
    RETURN
  L2:
    MOVE A2
    RETURN
  L3:
    MOVE A3
    RETURN
END
```

If the argument N is 1, 2 or 3, the robot will move to point A1, A2 or A3 respectively. If N is not 1, 2 or 3, program execution will be sent back to the main program without moving the robot.

HERE

Purpose

The HERE statement returns the current position of the robot.

Type

System variable (movement reference command)

Format

HERE

Examples

MOVE HERE

A1 = HERE

X = HERE.X

Analysis and advice

The HERE command returns the current position of the robot on the world coordinate system.

HERE can be handled just like any other POINT type data with the exception that you can only refer to the values contained inside and cannot change the values themselves.

If the HERE command is executed while the robot is in motion, the position after completing the currently executed motion is retrieved as the current position.

It is also possible to refer to each element such as HERE.X. For details, see [1] POINT type (position type) variables in "2.6.2 Vector type variables".

Sample

```
PROGRAM HERESAMPLE
   MOVE HERE + POINT(0, 0, 100)
   MOVE HERE + POINT(0, 50, 0)
   IF HERE.CONFIG == RIGHTY THEN DOUT(1)
END
```

Moves 100 mm in the +Z direction from the execution start position.

Moves 50 mm in the +Y direction.

Outputs signals with reference to configuration.

HEXIN( )

Purpose

This function reads input signals in the hexadecimal notation.

Type

I/O (Input/output) control command

Format

HEXIN(<signal name>, <signal length>)

Examples

K = HEXIN(1, 2)

J2 = HEXIN(N, N + 2)

GOTO(HEXIN(20, 2)) L1, L2, L3

Analysis and advice

This function reads the input signals of <signal length> in the hexadecimal notation, starting with the signal specified by <signal name>.

When K = HEXIN (1, 2), for instance, the state of two (2) input signals 1 and 2 is read as the HEX code.

The signal length can be specified in the range of 1 to 32. As the input signal number increases, it corresponds to higher-order bit accordingly. Each signal is coded as follows.

ON = 1, OFF = 0

Example) When the state of input signals 1 ~ 12 is as shown in the table below, the value of "HEXIN (1, 12)" is 809 (i.e., 329 in the hexadecimal notation).

Input signal number	12	11	10	9	8	7	6	5	4	3	2	1
Input signal state	OFF	OFF	ON	ON	OFF	OFF	ON	OFF	ON	OFF	OFF	ON
Binary expression	0	0	1	1	0	0	1	0	1	0	0	1
Hexadecimal expression				3				2				9

You can use a constant, variable, expression and calculation for <signal name> and <signal length>. You cannot use the vector type data, however.

This command is used in the expression.

Sample

```
PROGRAM HEXINSAMPL
  K = HEXIN(1, 7)          The state of input signals 1 to 7 is coded, the
  SPEED = K                motion speed is set according to such a value,
  MOVE A1                  then the robot moves at the set speed.
  MOVE A2
END
```

HEXOUT( )

Purpose

This function outputs signals by coding them in the hexadecimal notation.

Type

I/O (Input/output) control command

Format

HEXOUT(<signal name>, <signal length>, <expression>)

Examples

HEXOUT(1, 4, 3)

HEXOUT(N, N + 4, K)

Analysis and advice

This function codes the value of <expression> in the hexadecimal notation and outputs the digits specified by <signal length> to the number of the signals as specified by <signal length>, starting with the signal specified by <signal name>.

When HEXOUT (1, 4, 3) is commanded, for instance, the value of "3" which is coded in the hexadecimal notation is output to the four signals (1 to 4), and output signals 1 and 2 become ON.

The signal length can be specified in the range of 1 to 32. If the value of <expression> exceeds the value specified by <signal length>, the high-order digit or digits are ignored.

As the input signal number increases, it corresponds to higher-order bit.

Each signal is coded as follows.

ON = 1, OFF = 0

Example) When HEXOUT (1, 12, 952) is commanded, the output signals are as shown below.

Decimal expression										9	5	2
Hexadecimal expression				3				B				8
Binary expression	0	0	1	1	1	0	1	1	1	0	0	0
Output signal number	12	11	10	9	8	7	6	5	4	3	2	1
Output signal state	OFF	OFF	ON	ON	ON	OFF	ON	ON	ON	OFF	OFF	OFF

You can use a constant, variable, expression and calculation for <signal name>, <signal length> and <expression>. You cannot use the vector type data, however. If the same signals are output consecutively, the signal output last becomes effective.

Be careful of agreement's there being in the signal range by which it is possible to guarantee simultaneousness.
The simultaneousness of HEXOUT to the range in units of 16 bits from DOUT1, DOUT101, and DOUT301 can be guaranteed but the simultaneousness of HEXOUT which strides the boundary cannot be guaranteed.

Sample

```
PROGRAM HEXOUTSMPL
  FOR K = 1 TO 4
    J = 2 ^(K - 1)
    HEXOUT(1, 4, J)
    TIMER = 0.5
    WAIT TIMER == 0
    HEXOUT(1, 4, 0)
  NEXT K
END
```

Output signals 1 to 4 are output in turn at intervals of 0.5 second.

IF....THEN.....ELSE

Purpose

This executes a conditional statement.

Type

Program control command

Format

IF <logical expression> THEN <statement> [ELSE <statement>]

Examples

IF DIN(1) THEN K = K + 1 ELSE K = 0

Analysis and advice

If the conditions of the <logical expression> following IF are satisfied, the <statement> following THEN will be executed. If the conditions are not satisfied, the statement following ELSE will be executed.

An ELSE statement is not mandatory in an IF construction. If the IF condition is not satisfied and there is no ELSE statement, program execution will shift to the next step following the IF command.

The <statement> following the THEN or ELSE statement may not contain PROGRAM, END, IF, FOR, NEXT or WAIT.

IF ~ THEN ~ ELSE constructs are considered as a single unit, and for that reason they all have to be on the same line. (That means you must write everything following the ELSE statement on that line.)

Multiple lines of IF commands were added from SCOL2 language. For details, see the following pages.

Sample

```
PROGRAM IFSAMPLE
  IF DIN(1) THEN K = 1 ELSE K = 0        Should Input Signal 1 be ON, K will equal 1.
  MOVE A1                                Should Input Signal 1 be OFF, K will equal 0.
  MOVE A2
  MOVE A3
  PRINT TP, K, CR
END
```

IF...THEN...ELSE...ENDIF

Purpose

This executes a conditional statement.

Type

Program control command

Format

IF <logical expression> THEN
    <statement>
[ELSEIF <statement>
    <statement>]
[ELSE
    <statement>]
ENDIF

Examples

```
IF DIN(1) THEN
  K = K + 1
  B = 1
ELSE
  K = K - 1
ENDIF
```

Analysis and advice

If the conditions of the <logical expression> following IF are satisfied, the <statement> following THEN will be executed. If the conditions are not satisfied, the statement following ELSE will be executed.

An ELSE statement is not mandatory in an IF construction. If the IF condition is not satisfied and there is no ELSE statement, program execution will shift to the next step following the ENDIF command.

Besides ELSE, ELSEIF can also be used. (Up to 32 times)

After THEN, ELSE, a <statement> (process) can be specified using multiple lines.

In the 1-line IF command on the previous page, IF ... THEN ... (ELSE...) must be written in the same line, but in IF commands using multiple lines (when using ENDIF, ELSEIF), the statement cannot be written in a single line, and carriage returns in the form of the example only are possible. A carriage return cannot be used between IF ... THEN or between ELSEIF ... THEN.

Sample

```
PROGRAM IFSAMPLE
 K = 0
 B = 0
 IF DIN(1) THEN
  K = K + 1
  B = 1
 ELSE
  K = K - 1
  B = 2
 ENDIF
 PRINT “K =“, K, CR
 PRINT “B =“, B, CR
END
```

When input signal 1 is ON, 1 is added to K, B = 1 is set, and when input signal 1 is off, 1 is subtracted from K, and B = 2 is set.

```
PROGRAM IFSAMPLE
 IF DIN(5) THEN
  M = 5
 ELSEIF DIN(6) THEN
  M = 6
 ELSEIF DIN(7) THEN
  M = 7
 ELSE
  M = 0
 ENDIF
 PRINT “M =“, M, CR
END
```

When input signal 5 is ON, M=5 is set. When input signal 6 is ON, M = 6 is set. When input signal 7 is ON, M = 7 is set. In all other cases, M = 0 is set.

IGNORE

Purpose

The IGNORE command is used to cancel the monitoring of a condition specified by a previous ON command.

Type

Program control command

Format

IGNORE <monitoring condition>

Examples

IGNORE DIN(1)
IGNORE TIMER

Analysis and advice

The IGNORE command is used to cancel the monitoring of a condition specified by a previous ON command.
In the <monitoring condition> specification, use the exact same logical expression as you used for the corresponding ON statement. However, the ON condition statement that can be ignored should be specified before the IGNORE command line. At this time, the same ON condition should not be specified on two (2) or more lines.
When the IGNORE statement is executed, monitoring of the condition will cease.
For more information on condition monitoring, refer to the ON command.

Sample

```
PROGRAM MAIN
 IGNORESMPL
 MOVE P1
 MOVE P2
END
PROGRAM IGNORESMPL
 ON DIN(1) PAUSE DO RETURN
  MOVE A1
  MOVE A2
  WAIT MOTION >= 100
 IGNORE DIN(1)
 RETURN
END
```

Should Input Signal 1 go on, execution will return to the main program after the motion in progress at the time is completed. Monitoring of Input Signal 1 will cease when the movement from point A1 to point A2 is completed.

INERTIA

Purpose

The INERTIA command is used to specify the inertia data for the load on the robot hand.

Type

System variable (motion condition command)

Format

INERTIA = <moment of inertia>

Examples

INERTIA = 0.3
MOVE A1 WITH INERTIA = 0.1

Analysis and advice

System variable that specifies the moment of inertia [kgm^2] of the load on the robot hand.
The SCOL language allows load data to be set for the load applied to the robot end effector so that the robot can operate optimally for any load within the allowable load range of the robot.

In <moment of inertia>, the moments of inertia (around the center of rotation of Axis 4) of the tool attached to the robot and the workpiece being gripped shall be set in [kgm^2].

Sample

```
PROGRAM SAMPLE
  PAYLOAD = {5, 20}          Set load mass and offset
  INERTIA = 0.4              Set moment of inertia
  MOVE P1
  MOVE P2
  MOVE P3
END
```

System version

X8KCS-0 3C and later

INNER( )

Purpose

This finds the inner product of two TRANS type variables.

Type

Vector variable calculation function

Format

INNER(<TRANS type>, <TRANS type>)

Examples

A = INNER(T1, T2)

B = INNER(TRANS(100, 100, 100, 0) , T2)

Analysis and advice

This finds the inner product calculation result of the vectors indicated by the two TRANS type variables in ( ).

The inner product calculation result is a real number.

Sample

```
PROGRAM MAIN
  T1 = TRANS(100, 100, 50, 30)
  T2 = TRANS(100, -50, 100, 0)
  N = INNER (T1, T2)
  PRINT “INNER PRODUCT N = “, N, CR
END
```

This finds the inner product calculation result of the vectors indicated by the TRANS type variables in T1 and T2.

INPSTSTP,INPSTSIP1~8

Purpose

This references the timeout monitoring result of the INPUT command in the selected communication channel.

Type

System variable (Other)

Format

INPSTSTP
INPSTSIP1
INPSTSIP2

Examples

IF INPSTSTP == 1 THEN GOTO ERR

Analysis and advice

This references the timeout monitoring result of the INPUT command.
0: The INPUT command was executed successfully.
1: The INPUT command was executed by a timeout.

The timeout monitoring result is always 0 when a timeout is not set for the INPUT command.
When the timeout monitoring result is referenced, set the timeout time for the INPUT command.

The timeout monitoring result languages for the communication channels in the INPUT command are as follows.

Communication channel	Timeout time language	Timeout monitoring result language
TP	INPTMOTP	INPSTSTP
IP1	INPTMOIP1	INPSTSIP1
IP2	INPTMOIP2	INPSTSIP2
:	:	:
IP8	INPTMOIP8	INPSTSIP8

Sample

```
PROGRAM INPSTSSAMPLE
 X1 = 0.0
 INPTMOIP1 = 0.1
 INPUT IP1, X1
 IF INPSTSIP1 == 1 THEN
  PRINT “NG”, CR
 ELSE
  PRINT “OK”, CR
 ENDIF
END
```

The INPUT timeout time is set to 0.1 second, and after the INPUT command is executed, if the INPUT status information is 1, this is a timeout, and so NG (Fail) is displayed on the teach pendant.
If the INPUT status information is a value other than 1, this is successful, and OK (Pass) is displayed on the teach pendant.

INPTMOTP,INPTMOIP1~8

Purpose

This sets the timeout time in second units for the INPUT command in the selected communication channel.

Type

System variable (Other)

Format

INPTMOTP
INPTMOIP1
INPTMOIP2

Examples

INPTMOTP = 10.0
INPTMOIP1 = 0.01

Analysis and advice

This sets the timeout time as a value in second units for the INPUT command.
If the INPUT command is from the teach pendant, the timeout time is set to INPTMOTP.
After the INPUT command is executed, if the data is not read from the communication channel before the preset timeout time has elapsed, 0 is substituted into the variable specified in the INPUT command, and operation proceeds to the next step.
If the communication channel specified by the INPUT command does not match the communication channel for the timeout, the timeout time is not monitored.

If a value less than 0.001 is set, the timeout time is not monitored.

The timeout time languages for the communication channels in the INPUT command are as follows.

Communication channel	Timeout time language	Timeout monitoring result language
TP	INPTMOTP	INPSTSTP
IP1	INPTMOIP1	INPSTSIP1
IP2	INPTMOIP2	INPSTSIP2
:	:	:
IP8	INPTMOIP8	INPSTSIP8

Sample

```
PROGRAM INPTMOSAMPLE
  X1 = 0.0
  INPTMOIP1 = 0.1
  INPUT IP1, X
  IF INPSTSIP1 == 1 THEN
    PRINT “NG”, CR
  ELSE
    PRINT “OK”, CR
  ENDIF
END
```

The INPUT timeout time is set to 0.1 second, and after the INPUT command is executed, if the INPUT status information is 1, this is a timeout, and so NG (Fail) is displayed on the teach pendant.
If the INPUT status information is a value other than 1, this is successful, and OK (Pass) is displayed on the teach pendant.

INPUT

Purpose

The INPUT command reads in data from a specified communications channel.

Type

I/O (Input/output) control command

Format

INPUT [{TP | IP1 | IP2 | … | IP8 } ,] <variable> [, <variable>] …

Examples

INPUT K1, K2, K3

INPUT IP1, K

Analysis and advice

The INPUT command is used to read in data from a communication channel.
Data that can be read are INT, REAL, and STRING types.
One of TP, IP1, IP2, ..., and IP8 is selected as the communication channel.
TP is a communication channel for the teach pendant only. IP1 is for the IP1 communication channel of the controller.
If you do not specify a communication channel in your INPUT statement, data will be read in from the teach pendant communication channel.
When an INPUT command is executed, the program will wait until the data is read in from the communication channel. Data which has been read in will be placed in the assigned variable(s). If there is more data than there are variables, excess data will be ignored. If there is less data than there are variables, the program will wait until the remaining data comes in.
When inputting data from the teach pendant, keep real numbers separate with commas. When you are done entering the numbers, push the EXE key. When inputting data from anywhere other than the teach pendant, the data will be processed when transmission is completed. For information on data communication, refer to the “Communication Manual”.
After the moving arm has stopped, this command is not executed.

If 11 digits or more are entered for the integer data in the INPUT command, an error occurs.
The number of allowable digits for real number data is 9 digits for the integer part and 3 digits for the decimal part. If a value that exceeds this number of digits is entered in the INPUT command, an error occurs.
An INPUT command to INT or REAL type data will cause an error if a string that cannot be converted to a numerical value is input.
STRING type data will result in an error if a string longer than the size is entered.

Sample

```
PROGRAM INPUTSMPL
  N1 = 0.0
  N2 = 0.0
  N3 = 0.0
  PRINT TP, "*** INPUT N1, N2, N3 ***"
  INPUT TP, N1, N2, N3
  PRINT (N1 + N2 + N3) / 3, CR
END
```

This program will read in three values (N1, N2 and N3) from the teach pendant, find the average, and display the average on the teach pendant.

```
GLOBAL
  DIM A AS STRING(200)
END
PROGRAM MAIN
  INPUT A
  PRINT "A = ", A, CR
END
```

INSTR( )

Purpose

This searches for the specified character string within a given character string and returns the start position of the first character string that was found.

Type

Character string function

Format

INSTR(<character string expression>)>, <character string expression>)

Examples

A = INSTR(“ABCDE”, “CD”)

B = INSTR(TXT, KEY) ..... TXT and KEY are character string variables

Analysis and advice

INSTR (search target character string, search character string)

This searches for the search character string within a search target character string and returns the start position of the first character string that was found as an integer value.

If a search character is not found, 0 is returned. Also, if the search target character string length is 0, a value of 0 is returned.

If the search character string length is 0, this returns the position of the first character of the search target character string.

If the search target character string or search character string is an undefined variable or is not a character string (numerical value, etc.), a compile error occurs.

Sample

```
PROGRAM MAIN
 A = INSTR("ABCDE", "CD")        Searches for "CD" from "ABCDE" and returns
 PRINT TP, A, CR                 the start position "3" that was found first.
END
```

```
GLOBAL
  DIM TXT AS STRING(10)
  DIM KEY AS STRING(10)
END
PROGRAM MAIN
  TXT = "baseBALL"
  KEY = "BA"
  B = INSTR(TXT, KEY)             Searches for "BA" from "baseBALL" and returns
  PRINT TP, B, CR                 the start position "5" that was found first.
END
```

INSTRREV( )

Purpose

This searches from the last character for the specified character string within a given character string and returns the start position of the first character string that was found.

Type

Character string function

Format

INSTRREV(<character string expression>, <character string expression>)

Examples

A = INSTRREV(“ABAB”, “B”)

B = INSTRREV(TXT, KEY) ..... TXT and KEY are character string variables

Analysis and advice

INSTRREV (search target character string, search character string)

This searches from the last character position for the search character string within a search target character string and returns the start position of the first character string that was found as an integer value.

If a search character is not found, 0 is returned. Also, if the search target character string length is 0, a value of 0 is returned.

If the search character string length is 0, this returns the position of the first character of the search target character string.

If the search target character string or search character string is an undefined variable or is not a character string (numerical value, etc.), a compile error occurs.

Sample

```
PROGRAM MAIN
 A = INSTRREV(“ABAB”, “B”)
 PRINT TP, A, CR
END
```

Searches from the last character for “B” from “ABAB” and returns the start position “4” that was found first.

```
GLOBAL
  DIM TXT AS STRING(10)
  DIM KEY AS STRING(10)
END
PROGRAM MAIN
  TXT = “ABCABCabc”
  KEY = “ABC”
  B = INSTRREV(TXT, KEY)
  PRINT TP, B, CR
END
```

Searches from the last character for “ABC” from “ABCABCabc” and returns the start position “4” that was found first.

INT( )

Purpose

The INT command changes a numerical value into an integer.

Type

Conversion function

Format

INT(<expression>)

Examples

AK = INT(-20.345)
N = INT(K)
J1 = K - INT(N - 28.5)

Analysis and advice

The INT command converts the number or calculation result in the brackets ( ) to an integer.
Note that the INT command simply cuts off real numbers to the right of the decimal point and converts them into integer.
This command is used when one wants to specify the data type of a variable as an INT type.
You may use a constant, variable or result of calculation for the <expression> term. However, you may not use vector data.
This command must be used in an expression.

Sample

```
PROGRAM MAIN
 K = 0                                  ("DIM K AS INT" is also possible.)
 INTSAMPLE(2, 30, K)
 PRINT TP, K, CR
END
PROGRAM INTSAMPL(L, R, K)               This program takes in arguments L and R, finds
 K = INT(L * COS(R))                    the value of (L * COS (R)), cuts off any decimal
 RETURN                                 places and returns the result as argument K to
END                                     the main program.
```

INV( )

Purpose

This finds the inverse matrix of the TRANS type variable in the argument.

Type

Vector variable calculation function

Format

INV(<TRANS type>)

Examples

A = INV(T1)

B = INV(TRANS(100, 100, 100, 0))

Analysis and advice

This finds the inverse matrix of the TRANS type variable in ( ).

The inverse matrix that is found is returned as TRANS type data.

Sample

```
PROGRAM MAIN
  T1 = TRANS(50, 50, 50, 50)
  T1INV = TRANS(0, 0, 0, 0)
  T1INV = INV(T1)                          Finds the inverse matrix T1INV of T1.
  PRINT "T1INV X = ", T1INV.X, CR
  PRINT "T1INV Y = ", T1INV.Y, CR
  PRINT "T1INV Z = ", T1INV.Z, CR
  PRINT "T1INV C = ", T1INV.C, CR
END
```

IP1STATUS~IP8STATUS

Purpose

This references the communication status in the selected communication channel.

Type

System variable (Other)

Format

IP1STATUS
IP2STATUS
IP3STATUS

Examples

IF IP1STATUS == 0 THEN GOTO ERR

Analysis and advice

This references the communication status in the selected communication channel.

0: non	6: Fin Wait1
1: Closed	7: Fin Wait2
2: Listen	8: Close wait
3: Syn Sent	9: Closing
4: Syn Recived	10: Last ACK
5: Established	11: Time Wait

Sample

```
PROGRAM IPSTATUSSAMPLE
 STRT:
  X1 = 0.0
  IF IP1STATUS <> 5 THEN GOTO ERR      When the IP1 communication status is
  INPUT IP1, X1                        Established, the INPUT command is executed.
  GOTO STRT
 ERR:                                  When it is a status other than Established, it
  PRINT “IP1 NG”, CR                   jumps to ERR, and “IP1 NG” is displayed on the
END                                    teach pendant.
```

IP1~8

Purpose

Specifies the communication channel used for the PRINT and INPUT commands.

Type

System constant

Format

PRINT [{ IP1 | IP2 | … | IP8 | TP } ,] { <character string> | <expression> } [, { <character string> | <expression> }] … [, CR]

INPUT [{ IP1 | IP2 | … | IP8 | TP } ,] <variable> [, <variable>] …

Examples

PRINT IP1, "*** INPUT N ***"

PRINT IP2, N, N*10

INPUT IP2, K

Analysis and advice

These are used to specify the communication channel for the PRINT and INPUT commands.

IP1 to IP8 are for TCP/IP communication of the controller.

If the communication channel is not specified for the PRINT or INPUT commands, data input/output is performed using the dedicated communication channel of the teach pendant.

For details on the communication process, see the sections on the PRINT and INPUT commands.

Sample

```
PROGRAM IPSAMPLE
  N = 0                                  ("DIM N AS INT" is also possible.)
  PRINT IP1, "*** INPUT N ***"
  INPUT TP, N                            The numerical value entered from the teach
  PRINT IP1, N, CR                       pendant is output to IP1.
END
```

IPCLOSE( )

Purpose

Run the close processing for each channel.

Type

I/O (Input/output) control command

Format

IPCLOSE(<integer>)

Examples

IPCLOSE(1)

IPCLOSE(2)

IPCLOSE(3)

Analysis and advice

Run the close processing for each channel.

Use any of the following integer values to specify a relevant channel.

1: TCP/IP1

2: TCP/IP2

3: TCP/IP3

:

8: TCP/IP8

The system constants IP1, IP2, ..., IP8 cannot be used in this command.

To run the IPCLOSE command, ensure that the communication channel status is not CLOSED.

Example) To run IPOPEN(1), ensure that IP1STATUS > 1 (not closed).

* The communication channel status is not CLOSED after running the IPCLOSE command, check the cables for electrical discontinuity.

Sample

```
PROGRAM IPCLOSESAMPLE
 START:
  X1 = 0.0
  IF IP1STATUS > 1 THEN IPCLOSE(1)
  IF IP1STATUS <> 1 THEN GOTO START
  IF IP1STATUS == 1 THEN IPOPEN(1)
  IF IP1STATUS <> 2 AND IP1STATUS <> 3 THEN GOTO START
 INP:
  IF IP1STATUS == 5 THEN INPUT IP1, X1 ELSE GOTO INP
END
```

Set IP1 to CLOSE if the IP1 status is not non or Closed.
After running IPCLOSE(1), move to the START label if the IP1 status is not Closed.
Set IP1 to OPEN if the IP1 status is Closed.
After running IPOPEN(1), move to the START label if the IP1 status is not Listen or Syn_Sent.
Run INPUT IP1 if the IP1 status is Established.
Move to the INP label if the IP1 status is not Established.

* To reference the status of the communication channel, see IP1STATUS, IP2STATUS, ..., and IP8STATUS.

IPOPEN( )

Purpose

Run the open processing for each channel.

Type

I/O (Input/output) control command

Format

IPOPEN(<integer>)

Examples

IPOPEN(1)

IPOPEN(2)

IPOPEN(3)

Analysis and advice

Run the open processing for each channel.

Use any of the following integer values to specify a relevant channel.

1: TCP/IP1

2: TCP/IP2

3: TCP/IP3

:

8: TCP/IP8

The system constants IP1, IP2, ..., IP8 cannot be used in this command.

To run the IPOPEN command, ensure that the communication channel status is CLOSED.

Example) To run IPOPEN(1), ensure that IP1STATUS == 1 (closed).

* The communication channel status is Listen or Syn_Sent after running the IPOPEN command, check the cables for electrical discontinuity.

Sample

```
PROGRAM IPOPENSAMPLE
 START:
  X1 = 0.0
  IF IP1STATUS > 1 THEN IPCLOSE(1)
  IF IP1STATUS <> 1 THEN GOTO START
  IF IP1STATUS == 1 THEN IPOPEN(1)
  IF IP1STATUS <> 2 AND IP1STATUS <> 3 THEN GOTO START
 INP:
  IF IP1STATUS == 5 THEN INPUT IP1, X1 ELSE GOTO INP
END
```

Set IP1 to CLOSE if the IP1 status is not non or Closed.
After running IPCLOSE(1), move to the START label if the IP1 status is not Closed.
Set IP1 to OPEN if the IP1 status is Closed.
After running IPOPEN(1), move to the START label if the IP1 status is not Listen or Syn_Sent.
Run INPUT IP1 if the IP1 status is Established.
Move to the INP label if the IP1 status is not Established.

* To reference the status of the communication channel, see IP1STATUS, IP2STATUS, …, and IP8STATUS.

JOINT( )

Purpose

This generates JOINT type data.

Type

Conversion function

Format

JOINT(<expression>, <expression>, <expression>, <expression>, <expression>)

Examples

MOVE JOINT(90, 90, 150, 0)

MOVE P1 + JOINT(5, 5)

Analysis and advice

This generates JOINT type data. JOINT type data can be used to specify the movement command target position and offset amount.

The <expression> refers to the positions of the 1st to 5th axes in order from the left.

The joint position is specified in millimeter or degree units.

You may use a constant, variable or result of calculation for the <expression> term. However, you may not use vector data. Writing of the <expression> can also be omitted, but the omitted elements are treated as 0.

This command must be used in an expression.

Sample

```
PROGRAM MAIN
  MOVE JOINT(0, 0, 150, 0)
END
```

Moves the 1st, 2nd, and 4th axes to 0° and the 3rd axis to the 150 mm position.

JPSN()

Purpose

This obtains the joint position.

Type

Vector variable calculation function

Format

JPSN(<position data>, <axis>)

Examples

AX2 = JPSN(P1, 2)

Analysis and advice

This obtains the position in world coordinates of the joint for the <axis> number in the <position data>.

Sample

```
PROGRAM MAIN
  MOVE P1
  WAIT MOTION >= 100
  A = JPSN(PSNFBK, 2)        After moving to P1, this shows the 2nd axis joint
  PRINT A, CR                position.
END
```

KILL( )

Purpose

This function determines the multitask operation.

Type

Program control command

Format

KILL(<expression>)

Examples

KILL(TSKID)

Analysis and advice

The KILL command terminates the task which has the task number specified by the calculation result of the expression in brackets ( ).

If the task ID (settled on 1) of main task generated automatically at the start of program or the non-existing task ID is specified, NOP operation is effective. The task terminated by the KILL command starts by the TASK command. When this happens, a new number is assigned to the task number.

You may use a constant, variable or result of calculation for the <expression> term. However, you may not use vector data. The task for which stop is specified is to be deleted from the system at the time when it is executed. In other words, when the system switch is in the DISABLE status, task switching is not performed, and so other tasks cannot be killed. Execute the KILL command while the system switch is in the ENABLE status. This command is invalid during step execution.

Sample

```
GLOBAL
  MAXTASK = 2
  K = 0
END
PROGRAM MAIN
  TID1 = 0
  LOOP:
    IF DIN(1) AND TID1 == 0 THEN                Generates task when input signal 1 is ON.
      TID1 = TASK("SUB1")
    ENDIF
    IF DIN(-1) AND TID1 <> 0 THEN               Erases task when input signal 1 is OFF.
      KILL(TID1)
    ELSE
      GOTO LOOP1
    ENDIF
    TID1 = 0
  LOOP1:
    MOVEA 1, -90
    MOVEA 1, 90
    GOTO LOOP
END
PROGRAM SUB1
  K = K + 1
  PRINT K, CR
END
```

LCASE( )

Purpose

This returns a character string that was converted to lowercase.

Type

Character string function

Format

LCASE(<character string expression>)

Examples

TXT = LCASE(“Hello World! 123”) . . . . . TXT is a character string variable

Analysis and advice

This converts uppercase letters in a character string specified in ( ) to lowercase and returns it.

This affects uppercase letters only.

Sample

```
GLOBAL
  DIM TXT AS STRING(20)
END
PROGRAM MAIN
  TXT = LCASE(“Hello World! 123”)        Converts uppercase letters to lowercase and
  PRINT TP, TXT, CR                      returns “hello world! 123”.
END
```

LEFTY

Purpose

LEFTY is a system constant used to change over the configuration of the robot to a left handed system.

Type

System constant

Format

LEFTY

Examples

CONFIG = LEFTY
MOVE A1 WITH CONFIG = LEFTY

Analysis and advice

LEFTY is used in conjunction with CONFIG in order to set the robot configuration to a left handed system.
As a system constant, LEFTY has the value of 1. If you wanted to, you could use it in your program as a constant having the value 1. However, this is not a good idea since it makes your program unnecessarily complicated.
You cannot substitute into system constants.
For information on robot configuration, refer to the CONFIG command.

Sample

```
PROGRAM LEFTYSMPL
  CONFIG = LEFTY
  MOVE A1
  MOVE A2
END
```

This program will set the robot configuration to a left handed system before moving the robot on its way.

LN( )

Purpose

This function calculates the natural logarithm of a number.

Type

Calculator command (Calculation function)

Format

LN(<expression>)

Examples

K = LN(100)

J1 = 1 - LN(50 - D)

Analysis and advice

The LN command will return the natural logarithm of the number in the brackets ( ).
However, be warned that the result of LN (0) will be returned as 0 (when in fact it is undefined and would be expected to be returned as an error).
You may use a constant, variable or result of calculation for the <expression> term.
However, you may not use vector data.
The LN command must be used in an expression.

Sample

```
PROGRAM MAIN
  K = 0.0                          ("DIM K AS REAL" is also possible.)
  LNSMPLE(3, K)
  PRINT TP, K, CR
END
PROGRAM LNSMPLE(N, K)              This program will take the value of a constant
  K = 10 ^ N                       logarithm given by the argument N, convert this
  K = LN(K)                        value into a natural logarithm, and send the
  RETURN                           result back to the main program as argument K.
END
```

LOADCELL

Purpose

This obtains the read value of a load cell [N].

Type

System variable

Format

LOADCELL

Examples

A = LOADCELL

ON LOADCELL > 10.0 BREAK DO GOTO L1

Analysis and advice

This obtains the value of the current load cell.

If the A/D conversion board is not connected or the parameter is not set, it returns 0.

System version

X8KCS-02 I or later

LOADLIB

Purpose

This function reads a dynamic link library.

Type

Program control command

Format

LOADLIB file name

Examples

LOADLIB ABC.LIB

Analysis and advice

Be sure to declare the LOADLIB command in the GLOBAL area.

The library file name is ********.LIB.

A maximum of five libraries can be read in a program file.

For the dynamic library, see Para. “2.12 Library”.

Sample

```
GLOBAL
  LOADLIB ABC.LIB                    Library build-in declaration
END
PROGRAM MAIN
  INPUT N
  GOTOSAMPL2(N)
END
```

```
PROGRAM GOTOSAMPL2(N)                Example of library file
  GOTO(N) L1, L2, L3
  RETURN
  L1:
    MOVE A1
    RETURN
  L2:
    MOVE A2
    RETURN
  L3:
    MOVE A3
    RETURN
END
```

LOCUS

Purpose

This specifies the path precision of the robot.

Type

System variable (movement condition command)

Format

LOCUS = <expression>

Examples

LOCUS = FINE

LOCUS = 0

Analysis and advice

This is a system variable that specifies the path tracking precision of the robot.

When set to 0, the path tracking precision is COARSE (rough). When set to 1, it is FINE.

When the FINE setting for the path tracking precision is executed by the LOCUS command in a robot program, speed feed forward control is enabled by the servo control unit, and the path tracking precision is improved.

Models with LOCUS = 1 by default have feedforward enabled for all operations. For models with default LOCUS = 0, setting LOCUS = 1 will enable feedforward only in MOVES and MOVEC operations.

The system constant FINE or COARSE can be used to specify the path tracking precision.

The path tracking precision is set to FINE using LOCUS = FINE, and it is set to COARSE using LOCUS = COARSE.

You may use constants, variables or equations for the <expression> term. However, you may not use vector type data.

If the path tracking precision is set to a value less than 0 or more than 1, the setting is assumed to be 0 and 1, respectively.

Sample

```
PROGRAM LOCUSSMPL
 LOCUS = FINE                    Specifies FINE for the path tracking precision.
 MOVE A1
 MOVES A2                        This improves the path tracking precision when
 MOVES A3                        moving to A2 and A3 for the MOVES operation.
 MOVE A4
END
```

LOG10( )

Purpose

This function calculates the common logarithm of a number.

Type

Calculator command (Calculation function)

Format

LOG10(<expression>)

Examples

K = LOG10(100)

J1 = 1 - LOG10(50 - D)

Analysis and advice

The LOG10 command will return the common logarithm of the number in the brackets ( ). (However, be warned that the result of LOG10 (0) will be returned as some undefined number (when in fact it would be expected to be returned as an error).)

You may use a constant, variable or result of calculation for the <expression> term. However, you may not use vector data.

This command must be used in an expression.

Sample

```
PROGRAM MAIN
  K = 0.0                          ("DIM K AS REAL" is also possible.)
  LOG10SAMPLE(3, K)
  PRINT TP, K, CR
END
PROGRAM LOG10SAMPLE(N, K)
  K = EXP(N)                       This program will take the value of a natural
  K = LOG10(K)                     logarithm given by the argument N, convert this
  RETURN                           value into a common logarithm, and send the
END                                result back to the main program as argument K.
```

MAXTASK

Purpose

This function specifies the maximum number of tasks that can be executed at the same time in the program containing the multitask function.

Type

System variable (Other)

Format

MAXTASK

Examples

MAXTASK = 4

Analysis and advice

This variable can be used in the global section only.
Normally, specify the value of “No. of TASK commands + 1” for this variable. The maximum value is eight (8).
If a plural number of tasks are used, only the last value takes effect.
Unless the multitask function is used, this variable need not be used. When this happens, the default value is 1 and the work area of only the main task is maintained.
The work area of the controller is assigned to each task, divided equally by this variable. If a large value is specified, the work area that can be used by one (1) task reduces and a large-sized program cannot be executed.

Sample

```
GLOBAL
  A = 0
  MAXTASK = 2                    As one (1) subtask is used, “2” is set.
END
PROGRAM MAIN
  ID = 0
  ID = TASK(“SUB”)
  LOOP:                          A loop is formed in the GOTO statement to
  IF DIN(1) THEN A = 1 ELSE A = 0  prevent repeated call of the task command.
  GOTO LOOP
END
PROGRAM SUB
  ENABLE NOWAIT
  IF A == 0 THEN PRINT “A”, A, CR
END
```

MEMORYI

Purpose

This saves and references memory variables (INT type).

Type

System variable (Other)

Format

MEMORYI(<number>)

Examples

MEMORYI(1) = 100

A = MEMORYI(3)

Analysis and advice

This is an integer variable where the value is stored even when the power is shut off. Variables that are shared between programs or used after restarting the controller can be saved.

The valid range for the number is 1 to 1024.

This system variable is shared throughout the controller. Even if a different program is selected, the value that was written by the previous program can be referenced.

Sample

```
PROGRAM MAIN
  COUNT = MEMORYI(1)                Takes out the value that was saved.
  MOVE P1
  MOVE P2
  MOVE P3
  COUNT += 1
  MEMORYI(1) = COUNT                The value after execution is saved.
END
```

MEMORYJ

Purpose

This saves and references memory variables (JOINT type).

Type

System variable (Other)

Format

MEMORYJ(<number>)

Examples

MEMORYJ(1) = JOINT(90, 90, 0, 0)

J2 = MEMORYJ(2)

Analysis and advice

This is a JOINT type variable where the value is stored even when the power is shut off. Variables that are shared between programs or used after restarting the controller can be saved.

The valid range for the number is 1 to 128.

This system variable is shared throughout the controller. Even if a different program is selected, the value that was written by the previous program can be referenced.

Sample

```
PROGRAM MAIN
  FOR K = 1 TO 5
    MOVE MEMORYJ(K)
  NEXT K
END
```

Moves to the positions that were saved to numbers 1 to 5 of MEMORYJ in the indicated order.

MEMORYL

Purpose

This saves and references memory (PAYLOAD type).

Type

System variable (Other)

Format

MEMORYL(<number>)

Examples

MEMORYL(1) = PAYLOAD(2, 100)

PAYLOAD = MEMORYL(2)

Analysis and advice

This is a PAYLOAD type variable where the value is stored even when the power is shut off.

Variables that are shared between programs or used after restarting the controller can be saved.

The valid range for the number is 1 to 16.

This system variable is shared throughout the controller. Even if a different program is selected, the value that was written by the previous program can be referenced.

Sample

```
PROGRAM MAIN
  PAYLOAD = MEMORYL(1)                    This sets PAYLOAD to the stored value.
  MOVE P1
END
```

MEMORYP

Purpose

This saves and references memory variables (POINT type).

Type

System variable (Other)

Format

MEMORYP(<number>)

Examples

MEMORYP(1) = POINT(100, 50, 20)

P2 = MEMORYP(2)

MOVE MEMORYP(3)

Analysis and advice

This is a POINT type variable where the value is stored even when the power is shut off.

Variables that are shared between programs or used after restarting the controller can be saved.

The valid range for the number is 1 to 1024.

This system variable is shared throughout the controller. Even if a different program is selected, the value that was written by the previous program can be referenced.

Sample

```
PROGRAM MAIN
  FOR K = 1 TO 10
    MOVE MEMORYP(K)
    DELAY 0.5
  NEXT K
END
```

Moves to the positions that were saved to numbers 1 to 10 of MEMORYP in the indicated order.

MEMORYR

Purpose

This saves and references to memory variables (REAL type).

Type

System variable (Other)

Format

MEMORYR(<number>)

Examples

MEMORYR(1) = 3.5

B = MEMORYR(2)

Analysis and advice

This is a real number variable where the value is stored even when the power is shut off.

Variables that are shared between programs or used after restarting the controller can be saved.

The valid range for the number is 1 to 1024.

This system variable is shared throughout the controller. Even if a different program is selected, the value that was written by the previous program can be referenced.

Sample

```
PROGRAM MAIN
  MOVE P1 + POINT(0, 0, MEMORYR(1))
END
```

Moves to the position that was offset in the Z-axis direction by the amount for the value that was saved.

MEMORYT

Purpose

This saves and references memory variables (TRANS type).

Type

System variable (Other)

Format

MEMORYT(<number>)

Examples

MEMORYT(1) = TRANS(100, 0, 0)

T2 = MEMORYT(2)

MOVE P3 WITH TOOL = MEMORYT(3)

Analysis and advice

This is a TRANS type variable where the value is stored even when the power is shut off.

Variables that are shared between programs or used after restarting the controller can be saved.

The valid range for the number is 1 to 128.

This system variable is shared throughout the controller. Even if a different program is selected, the value that was written by the previous program can be referenced.

Sample

```
PROGRAM MAIN
 TOOL = MEMORYT(1)                    This sets TOOL to the stored value.
 MOVE P1
END
```

MINUSLIM

Purpose

This references the software limit in the negative direction.

Type

System variable (movement reference command)

Format

MINUSLIM

Examples

MOVEA 2, MINUSLIM.2

Analysis and advice

This is the JOINT type variable indicating the lower limit value of the robot movement range.

Sample

```
PROGRAM MAIN
  MOVEA 3, MINUSLIM.3                    Moves the 3rd axis to the lower limit position.
END
```

MOD

Purpose

The MOD function returns the remainder of a division operation.

Type

Calculator command (operand)

Format

<expression> MOD <expression>

Examples

N = K MOD 3

J = K + (L MOD M)

Analysis and advice

The MOD function will take the <expression> on the left, divide it by the <expression> on the right, and send back the remainder.

You may use constants, variables or equations for the <expression>. However, you may not use vector type data.

The MOD command must be used in an expression.

Sample

```
PROGRAM MAIN
 K = 0.0                          ("DIM K AS REAL" is also possible.)
 MODSAMPLE(5.0, 3.0, K)
 PRINT TP, K, CR
END
PROGRAM MODSAMPLE(N1, N2, K)
 K = N1 MOD N2                    This program takes argument N1, divides it by
 RETURN                           argument N2 and sends the result back to the
END                               main program as argument K.
```

MODE

Purpose

MODE is used to refer to the system operating mode

Type

System variable (movement reference command)

Format

MODE

Examples

IF MODE <> CONT THEN STOP

Analysis and advice

MODE is used to refer to the system operating mode.

Should the value of MODE be 0, the system is in the continuous operation mode. If 1, the system is in the cycle operation mode, and if 2, the system is in the segment operation mode.

When referring to the system operating mode, you may use the system constants CONT, CYCLE and SEGMENT. As you would expect, MODE = = CONT puts the system in the continuous operation mode, MODE = = CYCLE puts the system in the cycle operation mode, and MODE = = SEGMENT puts the system in the segment operation mode.

The monitor command MODE MOTION can be used to specify segment operation.

Sample

```
PROGRAM MODESAMPLE
  MOVE A1
  MOVE A2
  IF MODE <> CONT THEN STOP        This program will stop executing itself should
  MOVE A3                          the system change out of the continuous
END                                operation mode.
```

MOTION

Purpose

The MOTION statement is used to refer to the amount of a motion that has been completed.

Type

System variable (movement reference command)

Format

MOTION

Examples

K = MOTION
ON MOTION >= 50 DO DOUT(1)

Analysis and advice

The MOTION statement can be used to see what percentage of a robot motion has been completed.
The “amount of motion” is defined as the percentage of a motion completed by the robot with respect to the total distance to be covered by that motion. Calculations for the amount of motion are carried out for the axis that has the greatest distance to travel.
The amount of motion is returned as a real number.
By combining the MOTION statement with an ON command, the robot can be made to send out signals while a motion is still in progress. This statement must be used in an expression.

The amount of motion referenced with this command is the position commanded to the robot. Note that while the robot is moving, the current position of the robot has a delay from the command position. Be careful because == can't be used for the comparative operator.
With ENABLE PASS when using, be careful because an infinite loop is worked depending on how to use.
Because the path orbit formation wait to P2 has occurred from P1, the following example becomes an infinite loop. By replacing with the WAIT sentence, it is possible to avoid an infinite loop.

```
ENABLE PASS                          ENABLE PASS
PASS = 50                      PASS = 50
MOVE P1                        →     MOVE P1
LOOP1:                               WAIT MOTION >= 95
IF MOTION < 95 THEN GOTO LOOP1       MOVE P2
MOVE P2
```

Sample

```
PROGRAM MOTIONSMPL
  ENABLE NOWAIT
  ON MOTION >= 50 DO DOUT(1)          When the robot hand is 50% or more of the way
  MOVE A1                             to point A1, Signal 1 will be output.
  ON MOTION >= 80 DO DOUT(2)          When the robot hand is 80% or more of the way
  MOVE A2                             to point A2, Signal 2 will be output.
END
```

MOTIONT

Purpose

The MOTIONT statement is used to refer to the amount of time passed since a motion has begun.

Type

System variable (movement reference command)

Format

MOTIONT

Examples

K = MOTIONT
ON MOTIONT >= 5 DO DOUT(1)

Analysis and advice

The MOTIONT statement can be used to see how much time has passed since a certain motion started.
Execution time is given as a real number in units of seconds. The execution time will change to 0 when the robot has completed final positioning for that movement.
The MOTIONT command must be used in an expression.
By combining the MOTIONT statement with an ON command, the robot can be made to send out signals while a motion is still in progress. When this statement monitors travel time per one movement of the robot over the specified time, it can handle the error.
Be careful because == can't be used for the comparative operator.

With ENABLE PASS when using, be careful because an infinite loop is worked depending on how to use.
Because the path orbit formation wait to P2 has occurred from P1, the following example becomes an infinite loop. By replacing with the WAIT sentence, it is possible to avoid an infinite loop.

```
ENABLE PASS                              ENABLE PASS
PASS = 50                          PASS = 50
MOVE P1                        →         MOVE P1
LOOP1:                                   WAIT MOTIONT >= 5
IF MOTIONT < 5 THEN GOTO LOOP1           MOVE P2
MOVE P2
```

Sample

```
PROGRAM MOTIONTSMPL
  ENABLE NOWAIT
  ON MOTIONT >= 10 DO DOUT(1)        Should the robot take 10 seconds or more to
  MOVE A1                            complete a motion to A1, Signal 1 will be output
  MOVE A2                            immediately.
END
```

MOVE

Purpose

The MOVE command moves the robot to a specified position.

Type

Movement control command

Format

MOVE <position> [WITH clause]

Examples

MOVE A1

MOVE A1 WITH SPEED = 50

Analysis and advice

The MOVE command moves the robot to the specified position in synchronous motion.

All the robot joints will start and stop moving at the same time. The controller will adjust the speeds of the joints relative to the slowest joint for that motion accordingly. This is called synchronous motion (or sometimes joint angle interpolation).

POINT type data or JOINT type data can be used in <position>.

Also, the coordinate value can be specified directly as POINT type data and JOINT type data as shown below.

Note that TRANS type data and PAYLOAD type data cannot be used in <position>.

MOVE POINT(X, Y, Z, C, T, <configuration>)

MOVE JOINT(J1, J2, J3, J4, J5)

MOVE {X, Y, Z, C, T} WITH CONFIG = <configuration>

(Although { } can also be used to write as shown above, use of the POINT command is recommended for explicitly indicating the data type.)

X, Y, Z, C, T :	Coordinate values X, Y, Z, C and T are specified with real numbers (in units of millimeters or degrees).
<configuration>:	The configuration of the robot is specified by an integer value of 0, 1 or 2. (0 = undefined (FREE); 1 = left-handed system; 2 = right-handed system)
J1 to J5:	The value of each joint is specified as a real number. (in units of millimeters or degrees).

You may use a constant, a variable or a calculation for each individual element. However, you may not use vector type data for an element. Anything less than 0 which is entered as the <configuration> will be treated as 0, and anything greater than 2 will be treated as 2.
Individual data elements may be omitted, but these omitted elements will all be treated as 0. For example, the two statements below mean the same thing:

Example) MOVE POINT(100, 100, 0, 0, 0, RIGHTY)
MOVE POINT(100, 100)
(Everything from Z to <configuration> will be taken as 0.)

When entering positional data from the teaching pendant, work coordinate system data specified at the time of teaching will also be recorded. When a movement command is executed, the work coordinate system will change over to that specified at the time the positional data was taught. Note, however, that base and tool coordinates will stay as they were before the command was executed.
When directly specifying the coordinate values of <position> (sometimes along with creating or manipulating positional data with commands such as DEST, HERE and POINT), movements are performed with the work coordinate system in effect before the command was executed.
The controller will figure out movement conditions such as speed and acceleration using the system variable values in effect at the time. Should you wish to change a movement condition for one operation, use a use the WITH clause to specify that condition. Refer to the WITH command for how to use the WITH clause.

Sample

```
PROGRAM MOVESAMPLE
 MOVE A1                    This program will move the robot to point A1
                            with synchronous motion.
 MOVE A2                    This program will move the robot to point A2
END                         with synchronous motion.
```

MOVEA

Purpose

MOVEA moves a specified robot joint to a specified position.

Type

Movement control command

Format

MOVEA <axis>, <absolute position> [, <axis>, <absolute position> [, . . .]] [WITH clause]

Examples

MOVEA 1, 60

MOVEA 1, 90, 2, 90

MOVEA 3, 0 WITH GAIN = { , , ON}

Analysis and advice

The MOVEA command moves a specified robot joint to a specified position. Such movement is called “absolute single axis motion.”

The <axis> designation contains an integer from 1 to 5 and specifies the robot joint to be moved. All other axes besides that specified will not move.

The <absolute position> designation specifies the destination of that movement relative to the origin of that axis. For rotary joints, <absolute position> is in terms of degrees. For linear (direct drive) joints, <absolute position> is in terms of millimeters. Should you specify an <absolute position> outside of the range of that joint, the robot will move to the position just before the end of that limit.

Constants, variables or calculations may be used for the <axis> and <absolute position> designations. However, you may not use vector type data. Should you use anything other than 1 to 5 for the <axis> specification, or should you designate an axis which your robot does not have, the robot does not move.

Multiple combinations of <axis> and <absolute position> can be written to perform simultaneous movement of two or more axes.

The controller will figure out movement conditions such as speed and acceleration with the system variable values in effect at the time. Should you wish to change a movement condition for one operation, use the WITH clause to specify that condition. Refer to the WITH command for how to use the WITH clause.

Sample

```
PROGRAM MOVEASAMPL
  MOVEA 1, 0          This program will move Axis 1 to its 0 degree
                      position.
  MOVEA 2, 90         This program will move Axis 2 to its 90 degree
END                   position.
```

MOVEC

Purpose

MOVEC moves the robot hand to a specified position through a specified passing position in circular interpolation.

Type

Movement control command

Format

MOVEC <passing position> <position> [, <angle>] [WITH clause]
MOVEC <passing position>, <position> [, <angle>] [WITH clause]

Examples

MOVEC A1 A2
MOVEC A1, A2 WITH SPEED = 10
MOVEC A1, A2, 360

Analysis and advice

The tip of the robot hand is moved in a circular path connected among the current position, <passing position> and <position>. The tip of the hand is moved in the direction from the current position to <position> at a constant angular velocity.
POINT type data or JOINT type data are specified in <passing position> and <position>. Like the MOVE and MOVES commands, with the POINT command, a coordinate value can be directly specified.
Note that TRANS type data and PAYLOAD type data cannot be used in <passing position> and <position>.
In this case, <angle> can be specified to perform movement that passes through the arc path of the specified angle, not the angle up to <position>. Use this for tracing a circle path, for instance.
•WORK coordinate system
In the work coordinate system, the tip of the robot hand is moved as taught for both <passing position> and <position>. When the work coordinate system is specified with the WITH clause, the tip of the hand will be moved work coordinate system. When a coordinate value is directly specified for <passing position> and <position>, the tip of the hand will be moved in the work coordinate system when the command is executed.
•WITH clause
When the robot is moved, the moving conditions such as speed and acceleration will be determined depending on the setting values of the system variables at the time. To change the moving conditions for one motion, use the WITH clause.
Refer to the WITH command for how to use the WITH clause.

• Limitation of restoration after canceling circular interpolation

After a motion of circular interpolation is cancelled by “ON ~ BREAK ~ DO ~,” feed hold, emergency stop, or a trouble, when the execution is resumed without resetting the program (when a motion is cancelled by “ON ~ BREAK ~ DO ~” and resumed with the RESUME command), the cancelled command is re-executed. At that time, the motion of circular interpolation works as linear motion of interpolation to <position>.

• Limitation of position relationship of three points

When three or two positions of three points forming an arc (present position, <passing position>, and <position>) are the same or very close, the tip of the robot hand may be moved along an arc which differs from that expected. When a motion of circular interpolation is used during a short-cut motion, the path of the robot should be connected to the tangent of the circular. When the angle becomes sharp, at a joint of circular interpolation of the short-cut motion, an abrupt acceleration may be applied to the robot.

• Tool offset

When the hand is moved linearly or in circular interpolation with a tool being offset while changing the orientation of the tool, unless the tool offset is properly set, the specified motion may not be obtained. The tool offset is used in teaching positions. Before teaching the positions, it is necessary to check that the tool offset is correctly set (For details on how to select the tool coordinate system, see the "TP5000 Operation Manual "Setting the tool coordinates"".) If using a tool with offset in this way, generally, be sure to first set the tool coordinate system, and then teach all the positions. In addition, at the beginning of the robot language program, with the TOOL command, securely specify the correct tool offset.

```
Example)  PROGRAM MAIN
            TOOL = TOOL1      Use “TOOL1” as tool offset after that.
            …
            END
```

• About the 5th axis addition (option)

When adding (option) the 5th axis, be careful because it isn't possible for an arc interpolation to be worked about the 5th axis. Among the other axes, an arc interpolation is worked.

Sample

```
PROGRAM MOVECSAMPL
 MOVE A1
 MOVEC A2, A3                    Moves the hand from A1 to A2 in arc
END                              interpolation.
```

MOVECV

Purpose

This moves the robot with spline interpolation through specified points in a positional array.

Type

Motion control command

Format

MOVECV <name of positional array> [WITH clause]

Examples

MOVECV P

MOVECV P WITH SPEED = 10

Analysis and advice

The tip of the robot hand moves through a spline curve arc trajectory that passes through the current position and each point in <name of positional array>. The maximum number of elements that can be specified in a positional array is 256.

If there is no robot at the position of the first point in the positional array at the start of the motion, the tip moves through a spline curve including the current position. If the start position of the motion and the last position in the positional array are the same, the tip of the robot hand moves so that the trajectory draws a closed spline.

Basically, ensure that the orientation of C-axis does not change. If the orientation of C-axis changes drastically, an error may occur.

• WITH clause

When a robot moves, the operating conditions such as velocity and acceleration are determined by the system variable set at that moment. To change the operating conditions for only one operation, use the WITH clause and specify the operating conditions. Refer to the WITH command for how to use the WITH clause.

Sample

```
GLOBAL
   DIM P(10) AS POINT
END

PROGRAM MAIN
  MOVE P(1)
  MOVECV P                    This makes a motion through a spline curve
END                           trajectory that passes through each point of P
                              (10).
```

System version

X8KCS-02C or later

MOVEI

Purpose

MOVEI moves a specified robot joint by a specified amount from its present position.

Type

Movement control command

Format

MOVEI <axis>, <relative position> [, <axis> , <relative position> [, . . .]] [WITH clause]

Examples

MOVEI 1, 60

MOVEI 1, 5, 2, 10

MOVEI 3, 10 WITH SPEED = 50

Analysis and advice

The MOVEI command moves a specified robot joint by a specified amount from its position at the time. Such movement is called “relative single axis motion.”

The <axis> designation contains an integer from 1 to 5 and specifies the robot joint to be moved. All other axes besides that specified will not move.

The <relative position> designation specifies the amount of that movement relative to the position of that joint at the time. For rotary joints, <relative position> is in terms of degrees. For linear (direct drive) joints, <relative position> is in terms of millimeters. Should you specify an <relative position> outside of the range of that joint, the robot will move to the position just before the end of that limit.

Constants, variables or calculations may be used for the <axis> and <relative position> designations. However, you may not use vector type data.

Should you use anything other than 1 to 5 for the <axis> specification, or should you designate an axis which your robot does not have, the robot does not move.

Multiple combinations of <axis> and <absolute position> can be written to perform simultaneous movement of two or more axes.

The controller will figure out movement conditions such as speed and acceleration with the system variable values in effect at the time. Should you wish to change a movement condition for one operation, use the WITH clause to specify that condition. Refer to the WITH command for how to use the WITH clause.

Sample

```
PROGRAM MOVEISAMPL
 MOVEI 1, 30          This program will move Axis 1 to a position 30
                      degrees from its current position.
 MOVEI 2, 30          This program will move Axis 2 to a position 30
                      degrees from its current position.
 MOVEI 3, 30          This program will move Axis 3 to a position 30
                      degrees from its current position.
 MOVEI 4, 30          This program will move Axis 4 to a position 30
                      degrees from its current position.
END
```

MOVEJ

Purpose

Moves the robot to the specified position along an arch.

Type

Movement control command

Format

MOVEJ <position> <definition of arch> [WITH clause]

MOVEJ <position>, <definition of arch> [WITH clause]

Examples

MOVEJ A1 {50.0, 20.0, 30.0}

MOVEJ A1, AC WITH SPEED = 30

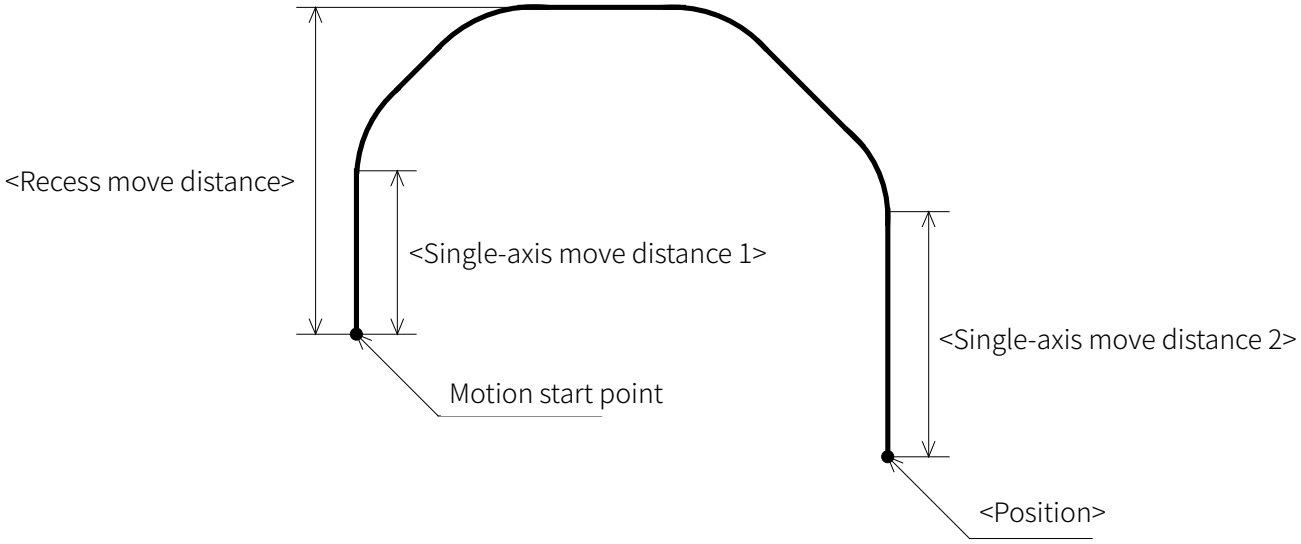
Analysis and advice

This function is used to move the robot to the specified position along an arch. Like the MOVE command, the robot moves by PTP (point-to-point) control. That is, at horizontal movement in the arch motion, the robot will not move along a straight line.

<Position> specifies the final target position in the MOVEJ motion. POINT type data or JOINT type data can be used in <position>.

<definition of arch> defines the profile of the arch motion by value. POINT type data can be used in <definition of arch>. Up to five elements are valid in <definition of arch>. (Values of elements 6 are ignored.) Elements 5 and 6 can be omitted.

<definition of arch> = {<recess move distance>, <Single-axis move distance 1>, <Single-axis move distance 2>}
[, <Upward movement speed>, <Downward movement speed>]}



The TRANS type data and PAYLOAD type data cannot be used for <position> and <definition of arch>.

<recess move distance> signifies the distance from the motion start point to the highest position in the Z-axis direction in units of “mm”. The value of <recess move distance> should be a real number larger than 0.0. If a negative value is specified for <recess move distance>, an error is generated.

<Single-axis move distance 1> designates the move distance of axis 3 only in the up direction in units of “mm”. The value of <Single-axis move distance 1> should be a real number larger than 0.0. If a negative value is specified for <Single-axis move distance 1>, an error is generated. If <Single-axis move distance 1> is larger than the move distance in the up direction, the system interprets that the move distance in the up direction is designated.

<Single-axis move distance 2> designates the move distance of axis 3 only in the down direction in units of “mm”. The value of <Single-axis move distance 2> should be a real number larger than 0.0. If a negative value is specified for <Single-axis move distance 2>, an error is generated. If <Single-axis move distance 2> is larger than the move distance in the down direction, the system interprets that the move distance in the down direction is designated.

<Upward movement speed> specifies the upward movement speed of axis 3 in mm/s units. <Downward movement speed> specifies the downward movement speed of axis 3 in mm/s units. <Upward movement speed> and <Downward movement speed> can be omitted. If they are omitted or set to a value less than or equal to 0, the movement speed is determined based on the set value of SPEED.

<Upward movement speed> and <Downward movement speed> can be set only in system version X8KCS-04C or higher.

It is also possible to directly designate the values for <position> and <definition of arch> in the following manner.

MOVEJ POINT(X, Y, Z, C, T, <configuration>) POINT(Z1, Z2, Z3)

MOVEJ {X, Y, Z, C, T} {Z1, Z2, Z3} WITH CONFIG = <configuration>

(Although { } can also be used to write as shown above, use of the POINT command is recommended for explicitly indicating the data type.)

X, Y, Z, C, T:	Specify a real number for each coordinate of X, Y, Z, C, T. (Unit: mm, degree)
<Configuration>:	Specify the robot configuration by an integer of 0 ~ 2. (0: Undefined, 1: Left-handed system, 2: Right-handed system)
Z1, Z2, Z3:	Specify a real number for defining the arch profile. (Unit: mm)

The MOVEJ command composes the movements in the up, horizontal and down directions, and will not draw an arc. Also, to give priority to the highest position in

the Z-axis direction, the movements in the up and down directions are not composed. If the move distance in the up or down direction is smaller than the movement in the horizontal direction, the single-axis move distance may be larger than the specified value.

If the configuration at the motion start point of the MOVEJ command differs from the target configuration, the configuration changes in the horizontal movement.

The value of already moved distance shown by the MOTION command at the execution of the MOVEJ command is the percentage of the lapse of time to the total movement time of the MOVEJ command. Likewise, the value of distance to go shown by the REMAIN command is the percentage of the remaining time to the total movement time of the MOVEJ command.

When the MOVEJ command has been interrupted by BREAK, the distance to go in the up direction, distance to go in the down direction, distance to go of single-axis in each direction and target position are maintained. To resume the operation, the MOVEJ command is created again based on these data. Therefore, if the robot has been moved by manual guide, etc. during interruption by BREAK, the midway pass cannot be assured.

The MOVEJ command cannot allow a short-cut movement in the interval with other motion command (including the MOVEJ command).

Sample

```
PROGRAM MOVEJSAMPL
  P1 = POINT(300.0, 350.0, 0.0, 0.0, 0.0)
  P2 = POINT(300.0, -350.0, 0.0, 0.0, 0.0)
  ARCH = POINT(100.0, 40.0, 50.0)
  MOVE P1
  MOVEJ P2, ARCH
  MOVEJ P1, {100.0, 100.0, 100.0, 50.0, 20.0}     Specifies an upward movement speed of
END                                               50 mm/s and a downward movement speed of
                                                  20 mm/s.
```

MOVES

Purpose

The MOVES command moves the robot by linear interpolation to a specified position.

Type

Movement control command

Format

MOVES <position> [WITH clause]

Examples

MOVES A1

MOVES A1 WITH GAIN = { , , ON}

Analysis and advice

The MOVES command moves the robot to the specified position along a path determined by linear interpolation.

This command will cause the robot hand to move from its current position to a specified position along a straight line which ties the two together. This kind of movement is called linear interpolated motion. Under such motion, the linear speed of the robot hand will remain constant (except during acceleration and deceleration).

POINT type data or JOINT type data can be used in <position>.

Also, the coordinate value can be specified directly as POINT type data as shown below.

You cannot use the TRANS type data or PAYLOAD type data for <position>.

MOVES POINT(X, Y, Z, C, T, <configuration>)

MOVES JOINT(J1, J2, J3, J4, J5)

MOVES {X, Y, Z, C, T} WITH CONFIG = <configuration>

(Although { } can also be used to write as shown above, use of the POINT command is recommended for explicitly indicating the data type.)

X, Y, Z, C, T :	Coordinate values X, Y, Z, C and T are specified with real numbers (in units of millimeters or degrees).
<configuration>:	The configuration of the robot is specified by an integer value of 0, 1 or 2. (0 = undefined (FREE); 1 = left-handed system; 2 = right-handed system)
J1 to J5:	The value of each joint is specified as a real number. (in units of millimeters or degrees).

You may use a constant, a variable or a calculation for each individual element. However, you may not use vector type data for an element. Anything less than 0 which is entered as the <configuration> will be treated as 0, and anything greater than 2 will be treated as 2.
Individual data elements may be omitted, but these omitted elements will all be treated as 0. For example, the two statements below mean the same thing:
Example) MOVES POINT(100, 100, 0, 0, 0, 0)
MOVES POINT(100, 100)
(Everything from Z to <configuration> will be taken as 0.)

When entering positional data from the teach pendant, data for the work coordinate system in effect at the time will also be recorded. When a movement command is executed, the current work coordinate system will change over to that in effect at the time the positional data was taught. Note, however, that base and tool coordinates will stay as they were before the command was executed.
When directly specifying the <position> values, (sometimes along with creating or manipulating positional data with commands such as DEST, HERE and POINT), movements are performed with the work coordinate system in effect before the command was executed.
The controller will figure out movement conditions such as speed and acceleration with the system variable values in effect at the time. Should you wish to change a movement condition for one operation, use the WITH clause to specify that condition. Refer to the WITH command for how to use the WITH clause.
•About the 5th axis addition (option)
When adding (option) the 5th axis, be careful because it isn't possible for a straight line interpolation to be worked about the 5th axis. Among the other axes, a straight line interpolation is worked.

Sample

```
PROGRAM MOVESSAMPL
  MOVE A1
  MOVES A2                    This program will move the robot to point A2
END                           with linear motion.
```

MOVESYNC

Purpose

Specifies the motion command synchronous mode or motion command asynchronous mode.

Type

System switch

Format

MOVESYNC

Examples

DISABLE MOVESYNC

Analysis and advice

Assume a motion during ENABLE NOWAIT in the SCOL2 program where multiple motion commands and signal input/output commands line up alternately. In the ENABLE MOVESYNC status (motion command synchronous mode), the system executes up to just before the next motion command and waits for the completion of positioning. Therefore, the second signal input/output is executed immediately after the second motion command starts and the third signal input/output command immediately after the third motion command starts. In this mode, however, the system does not get into the state of the system variable PASS, and short-cut motion cannot be executed. In the DISABLE MOVESYNC state (motion command asynchronous mode), the system pre-executes up to just before the maximum four motion commands ahead and waits for the completion of positioning. Therefore, the second and subsequent signal input/output commands may be executed during the first motion By enabling the system variable PASS, pass motion becomes possible. The value of this system variable when the SCOL2 program is actuated, is set by the user parameter [U03].

Sample

```
PROGRAM MAIN
  ENABLE NOWAIT
  ENABLE MOVESYNC                 Sets to movement command synchronization
                                  mode.
    MOVEA 1, 90                   Output signal 1 is turned on during movement
    DOUT(1)                       of 1st axis to +90° position.
    MOVEA 1, -90                  Output signal 2 is turned on during movement
    DOUT(2)                       of 1st axis to -90° position.
  DISABLE MOVESYNC                Sets to movement command asynchronization
                                  mode.
    MOVEA 2, 90                   Output signal 3 and output signal 4 are turned
    DOUT(3)                       on during movement of 2nd axis to +90°
    MOVEA 2, -90                  position.  (Output signal 4 is executed first, not
    DOUT(4)                       during movement of 2nd axis to -90° position.)
END
```

MOVET

Purpose

This moves the robot to the specified tool direction.

Type

Movement control command

Format

MOVET <coordinate axis>, <relative position> [, <coordinate axis>, <relative position> [. . .]] [WITH clause]

Examples

MOVET 1, 100
MOVET 1, 50, 2, 50
MOVET 2, 50 WITH SPEED = 50

Analysis and advice

This moves the robot to the specified tool direction. The tool direction is the direction in the Cartesian coordinate system where the C-axis direction for the SCARA robot is the X-axis.

The value for <coordinate axis> is an integer value from 1 to 5, and each value corresponds to the following axes.

1: X-axis, 2: Y-axis, 3: Z-axis, 4: C-axis, 5: T-axis (when using additional axis only)

A combination of <coordinate axis> and <relative position> can be repeatedly written to perform simultaneous movement in multiple axial directions.

When a robot performs movement, the speed, acceleration, and other movement conditions are determined based on the setting values of the system variables at that time. To change the movement conditions for a single movement only, use the WITH clause to specify the movement conditions. For details on how to use the WITH clause, see the description on the WITH command.

Sample

```
PROGRAM MAIN
  MOVE POINT(300, -50, 100, 90, 0, RIGHTY)      Moves to the specified point.
  MOVET 2, 50                                   Moves 50 mm in the tool Y-axis direction.
  PRINT HERE, CR                                → 250.000, -50.000, 100.000, 90.000, 0.000, 2
END
```

MOVEW

Purpose

This moves the robot to a specified world coordinates position.

Type

Movement control command

Format

MOVEW <coordinate axis>, <absolute position> [, <coordinate axis> , <absolute position> [. . .]] [WITH clause]

Examples

MOVEW 1, 100

MOVEW 1, 50, 2, 50

MOVEW 2, 50 WITH SPEED = 50

Analysis and advice

This moves the robot to a specified world coordinates position. If the system work coordinates are set, movement is performed in that coordinate system.

The value for <coordinate axis> is an integer value from 1 to 5, and each value corresponds to the following axes.

1: X-axis, 2: Y-axis, 3: Z-axis, 4: C-axis, 5: T-axis (when using additional axis only)

A combination of <coordinate axis> and <absolute position> can be repeatedly written to perform simultaneous movement in multiple axial directions.

When a robot performs movement, the speed, acceleration, and other movement conditions are determined based on the setting values of the system variables at that time. To change the movement conditions for a single movement only, use the WITH clause to specify the movement conditions. For details on how to use the WITH clause, see the description on the WITH command.

Sample

```
PROGRAM MAIN
  MOVE POINT(300, -50, 100, 90, 0, RIGHTY)      Moves to the specified point.
  MOVEW 2, 50                                   Moves to the Y-axis 50 mm position.
  PRINT HERE, CR                                → 300.000, 50.000, 100.000, 90.000, 0.000, 2
END
```

MOVEWI

Purpose

This moves the robot in a specified world coordinates direction.

Type

Movement control command

Format

MOVEWI <coordinate axis>, <relative position> [, <coordinate axis>, <relative position> [. . .]] [WITH clause]

Examples

MOVEWI 1, 100

MOVEWI 1, 50, 2, 50

MOVEWI 2, 50 WITH SPEED = 50

Analysis and advice

This moves the robot in a specified world coordinates direction. If the system work coordinates are set, movement is performed in that coordinate system.

The value for <coordinate axis> is an integer value from 1 to 5, and each value corresponds to the following axes.

1: X-axis, 2: Y-axis, 3: Z-axis, 4: C-axis, 5: T-axis (when using additional axis only)

A combination of <coordinate axis> and <relative position> can be repeatedly written to perform simultaneous movement in multiple axial directions.

When a robot performs movement, the speed, acceleration, and other movement conditions are determined based on the setting values of the system variables at that time. To change the movement conditions for a single movement only, use the WITH clause to specify the movement conditions. For details on how to use the WITH clause, see the description on the WITH command.

Sample

```
PROGRAM MAIN
  MOVE POINT(300, -50, 100, 90, 0, RIGHTY)     Moves to the specified point.
  MOVEWI 2, 50                                 Moves 50 mm in the Y-axis direction.
  PRINT HERE, CR                               → 300.000, 0.000, 100.000, 90.000, 0.000, 2
END
```

NOT

Purpose

NOT reverses the judgment of a logical expression.

Type

Calculator command (operand)

Format

NOT <logical expression>

Examples

IF NOT DIN(1) THEN STOP

Analysis and advice

NOT reverses the judgment of a logical expression.

The NOT statement must be used in a logical expression.

Sample

```
PROGRAM NOTSAMPLE
 IF NOT DIN(1) THEN DOUT(1)                    If input signal 1 is OFF, output signal 1 is ON.
END
```

NOWAIT

Purpose

NOWAIT is a system switch which directs the controller to continue processing I/O signals without waiting for the robot to finish positioning itself.

Type

System switch

Format

NOWAIT

Examples

DISABLE NOWAIT
ENABLE NOWAIT

Analysis and advice

NOWAIT is a system constant used to tell the controller not to wait for the robot to finish positioning itself before processing I/O (input/output) signals.
Signal output timing is described in detail in Section 5.
The ENABLE and DISABLE commands are used to turn system switches (such as NOWAIT) on and off.
ENABLE NOWAIT tells the controller not to wait for the robot to finish positioning itself before sending out (or taking in) signals. DISABLE NOWAIT tells the controller to wait for the robot to finish positioning itself before sending out (or taking in) signals.
The initial setting for the system is DISABLE NOWAIT.

Sample

```
PROGRAM NOWAITSMPL
  ENABLE NOWAIT
  MOVE A1
  DOUT(1)
  MOVE A2
  DOUT(2)
  MOVE A3
END
```

Here, the controller will send out external signals without waiting for the robot to finish positioning itself.

OFF

Purpose

OFF is a system constant used to specify axes for which the gain (servo control) is to be OFF.

Type

System constant

Format

OFF

Examples

GAIN = {OFF, OFF, ON, OFF, OFF}
MOVE A1 WITH GAIN = { , , OFF}

Analysis and advice

OFF is a system constant used in conjunction with GAIN in order to specify the gain (servo control) of a specific axis as off.
Should the GAIN be specified as OFF, servo control for that axis will stop the next time a movement command is executed. Axes for which servo control has been stopped are in the “servo free state” (in which positioning control is not carried out).
As a system constant, OFF has a value of 0. If you wanted to, you could use it in your program as a constant having the value 0. However, this is not unnecessarily hard to understand.
You cannot substitute into system constants.
For information on gains, refer to the GAIN command.

Sample

```
PROGRAM OFFSAMPLE
  MOVE A1
  WAIT MOTION >= 100
  GAIN = {OFF, OFF, ON, OFF, OFF}
  MOVE A2
  OPEN1
  DELAY 0.5
  MOVE A1
  WAIT MOTION >= 100
  GAIN = {ON, ON, ON, ON, ON}
  READY
END
```

This program turns off all gains except that for the Z-axis (Axis 3) before the robot moves to point A2.

OFFGAIN( )

Purpose

Turns off the gain of each axis (servo control).

Type

Library function (SCOL.LIB)

Format

OFFGAIN(<integer>, <integer>, <integer>, <integer>, <integer>)

Examples

OFFGAIN(1, 1, 0, 1, 1)

Analysis and advice

The gain of each axis (servo control) of the robot is turned off.
For details of the gain, see the descriptions of the GAIN command.
To turn on the gain, use the ONGAIN command. For each joint of axes 1 to 5, specify the gains by delimiting the values of the five axes with a comma. The values are specified with 1 or 0. When 1 is specified, the gain of the axis is turned off. When 0 is specified, the state of the axis remains unchanged.
The on/off state of the gain is changed after the current motion is completed.
This command can be executed only when file SCOL.LIB is present in the ROM drive of the controller.
In this command, the system constants ON and OFF cannot be used.

Sample

```
PROGRAM OFFGAINSMPL
  MOVE A1                          After the robot is moved to A1, except for the
  OFFGAIN(1, 1, 0, 1, 1)           axis 3, the gains are turned off.
  MOVE A2                          After the robot is moved to A2, the gains of all
  ONGAIN(1, 1, 1, 1, 1)            the axes are turned on.
  MOVE A3
END
```

ON

Purpose

ON is a system constant used to specify axes for which the gain (servo control) is to be ON.

Type

System constant

Format

ON

Examples

GAIN = {OFF, OFF, ON, OFF, OFF}
MOVE A1 WITH GAIN = { , , ON}

Analysis and advice

ON is a system constant used in conjunction with GAIN in order to specify the gain (servo control) of a specific axis as ON.
Should the GAIN be specified as ON, servo control for that axis will start the next time a movement command is executed. Axes for which servo control has been stopped are in the “servo free state” (in which positioning control is not carried out).
As a system constant, ON has a value of 1. If you wanted to, you could use it in your program as a constant having the value 1. However, this is not a good idea since it makes your program unnecessarily hard to understand.
You cannot substitute into system constants.
For information on gains, refer to the GAIN command.

Sample

```
PROGRAM ONSAMPLE
  MOVE A1
  WAIT MOTION >= 100
  GAIN = {OFF, OFF, ON, OFF, OFF}      Before moving to point A2, this program turns
  MOVE A2                              off all gains except that for the Z-axis (Axis 3).
  OPEN1
  DELAY 0.5
  MOVE A1
  WAIT MOTION >= 100
  GAIN = {ON, ON, ON, ON, ON}          Then, when all motions have been completed,
  READY                                the program will turn the gains back on for all
END                                    axes.
```

ON.....DO.....

Purpose

This monitors the conditions.

Type

Program control command

Format

ON <monitoring condition> [{BREAK | PAUSE}] DO <statement>

Examples

ON DIN(1) DO DOUT(1)

ON TIMER BREAK DO MOVE A1

Analysis and advice

Should the <monitoring condition> be satisfied, the statement following the DO command will be executed.

Condition monitoring is carried out no matter what movement the robot happens to be doing at the time.

The ON command is processed in parallel with robot motion commands. Should a MOTION, MOTIONT, REMAIN or REMAINT command be used as the monitoring condition, monitoring of conditions for subsequent movement commands will be performed. Should TIMER be used as the monitoring condition, conditions will be monitored independently of robot movement. When monitoring input signals with DIN or other such commands, the timing with which monitoring begins will vary depending on the setting of the NOWAIT system switch.

When an ENABLE NOWAIT statement is in effect, signals will be monitored independently of robot movement.

When the DISABLE NOWAIT statement is in effect, monitoring of the signal will start after the robot has completed the movement it was executing at the time.

The execution of the statement following the DO command will start immediately after the execution of the command in effect when the monitoring condition was satisfied. However, if you happen to be executing a WAIT command at the time the monitoring condition was satisfied, the WAIT command will be cancelled immediately and program control will shift to the statement following the DO command.

There are three types of execution timing you can specify for the robot while in operation:

BREAK: BREAK will immediately stop all robot movement and shift control to the statement following the DO command. Movement that was aborted by

BREAK can be resumed using the RESUME command.

PAUSE: The statement following the DO command is executed after the movement now in progress finishes. During arm movement, however, normal program execution continues, except for the subprogram call command, return command to main program and motion command. At execution of these commands, program execution stops until the arm has stopped.

Default: The default setting will cause the movement in progress to be completed while simultaneously executing statements following the DO command. When the statement following the DO command is a movement command, always include a BREAK or PAUSE statement in the ON command line.

If the statement following the DO command (i.e., DO statement) and the motion command in the DO statement were executed, after the arm movement has finished, program execution will restart in accordance with conditions just before the condition for the ON command was satisfied. (Should a WAIT command have been interrupted, program execution will restart from the beginning of that WAIT command, i.e., the WAIT command will be executed again). However, should a program branch to a label have been carried out with the statement following the DO command, execution will start from the statement having that label.
Ten sets of conditions can be monitored at once. Furthermore, a maximum of four input signals may be specified with a single ON command.
When multiple monitoring conditions become true at once, the DO statement corresponding to the ON command having the highest priority is executed. This priority is determined by the order in which the ON commands were encountered in the program, with the first ON command encountered having the highest priority. DO statements corresponding to all other ON commands are ignored.
Monitoring of a condition specified by an ON command will be cancelled should execution shift to a DO statement corresponding to another ON command. Also, conditions are not monitored while program execution is halted due to a STOP command or an error.
When a subprogram is specified with a statement following DO, two or more processes described in the subprogram can be executed when the condition is satisfied. When an ON statement is used in the executing program as a statement following DO, the monitoring of the condition becomes valid just after the subprogram is returned.

When the system timer is specified as the monitoring condition, the condition is checked only when the state of the timer changes. When monitoring an external signal, an error condition or a movement reference command (such as the amount of a motion remaining to be performed), the controller monitors the state, not the change, of that signal.

The IGNORE command will cancel the monitoring of conditions specified by an ON command. Monitoring of conditions will also stop when a condition is satisfied and a statement following a DO command is executed.

Note 1: At present, ON and DO command combinations may only be used in the ways shown below:

- ON TIMER DO <statement>
  When the timer becomes 0, execute the statement.
- ON DIN( ) DO <statement>
  When the state of the input signal(s) in the brackets ( ) becomes as specified, execute the statement. You cannot monitor more than four signals at once with one such statement. When four or more points are specified, the extra points exceeding four points are ignored.
- ON MOTION >= <expression> DO <statement>
  Execute the statement when the amount of a motion which is to be executed next to this command exceeds the specified value. The only relational operand you can use with MOTION is ">=".
- ON MOTIONT >= <expression> DO <statement>
  Execute the statement when the time required for a motion which is to be executed next to this command exceeds the specified time. The only relational operand you can use with MOTIONT is "> =".
- ON REMAIN <= <expression> DO <statement>
  Execute the statement when the remaining amount of a motion which is to be executed next to this command is smaller than the specified value. The only relational operand you can use with REMAIN is "< =".
- ON REMAINT <= <expression> DO <statement>
  Execute the statement when the remaining time required for a motion which is to be executed next to this command is smaller than the specified time.
  The only relational operand you can use with REMAINT is "< =".

Note 2: The following command relating to the task control cannot be used in the area following the DO statement.
TASK, KILL, SWITCH
When using these commands after the DO statement, they are not executed. Note that monitoring of the conditions specified by the subtask ON command cannot be executed.

Note 3: If a motion monitored under the condition of ON MOTION, ON MOTIONT, ON REMAIN or ON REMAINT has been stopped, or if the slow speed command has been specified during execution of a monitored motion, the ON condition is cancelled.

Sample

```
PROGRAM MAIN
  ONSAMPLE
  MOVE P
END
PROGRAM ONSAMPLE
  ON DIN(1) PAUSE DO RETURN
  MOVE A1
  MOVE A2
  MOVE A3
  WAIT MOTION >=100
  IGNORE DIN(1)
  RETURN
END
```

Should Signal 1 turn ON while a movement is being executed, control will be returned to the main program after that movement has been completed.

Cautions on DO statement:

For ON ~ DO command, the ON conditions to be monitored and the DO statement which starts when the conditions are satisfied are registered.

```
PROGRAM MAIN
  SIG = 1
  ON DIN(1) DO INPUT SIG
  SUB
  IGNORE DIN(1)
  PRINT SIG
END
PROGRAM SUB
  MOVE P
  WAIT MOTION >= 100
END
```

In the above SCOL2 program, if DIN(1) is set ON during traverse to P, the DO statement cannot be executed because the variable SIG is not defined in the program SUB and there is no space for saving the variable as input by the INPUT command. In this case, the relevant DO statement can be executed normally by defining the variable SIG as the global variable.

```
GLOBAL
  SIG = 0
END
PROGRAM MAIN
  SIG = 1
  ON DIN(1) DO INPUT SIG
  SUB
  IGNORE DIN(1)
  PRINT SIG
END
PROGRAM SUB
  MOVE P
  WAIT MOTION>=100
END
```

In the DO statement, even if the task changeover conditions are established or the SWITCH command is executed, the task cannot be changed over. If the TASK command or KILL command is executed, an error occurs.

ONGAIN( )

Purpose

Turns on the gain of each axis (servo control).

Type

Library function (SCOL.LIB)

Format

ONGAIN(<integer>, <integer>, <integer>, <integer>, <integer>)

Examples

ONGAIN(0, 0, 1, 0, 0)

Analysis and advice

The gain of each axis (servo control) of the robot is turned on.
For details of the gain, see the descriptions of the GAIN command.
To turn off the gain, use the OFFGAIN command. For each joint of axes 1 to 5, specify the gains by delimiting the values of the five axes with a comma. The values are specified with 1 or 0. When 1 is specified, the gain of the axis is turned on. When 0 is specified, the state of the axis remains unchanged.
The on/off state of the gain is changed after the current motion is completed.
This command can be executed only when file SCOL.LIB is present in the ROM drive of the controller.
In this command, the system constants ON and OFF cannot be used.

Sample

```
PROGRAM ONGAINSMPL
  MOVE A1                    After the robot is moved to A1, except for the
  OFFGAIN(1, 1, 0, 1, 1)     axis 3, the gains are turned off.
  MOVE A2                    After the robot is moved to A2, the gains of all
  ONGAIN(1, 1, 1, 1, 1)      the axes are turned on.
  MOVE A3
END
```

OPEN1 , OPEN2 , OPENI1 , OPENI2

Purpose

These commands open the robot hand.

Type

Library function (SCOL.LIB)

Format

OPEN1
OPEN2
OPENI1
OPENI2

Examples

OPEN1
OPENI2

Analysis and advice

These commands are used to open the hand. The numbers 1 and 2 refer to Hand 1 and Hand 2.
These commands open the hand by changing the state of the output signal which controls the robot hand. In the initial setting of SCOL.LIB, DOUT(203,-204) is executed when OPEN1 is triggered.
The OPEN command directs the robot to open its hand after it completes the motion in progress.
The OPENI command directs the robot to open its hand immediately.
Note that these commands will not work if the file SCOL.LIB is not in the controller ROM drive.
Also, keep in mind that there is a slight delay from when an OPEN command is output until the robot actually opens its hand.
Corresponding commands CLOSE1, CLOSE2, CLOSEI1 and CLOSEI2 are provided in order to close the hand.

These commands execute a program written in the system library (SCOL. LIB). The data of SCOL. LIB should be changed according to the robot hand specifications.

Sample

```
PROGRAM OPENSAMPLE
 CLOSE1
 MOVE A1                    This program opens the hand after the robot
                            has finished moving to point A1.
 OPEN1                      The robot waits 0.5 second until the hand is
 DELAY 0.5                  open completely after the OPEN1 command has
 MOVE A2                    been executed.
END
```

```
PROGRAM OPENISMPLE
 ENABLE NOWAIT
 OPENI1                     Here, the robot will open its hand1 while
 DELAY 2                    moving to point A1.
 MOVE A1
 CLOSEI1
 DELAY 2
 MOVE A2
END
```

OR

Purpose

OR is used to find the logical sum of two logical expressions.

Type

Calculator command (operand)

Format

<logical expression> OR <logical expression>

Examples

IF DIN(1) OR K <= 3 THEN J = 0

WAIT DIN(5) OR TIMER == 0

Analysis and advice

The OR statement calculates the logical sum of the expressions to the right and left.

If even one of the two statements is true, OR will return a TRUE.

The OR statement must be used in a logical expression.

Sample

```
PROGRAM ORSAMPLE
  FOR K = 1 TO 50
    IF DIN(1) OR K == 50 THEN J = 1 ELSE J = 0
    PRINT TP, J, CR
  NEXT K
END
```

OVERRIDE

Purpose

Specifies the override of the robot.

Type

System variable (movement condition command)

Format

OVERRIDE = <expression>

Examples

OVERRIDE = 50

Analysis and advice

This is a system variable for specifying the override for the robot operation speed.
OVERRIDE is specified as a positive integer value. If a value of 0 or less is specified, it is processed as if a value of 1 was specified. Also, even if a value of 100 or more is specified, it is processed as a value of 100%.
If the upper limit of the OVERRIDE is restricted due to the manual mode etc., a value higher than the limit cannot be set (a value exceeding the limit is treated as the upper limit value).
The <Equation> section can be used for constants, variables, or calculation equations. However, it cannot be used for vector type data.

Sample

```
PROGRAM SAMPLE
  SPEED = 100
  MOVE A1
  ON MOTION >= 25 DO OVERRIDE = 50
  MOVE A2
  WAIT MOTION >= 100
  OVERRIDE = 100
  ON MOTION >= 50 DO OVERRIDE = 50
  MOVE A3
  WAIT MOTION >= 100
  OVERRIDE = 100
  ON MOTION >= 75 DO OVERRIDE = 50
  MOVE A1
  WAIT MOTION >= 100
  MOVE A2
END
```

PAI

Purpose

PAI (normally written “~” or “pi “) is a system constant having a value of 3.14159

Type

System constant

Format

PAI

Examples

R = D * PAI / 180

D = N * PAI * 2

Analysis and advice

PAI is a system constant having a value beginning with 3.14159....

You can use it to represent “pi” when calculating the length of an arc, the area of a circle, etc.

You cannot substitute into system constants including PAI.

Sample

```
PROGRAM MAIN
  D = 0.0                          (“DIM D AS REAL” is also possible.)
  PAISAMPLE(5, D)
  PRINT TP, D, CR
END
PROGRAM PAISAMPLE(R, D)            This subprogram converts the value of the
  D = R / PAI * 180                argument from radians to degrees, and sends
  RETURN                           the result back to the main program as
END                                argument D.
```

PALLET

Purpose

This defines the pallet type variable.

Type

Conversion function

Format

<variable> = PALLET(<number of elements> [, <number of elements> [, <number of elements>]] , <position data>, ...)

Examples

PLT = PALLET(10, P1, P2)

PLT = PALLET(5, 5, P1, P2, P3)

PLT = PALLET(3, 3, 3, P1, P2, P3, P4, P5, P6, P7, P8)

Analysis and advice

This creates a pallet type variable.

The pallet can be defined as one with 1 to 3 dimensions. The number of dimensions for the pallet is determined by the written number of elements for each direction.

The quantity for <position data> that defines the pallet type variables corresponds to the following dimensions.

1 dimension: 2 items

2 dimensions: 3 or 4 items

3 dimensions: 4, 5, or 8 items

The defined pallet specifies the number in ( ) after the variable name in the same way as the array type position data for handling as POINT type data for the cell.

The number in ( ) is specified using the same number of dimensions as the pallet or a single numerical value.

If specified using a single numerical value, the cell assigned in order from the first direction is referenced.

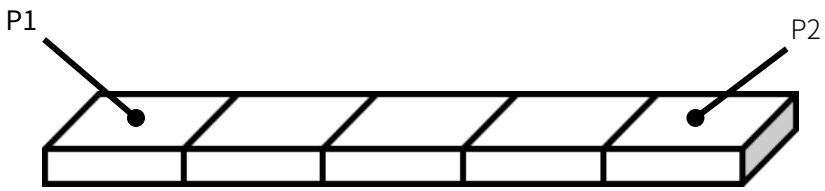
Details on how to create pallet type variables and how to access the positions for the created pallet type variables are provided after the sample program.

Sample

```
PROGRAM MAIN
  PLT = PALLET(3, 3, P1, P2, P3, P4)          Defines a 3×3 cell pallet.
  FOR J = 1 TO 3
    FOR I = 1 TO 3
      MOVE PLT(I, J) + POINT(0, 0, 100)       Moves to the points of the pallet.
      MOVE PLT(I, J)
      MOVE PLT(I, J) + POINT(0, 0, 100)
    NEXT I
  NEXT J
END
```

Creating a pallet type variable

•1-dimensional pallet

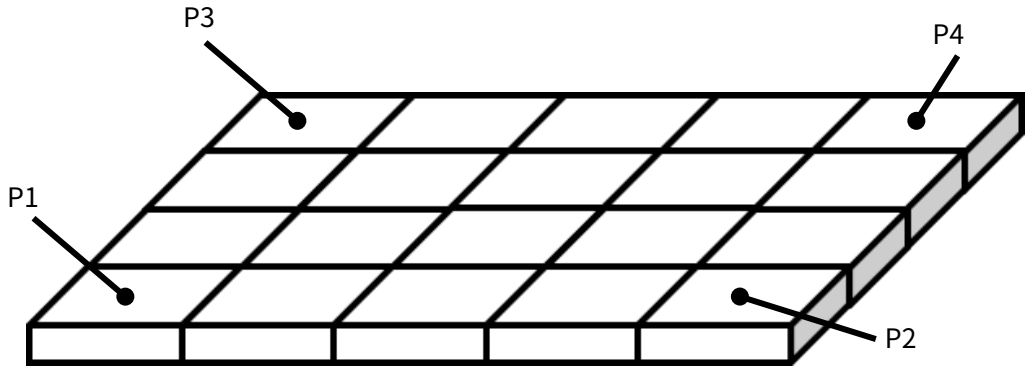


Both pallet edge points are taught, and this is written as follows.

PLT1 = PALLET(5, P1, P2)

There are no restrictions on the teaching point names or the name of the pallet type variables. Any name can be added in the same way as general variables.

•2-dimensional pallet



Each corner point of the pallet edge is taught, and this is written as follows.

PLT2 = PALLET(5, 4, P1, P2, P3, P4)

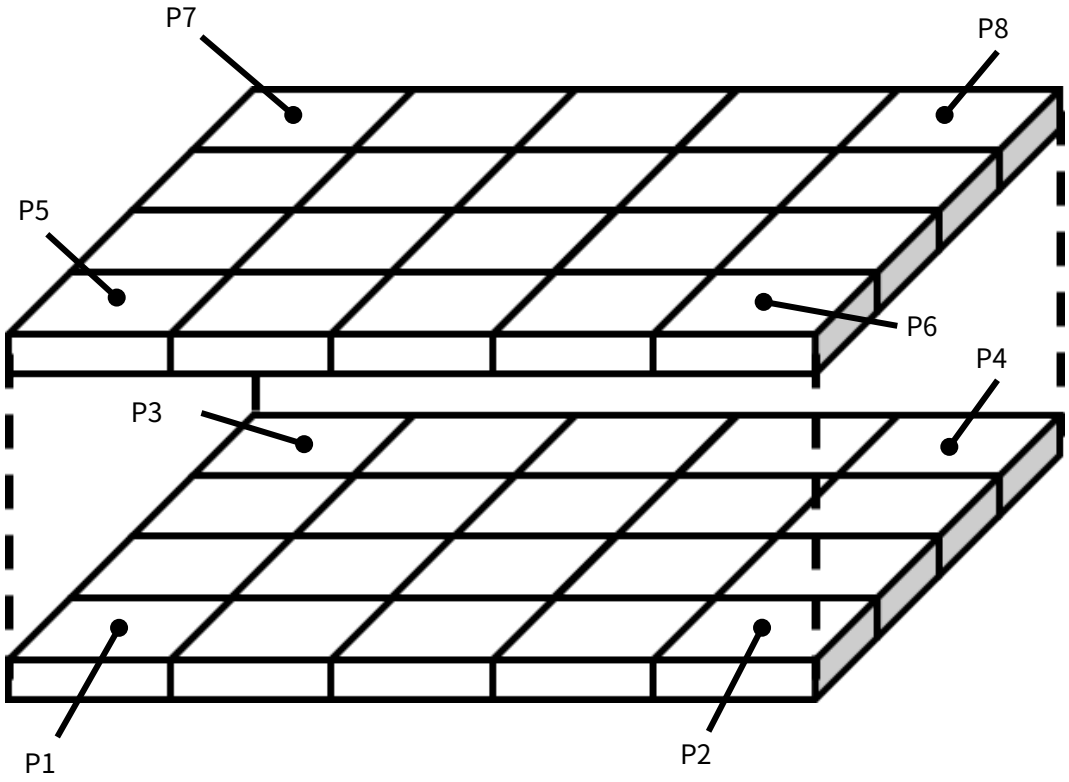
The order of the teaching points is fixed. This is written in the order of pallet origin point (P1), point in the direction of the first element count (5) (P2), point in the direction of the second element count (4) (P3), and point diagonally opposite the pallet origin (P4).

The point that is diagonally opposite (P4) can also be omitted. In this case, the teaching points are written as follows.

PLT2 = PALLET(5, 4, P1, P2, P3)

However, the accuracy of each pallet position is generally higher when taught using four points.

•3-dimensional pallet



Each corner point of the pallet edge is taught, and this is written as follows.

PLT3 = PALLET(5, 4, 2, P1, P2, P3, P4, P5, P6, P7, P8)

The order of the teaching points is fixed. This is written in the order of pallet origin point (P1), point in the direction of the first element count (5) (P2), point in the direction of the second element count (4) (P3), and point diagonally opposite the pallet origin on the same plane (P4), point in the direction of the third element count (2) (P5), point in the P5 direction of P2 (P6), point in the P5 direction of P3 (P7), and point in the P5 direction of P4 (P8).

The points P4, P6, P7, and P8 that are diagonally opposite can be omitted. In this case, the teaching points are written as follows.

PLT3 = PALLET(5, 4, 2, P1, P2, P3, P4, P5) *Points in P5 direction (written by omitting P6, P7, and P8)

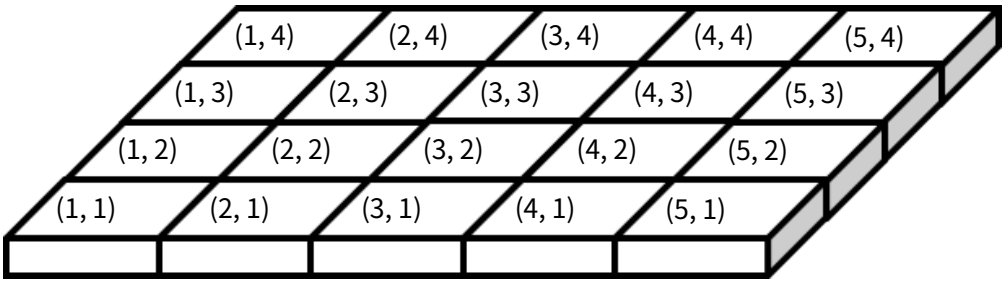
PLT3 = PALLET(5, 4, 2, P1, P2, P3, P5) *Written using minimum number of teaching points

However, the accuracy of each pallet position is generally higher when taught using eight points.

Accessing the pallet positions

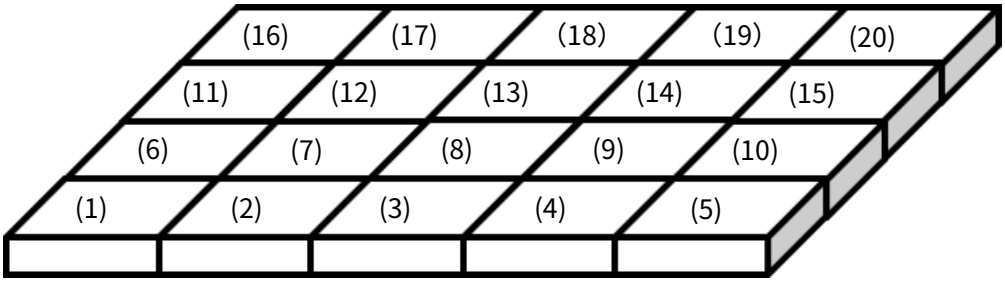
Access the pallet positions by specifying a number in the ( ) after the pallet type variable that is created. The number in ( ) is specified using the number of elements that is the same as the number of dimensions of the pallet or an element. The elements can also be specified using a variable or expression.

•Specifying using the same number of elements



•Specifying using a single element

When specified using a single element, numbers are assigned in order from the first direction.



If information other than the above is specified (such as specifying of two elements for a 3-dimensional pallet or specifying of two or more elements for a 1-dimensional pallet), an error occurs.

Accessing the array type pallet type variables

Pallet type variables can be defined as an array.

Sample

```
GLOBAL
  DIM PLT(3) AS PALLET
END
PROGRAM MAIN
  PLT(1) = PALLET(5, 4, P1, P2, P3, P4)
  PLT(2) = PALLET(5, P5, P6)
  PLT(3) = PALLET(3, 3, P7, P8, P9)
  MOVE PLT(3, 2, 2)
END
```

When defining a pallet, specify the array number only in ( ) after the pallet type variable name. To access the pallet positions, specify the array number and element number in order in ( ) after the pallet type variable name. Pallet type variables can be arrays with a maximum of 2 dimensions.

PALLETZ

Purpose

This defines a zigzag pallet.

Type

Conversion function

Format

<variable name> = PALLETZ(<number of elements> [, <number of elements> [, <number of elements>]] , <position data>, ...)

Examples

PLZ = PALLETZ(3, 3, P1, P2, P3)

PLZ = PALLETZ(5, 5, P1, P2, P3, P4)

Analysis and advice

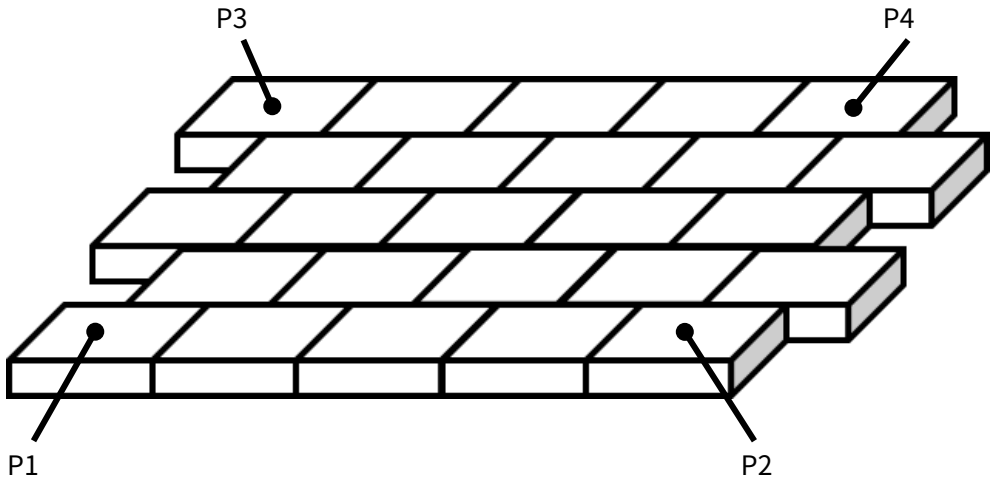
This defines a zigzag pallet.

A zigzag pallet is one where the order of the cells in the second direction are staggered.

Sample

```
PROGRAM MAIN
  PLT = PALLETZ(3, 3, P1, P2, P3, P4)
  FOR J = 1 TO 3
    FOR I = 1 TO 3
      MOVE PLT(I, J) + POINT(0, 0, 100)
      MOVE PLT(I, J)
      MOVE PLT(I, J) + POINT(0, 0, 100)
    NEXT I
  NEXT J
END
```

Creating a zigzag pallet

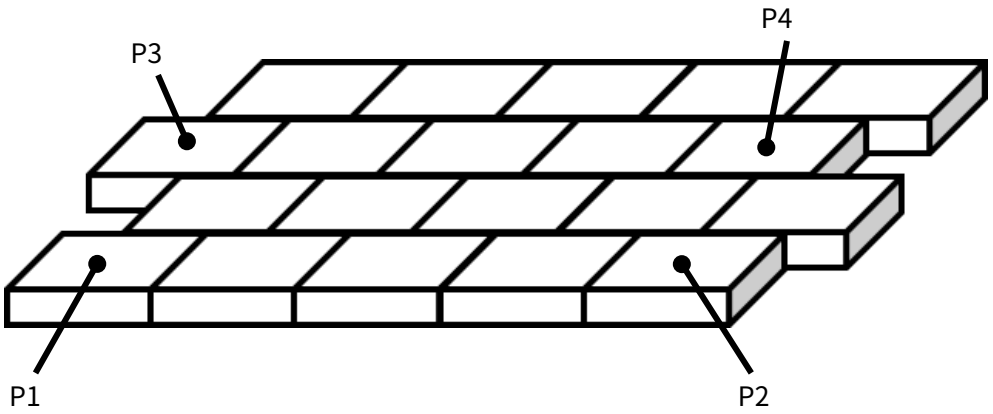


As shown in the figure above, a zigzag pallet is one where the cells in the second direction are staggered. The points are determined, and teaching is performed so that the staggered direction is the same as the first direction, and this is written as follows.

PLT = PALLETZ(5, 5, P1, P2, P3, P4)

The point P4 can be omitted. For details, see the description for PALLET.

If the second direction has an even number of cells, perform teaching for P3 and P4 at one position before instead of at the angle point.



The teaching points are written as follows.

PLT = PALLETZ(5, 4, P1, P2, P3, P4)

If the number of elements in the second direction is two only, create a pallet type variable using a normal PALLET command.

PASS

Purpose

PASS is a system switch used to specify short-cut movement.
(For setting short-cut motion parameters, see the next page.)

Type

System switch

Format

PASS

Examples

DISABLE PASS
ENABLE PASS

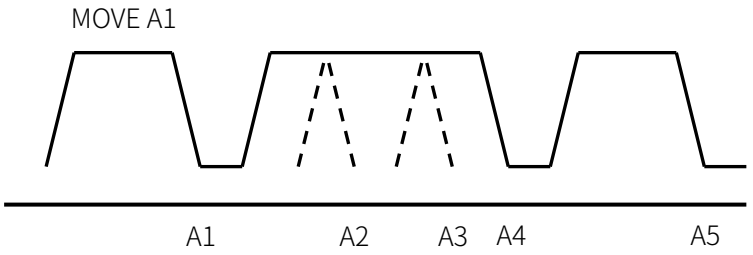
Analysis and advice

PASS is a system switch used to invoke short-cut movement.
Short-cut movement is an operating mode in which the robot is directed to begin its next move before completing the positioning of its previous move. The timing for switching over from the present movement to the next movement is specified with the system variable PASS command. Short-cut movement allows you to reduce the time it takes the robot to get from one place to another. For more information, refer to Section 5.
The ENABLE and DISABLE commands are used to turn on and off system switches such as PASS.
An ENABLE PASS statement will activate short-cut movement, and DISABLE PASS will cancel short-cut movement.
The initial setting for the controller is DISABLE PASS.

Sample

```
PROGRAM PASSSAMPLE
  MOVE A1
  PASS = 80
  ENABLE PASS
    MOVE A2
    MOVE A3
  DISABLE PASS
    MOVE A4
    MOVE A5
END
```

This program moves the robot with short-cut movement from A1 to A4, and for the movement from point A4 onwards, cancels the short-cut movement.



PASS

Purpose

PASS is a system variable used to set short-cut movement parameters.

Type

System variable (movement condition command)

Format

PASS = <expression>

Examples

PASS = 80

PASS = PASS * 0.8

Analysis and advice

PASS is a system variable used to specify parameters for short-cut movement.
Short-cut movement is an operating mode in which the robot is directed to begin its next move before completing the positioning of its previous move. The timing for switching over from the present movement to the next movement is specified with the system variable PASS command. The parameter for short-cut movement is expressed as a percentage of a motion completed by a robot relative to the entire motion. When the robot movement has exceeded that percentage, the motion being performed at that time and the following motion are superimposed.
The travel amount refers to a position that the robot is directed to, at which the next movement is started even if the actual robot cannot move because of interference between the robot and the controller.
An integer value of 0 to 100 may be specified for PASS. Numbers less than 50 will be treated as 50%, and numbers greater than 100 will be treated as 100%.
The <expression> designation may contain a constant, variable, or calculation. However, you may not use vector type data.
When referring to the PASS system variable, you can refer to the parameter of the current short-cut movement.
Short-cut movement allows you to reduce the time it takes the robot to get from one place to another. For more information, see Section 5.
The ENABLE and DISABLE commands are used to turn on and off system switches such as PASS. An ENABLE PASS statement will activate short-cut movement, and DISABLE PASS statement will cancel short-cut movement.
The initial setting for the controller is DISABLE PASS.

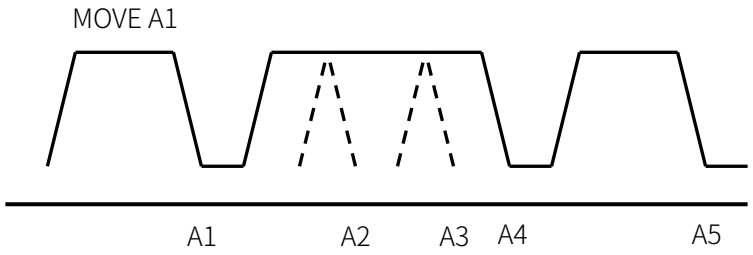
If the WAIT command and I/O command have been specified while a movement

command is executed before the PASS movement stats the execution, the PASS movement may not be executed.

Sample

```
PROGRAM PASSSAMPLE
  MOVE A1
  PASS = 80
  ENABLE PASS
    MOVE A2
    MOVE A3
  DISABLE PASS
    MOVE A4
    MOVE A5
END
```

This program moves the robot with short-cut movement from A1 to A4, and for the movement from point A4 onwards, cancels the short-cut movement.



PASSMM

Purpose

This specifies the shortcut amount (pass amount) of the shortcut movement in millimeters (mm).

Type

System variable (movement condition command)

Format

PASSMM = <expression>

Examples

PASSMM = 50
MOVES P1 WITH PASSMM = 30

Analysis and advice

This specifies the shortcut amount (pass amount) of the shortcut movement in millimeters (mm).
The value specified by this system variable is enabled for linear interpolation movement and direct-acting axis movement. The PASS value is referenced for all other movements.

Sample

```
PROGRAM MAIN
  PASSMM = 30
  MOVE A1
  ENABLE PASS
    MOVE A1 + POINT(0, 0, 100)
    MOVE A2 + POINT(0, 0, 100)
  DISABLE PASS
    MOVE A2
END
```

Moves the positions 100 mm above A1 and A2 by a pass amount of 30 mm.

PAUSE

Purpose

This waits for the end of robot movement and executes the command following DO.

Type

Program control command

Format

ON <monitoring condition> [{BREAK | PAUSE}] DO <statement>

Examples

ON DIN(1) PAUSE DO SUB

Analysis and advice

When the monitoring conditions specified by the ON statement are established, the PAUSE directs the controller to wait until the robot finishes the motion in progress before executing the DO statement. For details, see the “ON” command.

Sample

```
PROGRAM PAUSESMPL
  ENABLE NOWAIT
  REMARK *** MAIN PROGRAM ***
  ON DIN(24) PAUSE DO STOP
    MOVE A1
    MOVE A2
    MOVE A3
    WAIT MOTION >= 100
  IGNORE DIN(24)
END
```

Here, if something goes wrong with the system and Input Signal 24 turns ON, the robot will stop moving after completing the movement in progress at the time.

PAYLOAD

Purpose

PAYLOAD is a system variable used to set data for loads acting on the end of the robot hand.

Type

System variable (movement condition command)

Format

PAYLOAD = {<mass>, <center of gravity offset>}

Examples

PAYLOAD = {10, 10}
MOVE A1 WITH PAYLOAD = MOTOR

Analysis and advice

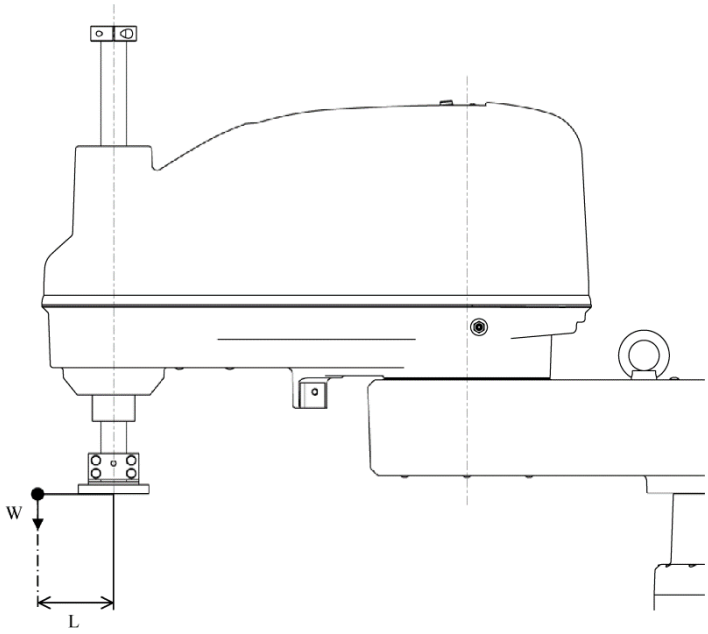
PAYLOAD is a system variable used to set data for loads acting on the end of the robot hand.
The SCOL2 language allows load data to be set for the load applied to the robot end effector so that the robot can operate optimally for any load within the allowable load range of the robot.
Loads acting on the robot hand are set with the system variable PAYLOAD. The controller uses these values to calculate control constants for robot acceleration and deceleration that are appropriate for the load. The load data include the weight of the load on the end effector and the value of the distance from the mechanical interface to the center of gravity.
The <mass> specification designates the weight of the load applied to the tip of the robot hand in units of kilograms.
The <center of gravity offset> designates the distance between the center of gravity of the load applied to the tip of the robot hand and the center of the tool of the hand in units of millimeters.
Constants, variables and calculations may be used for the <mass> and <center of gravity offset> designations. Also, PAYLOAD type data may be used for the {<mass>, <center of gravity offset>} specification.

<variable> = {<mass>, <center of gravity offset>}
<variable> = PAYLOAD(<mass>, <center of gravity offset>)
Example) MOTOR = {5, 10}
HAND = PAYLOAD(2, 0)
WORKA = HAND + MOTOR

The FREELOAD command is available to set load data to zero.

Sample

```
PROGRAM PAYLOADSMP
  PAYLOAD = HAND                          HAND is assigned as the load data before the
    MOVE A1                               robot moves to point A1.
  CLOSE1                                  The robot grabs something at point A1,
    DELAY 0.5
    MOVE A2 WITH PAYLOAD = HAND + MOTOR   and the sum of HAND + MOTOR is assigned as
  OPEN1                                   the new load data.
    DELAY 0.5
    MOVE A3
  FREELOAD                                Sets the load data to 0 and performs
    MOVE A2                               movement.
    MOVE A3
END
```



Load data is entered in term of:

Mass	W	[unit:	kg]
Center of gravity offset	L	[unit:	mm].

PLCDATAR1~8

Purpose

This is a system variable that receives data from the robot's internal PLC.

Type

System variable (Other)

Format

PLCDATAR1

Examples

A = PLCDATAR1

B = PLCDATAR5

Analysis and advice

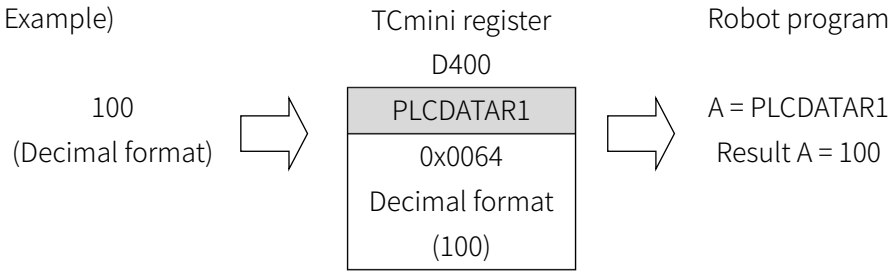
PLCDATAR1 ~ 8 are the read-only system variables.

Values set by the internal PLC can be read.

(The internal PLC is an optional function.)

The value these system variables can receive are 0 ~ 65535. (Neither a negative value nor a decimal point can be used.)

SCOL2 language name	Data register
PLCDATAR1	D400
PLCDATAR2	D401
PLCDATAR3	D402
PLCDATAR4	D403
PLCDATAR5	D404
PLCDATAR6	D405
PLCDATAR7	D406
PLCDATAR8	D407



Sample

```
PROGRAM PLCDIN
  A = PLCDATAR1
  B = PLCDATAR2
END
```

PLCDATAW1~8

Purpose

This is a system variable that writes values to the robot's internal PLC.

Type

System variable (Other)

Format

PLCDATAW1 = <expression>

Examples

PLCDATAW1 = 1

PLCDATAW5 = A + B

Analysis and advice

PLCDATAW1 ~ 8 are the write-only system variables.

It can pass values to the internal PLC. (The internal PLC is an optional function.)

The value these system variables can transfer are 0 ~ 65535. (Neither a negative value nor a decimal point can be used.)

If the internal PLC function option is selected, values of PLCDATAR1 to PLCDATAR8 can be freely handled in the sequence program.

SCOL2 language name	Data register
PLCDATAW1	D600
PLCDATAW2	D601
PLCDATAW3	D602
PLCDATAW4	D603
PLCDATAW5	D604
PLCDATAW6	D605
PLCDATAW7	D606
PLCDATAW8	D607

Example)

Robot program

PLCDATAW1 = 20



TCmini register

D600

PLCDATAW1
0x0014
Decimal format (20)

Sample

```
PROGRAM PLCDIN
  A = 10
  PLCDATAW1 = 1
  PLCDATAW2 = A
END
```

PLCDREGDW

Purpose

This variable allows to access the robot's internal PLC (D register) as signed 4-byte data.

Type

System variable (Other)

Format

PLCDREGDW(<number>)

Examples

PLCDREGDW(0x100) = 100

A = PLCDREGDW(0x300)

Analysis and advice

Numerical values can be read and written to and from the data register (D register) of the robot's internal PLC.

For <number>, specify the register name of the D register by describing as a hexadecimal number.

For example, to access D27E, write PLCDREGDW(0x27E).

The valid range of numbers is 0x000 to 0x77E, but 0x400 to 0x47F, 0x600 to 0x67F, and 0x700 to 0x77F are interface registers and can only be referenced.

Data to be read/written is handled as signed 4-byte data, and data in the range of -2147483648 to 2147483647 can be handled.

Since the data register’s word length is 2 bytes, the specified register and the succeeding register are accessed.

Sample

```
PROGRAM MAIN
  A = PLCDREGDW(0x000)          Acquires the values of D000 and D001.
  B = PLCDREGDW(0x002)          Acquires the values of D002 and D003.
  PLCDREGDW(0x004) = A + B      Sets the values of D004 and D005.
END
```

System version

X8KCS-05C or later

PLCDREGF

Purpose

This variable allows to access the robot's internal PLC (D register) as floating-point data.

Type

System variable (Other)

Format

PLCDREGF(<number>)

Examples

PLCDREGF(0x100) = 123.456

A = PLCDREGF(0x300)

Analysis and advice

Numerical values can be read and written to and from the data register (D register) of the robot's internal PLC.

For <number>, specify the register name of the D register by describing as a hexadecimal number.

For example, to access D27E, write PLCDREGDW(0x27E).

The valid range of numbers is 0x000 to 0x77E, but 0x400 to 0x47F, 0x600 to 0x67F, and 0x700 to 0x77F are interface registers and can only be referenced.

Data to be read/written is handled as 4-byte floating-point number (REAL type data).

If referenced data is NaN, Inf or the like, an alarm occurs and the program stops.

Since the data register’s word length is 2 bytes, the specified register and the succeeding register are accessed.

Sample

```
PROGRAM MAIN
  A = PLCDREGF(0x000)          Acquires the values of D000 and D001.
  B = PLCDREGF(0x002)          Acquires the values of D002 and D003.
  PLCDREGF(0x004) = A + B      Sets the values of D004 and D005.
END
```

System version

X8KCS-05C or later

PLCDREGUW

Purpose

This variable allows to access the robot's internal PLC (D register) as unsigned 2-byte data.

Type

System variable (Other)

Format

PLCDREGUW(<number>)

Examples

PLCDREGUW(0x100) = 100

A = PLCDREGUW(0x300)

Analysis and advice

Numerical values can be read and written to and from the data register (D register) of the robot's internal PLC.

For <number>, specify the register name of the D register by describing as a hexadecimal number.

For example, to access D27E, write PLCDREGUW(0x27E).

The valid range of numbers is 0x000 to 0x77F, but 0x400 to 0x47F, 0x600 to 0x67F, and 0x700 to 0x77F are interface registers and can only be referenced.

Data to be read/written is handled as unsigned 2-byte data and can range from 0 to 65535.

Sample

```
PROGRAM MAIN
  A = PLCDREGUW(0x000)          Acquires the value of D000.
  B = PLCDREGUW(0x001)          Acquires the value of D001.
  PLCDREGUW(0x002) = A + B      Sets the value of D002.
END
```

System version

X8KCS-05C or later

PLCDREGW

Purpose

This variable allows to access the robot's internal PLC (D register) as signed 2-byte data.

Type

System variable (Other)

Format

PLCDREGW(<number>)

Examples

PLCDREGW(0x100) = 100

A = PLCDREGW(0x300)

Analysis and advice

Numerical values can be read and written to and from the data register (D register) of the robot's internal PLC.

For <number>, specify the register name of the D register by describing as a hexadecimal number.

For example, to access D27E, write PLCDREGW(0x27E).

The valid range of numbers is 0x000 to 0x77F, but 0x400 to 0x47F, 0x600 to 0x67F, and 0x700 to 0x77F are interface registers and can only be referenced.

Data to be read/written is handled as unsigned 2-byte data and can range from -32768 to 32767.

Sample

```
PROGRAM MAIN
  A = PLCDREGW(0x000)          Acquires the value of D000.
  B = PLCDREGW(0x001)          Acquires the value of D001.
  PLCDREGW(0x002) = A + B      Sets the value of D002.
END
```

System version

X8KCS-05C or later

PLCSLR01~08

Purpose

This is a system variable that receives data from the robot's internal PLC.

Type

System variable (Other)

Format

PLCSLR01

Examples

A = PLCSLR01

B = PLCSLR05

Analysis and advice

PLCSLR01 to 08 are read-only system variables.

The value set by the internal PLC can be read.

(The internal PLC is an optional function.)

The value that is received is in the range from -2147483648 to 2147483647.

SCOL2 language name		Data register
PLCSLR01	L	D410
	H	D411
PLCSLR02	L	D412
	H	D413
PLCSLR03	L	D414
	H	D415
PLCSLR04	L	D416
	H	D417
PLCSLR05	L	D418
	H	D419
PLCSLR06	L	D41A
	H	D41B
PLCSLR07	L	D41C
	H	D41D
PLCSLR08	L	D41E
	H	D41F

Example)

-10

(decimal format)



TCmini register

D411	D410
PLCSLR01H	PLCSLR01L
0xFFFF	0xFFF6
Decimal format (-10)	



Robot program

A = PLCSLR01

Result A = -10

Sample

```
PROGRAM PLCSLR
  A = PLCSLR01
  B = PLCSLR02
END
```

PLCSLW01~08

Purpose

This is a system variable that writes values to the robot's internal PLC.

Type

System variable (Other)

Format

PLCSLW01 = <expression>

Examples

PLCSLW01 = 1

PLCSLW05 = A + B

Analysis and advice

PLCSLW01 ~ 8 are the write-only system variables.

It can pass values to the internal PLC.

(The internal PLC is an optional function.)

The value these system variables can transfer are -2147483648 to 2147483647.

SCOL2 language name		Data register
PLCSLW01	L	D610
	H	D611
PLCSLW02	L	D612
	H	D613
PLCSLW03	L	D614
	H	D615
PLCSLW04	L	D616
	H	D617
PLCSLW05	L	D618
	H	D619
PLCSLW06	L	D61A
	H	D61B
PLCSLW07	L	D61C
	H	D61D
PLCSLW08	L	D61E
	H	D61F

Example)

Robot program

PLCSLW01 = 70000



TCmini register

D611	D610
PLCSLW01H	PLCSLW01L
0x0001	0x1170
Decimal format (70000)	

Sample

```
PROGRAM PLCSLW
  A = 10
  PLCSLW01 = 1
  PLCSLW02 = A
END
```

PLCSSR01~08

Purpose

This is a system variable that receives data from the robot's internal PLC.

Type

System variable (Other)

Format

PLCSSR01

Examples

A = PLCSSR01

B = PLCSSR05

Analysis and advice

PLCSSR01 to 08 are read-only system variables.

The value set by the internal PLC can be read.

(The internal PLC is an optional function.)

The value that is received is in the range from –32768 to 32767.

SCOL2 language name	Data register
PLCSSR01	D408
PLCSSR02	D409
PLCSSR03	D40A
PLCSSR04	D40B
PLCSSR05	D40C
PLCSSR06	D40D
PLCSSR07	D40E
PLCSSR08	D40F

Example)

100

(decimal format)



TCmini register

D408

PLCSSR01
0x0064 Decimal format (100)



Robot program

A = PLCSSR01

Result A = 100

Sample

```
PROGRAM PLCSSR
  A = PLCSSR01
  B = PLCSSR02
END
```

PLCSSW01~08

Purpose

This is a system variable that writes values to the robot's internal PLC.

Type

System variable (Other)

Format

PLCSSW01 = <expression>

Examples

PLCSSW01 = 1

PLCSSW05 = A + B

Analysis and advice

PLCSSW01 ~ 8 are the write-only system variables.

It can pass values to the internal PLC.

(The internal PLC is an optional function.)

The value these system variables can transfer are -32768 to 32767.

When the internal PLC function option is selected, the values of PLCSSW01 to 08 can be freely handled in the sequence program.

SCOL2 language name	Data register
PLCSSW01	D608
PLCSSW02	D609
PLCSSW03	D60A
PLCSSW04	D60B
PLCSSW05	D60C
PLCSSW06	D60D
PLCSSW07	D60E
PLCSSW08	D60F

Example)

Robot program

PLCSSW01 = -10



TCmini register

D608

PLCSSW01
0xFFF6 Decimal format (-10)

Sample

```
PROGRAM PLCSSW
  A = 10
  PLCSSW01 = 1
  PLCSSW02 = A
END
```

PLUSLIM

Purpose

This references the software limit in the positive direction.

Type

System variable (movement reference command)

Format

PLUSLIM

Examples

MOVEA 1, PLUSLIM.1

PRINT PLUSLIM, CR

Analysis and advice

This is the JOINT type variable indicating the upper limit value of the robot movement range.

Sample

```
PROGRAM MAIN
  MOVEA 3, PLUSLIM.3                    Moves the 3rd axis to the upper limit position.
END
```

POINT( )

Purpose

POINT creates POINT type data.

Type

Conversion function

Format

POINT(<expression>, <expression>, <expression>, <expression>, <expression>, <configuration>)

Examples

A = POINT(100, 100, 0, 0, 0, 0)
MOVE POINT(100, 100)

Analysis and advice

The POINT command creates POINT type data.
From left to right, the <expression> designations correspond to the X, Y, Z, C and T elements. These elements are specified in units of millimeters or degrees.
The <configuration> element is to contain an integer from 0 to 2 that specifies robot configuration. The robot configuration is undefined (free) at 0, left handed at 1 and right handed at 2. In order to specify the system configuration, you may use these numbers or the system constants FREE, LEFTY and RIGHTY. As you would expect, the configuration is undefined at CONFIG = FREE, left handed at CONFIG = LEFTY, and right handed at CONFIG = RIGHTY. Anything less than 0 which is entered as the <configuration> will be treated as 0, and anything greater than 2 will be treated as 2.
Constant, variables or calculations may be used for the <expression> and <configuration> terms. However, you may not use vector type data. Furthermore, any omitted <expression> or <configuration> terms will be taken as 0.
This command must be used in an expression.

Sample

```
PROGRAM POINTSMPL
  MOVE POINT(300,300)
END
```

Moves to the position X = 300, Y = 300, Z = 0, C = 0, T = 0, CONFIG=FREE(0).

PRESS

Purpose

This performs press control with the specified stress.

Type

Motion control command

Format

PRESS (<target stress>, <travel completion distance>, <completion time>) [WITH clause]

Examples

PRESS(50, 100, 10)
PRESS(5, 0, 0) WITH TORQUE = {0, 0, 20, 0, 0}

Analysis and advice

To use this command, an optional A/D conversion board is required for the controller.
If the A/D conversion board is not mounted or the parameters required for press control are not set, an error occurs during execution.
Starting at the current position, the hand performs press control with the specified stress [N].
Axis 3 moves so that the value of the load cell attached to the robot equals the target stress.
The value that can be specified in <target stress [N]> is 95% of the upper limit of the range that can be measured with an A/D conversion board, or a value less than or equal to the pressing force determined for each model.
When the robot moves the distance set in < travel completion distance [mm]> or the time set in <completion time[sec]> has elapsed, the press control will be completed.
If a value less than or equal to 0 is set in < travel completion distance [mm]> or <completion time [sec]>, the completion condition will be ignored. For the setting to ignore both completion conditions, press control continues until an interrupt occurs with ON BREAK DO, etc., or the program is stopped by STOP, BREAK, etc.
The maximum value of the motion speed is limited t by Override and SPEED.

System version

X8KCS-0 2I or later

PRINT.....CR

Purpose

The PRINT command outputs data to a specified communications channel.
This outputs a CR (carriage return) code to a communication channel.

Type

I/O (Input/output) control command

Format

PRINT [{ TP | IP1 | IP2 | … | IP8 } ,] {<character string> | <expression>} [, {<character string> | <expression>}] … [, CR]

Examples

PRINT “X =“, A1.X
PRINT IP1, K, CR

Analysis and advice

The PRINT command is used to output data to a communication channel.
One of TP, IP1, IP2, ..., and IP8 is selected as the communication channel.
TP is a communication channel for the teach pendant only.
If you do not specify a communication channel in your PRINT statement, data will be output to the teach pendant communication channel.
When a PRINT command is executed, the data will be output to the specified communication channel. Data contained in a <character string> will be output as it is.
Data included in an <expression> will be output in solid blocks having a fixed length of 12 characters aligned on the right. If the value in the expression is an integer, a maximum of 10 digits in base 10 format is output. If the value is a real number, a maximum of 7 digits with 4 digits for the integer part and 3 digits for the decimal part (8 digits including the decimal point) is output. A one character space is provided in front of the number for a plus or minus sign, although the sign itself is omitted when it is +. Numbers will be pushed over to the right in the 12 character space, and any unused spaces will be left blank.
If you output data to the teach pendant, that data will be displayed on the teach pendant.
All data is in ASCII code. Should you write CR (Carriage Return) (0DH)at the end of the PRINT command, output from a subsequent PRINT command will be displayed on the next line.
If output to TP, the screen displays a carriage return.
For information of data communication, refer to the “Communication Manual”.

After the moving arm has stopped, this command cannot be executed.
Also, undefined variables cannot be used, and so be sure to declare variables within the program.

Sample

```
PROGRAM PRINTSMPL
  N1 = 0                                  ("DIM N1 AS INT" is also possible.)
  N2 = 0                                  ("DIM N2 AS INT" is also possible.)
  N3 = 0                                  ("DIM N3 AS INT" is also possible.)
  PRINT TP, "*** INPUT N1, N2, N3 ***"
  INPUT TP, N1, N2, N3                    This program will read in three values (N1, N2
  PRINT(N1 + N2 + N3) / 3, CR             and N3) from the teach pendant, find the
END                                       average, and display the average on the teach
                                          pendant.
```

```
PROGRAM CRSAMPLE
  N = 0                                   ("DIM N AS INT" is also possible.)
  PRINT TP, "*** INPUT N ***", CR         Characters are displayed on the teach pendant,
  INPUT TP, N                             and a carriage return is used.
  PRINT TP, N, CR                         The N value is displayed on the teach pendant,
END                                       and a carriage return is used.
```

PROGRAM.....END

Purpose

This shows the program section.

Type

Program control command

Format

PROGRAM <program name> [(<variable name> , ...)]

Examples

PROGRAM SAMPLE
PROGRAM MAIN
PROGRAM SUB1(N1, N2, N3)

Analysis and advice

The name of the program is designated by an identifier in the <program name> specification.
The program text itself is sandwiched between a PROGRAM statement and an END statement.
If there is no END statement, an error occurs.
When designating a sub program, it is necessary to specify an argument in parentheses when required.
For details of sub programs and arguments, see “2.11 Subprograms.”
During execution in cycle operation mode, program execution is ended by an END for the main task that is generated automatically when a program is started. In continuous operation mode, execution continues by returning to the start of the program.
If a program is executed as a subprogram, program processing returns to the main program by an END even without a RETURN statement before END.
After the END statement of the main program is executed, the internal variable value is cleared.

Sample

```
PROGRAM ENDSAMPLE
  MOVE A1                    Everything from the PROGRAM statement to the
  MOVE A2                    END statement will be executed as a single
  MOVE A3                    program.
END
```

PSNCMD

Purpose

Exports the current position in the world coordinate system based on the position command value.

Type

System variable (movement reference command)

Format

PSNCMD

Examples

A1 = PSNCMD
X = PSNCMD.X

Analysis and advice

This exports the current position in the world coordinate system.
PSNCMD can be used in the same way as regular POINT type data, but it is reference only, and substitution is not possible.
If a PSNCMD command is executed while the robot is operating, the command position to the robot when the PSNCMD command was issued is exported as the current position.
The position exported by the PSNCMD command is the command position to the robot that includes the correction values for the conveyor synchronization function and other parameters. During robot operation, be careful because the actual current position of the robot is delayed from the command position.
It is also possible to refer to each element such as PSNCMD.X. For details, see [1] POINT type (position type) variables in "2.6.2 Vector type variables".

Sample

```
PROGRAM PSNCMDSAMPLE
  AA = A
  MOVE A
  ON DIN(1) DO AA = PSNCMD
  MOVE A1
  MOVE A2
  MOVE A3
  MOVE A4
  IGNORE DIN(1)
  PRINT “POSITION DATA =“, AA.X, AA.Y, AA.Z
END
```

Input signal 1 is monitored during movement from A1 to A4, and the current position in the world coordinate system based on the position command value when the signal was turned on is displayed on the teach pendant.

PSNCMDJ

Purpose

Exports the current position of the joint axis based on the position command value.

Type

System variable (movement reference command)

Format

PSNCMDJ

Examples

A1 = PSNCMDJ

X = PSNCMDJ.1

Analysis and advice

This exports the current position at the joint axis.

PSNCMDJ can be used in the same way as regular POINT type data, but it is reference only, and substitution is not possible.

If a PSNCMDJ command is executed while the robot is operating, the command position to the robot when the PSNCMDJ command was issued is exported as the current position.

The position exported by the PSNCMDJ command is the command position to the robot that includes the correction values for the conveyor synchronization function and other parameters. During robot operation, be careful because the actual current position of the robot is delayed from the command position.

It is also possible to refer to each element such as PSNCMDJ.J1. For details, see [4] JOINT type variables in "2.6.2 Vector type variables".

Sample

```
PROGRAM PSNCMDJSAMPLE
  AA = A
  MOVE A
  ON DIN(1) DO AA = PSNCMDJ
  MOVE A1
  MOVE A2
  MOVE A3
  MOVE A4
  IGNORE DIN(1)
  PRINT “POSITION DATA =“, AA.1, AA.2, AA.3
END
```

Input signal 1 is monitored during movement from A1 to A4, and the joint axis current position based on the position command value when the signal was turned on is displayed on the teach pendant.

PSNCMDW

Purpose

Exports the current position in the workpiece coordinate system based on the position command value.

Type

System variable (movement reference command)

Format

PSNCMDW

Examples

A1 = PSNCMDW

X = PSNCMDW.X

Analysis and advice

This exports the current position in the workpiece coordinate system.

PSNCMDW can be used in the same way as regular POINT type data, but it is reference only, and substitution is not possible.

If a PSNCMDW command is executed while the robot is operating, the command position to the robot when the PSNCMDW command was issued is exported as the current position.

The position exported by the PSNCMDW command is the command position to the robot that includes the correction values for the conveyor synchronization function and other parameters. During robot operation, be careful because the actual current position of the robot is delayed from the command position.

It is also possible to refer to each element such as PSNCMDW.X. For details, see [1] POINT type (position type) variables in "2.6.2 Vector type variables".

Sample

```
PROGRAM PSNCMDWSAMPLE
  AA = A
  MOVE A
  ON DIN(1) DO AA = PSNCMDW
  MOVE A1
  MOVE A2
  MOVE A3
  MOVE A4
  IGNORE DIN(1)
  PRINT “POSITION DATA =“, AA.X, AA.Y, AA.Z
END
```

Input signal 1 is monitored during movement from A1 to A4, and the current position in the workpiece coordinate system based on the position command value when the signal was turned on is displayed on the teach pendant.

PSNFBK

Purpose

Exports the current position in the world coordinate system based on the position feedback.

Type

System variable (movement reference command)

Format

PSNFBK

Examples

A1 = PSNFBK

X = PSNFBK.X

Analysis and advice

This exports the current position in the world coordinate system.

PSNFBK can be used in the same way as regular POINT type data, but it is reference only, and substitution is not possible.

If a PSNFBK command was executed during movement by the robot, the current position is taken out based on the position feedback of the robot at the time of the PSNFBK command.

It is also possible to refer to each element such as PSNFBK.X. For details, see [1] POINT type (position type) variables in "2.6.2 Vector type variables".

Sample

```
PROGRAM PSNFBKSAMPLE
 AA = A
 MOVE A
 ON DIN(1) DO AA = PSNFBK
 MOVE A1
 MOVE A2
 MOVE A3
 MOVE A4
 IGNORE DIN(1)
 PRINT “POSITION DATA =“, AA.X, AA.Y, AA.Z
END
```

Input signal 1 is monitored during movement from A1 to A4, and the current position in the world coordinate system based on the position feedback when the signal was turned on is displayed on the teach pendant.

PSNFBKJ

Purpose

Exports the current position of the joint axis based on the position feedback.

Type

System variable (movement reference command)

Format

PSNFBKJ

Examples

A1 = PSNFBKJ

X = PSNFBKJ.1

Analysis and advice

This exports the current position at the joint axis.

PSNFBKJ can be used in the same way as regular POINT type data, but it is reference only, and substitution is not possible.

If a PSNFBKJ command was executed during movement by the robot, the current position is taken out based on the position feedback of the robot at the time of the PSNFBKJ command.

It is also possible to refer to each element such as PSNFBKJ.J1. For details, see [4] JOINT type variables in "2.6.2 Vector type variables".

Sample

```
PROGRAM PSNFBKJSAMPLE
 AA = A
 MOVE A
 ON DIN(1) DO AA = PSNFBKJ
 MOVE A1
 MOVE A2
 MOVE A3
 MOVE A4
 IGNORE DIN(1)
 PRINT “POSITION DATA =“, AA.1, AA.2, AA.3
END
```

Input signal 1 is monitored during movement from A1 to A4, and the joint axis current position based on the position feedback when the signal was turned on is displayed on the teach pendant.

PSNFBKW

Purpose

Exports the current position in the workpiece coordinate system based on the position feedback.

Type

System variable (movement reference command)

Format

PSNFBKW

Examples

A1 = PSNFBKW

X = PSNFBKW.X

Analysis and advice

This exports the current position in the workpiece coordinate system.

PSNFBKW can be used in the same way as regular POINT type data, but it is reference only, and substitution is not possible.

If a PSNFBKW command was executed during movement by the robot, the current position is taken out based on the position feedback of the robot at the time of the PSNFBKW command.

It is also possible to refer to each element such as PSNFBKW.X. For details, see [1] POINT type (position type) variables in "2.6.2 Vector type variables".

Sample

```
PROGRAM PSNFBKWSAMPLE
 AA = A
 MOVE A
 ON DIN(1) DO AA = PSNFBKW
 MOVE A1
 MOVE A2
 MOVE A3
 MOVE A4
 IGNORE DIN(1)
 PRINT “POSITION DATA =“, AA.X, AA.Y, AA.Z
END
```

Input signal 1 is monitored during movement from A1 to A4, and the workpiece coordinate current position based on the position feedback when the signal was turned on is displayed on the teach pendant.

PULOUT( )

Purpose

The PULOUT command directs the controller to output an external signal as a pulse.

Type

I/O (Input/output) control command

Format

PULOUT(<signal name> [, <signal name>] …)

Examples

PULOUT(1, 2, -3)

PULOUT(J, J + 1, J + 2)

Analysis and advice

This command outputs the specified output signal in pulses.

Specify the pulse width by the system variable PULTIME. (The default is 0.2 seconds.)

<signal name> is to contain the number of the signal to be output. If the sign of <signal name> is positive, the signal will be modulated as OFF ~ ON ~ OFF. If the sign of <signal name> is negative, the signal will be modulated as ON ~ OFF ~ ON. Should the signal be ON and the sign of the <signal name> be positive, the signal will not be modulated. Likewise, should the signal be OFF and sign of the <signal name> be negative, the signal will not be modulated. Up to ten <signal name> designations can be made with one PULOUT command.

You may use constants, variables or calculations for the <signal name>. However, you may not use vector type data.

By using an ENABLE NOWAIT statement, it is possible to output pulse signals in parallel with (at the same time as) such operations as robot movement and processing of other, non-pulse output signals.

Should a DISABLE NOWAIT statement be in effect, processing of any commands which follow a PULOUT command will not begin until the pulse signal is completely output.

When the same signal is output consecutively after the execution of the PULOUT command while an ENABLE NOWAIT statement is in effect, the pulse output is not guaranteed.

Sample

```
PROGRAM PULOUTSMPL
  DISABLE NOWAIT
  FOR K = 1 TO 16
    PULOUT(K)
  NEXT K
END
```

Output Signals 1 to 16 will turn ON one after the other at an interval of 0.2 second.

PULTIME

Purpose

This specifies the PULOUT time.

Type

System variable (Other)

Format

PULTIME = <time>

Examples

PULTIME = 1.0

Analysis and advice

This specifies the width of the pulses output by PULOUT in second units.
The valid range is 0.2 to 300 seconds, and the default value is 0.2 second.
If a value is set outside the valid range, it is processed as an upper limit or lower limit value.

Sample

```
PROGRAM MAIN
  PULTIME = 1.0
  PULOUT(1)          Turns on output signal 1 for 1 second.
  MOVE P1            The next command is started before PULOUT is
END                  completed.
```

RCYCLE:

Purpose

RCYCLE is a label used for cycle resetting.

Type

Program control command

Format

RCYCLE:

Examples

RCYCLE:

Analysis and advice

Define the line on which the program resumes upon execution of a cycle reset.
If a program has been executed and a line labeled RCYCLE has also been executed, the program resumes from the line labeled RCYCLE upon cycle reset. If it has not been executed, the program resumes from the first line of the program.
When using the RCYCLE command, be sure to observe the following cautions.
You can only use RCYCLE once in the main program.
Be sure to program so that at least one (1) line describing “RCYCLE :” is executed.
This function cannot be used for the multitask program. An error occurs at cycle reset.
For the ON ~ DO command, this function cannot be used during execution of the DO statement. An error occurs at cycle reset.
For more information on cycle resetting, refer to the “TP5000 Operation Manual”.

Sample

```
PROGRAM RCYCLESMPL
  COUNT = 0
  RCYCLE:
    MOVE A1
    MOVE A2
    COUNT = COUNT + 1
  IF COUNT <= 10 THEN GOTO RCYCLE
END
```

Here, even should execution be interrupted partway through, operation can be continued since the value of counter COUNT is preserved.

REACH

Purpose

This determines whether movement is possible at the specified position data.

Type

Vector variable calculation function

Format

REACH(<position data>)

Examples

IF REACH(P1) THEN MOVE P1

Analysis and advice

This function returns 1 when movement is possible to the specified position data, and it returns 0 when movement is not possible.

The determination of whether movement is possible is an evaluation using the MOVE command. The path information is not evaluated. For this reason, even if position data was evaluated as 1 by this function, a movement range exceeded error can still occur using MOVES, MOVEC, …, or MOVEJ.

Sample

```
PROGRAM MAIN
 X = 0.0                          (“DIM X AS REAL” is also possible.)
 Y = 0.0                          (“DIM Y AS REAL” is also possible.)
 Z = 0.0                          (“DIM Z AS REAL” is also possible.)
 C = 0.0                          (“DIM C AS REAL” is also possible.)
 INPUT X, Y, Z, C
 P1 = POINT(X, Y, Z, C)
 IF REACH(P1) THEN                Moves if movement is possible to the entered
  MOVE P1                         position, and if movement is not possible, an
 ELSE                             error message is output.
  PRINT “OUT OF RANGE”, CR
 ENDIF
END
```

READY

Purpose

The READY command returns the robot to its mechanical origin.

Type

Library function (SCOL.LIB)

Format

READY

Examples

READY

Analysis and advice

The READY command moves each axis of the robot to its mechanical origin.
Note that this command will not work if you do not have the file SCOL.LIB in the controller ROM drive.
In the horizontal rotating type robot, the axes are moved in the order of axis 3, axis 4, axis 2, axis 1, and axis 5. Note that the machine zero point of axis 3 is near the lower motion limit.
SCOL.LIB can be edited to change to movement in any selected order or to any selected position.

Sample

```
PROGRAM READYSMPL
  READY                    This program will return the robot to its
END                        mechanical origin.
```

REAL( )

Purpose

The REAL command changes a numerical value into a real number.

Type

Conversion function

Format

REAL(<expression>)

Examples

AK = REAL(-20)

N = REAL(K)

J1 = K - REAL(N - 28)

Analysis and advice

The REAL command converts the number or calculation result in the brackets ( ) to a real number.

This command is used to specify the data type for a variable as a real number value.

You may use a constant, variable or result of calculation for the <expression> term.

However, you may not use vector data.

The REAL command must be used in an expression.

Sample

```
PROGRAM REALSAMPL
  K = REAL(0.12345)                    Defines the variable K as real number data.
  PRINT TP, K, CR
END
```

REMAIN

Purpose

The REMAIN statement is used to refer to the amount of a motion remaining to be completed.

Type

System variable (movement reference command)

Format

REMAIN

Examples

K = REMAIN
ON REMAIN <= 50 DO DOUT(1)

Analysis and advice

The REMAIN statement can be used to see what percentage of a robot motion remains to be completed.
The “amount of motion remaining” is defined as the percentage of a motion not yet completed by the robot with respect to the total distance to be covered by that motion. Calculations for the amount of motion remaining are carried out for the axis that has the greatest distance to travel. REMAIN returns a real number.
By combining the REMAIN statement with an ON command, the robot can be made to send out signals while a motion is still in progress.
This statement must be used in an expression.

The amount of motion referenced with the REMAIN command is the position commanded to the robot. Note that the current position of the robot has a delay to the commanded position while the robot is moving.
Be careful because == can't be used for the comparative operator
Because the path orbit formation wait to P2 has occurred from P1, the following example becomes an infinite loop. By replacing with the WAIT sentence, it is possible to avoid an infinite loop.

```
ENABLE PASS                          ENABLE PASS
PASS = 50                            PASS = 50
MOVE P1                    →         MOVE P1
LOOP1:                               WAIT REMAIN < 5
IF REMAIN > 5 THEN GOTO LOOP1        MOVE P2
MOVE P2
```

Sample

```
PROGRAM REMAINSMPL
  ENABLE NOWAIT
  ON REMAIN <= 50 DO DOUT(1)
  MOVE A1
  ON REMAIN <= 20 DO DOUT(2)
  MOVE A2
END
```

When the robot hand is 50% or more of the way to point A1, Signal 1 will be output.
When the robot hand is 80% or more of the way to point A2 (or, in other words, when 20% or less of the motion remains to be completed), Signal 2 will be output.

REMAINT

Purpose

The REMAINT statement is used to refer to the amount of time remaining before a motion is to be completed.

Type

System variable (movement reference command)

Format

REMAINT

Examples

K = REMAINT

ON REMAINT <= 1 DO DOUT(1)

Analysis and advice

The REMAINT statement can be used to see how much time remains before a certain motion will be completed.

Remaining time is given as a real number in units of seconds. The remaining time will become 0 when the robot has completed final positioning for that movement.

By combining the REMAINT statement with an ON command, the robot can be made to send out signals while a motion is still in progress.

This statement must be used in an expression. Be careful because == can't be used for the comparative operator.

Because the path orbit formation wait to P2 has occurred from P1, the following example becomes an infinite loop. By replacing with the WAIT sentence, it is possible to avoid an infinite loop.

```
ENABLE PASS                          ENABLE PASS
PASS = 50                            PASS = 50
MOVE P1                    →         MOVE P1
LOOP1:                               WAIT REMAINT < 1
IF REMAINT > 1 THEN GOTO LOOP1       MOVE P2
MOVE P2
```

Sample

```
PROGRAM REMAINTSMPL
  ENABLE NOWAIT
  ON REMAINT <= 1 DO DOUT(1)          Signal 1 will be output one second before the
  MOVE A1                             robot reaches point A1.
  MOVE A2
END
```

REMARK

Purpose

The REMARK statement is used to mark comments.

Type

Program control command

Format

REMARK [<comment>]

Examples

REMARK *** SCOL SAMPLE ***

Analysis and advice

Comments are used in the program to make it easier to read and understand.
REMARK statements themselves are interpreted as comments.
The comments are not executed.
The symbol (') has the same meaning as the REMARK statement. When writing a comment following other commands, this symbol (') is used. The characters following the symbol (') are all interpreted as comments.

Sample

```
PROGRAM REMARKSMPL
  REMARK ***SAMPLE PROGRAM***          “MOVES TO A1” is programmed as the
  MOVE A1 ’MOVES TO A1                 comment.
  MOVE A2
END
```

REPLACE( )

Purpose

This returns a character string where a specified part of a character string is replaced by a specified partial character string.

Type

Character string function

Format

REPLACE(<character string expression>, <character string expression>, <character string expression>)

Examples

TXT = REPLACE(“Shopping”, “p”, “o”) ..... TXT is a character string variable

Analysis and advice

REPLACE (target character string, search character string, replacement character string)

This returns a character string where the search character string from the target character string is replaced with the replacement character string.

The replacement count cannot be specified. All replaceable locations are replaced.

If the search character string length or replacement character string length is 0, the target character string is returned unchanged.

Sample

```
GLOBAL
  DIM TXT AS STRING(10)
END
PROGRAM MAIN
  TXT = REPLACE(“Shopping”, “p”, “o”)
  PRINT TP, TXT, CR
END
```

Character string “Shoooing” is returned with the “p” of “Shopping” replaced with “o”.

RESET

Purpose

The RESET command is used to reset certain controller conditions such as the state of output signals.

Type

Program control command

Format

RESET <Status [, <Status>]

Examples

RESET DOUT
RESET RESUME

Analysis and advice

The RESET command is used to reset the state of such things as the controller output signals. The following three can be specified as < Status >.

(1) DOUT

DOUT will turn OFF all user output signals.

(2) RESUME

RESUME will reset the robot movement suspended by a BREAK command in an ON construct. After resetting, the movement interrupted by the BREAK command cannot be resumed by the RESUME command.

(3) TORQUE (System version: X8KCS-03C or higher)

This returns all torque limits to the default status.

Sample

```
PROGRAM RESETSMPL
 RESET DOUT                    This program will turn all user output signals
END                            OFF.
```

RESUME

Purpose

The RESUME command restarts robot movement interrupted by a BREAK command.

Type

Program control command

Format

RESUME

Examples

RESUME

Analysis and advice

The RESUME command is used to restart robot motion suspended by a BREAK command in an ON construct.
Movement is restarted (resumed) from the location the robot movement was suspended. Therefore, should you restart in the circular interpolation mode, the path the robot takes may vary by quite a bit depending on the relation between the current position, the interpolation points and the destination.
When multiple BREAK commands have been executed (and are still in effect), only the movement interrupted by the last BREAK command may be resumed.
Should you execute a RESET RESUME statement, the suspended motion will be reset.
The suspended motion will also be reset should a PAUSE command be executed by the ON command.
This command is effective only in the DO statement. If executed in a statement other than the DO statement, this command is ignored.

Sample

```
PROGRAM RESUMESMPL
  ENABLE NOWAIT
  REMARK *** MAIN PROGRAM ***
  ON DIN(24) BREAK DO BREAKSUB
  MOVE A1
  MOVE A2
  MOVE A3
  WAIT MOTION >= 100
  IGNORE DIN(24)
END
PROGRAM BREAKSUB
  REMARK *** SUBROUTINE ***
  WAIT DIN(-24)
  RESUME
END
```

Should something go wrong with the system and Input Signal 24 turn ON, the controller will stop robot movement immediately and shift program execution to subprogram BREAKSUB.

The subprogram BREAKSUB will wait until Input Signal 24 turns OFF. When the error is resolved, the interrupted motion will be resumed.

RETURN

Purpose

The RETURN statement directs program execution to return to the main program from a subprogram.

Type

Program control command

Format

RETURN

Examples

RETURN

Analysis and advice

The RETURN statement is used to return the program from the subprogram to the main program.

Even should you forget to include a RETURN statement in your subprogram, the controller will return the control to the main program through the execution of the END statement.

You will get an error should you put a RETURN statement in your main program.

Sample

```
PROGRAM MAIN
  K = 0                                  ("DIM K AS INT" is also possible.)
  RETURNSMPL(5, K)
  PRINT TP, K, CR
END
PROGRAM RETURNSMPL(N, K)                 This program will take argument N, multiply it
  K = N * N                              by itself, and send the result back to the main
  RETURN                                 program as argument K.
END
```

RIGHTY

Purpose

RIGHTY is a system constant used to change over the configuration of the robot to a right handed system.

Type

System constant

Format

RIGHTY

Examples

CONFIG = RIGHTY
MOVE A1 WITH CONFIG = RIGHTY

Analysis and advice

RIGHTY is used in conjunction with CONFIG in order to set the robot configuration to the right-handed system.
As a system constant, RIGHTY has the value of 2. If you wanted to, you could use it in your program as a constant having the value 2. However, this is not a good idea since it makes your program unnecessarily complicated.
You cannot substitute into system constants including RIGHTY.
For information on robot configuration, see the CONFIG command.

Sample

```
PROGRAM RIGHTYSMPL
  CONFIG = RIGHTY          This program will set the robot configuration to
  MOVE A1                  the right-handed system before moving the
  MOVE A2                  robot on its way.
END
```

SAVEF1~8

Purpose

These are REAL type system variables that can be retained without being cleared when the main power is shut off.

Type

System variable (Other)

Format

SAVEF1 = <expression>
SAVEF2 = <expression>
SAVEF3 = <expression>
SAVEF4 = <expression>

Examples

SAVEF1 = 1.0
A = SAVEF2

Analysis and advice

When the main power is shut off, the values for SAVEF1 to SAVEF8 are saved to the backup memory.
The REAL type variable can use the four memory slots from SAVEF1 to 8.
When the power is turned on the next time, the value saved to the backup memory is restored.
The values handled as real numbers are the range of numerical values with absolute value from approximately $5.87\times10^{(-39)}$ (2^{-127}) to 6.80×10^{38} ((2^{23}-1)$\times2^{106}$).
The values for SAVEF1 to SAVEF8 are cleared when the following operations are performed.

- Program select
- Program reset

Sample

```
PROGRAM SAVEFSAMPLE
  FOR A = 1 TO 100
    PRINT “SAVEF1 =“, SAVEF1, CR
    DELAY 0.5
    SAVEF1 = SAVEF1 + 1.0
  NEXT A
END
```

SAVEI1~8

Purpose

These are INT type system variables that can be retained without being cleared when the main power is shut off.

Type

System variable (Other)

Format

SAVEI1 = <expression>
SAVEI2 = <expression>
SAVEI3 = <expression>
SAVEI4 = <expression>

Examples

SAVEI1 = 10
A = SAVEI2

Analysis and advice

When the main power is shut off, the values for SAVEI1 to SAVEI8 are saved to the backup memory.
The INT type variable can use the four memory slots from SAVEI1 to 8.
When the power is turned on the next time, the value saved to the backup memory is restored.
The values handled as integers are the range of numerical values from -2147483648 to +2147483647.
The values for SAVEI1 to SAVEI8 are cleared when the following operations are performed.

- Program select
- Program reset

Sample

```
PROGRAM SAVEISAMPLE
  FOR A = 1 TO 100
    PRINT “SAVEI1 =“, SAVEI1, CR
    DELAY 0.5
    SAVEI1= SAVEI1 + 1.0
  NEXT A
END
```

SEGMENT

Purpose

SEGMENT is a system constant which is used to refer to the system operating mode.

Type

System constant

Format

SEGMENT

Examples

IF MODE == SEGMENT THEN RETURN

Analysis and advice

SEGMENT is used along with the MODE command to refer to the system operation mode. When MODE = = SEGMENT, the system is operating in the segment operation mode.

As a system constant, SEGMENT has a value of 2. If you wanted to, you could use it in your program as a constant having the value 2. However, don't do it since it will make your program hard to understand.

You cannot substitute into system constants.

The monitor command MODE MOTION can be used to specify segment operation.

For information on operating modes, see the MODE command.

Sample

```
PROGRAM MAIN
  SEGMENTSAMPLE
END
PROGRAM SEGMENTSAMPLE
  IF MODE == SEGMENT THEN RETURN        If the mode is set to segment operation,
    MOVE A1                             program execution will return to the main
    MOVE A2                             program without execution.
    MOVE A3
END
```

SELECT.....CASE.....ENDSELECT

Purpose

This performs multi-branching based on the variable value.

Type

Program control command

Format

```
SELECT <variable(INT type)>
  CASE <value(integer value)>
    <process>
  CASE <value, value, … >
    <process>
  [ CASEELSE ]
    [ <process> ]
ENDSELECT
```

Examples

```
SELECT A
 CASE 0
  PRINT “A is 0”, CR
 CASE 1, 2, 3, 4, 5
  PRINT “1 <= A <= 5”, CR
ENDSELECT
```

Analysis and advice

This performs multi-branching based on the variable value.

The value is an integer, and up to 10 values can be specified using comma delimiters for a single CASE.

Up to 64 values can be written using CASE.

CASEELSE (when the above CASE does not apply) can also be used. If CASEELSE is not needed, it may be omitted.

A GOTO command from within CASE can be used to jump outside the range enclosed from SELECT to ENDSELECT.

Sample

```
PROGRAM MAIN
  A = 0
  INPUT A
  SELECT A
    CASE 0
      PRINT “A is 0”, CR
    CASE 1, 2, 3, 4, 5
      PRINT “1 <= A <= 5”, CR
    CASE 6, 7, 8, 9, 10
      PRINT “6 <= A <= 10”, CR
  ENDSELECT
  B = 0
  INPUT B
  SELECT B
    CASE -1, 1
      PRINT “B is -1 or 1”, CR
    CASEELSE
      PRINT “IN CASEELSE”, CR
  ENDSELECT
END
```

SETGAIN( )

Purpose

The SETGAIN command is used to specify whether the gain (for servo control) is to be ON or OFF for each axis.

Type

Library function (SCOL.LIB)

Format

SETGAIN(<integer>, <integer>, <integer>, <integer>, <integer>)

Examples

SETGAIN(0, 0, 1, 0, 0)

Analysis and advice

The SETGAIN command is used to specify whether the gain (servo control) of each axis is to be ON or OFF.

The SETGAIN command will take effect upon the completion of the robot motion being performed at the time.

For more information on gains, see the “GAIN” command.

The SETGAIN command will not work if you do not have the file SCOL.LIB in the controller ROM drive.

With this command, a system constant cannot be turned on/off. Two commands ONGAIN and OFFGAIN are provided in the library file for turning on and off the gain.

Sample

```
PROGRAM SETGAINSMPL
  MOVE A1
  SETGAIN(0, 0, 1, 0, 0)
  MOVE A2
  SETGAIN(1, 1, 1, 1, 1)
  MOVE A3
END
```

This program turns off all gains except that the Axis 3 when the robot reaches point A1. Program will then turn on all gains when the reaches point A2.

SGN( )

Purpose

The SGN function returns the sign of a number value.

Type

Calculator command (Calculation function)

Format

SGN(<expression>)

Examples

S = SGN(-20.345)

N = ABS(K) * SGN(L)

Analysis and advice

The SGN function will return the sign of the <expression> in the brackets.

SGN will return 1 if the <expression> is positive, –1 if negative, and 0 if 0.

You may use constants, variables or equations for the <expression> term. However, you may not use vector type data.

The SGN command must be used in an expression.

Sample

```
PROGRAM MAIN
 K = 0                                  ("DIM K AS INT" is also possible.)
 SGNSAMPLE(5, 3, K)
 PRINT TP, K, CR
END                                     This subprogram will take in two arguments K1
PROGRAM SGNSAMPLE(K1, K2, K)            and K2 and, as argument K, will return a 1 if K1
 K = SGN(K1 - K2)                       is bigger than K2, a 0 if the same, and a –1 if
 RETURN                                 smaller to the main program.
END
```

SHIFT( )

Purpose

This specifies the offset amount in the tool direction.

Type

Vector variable calculation function

Format

SHIFT(<expression>, <expression>, <expression>, <expression>, <expression>)

Examples

MOVE P1 + SHIFT(100, 0, 0)

P2 = P3 - SHIFT(0, 50, 20)

Analysis and advice

This is a function that specifies the offset amount in the tool direction.

An amount can be added or subtracted to the position data to specify the position that was offset.

The <expression> refers to the X, Y, Z, C, and T elements in order from the left. X, Y, Z, C, and T are specified in millimeter or degree units.

Sample

```
PROGRAM MAIN
  MOVE P1 - SHIFT(100, 0, 0)
  MOVES P1
  MOVES P1 - SHIFT(100, 0, 0)
END
```

Approaches P1 from 100 mm in front of the X direction of the tool.

SIN( )

Purpose

This function returns the sine of an entered value.

Type

Calculator command (Calculation function)

Format

SIN(<expression>)

Examples

K = SIN(60)

J1 = 1 - SIN(180 - D)

Analysis and advice

This function returns the sine of the value in the brackets ( ).

Calculations are handled in units of degrees.

You may enter a constant, variable or calculation for the <expression> term.

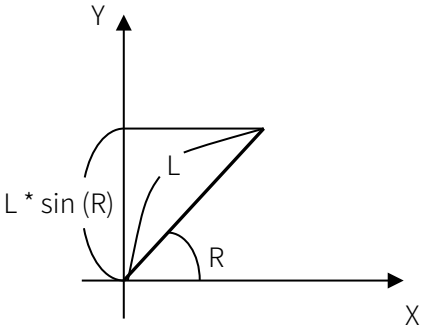
However, you may not enter vector type data.

This command must be used in an equation.

Sample

```
PROGRAM MAIN
 Y = 0.0
 SINSAMPLE(2.0, 60.0, Y)
 PRINT TP, Y, CR
END
PROGRAM SINSAMPLE(L, R, Y)
 LOOP:
 IF R > 180 THEN R = R - 360
 IF R < -180 THEN R = R + 360
 IF R > 180 OR R < -180 THEN GOTO LOOP
 Y = L * SIN(R)
 RETURN
END
```

("DIM Y AS REAL" is also possible.)



Given (as arguments) a line segment with a length L and forming an angle R with the X-axis, this program finds the length of the Y-axis component of the line segment L and sends it back to the main program as argument Y.

SLOWDOWN

Purpose

Specifies whether the slowdown operation is performed.

Type

System switch

Format

SLOWDOWN

Examples

DISABLE SLOWDOWN

ENABLE SLOWDOWN

Analysis and advice

This specifies the slowdown operation.

In the slowdown operation, the speed during movement of the current operation can be adjusted (slowed).

The ENABLE and DISABLE commands are used to turn system switches on and off.

The operation commands following ENABLE SLOWDOWN change the speed during movement based on the specified parameter.

The slowdown operation is disabled by DISABLE SLOWDOWN.

In the initial state, DISABLE SLOWDOWN is set.

Sample

```
PROGRAM SAMPLE
  MOVE A1
  SLOWDOWN = 25
  SLWSPD = 50
  ENABLE SLOWDOWN
    MOVE A2
    SLOWDOWN = 50
    MOVE A3
    SLOWDOWN = 75
    MOVE A1
  DISABLE SLOWDOWN
  MOVE A2
END
```

SLOWDOWN

Purpose

This sets the parameters for the slowdown operation.

Type

System variable (movement condition command)

Format

SLOWDOUN = <expression>

Examples

SLOWDOWN = 80

Analysis and advice

This is a system constant that specifies the slowdown operation parameter.
In the slowdown operation, the speed during movement of the current operation can be adjusted (slowed).
The timing when slowdown is started from the speed specified by the SPEED variable is specified using the system variable SLOWDOWN. The slowdown operation parameters are specified as a percentage of the travel amount of the robot for the operation. If the travel amount exceeds the percentage specified by the system variable SLOWDOWN, the robot starts to slow down to the speed specified by SLWSPD.

Sample

```
PROGRAM SAMPLE
  MOVE A1
  SLOWDOWN = 25
  SLWSPD = 50
  ENABLE SLOWDOWN
    MOVE A2
    SLOWDOWN = 50
    MOVE A3
    SLOWDOWN = 75
    MOVE A1
  DISABLE SLOWDOWN
  MOVE A2
END
```

SLOWSTART

Purpose

Slow movement start

Type

System variable (movement condition command)

Format

SLOWSTART = <expression>

Examples

SLOWSTART = 20

MOVE P1 WITH SLOWSTART = 10

Analysis and advice

This specifies slow movement.

When slow movement start is enabled, movement is executed at the value that was set to SLWSPD until the movement execution amount reaches the value that was set in SLOWSTART.

Slow movement start is enabled by setting the system switch SLOWSTART to ENABLE, and it is disabled by setting the switch to DISABLE. The initial status is DISABLE.

SLOWSTART can be specified in a WITH clause to enable slow movement start for the movement regardless of the setting status of the system switch.

Sample

```
PROGRAM MAIN
  SLWSPD = 50                      Moves at a speed equivalent to an override of
  MOVE P1 WITH SLOWSATRT = 20      50% up to 20% of the movement.
END
```

SLWSPD

Purpose

Specifies the speed at the low-speed setting for the slowdown operation.

Type

System variable (movement condition command)

Format

SLWSPD = <expression>

Examples

SLWSPD = 20

Analysis and advice

This is a system variable for specifying the speed at the low-speed setting for the slowdown operation as a percentage of the highest speed.
SLWSPD is specified as a positive real number value. If a value of 0 or less is specified, this is treated as a value of 1.
The speed cannot be increased in the slowdown operation. Therefore, if a value higher than the speed that was set in the SPEED variable is specified, the value is invalid.
You may use a constant, variable or result of calculation for the <expression> term. However, you may not use vector data.
The initial value of SLWSPD is 100%.

Sample

```
PROGRAM SAMPLE
  SLWSPD = 50
  SLOWDOWN = 80
  SLWSPD = 30
  MOVE A1
  ENABLE PASS
  ENABLE SLOWDOWN
    MOVE A2
  DISABLE PASS
  DISABLE SLOWDOWN
    MOVE A3
  ENABLE SLOWDOWN
    MOVE A4
  DISABLE SLOWDOWN
END
```

SMOOTH (Option function)

Purpose

This function designates a smooth motion.

Type

System switch

Format

SMOOTH

Examples

DISABLE SMOOTH

ENABLE SMOOTH

Analysis and advice

The motion command specified as SMOOTH allows the robot to move to the target position without decelerating. With the successive motion command, the robot starts moving without accelerating, irrespective of presence or absence of SMOOTH.

To turn on and off the system switch, use the ENABLE and DISABLE commands.

With the ENABLE SMOOTH command, the smooth motion starts. With the DISABLE SMOOTH command, the smooth motion is cancelled.

The initial setting is DISABLE SMOOTH.

The smooth function is effective only for the interpolation commands (MOVES, MOVEC).

During the ENABLE SMOOTH mode, COARSE is automatically selected for the positioning accuracy.

If the speed at smooth connection has not reached the specified speed for that motion command, the robot accelerates (or decelerates) to the specified speed at maximum acceleration.

If the smooth motion command is specified while the C-axis travel distance of the smooth motion command is not enough, compared with the X, Y and Z travel distances, an error occurs.

When the movement command including the travel of the T axis is specified as SMOOTH, an error occurs.

If the DISABLE SMOOTH motion command does not have a sufficient travel distance for deceleration and stop, speed control at deceleration and stop cannot be guaranteed.

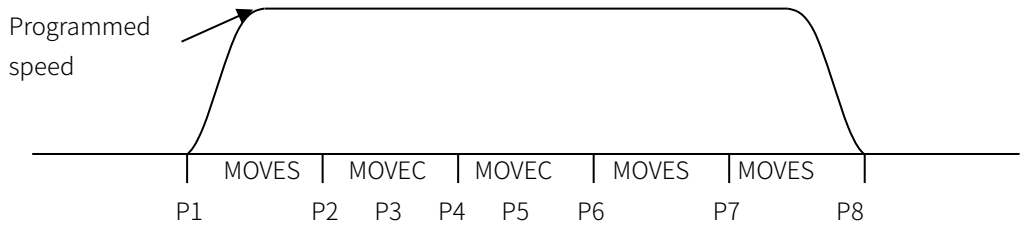
The PASS function cannot be used together with the SMOOTH function. Also, the PASS motion cannot be connected with the SMOOTH motion. (See the sample programs SMPL04 and SMPL05.)

This function is initially set to disabled because it may affect the life of the machine depending on the setting conditions. Prior to using this function, please contact us in advance.

Sample

```
PROGRAM SMPL01
   MOVE P1
  ENABLE SMOOTH
   MOVES P2
   MOVEC P3 P4
   MOVEC P5 P6
   MOVES P7
  DISABLE SMOOTH
   MOVES P8
END
```

By connecting teaching points P2, P4, P6 and P7 by smooth motion, the robot decelerates and stops at point P8.



Bad example 1

```
 PROGRAM SMPL02
  PASS = 50
  ENABLE SMOOTH
    MOVES P1
    MOVES P2
    MOVES P3
  ENABLE PASS                    ← An alarm is generated.
    MOVES P4                        2–039 “PASS command prohibit”
  DISABLE SMOOTH
    MOVES P5
    MOVES P6
    MOVES P7
  DISABLE PASS
    MOVES P8
 END
```

If ENABLE PASS is specified in the ENABLE SMOOTH mode, an alarm occurs and the robot stops moving.

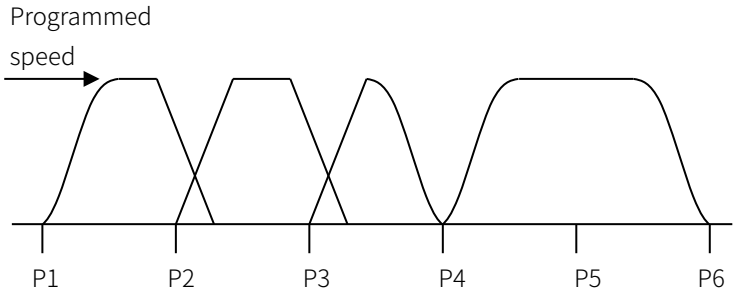
Bad example 2

```
PROGRAM SMPL03
  PASS = 50
  ENABLE PASS
    MOVES P1
    MOVES P2
    MOVES P3
  ENABLE SMOOTH                    ← An alarm is generated.
    MOVES P4                           2–040 “SMOOTH command prohibit”
  DISABLE PASS
    MOVES P5
    MOVES P6
    MOVES P7
  DISABLE SMOOTH
    MOVES P8
END
```

If ENABLE SMOOTH is specified in the ENABLE PASS mode, an alarm occurs and the robot stops moving.

Changeover from PASS to SMOOTH

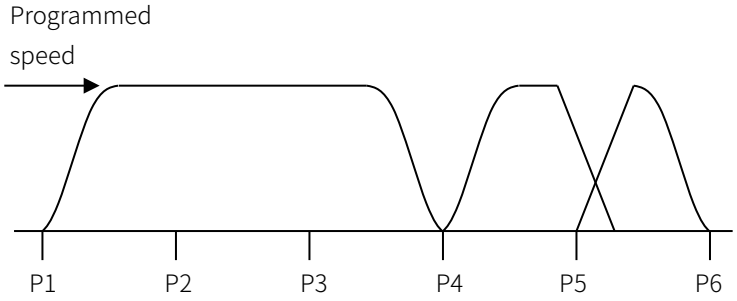
```
PROGRAM SMPL04
 PASS = 50
   MOVE P1
 ENABLE PASS
   MOVES P2
   MOVES P3
 DISABLE PASS
 ENABLE SMOOTH
   MOVES P4
   MOVES P5
 DISABLE SMOOTH
   MOVES P6
END
```



As the robot slows down and stops at the time of DISABLE PASS, SMOOTH designation for teaching point P4 becomes invalid.
That is, P2 and P3 are connected by short-cut, P4 by deceleration and stop, and P5 by smooth motion. At P6, the robot slows down and stops. When this happens, alarm 1–018 of “SMOOTH connect invalid” is generated at P4.

Changeover from SMOOTH to PASS

```
PROGRAM SMPL05
 PASS = 50
   MOVE P1
 ENABLE SMOOTH
   MOVES P2
   MOVES P3
 DISABLE SMOOTH
 ENABLE PASS
   MOVES P4
   MOVES P5
 DISABLE PASS
   MOVES P6
END
```



As the robot slows down and stops at the time of DISABLE SMOOTH, PASS designation for teaching point P4 becomes invalid.
That is, P2 and P3 are connected by smooth motion, P4 by deceleration and stop, and point P5 by short-cut. At P6, the robot slows down and stops. When this happens, alarm 1–017 of “PASS connect invalid” is generated at P4.

SPEED

Purpose

SPEED is a system variable used to specify the movement speed of the robot.

Type

System variable (movement condition command)

Format

SPEED = <expression>

Examples

SPEED = 50

MOVE A1 WITH SPEED = SPEED * 0.8

Analysis and advice

SPEED is a system variable which is used to specify the movement speed of the robot. It is expressed in terms of percent of the maximum speed (allowed by the controller). SPEED is specified with a positive integer number. When a numeric value of 0 or less is specified, the specification is treated as 1. When a value of 100 or more is specified, the movement speed is suppressed to the maximum speed designated in the system.

You may use constants, variables or equations for the <expression> term. However, you may not use vector type data.

By referring to this system variable, you can find the speed setting in effect at the time.

The initial setting for SPEED is 100%.

Sample

```
PROGRAM SPEEDSMPL
  SPEED = 50
  MOVE A1
  MOVE A2
  MOVE A3 WITH SPEED = 100
  MOVE A4
END
```

Here, the robot will move to all points at 50% of full speed with the exception of point A3, to which the robot will move at full speed.

SPURT

Purpose

This specifies enable or disable for the acceleration automatic adjustment function of the robot.

Type

System variable (movement condition command)

Format

SPURT = <expression>

Examples

SPURT = ON

Analysis and advice

This system variable specifies enable or disable for the acceleration automatic adjustment function of the robot.
It can be enabled by SPURT = ON and disabled by SPURT = OFF.
The initial status is disabled.
The acceleration automatic adjustment function is enabled for PTP movement (MOVE, MOVEA, MOVEI, MOVEJ). It is disabled for CP movement (MOVES, MOVEC, MOVEW, MOVEWI, MOVET, MOVECV).
This function forecasts the load of the motor due to movement and determines the acceleration. As a result, it is effective for reducing the cycle time in movements with small loads such as movements performed when the arm is folded. On the other hand, for movements with large loads, the cycle time may be unchanged or may be slightly longer compared to when the function is disabled.
When this function is enabled, it improves the overall motor load, the product lifespan may be shortened compared to usage in the normal state. Also, the overshoot and vibrations of the hand during positioning may be larger than the normal state. Be aware that the repeat positioning accuracy and overshoot amount when using this function are outside the scope of the warranty.
If this function is enabled for many movements, a motor load factor error (such as 002-411 Axis1 Motor overload(15m)) may occur.
If that happens, limit the movements where this function is used so that the motor load factor does not exceed the allowable value.

Sample

```
PROGRAM SPURTSMPL
  SPURT = ON                    Specifies enable for the acceleration automatic
  MOVE A1                       adjustment function.
  MOVES A2                      Improves the movement acceleration when
  SPURT = OFF                   moving to A1 for the PTP movement.
  MOVE A3
END
```

SQRT( )

Purpose

The SQRT function will return the square root of a given number.

Type

Calculator command (Calculation function)

Format

SQRT(<expression>)

J1 = SQRT(L1 ^ 2 + L2 ^ 2)

Examples

K = SQRT(60)

J1 = 1 - SIN(180 - D)

Analysis and advice

The SQRT function will return the square root of the value in the ( ) brackets.

You may use a constant, variable or result of calculation for the <expression> term.

However, you may not use vector data.

When the value of <expression> is negative, an error occurs and take care.

This command must be used in an expression.

Sample

```
PROGRAM MAIN
 L = 0.0                                  ("DIM L AS REAL" is also possible.)
 SQRTSAMPLE(3, 5, L)
 PRINT TP, L, CR
END
PROGRAM SQRTSAMPLE(X, Y, L)              This subprogram takes in arguments X and Y
 L = SQRT(X ^ 2 + Y ^ 2)                 (the lengths of the two perpendicular sides of a
 RETURN                                  right triangle), finds the length of the
END                                      hypotenuse of that triangle, and returns that
                                         length as argument L to the main program.
```

STOP

Purpose

The STOP command is used to stop execution of the program.

Type

Program control command

Format

STOP

Examples

STOP

Analysis and advice

The program will stop executing when a STOP command is encountered no matter what the system operating mode is at the time.
There is no way to restart a program thus stopped.
Instead, you have to restart the program all over again.
When the program is restarted, the robot movement is restored from the subsequent step.

Sample

```
PROGRAM STOPSAMPLE
  ENABLE NOWAIT
  ON DIN(10) DO STOP
    MOVE A1
    MOVE A2
    MOVE A3
END
```

This program will stop executing itself should Input signal 10 turn ON while the robot is moving.

STR( )

Purpose

This converts and returns the numerical value passed by the argument to a character string.

Type

Character string function

Format

STR(<expression>)

Examples

A = STR(1234) .....A is a character string variable
B = STR(-0.1234) .....B is a character string variable

Analysis and advice

This converts the numerical value specified in ( ) to a character string.
A space is provided for displaying the sign at the start of the converted character string. If the numerical value is a negative number, "-" is inserted. If the value is a positive number, a single-byte space is inserted.
If the specified numerical value is a real number, it is expressed as a number to 6 decimal places.

Sample

```
GLOBAL
  DIM A AS STRING(10)
END
PROGRAM MAIN
  A = STR(1234)                    Returns the character string "␣1234"
  PRINT TP, A, CR                  (where ␣ indicates a space)
END
```

```
GLOBAL
  DIM B AS STRING(10)
END
PROGRAM MAIN
  B = STR(-0.1234)                 Returns the character string "-0.123400".
  PRINT TP, B, CR
END
```

STRCAT( )

Purpose

This returns a character string formed by joining two character strings passed by an argument.

Type

Character string function

Format

STRCAT(<character string expression>)>, <character string expression>)

Examples

A = STRCAT(“ABCDE”, “FGH”) .....A is a character string variable

B = STRCAT(TXT1, TXT2) .....B, TXT1, and TXT2 are character string variables

Analysis and advice

This returns a character string formed by joining two character strings specified in ( ). This joins the character string on the right side to the character string on the left side in ( ).

Sample

```
GLOBAL
  DIM A AS STRING(10)
END
PROGRAM MAIN
  A = STRCAT(“ABCDE”, “FGH”)
  PRINT TP, A, CR
END
```

Returns the character string “ABCDEFGH” formed by joining “ABCDE” and “FGH”.

```
GLOBAL
  DIM B AS STRING(20)
  DIM TXT1 AS STRING(10)
  DIM TXT2 AS STRING(10)
END
PROGRAM MAIN
  TXT1 = “Shibaura”
  TXT2 = “Machine”
  B = STRCAT(TXT1, TXT2)
  PRINT TP, B, CR
END
```

Returns the character string “ShibauraMachine” formed by joining “TXT1” and “TXT2”.

STRCOMP( )

Purpose

This compares two character strings and returns the comparison result using a value of 0, 1, or -1.

Type

Character string function

Format

STRCOMP(<character string expression>, <character string expression>)

Examples

IF STRCOMP(“ABCDE”, “CD”) == 0
A = STRCOMP(“ABCDE”, “CD”)

Analysis and advice

This compares two character strings specified in ( ) and returns the comparison result using a value of 0, 1, or -1.
STRCOMP(character string 1, character string 2)
0: Character string 1 and character string 2 are equal.
1: Character string 1 comes after character string 2 in the character code order.
-1: Character string 1 comes before character string 2 in the character code order.

Sample

```
PROGRAM MAIN
 IF STRCOMP(“ABCDE”, “CD”) == 0 THEN
  DOUT(1)
 ELSE
  DOUT(2)
 ENDIF
 DELAY 2.0
 RESET DOUT
END
```

If the comparison result of two character strings is 0, output signal 1 turns on.
If the result is not 0, output signal 2 turns on.

```
PROGRAM MAIN
 A = STRCOMP(“ABCDE”, “CD”)
 PRINT TP, A, CR
END
```

Compares “ABCDE” and “CD”.
Because “ABCDE” comes before “CD”, a value of “-1” is returned.

STRLEFT( )

Purpose

This returns a character string having the specified number of characters from the left end of the character string.

Type

Character string function

Format

STRLEFT(<character string expression>, <expression>)

Examples

A = STRLEFT(“ABCDE”, 2) .....A is a character string variable

B = STRLEFT(TXT, N) .....B and TXT are character string variables

Analysis and advice

This returns a character string having the specified number of characters from the left end of the character string specified in ( ).

If the specified number of characters exceeds the character string length, the entire character string is returned.

If 0 or a negative value is set for the number of returned characters, a character string of length 0 (“”) is returned.

Sample

```
GLOBAL
  DIM A AS STRING(10)
END
PROGRAM MAIN
  A = STRLEFT(“ABCDE”, 2)
  PRINT TP, A, CR
END
```

Two characters are specified from the left end of “ABCDE”, and so “AB” is returned.

```
GLOBAL
  DIM B AS STRING(10)
  DIM TXT AS STRING(20)
END
PROGRAM MAIN
  TXT = “Hello World!”
  X = 8
  B = STRLEFT(TXT, X)
  PRINT TP, B, CR
END
```

Eight characters are specified from the left end of "Hello World!", and so "Hello Wo" is returned. (There is a space between "Hello" and "Wo".)

STRLEN( )

Purpose

This returns the length of a specified character string.

Type

Character string function

Format

STRLEN(<character string expression>)

Examples

A = STRLEN(“ABCDE”)

B = STRLEN(TXT) .....TXT is a character string variable

Analysis and advice

This returns the length of a character string (number of characters) specified in ( ).

The return value is given as an integer value.

Sample

```
PROGRAM MAIN
 A = STRLEN(“ABCDE”)
 PRINT TP, A, CR
END
```

Returns “5” for the character string length of “ABCDE”.

```
GLOBAL
  DIM TXT AS STRING(15)
END
PROGRAM MAIN
  TXT = “Hello World”
  B = STRLEN(TXT)
  PRINT TP, B, CR
END
```

Returns “11” for the character string length of “Hello World”.
(There is a space between “Hello” and “World”.)

STRMID( )

Purpose

This returns a specified number of characters from a specified position of the character string.

Type

Character string function

Format

STRMID(<character string expression>, <expression> [, <expression>])

Examples

A = STRMID(“ABCDEFGHIJK”, 3, 2) .....A is a character string variable

Analysis and advice

STRMID (target character string, start position, return character count (can be omitted))

This returns a character string having the specified number of characters from a specified position of the character string specified in ( ).

If the character string length from the start position is shorter than the return character count, the entire character string from the start position to the end is returned.

If the character string length from the start position is larger than the character string length, a character string of length 0 (“”) is returned.

If no return character count is specified, the entire character string from the start position to the end is returned.

Sample

```
GLOBAL
  DIM A AS STRING(15)
END
PROGRAM MAIN
  A = STRMID(“ABCDEFGHIJK”, 3, 2)
  PRINT TP, A, CR
END
```

Returns the two characters “CD” from the 3rd character of the character string “ABCDEFGHIJK”.

```
GLOBAL
  DIM B AS STRING(15)
  DIM TXT AS STRING(15)
END
PROGRAM MAIN
  TXT = “Hello World!”
  B = STRMID(TXT, 7)
  PRINT TP, B, CR
END
```

Returns “World!” as the character string from the 7th character to the end of “Hello World!”

STRREAD( )

Purpose

This takes out a character string having the specified number of characters within the character string.

Type

Character string function

Format

STRREAD(<character string expression>, <expression>)

Examples

A = STRREAD(TXT, 3) .....A and TXT are character string variables

Analysis and advice

This takes out a character string having the specified number of characters from the character string specified in ( ).

After this take-out process, the character string following the specified character count only remains as the target character string.

If the specified number of characters is larger than the character string length, the entire character string is returned.

If 0 or a negative value is set for the number of specified characters, a character string of length 0 (“”) is returned.

Sample

```
GLOBAL
  DIM A AS STRING(10)
  DIM TXT AS STRING(10)
END
PROGRAM MAIN
  TXT = “ABCDEFG”
  A = STRREAD(TXT, 3)          Takes out and returns the three characters
  PRINT TP, A, CR              “ABC” from the character string “ABCDEFG”.
  PRINT TP, TXT, CR             “DEFG” remains for TXT.
END
```

STRRIGHT( )

Purpose

This returns a character string having the specified number of characters from the right end of the character string.

Type

Character string function

Format

STRRIGHT(<character string expression>, <expression>)

Examples

A = STRRIGHT(“ABCDE”, 2) .....A is a character string variable

B = STRRIGHT(TXT, N) .....B and TXT are character string variables

Analysis and advice

This returns a character string having the specified number of characters from the right end of the character string specified in ( ).

If the number of characters exceeds the character string length, the entire character string is returned.

If 0 or a negative value is set for the number of returned characters, a character string of length 0 (““) is returned.

Sample

```
GLOBAL
  DIM A AS STRING(10)
END
PROGRAM MAIN
  A = STRRIGHT(“ABCDE”, 2)
  PRINT TP, A, CR
END
```

Two characters are specified from the right end of “ABCDE”, and so “DE” is returned.

```
GLOBAL
  DIM B AS STRING(10)
  DIM TXT AS STRING(20)
END
PROGRAM MAIN
  TXT = “Hello World!”
  X = 8
  B = STRRIGHT(TXT, X)
  PRINT TP, B, CR
END
```

Eight characters are specified from the right end of “Hello World!”, and so “o World!” is returned.

STRSPLIT( )

Purpose

This takes out a character string up to the specified delimiter character within the character string.

Type

Character string function

Format

STRSPLIT(<character string expression>, <character string expression>)

Examples

A = STRSPLIT(TXT, " ") .....A and TXT are character string variables

B = STRSPLIT(TXT1, TXT2) .....B, TXT1, and TXT2 are character string variables

Analysis and advice

STRSPLIT (target character string, delimiter character string)

This takes out a character string up to the specified delimiter character within the target character string.

After this take-out process, the character string following the delimiter character only remains, and the delimiter character within the target character string is deleted. If the delimiter character cannot be found, the entire target character string is taken out.

The target character string for this function is character string type variables only, and character string type constants cannot be specified.

If the first character of the target character string is a delimiter character, a character string of length 0 ("") is returned, and the character string following the delimiter character only remains as the target character string.

If the end character of the target character string is a delimiter character, the character string up to the delimiter character is taken out, and a character string of length 0 ("") is returned as the target character string.

Sample

```
GLOBAL
  DIM A AS STRING(20)
  DIM TXT AS STRING(20)
END
PROGRAM MAIN
  TXT = “Hello World!”
  A = STRSPLIT(TXT, “ “)          Because “Hello World!” is delimited by “␣”
  PRINT TP, A, CR                 (space), “Hello” is returned.
  PRINT TP, TXT, CR               “World!” only remains as the target character
END                               string.
```

```
GLOBAL
  DIM B AS STRING(20)
  DIM TXT1 AS STRING(20)
  DIM TXT2 AS STRING(20)
END
PROGRAM MAIN
  TXT1 = “ABCDE”
  TXT2 = “CD”
  B = STRSPLIT(TXT1, TXT2)        Because “ABCDE” is delimited by “CD”, “AB” is
  PRINT TP, B, CR                 returned.
  PRINT TP, TXT1, CR              “E” only remains as the target character string.
END
```

SWITCH

Purpose

This command prohibits or allows the task change-over.

Type

System switch

Format

SWITCH

Examples

ENABLE SWITCH

DISABLE SWITCH

Analysis and advice

While this system switch is changed to DISABLE, even if the SWITCH command is executed or the task change-over conditions predetermined in the system are satisfied, the task is not changed over. The initial value of this system variable is ENABLE.

Sample

```
GLOBAL
  MAXTASK = 2
END
PROGRAM MAIN
  ID = 0
  ID = TASK("SUB1")
  LOOP:
  IF DIN(1) THEN
    DISABLE SWITCH
  ELSE
    ENABLE SWITCH
  ENDIF
  MOVEA 1, 90
  MOVEA 1, -90
GOTO LOOP
END
PROGRAM SUB1
  ENABLE NOWAIT
  DOUT(-1, -2)
  TIMER = 1
  WAIT TIMER == 0
  DOUT(1)
  TIMER = 1
  WAIT TIMER == 0
  DOUT(2)
  TIMER = 1
  WAIT TIMER == 0
END
```

At the start of the main task loop, task changeover ENABLE or DISABLE is selected by the input of DIN (1).
If the task changeover is disabled, the subtask will not run and DOUT remains unchanged.

SWITCH

Purpose

This command compulsively changes over to other task in the multitask operation.

Type

Program control command

Format

SWITCH

Examples

SWITCH

Analysis and advice

If the single task is effective or the system variable “SWITCH” is set to “DISABLE” and STEP command is effective, this command is invalid.

Sample

```
GLOBAL
  MAXTASK = 2
END
PROGRAM MAIN
  ID = 0
  ID = TASK(“SUB1”)
  LOOP:
  IF DIN(1) THEN SWITCH
    PRINT “ TASK1 “, CR
  GOTO LOOP
END
PROGRAM SUB1
  ENABLE NOWAIT
  LOOP1:
  IF DIN(1) THEN SWITCH
    PRINT “ TASK2 “, CR
  GOTO LOOP1
END
```

As the two (2) tasks use the same I/O, if DIN(1) is OFF, either task will occupy the I/O.

When DIN(1) is set ON, the task is changed over compulsively to prevent one-sided occupation of I/O.

TAN( )

Purpose

This function returns the tangent of the number entered.

Type

Calculator command (Calculation function)

Format

TAN(<expression>)

Examples

K = TAN(60)

J1 = 1 - TAN(180 - D)

Analysis and advice

This function returns the tangent of the value in the brackets ( ).

Calculations are handled in units of degrees.

You may use a constant, variable or result of calculation for the <expression> term.

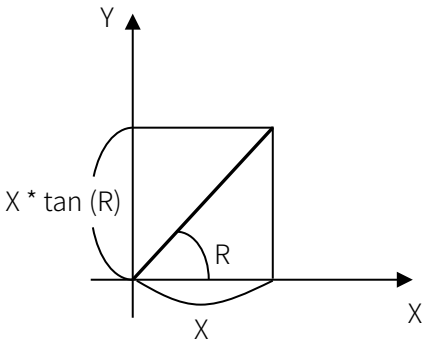
However, you may not use vector data.

This command must be used in an expression.

Sample

```
PROGRAM MAIN
 Y = 0.0
 TANSAMPLE(2, 60, Y)
 PRINT TP, Y, CR
END
PROGRAM TANSAMPLE(X, R, Y)
 LOOP:
  IF R > 180 THEN R = R - 360
  IF R < -180 THEN R = R + 360
  IF R > 180 OR R < -180 THEN GOTO LOOP
  Y = X * TAN(R)
 RETURN
END
```

("DIM Y AS REAL" is also possible.)



From the X-axis component X and the value of angle R of the line segment that makes an angle between the X-axis and R, this program finds the Y-axis component of the line segment and return the result to the main program as argument Y.

TASK( )

Purpose

This command executes the multitask.

Type

Program control command

Format

TASK (“<program name>“)

Examples

ID = TASK (“SUB”)

Analysis and advice

A program parenthesized starts as the task.

The return value is the number characteristic of the started subtask (task ID). The task ID is used as an argument at the stop of task.

When using the I/O command in the subtask, declare ENABLE NOWAIT in the subtask.

Sample

```
GLOBAL
  ID = 0
  MAXTASK = 2
END
PROGRAM MAIN
  IF ID == 0 THEN ID = TASK(“SUB”)        The subprogram starts as the task.
  LOOP:
  MOVEA 1,90
  MOVEA 1, -90
END
PROGRAM SUB
  ENABLE NOWAIT
  WAIT DIN(1)                             Asynchronous with the motion in the main task,
  DOUT(1)                                 the signal is output in reply to the signal input.
  WAIT DIN(-1)
  DOUT(-1)
END
```

TID

Purpose

This command refers to the task ID (number) of own task.

Type

System variable (Other)

Format

TID

Examples

MAINID = TID

Analysis and advice

This system variable is characteristic of each task started by the TASK command.

Writing of this variable is not possible.

Sample

```
GLOBAL
  MAXTASK = 2
END
PROGRAM MAIN
  TASK("SUB")
  LOOP:
  SWITCH
  PRINT "MAINID =", TID, CR          TID is expressed in the main task and subtask.
  GOTO LOOP                          Both values differ.
END
PROGRAM SUB
  ENABLE NOWAIT
  SWITCH
  PRINT "SUBID =", TID, CR
END
```

TIMER,TIMER1 to 4

Purpose

The TIMER system variable is a timer that can be changed by the system.

Type

System variable (movement reference command)

Format

TIMER

TIMER1

Examples

```
TIMER = 20
WAIT TIMER == 0
```

Analysis and advice

TIMER is a timer that can be used inside of SCOL2 programs.

The TIMER system variable can be set in units of seconds.

When a value of 0 or less is specified, the timer will not operate correctly.

The value of the TIMER system variable counts down at the same time as it has been set. When it reaches 0, counting down cannot be executed any longer.

By referring to TIMER in your program, you can see how much time is remaining at that time in your program.

Sample

```
PROGRAM TIMERSMPL
 TIMER = 5
 WAIT TIMER == 0
 PRINT TIMER, CR
END
```

The program waits until the set value becomes 0 after it has been set to 5 seconds.

```
PROGRAM TIMERSMPL2
 FOR J = 1 TO 1000
  PRINT "*", CR
  TIMER = 1
  WAIT TIMER == 0
 NEXT J
END
```

This program will display one "*" on the teach pendant every second for 1000 seconds.

TOOL

Purpose

TOOL is a system variable used to specify the tool coordinate system.

Type

System variable (movement reference command)

Format

TOOL

Examples

```
TOOL = TRANS(0, 0, 0, 0)
TOOL1 = TOOL
MOVE A1 WITH TOOL = TOOL + TRANS( , , 100)
```

Analysis and advice

TOOL is a system variable used to specify the tool coordinate system.
The system variable TOOL can be handled as normal TRANS type data.
By referring to TOOL, you can find the values (location) of the current tool coordinate system.

You can directly designate values for tool offset with one of the following two methods:

```
TOOL = TRANS(X, Y, Z, C)
TOOL = {X, Y, Z, C}
```

(Although { } can also be used to write as shown above, use of the TRANS command is recommended for explicitly indicating the data type.)

X, Y, Z, C: X, Y, Z and C are coordinate values in real number (unit: mm or deg).

The TOOL coordinate system is created by “sliding” a distance of X, Y and Z along the respective axes of the mechanical interface coordinate system and then twisting the new Z axis by an amount C.
TOOL must be used in an expression.
Be aware that if you change tool offset within a program, there may be some misalignment between the positions as taught and the robot movement.

Sample

```
PROGRAM TOOLSAMPLE
   MOVE A1
   MOVE A2
  TOOL = TOOL + TRANS( , , 500)
   MOVE A1
   MOVE A2
  TOOL = TRANS()
END
```

TOOL command is used to move to a point 500 mm above the position taught for the movements after the Z-axis of the tool coordinate system.

TORQUE

Purpose

The TORQUE command is used to specify the limit value of torque for each axis.

Type

System variable (movement condition command)

Format

TORQUE = {<expression>, <expression>, <expression>, <expression>, <expression>}

Examples

TORQUE = {300, 300, 100, 300, 300}

MOVE A1 WITH TORQUE = {T, T, T, T, T}

Analysis and advice

The TORQUE command is used to specify the limit value of torque for each axis.

TORQUE is a system variable having five data elements corresponding to the five axes.

The TORQUE command specifies the limit values of torque for five axes in the { } bracket following the TORQUE statement. Such data specifications are divided by commas with the first specification corresponding to Axis 1, the second to Axis 2, and so on.

The torque control value is expressed as a percentage of the torque rating of the corresponding motor. When there are no restrictions on torque, a motor may output up to 100% of its torque rating. Should torque control values be abbreviated, the controller will assume all non-specified torques to be 0.

When a value less than 100 is specified, a detection level of step-out and motor lock error will be decreased.

You may use constants, variables or calculations for the <expression> terms.

However, you may not use vector type data.

When the limit value of torque not more than 0 is specified the value is taken as 0.

Note that if the torque setting is too low, the robot will not be able to move properly and you will get an error.

Sample

```
PROGRAM TORQUESMPL
  MOVE A1 +POINT( , , 100)
  TORQUE = {300, 300, 100, 300, 300}
  ON DIN(1) BREAK DO GOTO L1
    MOVE A1
  L1:
    IGNORE DIN(1)
    MOVE PSNFBK
    WAIT MOTION >= 100
  TORQUE = {300, 300, 300, 300, 300}
  MOVE A1 +POINT( , , 100)
END
```

This program limits the torque of the Z axis (Axis 3) to 100% of the motor torque rating as the robot approaches point A1.
If input signal 1 is turned on during movement to A1, movement is stopped, the torque that was limited is returned to its original setting, and the next movement is performed.

NOTICE

To increase the torque that was limited, be sure to use the PSNFBK command to perform a movement command to the current position. A robot that has performed an unexpected motion may crash into a peripheral device.

TP

Purpose

The TP command is used to specify the communications channel to be used in the PRINT and INPUT commands to the teach pendant.

Type

System constant

Format

PRINT [{ TP | IP1 | IP2 | … | IP8 } ,] {<character string> | <expression>} [, {<character string> | <expression>}] … [, CR]

INPUT [{TP | IP1 | IP2 | … | IP8 } ,] <variable> [, <variable>] …

Examples

PRINT TP , “*** INPUT N ***”, CR

PRINT TP, N, N * 10, CR

INPUT TP, K

Analysis and advice

The TP command is used to specify the communications channel to be used in the PRINT and INPUT commands to the teach pendant.

When TP is specified as the communications channel, the controller will input and output data with the teach pendant on the communications channel allocated to the teach pendant.

Even if you do not specify a communication channel in the PRINT or INPUT command, the controller will output (or input) your data to (or from) the communications channel exclusively for the teach pendant.

See the PRINT and INPUT commands for more information on communication processing.

Sample

```
PROGRAM TPSAMPLE
  N = 0                                  (“DIM N AS INT” is also possible.)
  PRINT TP, “*** INPUT N ***”, CR
  INPUT TP, N                            This program will input a number from the
  PRINT TP, N, CR                        teach pendant and output it right back to the
END                                      teach pendant.
```

TRANS( )

Purpose

The TRANS command is used to create TRANS type data.

Type

Conversion function

Format

TRANS(<expression>, <expression>, <expression>, <expression>)

Examples

A = TRANS(100, 100, 0, 0)

WORK = WORK + TRANS(100, 100)

Analysis and advice

The TRANS command is used to create TRANS type variables.

The <expression> refers to the X, Y, Z, and C elements of the TRANS type data in order from the left. Elements X, Y and Z are in units of millimeters and element C is in units of degrees.

You may use a constant, variable or result of calculation for the <expression> term. However, you may not use vector data. Any omitted <expression> terms will be taken as 0.

This command must be used in an expression.

Sample

```
PROGRAM TRANSSMPL
  MOVE A1
  MOVE A2
  WORK1 = TRANS( , , 300)                  This sets the values of the work coordinate
  MOVE A1 WITH WORK = WORK1                system to Z=300, X, Y, C = 0.
  MOVE A2 WITH WORK = WORK1
END
```

TRIM( )

Purpose

This returns a character string excluding the spaces at the beginning or end from the specified character string.

Type

Character string function

Format

TRIM(<character string expression>)

Examples

A = TRIM(“ ABCD “) .....A is a character string variable

B = TRIM(TXT) .....B and TXT are character string variables

Analysis and advice

This returns a character string excluding the spaces at the beginning or end from the character string specified in ( ).

The spaces within the character string are not deleted.

If there are no spaces before or after, the character string is returned unchanged.

If the character string is spaces only, a character string of length 0 (““) is returned.

Sample

```
GLOBAL
  DIM A AS STRING(10)
END
PROGRAM MAIN
  A = TRIM("    ABCD ")
  PRINT TP, A, CR
END
```

The □ (spaces) in "□□ABCD□" are removed, and so "ABCD" is returned.

```
GLOBAL
  DIM B AS STRING(10)
  DIM TXT AS STRING(10)
END
PROGRAM MAIN
  TXT = "< TRIM > "
  B = TRIM(TXT)
  PRINT TP, B, CR
END
```

The □ (spaces) in "<□TRIM □>□" are removed, and so "<□TRIM □>" is returned. (The spaces within the character string remain.)

UCASE( )

Purpose

This returns a character string after converting to uppercase

Type

Character string function

Format

UCASE(<character string expression>)

Examples

TXT = UCASE(“Hello World! 123”) .....TXT is a character string variable

Analysis and advice

This converts lowercase letters in a character string specified in ( ) to uppercase and returns it.

This affects lowercase letters only.

Sample

```
GLOBAL
  DIM TXT AS STRING(20)
END
PROGRAM MAIN
  TXT = UCASE(“Hello World! 123”)
  PRINT TP, TXT, CR
END
```

Returns “HELLO WORLD! 123” where the lowercase letters have been converted to uppercase.

WAIT

Purpose

This function waits until the condition is established.

Type

Program control command

Format

WAIT <logical expression>

Examples

WAIT DIN(1)

WAIT TIMER == 0

WAIT MOTION >= 100

Analysis and advice

This function delays program execution until the condition of <logical expression> is established.

The condition is monitored, irrespective of the ongoing robot motion.

Sample

```
PROGRAM WAITSAMPLE
  WAIT DIN(1)                    Waits until input signal 1 is ON.
  MOVE A1
  MOVE A2
  MOVE A3
END
```

WHILE...ENDWHILE

Purpose

This specifies repeating of program.

Type

Program control command

Format

WHILE <logical expression>
:
(IF <logical expression> THEN EXIT)
ENDWHILE

Examples

WHILE A < 3
WHILE 1

Analysis and advice

The program enclosed by WHILE and ENDWHILE is repeated while the conditional expression is TRUE.
If the conditional expression is FALSE, the WHILE statement is exited, and operation proceeds to the next statement.
If EXIT is used, the loop can be exited even during repeating of the WHILE statement.
If WHILE 1 is written, the condition is always TRUE. (This results in an endless loop, and so EXIT must be used to escape.)

Sample

```
PROGRAM WHILESAMPLE
 A = 0
 WHILE A < 5                          Repeats while the A value is less than 5.
  PRINT “A =“, A, CR
  A = A + 1
 ENDWHILE
END
```

```
PROGRAM WHILESAMPLE
 WHILE 1
  PRINT “A =“, A, CR
  IF A >= 5 THEN EXIT                 If the A value becomes 5 or more, the WHILE
  A = A + 1                           statement is exited, and operation proceeds to
 ENDWHILE                             the next statement.
END
```

WITH

Purpose

The WITH statement is used to add a conditional statement to a movement command.

Type

Program control command

Format

WITH <statement> [, <statement>] …

Examples

MOVE A1 WITH SPEED = 50

MOVE A1 WITH TOOL = TOOL1, PASS = 80

Analysis and advice

The WITH statement is used to specify movement conditions corresponding to individual movement commands.

The movement conditions, such as speed, acceleration, and deceleration, are determined according to the set values of the system variables required for robot movement. In order to change movement conditions for a single motion, the corresponding movement condition is specified using the WITH clause. The movement condition specified by the WITH clause becomes valid only in the movement command where the WITH clause has been specified. The values of the system variables remain unchanged after and before the execution of the movement command.

The following movement conditions may be specified in the <statement> term. Should you wish to specify more than one such condition, be sure to keep the terms separate with commas.

Movement condition	System variable name
Robot configuration	CONFIG
Positioning accuracy	ACCUR
Acceleration (during acceleration)	ACCEL
Deceleration (during deceleration)	DECEL
Speed	SPEED
CP acceleration	CPACCEL
CP deceleration	CPDECEL
CP movement speed	CPSPEED
Override	OVERRIDE
Short-cut movement parameter	PASS
Specifying of pass amount in mm	PASSMM
Slow movement start	SLOWSTART
Max. torque for each axis	TORQUE
Servo gain for each axis	GAIN
Tool coordinate system	TOOL
Base coordinate system	BASE
Work coordinate system	WORK
Robot load	PAYLOAD

Sample

```
PROGRAM WITHSAMPLE
  SPEED = 50
  MOVE A1
  MOVE A2 WITH SPEED = 100
  MOVE A3
  MOVE A4
END
```

This program will move to all points at half (50%) speed with the exception of point A2, to which the robot will move at full (100%) speed.

WORK

Purpose

WORK is a system variable used to specify the work coordinate system.

Type

System variable (movement reference command)

Format

WORK

Examples

MOVE A1 WITH WORK = WORK + TRANS(, , 100)

WORK WK1

Analysis and advice

WORK is a system variable used to specify the work coordinate system.

The WORK coordinate system is created by “sliding” a distance of X, Y and Z along the respective axes of the WORLD coordinate system and then twisting the new Z axis by an amount C.

WORK must be used in an expression.

When positional data is entered with the teach pendant, the work coordinate system in effect at that time is also entered. Afterwards, when a movement command is executed using that positional data, the work coordinate system will automatically switch over to that in effect when the positional data was recorded. When you wish to move the robot in terms of two (or more) work coordinate systems (e.g., the work coordinate system in effect during teaching and a different coordinate system), you may either:

(1) Use the WITH clause to specify the work coordinate system in effect for that movement;

or

(2) Change the work coordinate data itself.

Be aware that if you change work coordinate systems within a program, there may be some misalignment between the positions as taught and the robot movement.

Sample

```
PROGRAM WORKSAMPLE
  MOVE A1
  WK1 = WORK + TRANS( , , 300)
  MOVE A1 WITH WORK = WK1
END
```

The WORK command moves 300 mm along the Z axis of the work coordinate system, and after that the robot moves to a point above 300 mm from the taught position.

```
PROGRAM WORKSAMPLE2
  MOVE A2
  WK1 = WORK + TRANS( , , 300)
  MOVE A2
END
```

```
DATA
TRANS WK1 = 0, 0, 0, 0
WORK WK1
POINT A2 = 500, 0, 100, 0, 0 / FREE
END
```

Data section

XOR

Purpose

This command computes exclusive OR.

Type

Calculation command (operator)

Format

<logical expression> XOR <logical expression>

Examples

IF DIN(1) XOR DIN(2) THEN J = 0

WAIT DIN(5) XOR TIMER == 0

Analysis and advice

This command calculates the exclusive OR of the right and left side logical expressions of XOR.

If one of the logical expressions on both sides of XOR is true, the result will be true.

If both are true or both are false, the result will be false.

This command is used in logical expressions.

Sample

```
PROGRAM MAIN
   WAIT DIN(1) XOR DIN(2)          Waits until only one of DIN(1) and DIN(2) is
   PRINT TP, "WAIT END", CR        turned on.
   DELAY 1.0
END
```

System version

X8KCS-05C or later

4. Program Examples

In this section, we explain various programming examples using the SCOL2 language. When applying these programs to an actual robot, be sure to modify the programs accordingly to match individual robot operating conditions such as the work environment and range of movement.

4.1 Program to move robot back to its mechanical origins

This program moves the robot back to its mechanical origins (zero-points for each axis). Using absolute single shaft movement, the robot will zero itself starting with the Z-axis (Axis 3) and then working in from the tip to the base.

```
PROGRAM ORIGIN
  MOVEA 3 , 0
  MOVEA 5 , 0
  MOVEA 4 , 0
  MOVEA 2 , 0
  MOVEA 1 , 0
END
```

4.2 Warm-up program

This program is used to warm up the robot before beginning work. The robot will start out slowly and gradually speed up.

```
PROGRAM WARMINGUP
  FOR K = 1 TO 100
    SPEED = K
    MOVEA 3 , 100
    MOVEA 3 , 0
    MOVEA 5 , 50
    MOVEA 5 , 0
    MOVEA 4 , 50
    MOVEA 4 , 0
    MOVEA 2 , 50
    MOVEA 2 , 0
    MOVEA 1 , 50
    MOVEA 1 , 0
  NEXT K
END
```

4.3 Robot motion

A program for causing the robot to be moved from position A1 to position A2. The robot movement speed is set to 20 % of the maximum speed.

(a) When the PTP motion (MOVE command) is used:

```
PROGRAM MOVEA1A2
  SPEED = 20
  MOVE A1
  MOVE A2
END
```

(b) When the linear interpolated motion (MOVES command) is used:

```
PROGRAM MOVESA1A2
  SPEED = 20
  MOVES A1
  MOVES A2
END
```

4.4 I/O signals

A program where turning on input signals 1 to 4 causes output signals 1 to 4 to be turned on, while turning off input signals 1 to 4 causes output signals 1 to 4 to be turned off.

```
PROGRAM SAMPLE
  IF DIN(1) THEN DOUT(1) ELSE DOUT(-1)
  IF DIN(2) THEN DOUT(2) ELSE DOUT(-2)
  IF DIN(3) THEN DOUT(3) ELSE DOUT(-3)
  IF DIN(4) THEN DOUT(4) ELSE DOUT(-4)
END
```

4.5 Interlock

A program for stopping the motion of the robot while input signal 1 is turned off in positions A1 to A4.
The movement speed of the robot is set to 20% of the maximum speed.

(a) When WAIT command is used:

```
PROGRAM SAMPLE
  SPEED = 20
  WAIT DIN(1)
  MOVE A1
  WAIT DIN(1)
  MOVE A2
  WAIT DIN(1)
  MOVE A3
  WAIT DIN(1)
  MOVE A4
END
```

(b) When the robot motion is stopped with the ON command and waits until input signal 1 is turned on:

```
PROGRAM SAMPLE
  SPEED = 20
  ENABLE NOWAIT
  ON DIN(-1) BREAK DO SUB
  WAIT DIN(1)
  MOVE A1
  MOVE A2
  MOVE A3
  MOVE A4
END
PROGRAM SUB
  WAIT DIN(1)
  RESUME
  ON DIN(-1) BREAK DO SUB
END
```

4.6 Pick and place

A program for picking up a workpiece at position A1 and placing it at position A2. The hand is opened/closed with output signal 201. The movement speed of the robot is set to 20 % of the maximum speed.

(a) When positions just above positions A1 and A2 are taught as A3 and A4, respectively:

```
PROGRAM SAMPLE
  SPEED = 20
  DOUT(-201)
  MOVE A3
  MOVE A1
  DOUT(201)
  DELAY 0.5
  MOVE A3
  MOVE A4
  MOVE A2
  DOUT(-201)
  DELAY 0.5
  MOVE A4
END
```

(b) When the robot is moved to a position over the taught position in the format of “MOVE A1+POINT(0,0,20)”.

```
PROGRAM SAMPLE
  SPEED = 20
  DOUT(-201)
  MOVE A1 + POINT(0 , 0 , 20)
  MOVE A1
  DOUT(201)
  DELAY 0.5
  MOVE A1 + POINT(0 , 0 , 20)
  MOVE A2 + POINT(0 , 0 , 20)
  MOVE A2
  DOUT(-201)
  DELAY 0.5
  MOVE A2 + POINT(0 , 0 , 20)
END
```

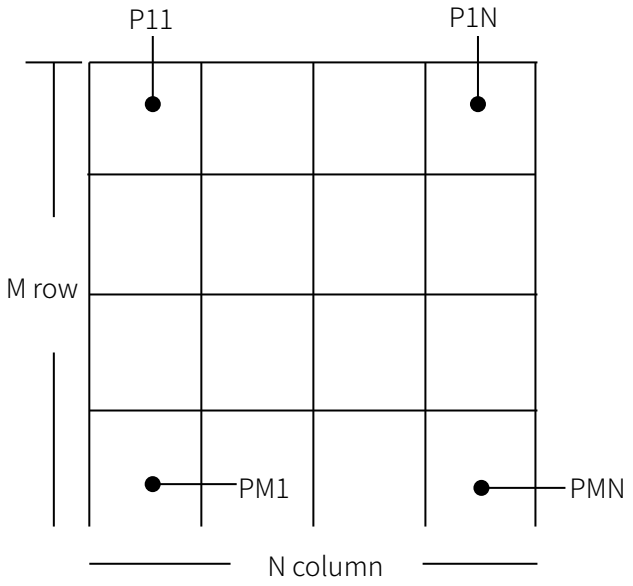
(c) When a short-cut motion is executed:

```
PROGRAM SAMPLE
  SPEED = 20
  PASS = 80
    DOUT(-201)
  ENABLE PASS
    MOVE A1 + POINT(0 , 0 , 50)
  DISABLE PASS
    MOVE A1
    DOUT(201)
    DELAY 0.5
  ENABLE PASS
    MOVE A1 + POINT(0 , 0 , 50)
    MOVE A2 + POINT(0 , 0 , 50)
  DISABLE PASS
    MOVE A2
    DOUT(-201)
    DELAY 0.5
  ENABLE PASS
    MOVE A2 + POINT(0 , 0 , 50)
END
```

4.7 Palletize

Consider a program for placing parts on a pallet as shown in the right hand figure.

Pallet size: M rows x N columns



Teaching positions:

P11 : Position of row 1 and column 1 of pallet
P1N : Position of row 1 and column N of pallet
PM1 : Position of row M and column 1 of pallet
PMN : Position of row M and column N of pallet
PP : Part unloading position
P0 : Standby position

Designation of I/O signals:

DIN(1) : Pallet standby (This signal is turned on when the pallet is in the operating position)
DIN(2) : Part standby (This signal is turned on when a part can be picked up.)
DOUT(1) : Hand close (This signal is turned on when the hand is closed.)
DOUT(2) : One pallet operation completion (This signal is turned on when all parts are placed on the pallet. When the pallet is removed and then the pallet standby signal is turned off.)

```
PROGRAM PALLET
 MOVE P0
 RESET DOUT
 M = 10                                  Specifies the number of lines and rows.
 N = 15
 PLT = PALLET(M, N, P11, PM1, P1N, PMN)  Defines a pallet.
 PASS = 80
 ENABLE PASS
 FOR I = 1 TO M                          Repeats the operation for each row
   FOR J = 1 TO N                        Repeats the operation for each column.
     MOVE PP + POINT(0 , 0 , 50)
     WAIT DIN(2)
     MOVE PP                             Unloads the part.
     WAIT MOTION > = 100
     DOUT(1)
     DELAY 0.5
     MOVE PP + POINT(0 , 0 , 50)

     MOVE PLT(I, J) + POINT(0, 0, 50)
     WAIT DIN(1)
     MOVE PLT(I, J)                      Places the part.
     WAIT MOTION > = 100
     DOUT(-1)
     DELAY 0.5
     MOVE PLT(I, J) + POINT(0, 0, 50)
   NEXT J
 NEXT I

 WAIT MOTION > = 100
 DOUT(2)
 MOVE P0
 END
```

4.8 Creating a program for monitoring an insertion error

A program for detecting a part insertion error and for an error handling is as follows.

A1: Position to insert the part

```
PROGRAM SAMPLE
   :
ACCUR = COARSE                          Sets the positioning accuracy to coarse.   ①
MOVE A1 + POINT(0 , 0 , 50)             Moves the hand to a position 50 mm just
                                        over the part insertion point.
SETGAIN(0 , 0 , 1 , 0 , 0)              Turns off the servo controls except for    ②
                                        axis Z.
TORQUE = { 300 , 300 , 50 , 300 , 300 } Limits the torque of axis Z to 50%         ③
MOVE A1                                 Inserts all the parts.
WAIT MOTION > = 100
SETGAIN(0 , 0 , 0 , 0 , 0)              Turns off the servo control for all the axes. ④
SETGAIN(0 , 0 , 1 , 0 , 0)              Turns on the servo control for axis Z.     ⑤
MOVE HERE
HS = HERE .Z - A1 .Z                    When the difference of the height
IF HS < 5 THEN GOTO OK                  between the current position and the       ⑥
                                        inserted position is less than 5 mm, it is
                                        determined that the inserted position is
                                        normal.
SETGAIN(1 , 1 , 1 , 1 , 1)              Turns on the servo controls for all the
DELAY 0.1                               axes, returns the torque to 300% and
TORQUE = { 300 , 300 , 300 , 300 , 300 } moves the hand to a position 50 mm just
DELAY 0.1                               over the part insertion point.
MOVE A1 + POINT(0 , 0 , 50)

   Processing upon a part insertion error

OK :
SETGAIN(1 , 1 , 1 , 1 , 1)              Turns on the servo controls for all the
DELAY 0.1                               axes, returns the torque to 300% and
TORQUE = { 300 , 300 , 300 , 300 , 300 } moves the hand to a position 50 mm just
DELAY 0.1                               over the part insertion point.
MOVE A1 + POINT(0 , 0 , 50)

   Processing when part insertion is successful

   :
END
```

Comments for program description:

① Unless the positioning accuracy is set to COARSE, the robot system may detect an error and thereby cause the automatic operation to be stopped.

② By turning off the servo controls except for axis Z, the robot can be freely moved on the X–Y plane. Thus, when workpiece or part has been chamfered, it can be inserted along the chamfered surface.

③ With the TORQUE command, the torque which occurs in the motor is limited. When an insertion error occurs, the force applied to the work and hand is suppressed. (Normally, the torque is set to 300 % of the rating value.)

By suppressing the torque to less than 100 %, the error detection level of the controller will be decreased. Thus, when monitoring the insertion error, it is necessary to set the torque to less than 100%. When the torque setting value is excessively decreased, since the torque necessary for moving the robot cannot be obtained, an error takes place.

④ The position being read with the HERE command is the position commanded to the robot. To read the present position of the robot with the HERE command, it is necessary to turn off the servo control and then turn it on again. Thus, with the GAIN command, the servo controls for all the axes are turned off.

⑤ The servo control which was turned off in ④ is turned on.

⑥ The difference of height between the current position and the insertion position should be specified to a proper value.

4.9 Program example of short-cut (PASS) movement

The following programming example uses short-cut movement to carry out “pick and place” operation.



A: Part pick-up location

D: Part insertion location

F: Dump location for defective parts

The robot will take a part from the part pick-up location (A), move through points B and C, and try to insert the part at the part insertion location (D). If the part is defective (i.e., cannot be inserted), the robot will move through points C and E, and place the part at the dump location for defective parts (F).
Short-cut movement is used throughout the program.

The following signals are used in this program:

Input signals:

DI1: Completion of pick-up preparation
Turns ON when preparation of the part to be picked up is completed.

DI2: Completion of insertion preparation
Turns ON when preparation of the part insert position is completed.

DI3: Defective part
Turns ON when the grasped part is determined to be defective.

DI4: Completion of defective part dumping preparation
Turns ON when preparation of the part dump position is completed.

Output signals:

DO1: Completion of part pick-up
Turns ON when part pick-up is completed and preparation of the next part may be begun.

DO2: Completion of part insertion
Turns ON when part insertion is completed and preparation for the insertion of the next part may be begun.

DO3: Completion of defective part dumping
Turns ON when dumping of a defective part is completed and preparation for dumping the next defective part may be begun.

The example program is presented below:

```
PROGRAM PICKPLACE
‘* PICK AND PLACE SAMPLE PROGRAM *
 INPUT :                                  Initial settings
    OPEN1
    DOUT(1 , 2 , 3)                       Part pick-up, part insertion and
    ENABLE NOWAIT                         completion of disposal for defective part
    SPEED = 80
    PASS = 80
    ENABLE PASS
    MOVE B                                Move to position above part pick-up location

 PICKUP :                                 Section for picking up part
    DOUT(-1)                              Begin picking up part
    WAIT DIN(1)                           Wait until pick-up preparations are completed
    MOVE A                                Lower down to A
    WAIT MOTION > = 100
    CLOSE1                                Close hand
    DELAY 0.3
    MOVE B                                Move up to B

 PLACE :                                  Section for placing part
    MOVE C                                Move to position above part insertion location
    DOUT(1 , -2)                          Part pick-up complete – begin part insertion
    WAIT DIN(2)                           Wait until insertion preparations are completed
    MOVE D                                Lower down to D
    WAIT MOTION > = 100
    IF DIN(3) THEN GOTO ERR               Judgment of part as defective
    OPEN1                                 Open hand
    DELAY 0.3
    MOVE C                                Move up to
    MOVE B                                Move up to position above part pick-up location
    DOUT(2)                               Completion of part insertion
    IF MODE = = CYCLE THEN GOTO CYCLEEND  Mode check
    GOTO PICKUP
```

```
 ERR :                                  Section for handling defective part
    MOVE C                              Move to position above part insertion location
    MOVE E                              Move to position above defective part dump
                                        location
    DOUT(-3)                            Begin dumping of defective part
    WAIT DIN(4)                         Wait until dumping preparations are completed
    MOVE F                              Lower down to F
    WAIT MOTION > = 100
    OPEN1                               Open hand
    DELAY 0.3
    MOVE E                              Move up to E
    MOVE B                              Move to position above part pick-up location
    DOUT(3)                             Completion of dumping of defective part
    IF MODE = = CYCLE THEN GOTO CYCLEEND Mode check
    GOTO PICKUP

 CYCLEEND :                             Processing for end of cycle
    MOVE R                              Move to wait station
END
```

5. Programming Hints and Warnings

This section explains timing considerations, things not to do, and things to watch out for when writing up a program.

5.1 Program Execution Timing

Robot programs are executed one line at a time starting from the top. Normally, when a robot carries out a movement command, the next command is not executed until final positioning for that movement is completed. However, when handling I/O signals and other operations not directly related to robot movement, this causes time to be wasted (waiting for the movement to be completed).
In order to efficiently utilize time and speed up the robot as much as possible, the SCOL2 language allows you to input/output signals and to process communications while the robot is moving.

5.1.1 Arm Movement and Signal I/O Timing

When inputting or outputting a signal, the robot program instructions written in the robot language specify whether or not to wait for the arm to complete the motion.
The system switch NOWAIT is used to tell the controller whether to wait for the robot to stop moving (finish positioning itself) before inputting or outputting signals.

ENABLE NOWAIT	This tells the controller not to wait for the arm to finish positioning itself before inputting or outputting signals.
DISABLE NOWAIT	This tells the controller to wait for the arm to finish positioning itself before inputting or outputting signals.

The initial (default) setting is DISABLE NOWAIT.
An example program is shown below. Using this program, we will describe I/O signal timing relative to command processing and arm movement.

Example) Let's consider the case of executing the following commands.

```
PROGRAM SAMPLE
  MOVE P1
  DOUT(1)
  MOVE P2
  DOUT(2)
  MOVE P3
  DOUT(3)
  MOVE P4
  DOUT(4)
  MOVE P5
  DOUT(5)
END
```

(1) Timing when DISABLE NOWAIT is in effect

Command processing	MOVE P1		DOUT(1)	MOVE P2	Wait for finish of motion Wait for finish of motion	DOUT(2)	MOVE P3	Wait for finish of motion Wait for finish of motion	DOUT(3)	MOVE P4	Wait for finish of motion Wait for finish of motion	DOUT(4)	MOVE P5	Wait for finish of motion Wait for finish of motion
Arm motion	Move to P1		Move to P2			Move to P3			Move to P4			Move to P5		

As shown above, signals are output after the arm has stopped moving.

When ACCURE=COARSE is specified, since the subsequent command is executed before the positioning operation is completed, a signal may be output before the robot motion is completely stopped.

(2) Timing when ENABLE NOWAIT is in effect

Command processing	MOVE P1	DOUT(1)	MOVE P2	DOUT(2)	MOVE P3	DOUT(3)	MOVE P4	DOUT(4)	Wait for finish of motion Wait for finish of motion	MOVE P5	DOUT(5)				
Arm motion	Move to P1									Move to P2		Move to P3		Move to P4	Move to P5

The arm motion by the previous command completes, and the arm motion by the next command starts. Even if the arm is moving, however, processing of signal output described next to that motion command is executed, which is called “pre-reading of motion command.” This robot controller pre-reads up to four (4) motion commands. (While the arm is moving to point P1, the controller pre-reads motion commands up to point P4 and waits for completion of the motion just before the arm executes the motion command to point P5.)

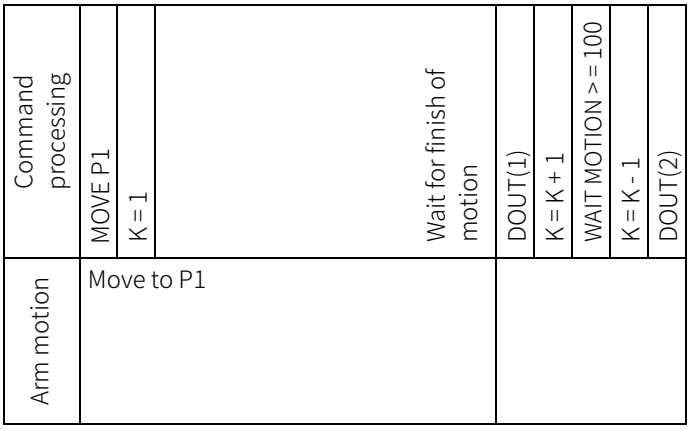
5.1.2 Synchronization of Arm Movement and Program Execution

In SCOL2 programs, arm movements are executed simultaneously with other command languages. Therefore, the order described in the program and the actual order of movements may differ.
The command "WAIT MOTION>=100" is used to synchronize the order described in the program with the actual order of movements.
The following describes the processing of command language, arm movements and signal output timing.

Example) Let's consider the case of executing the following commands.

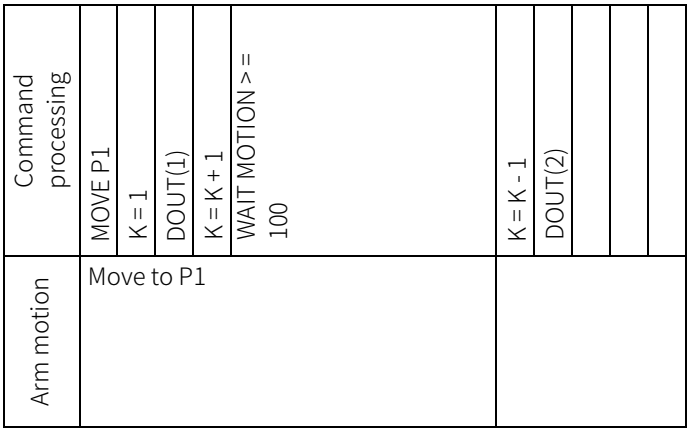
```
PROGRAM SAMPLE
  MOVE P1
  K = 1
  DOUT(1)
  K = K + 1
  WAIT MOTION > = 100
  K = K - 1
  DOUT(2)
END
```

(1) Timing when DISABLE NOWAIT is in effect



The DOUT command waits until the arm movement finishes.

(2) Timing when ENABLE NOWAIT is in effect



The command of “WAIT MOTION >=100” waits until the arm movement finishes.

In addition to signal input and output, processing of commands other than those given below goes on, irrespective of the robot movement.
DIN , BCDIN , INPUT , DOUT , BCDOUT , PULOUT , PRINT

5.1.3 DELAY Command and WAIT Command

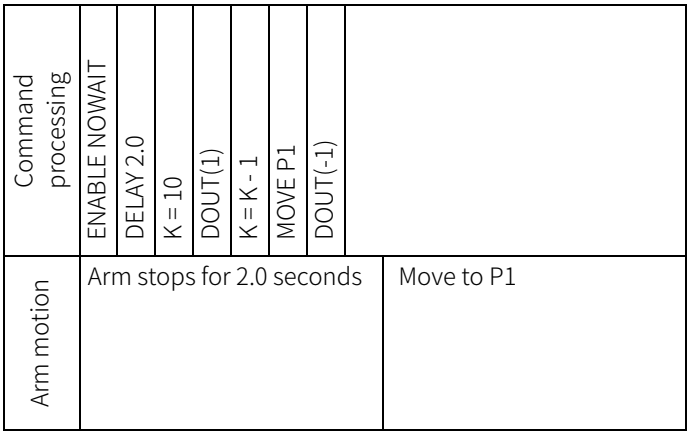
To stop the arm motion for the specified time during program execution, there are two ways. One method utilizes the DELAY command and the other method utilizes the WAIT command in conjunction with the TIMER command. When writing a program, you should keep in mind that there is a difference in execution timing between the two.

(1) DELAY command

The DELAY command is a kind of motion control commands and stops the arm motion for the predetermined time. However, as the SCOL2 language program is processed in synchronization with ongoing arm motion, the program is executed even during execution of the DELAY command.

Example) Let's consider the case of executing the following commands.

```
PROGRAM SAMPLE
  ENABLE NOWAIT
  DELAY 2.0
  K = 10
  DOUT(1)
  K = K - 1
  MOVE P1
  DOUT(-1)
END
```



Since the program continues to be executed even after the DELAY command causes the robot arm to pause, Signal 1 will be output while the arm is stopped.

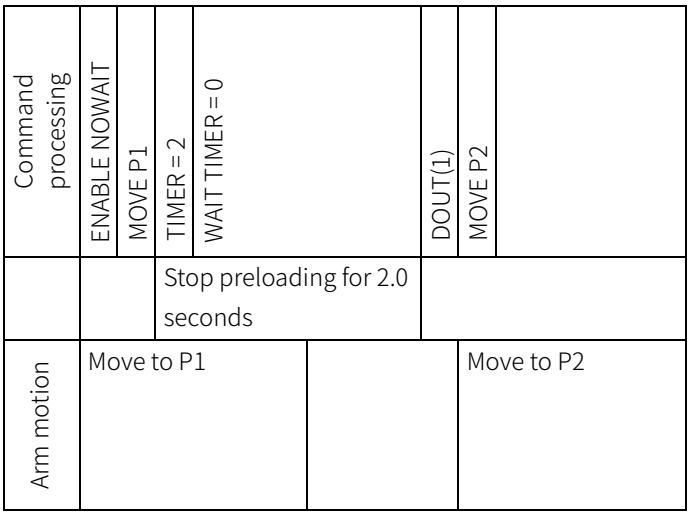
Should you wish to delay program execution as well, you should insert a “WAIT MOTION >= 100” command between the DELAY command and DOUT command.

(2) WAIT command

The WAIT command can be used with the TIMER command to delay program execution for a specified period of time. However, as with the DELAY command described above, the program is processed in parallel with the robot movement and one must be careful of the execution timing.

Example) Let's consider the case of executing the following commands.

```
PROGRAM SAMPLE
  ENABLE NOWAIT
  MOVE P1
  TIMER = 2
  WAIT TIMER = = 0
  DOUT(1)
  MOVE P2
END
```



The delay in program execution specified by the WAIT command comes to an end while the robot is still moving. When it is necessary for the robot to wait also, insert a WAIT MOTION statement as shown below:

```
  MOVE P1
  WAIT MOTION > = 100
  TIMER = 2
  WAIT TIMER = = 0
```

Command processing	ENABLE NOWAIT MOVE P1 WAIT MOTION > = 100	TIMER = 2 WAIT TIMER = 0	DOUT(1) MOVE P2
		Stop preloading for 2.0 seconds	
Arm motion	Move to P1		Move to P2

5.2 Things Not to Do When Programming

This paragraph presents restrictions and prohibitions in effect when writing programs. Refer to the command descriptions in Section 3 for information on individual commands.

5.2.1 Variables

(1) Referring to undefined variables

The data type of a variable is first defined when something is substituted into it. Therefore, do not refer to variables which appear for the first time in your program. If a variable is referenced before it is assigned in the program, the type and value of the variable will be undefined and a compile error will result.

Example)

```
PROGRAM SAMPLE
  IF K < > 0 THEN GOTO RESTART
  K = 1
  A1 = A
  RESTART :
  FOR N = 1 TO 3
  MOVE A1
  A1 = A1 + POINT(100 , 100)
  NEXT N
END
```

Here, when the program is first executed, variable K is referenced in the IF statement. However, variable K has never been used (substituted into) before, so it becomes undefined. After that, 1 is substituted in K, and so the variable K takes the value 1.

5.3 Things to Watch Out for When Writing a Program

This paragraph describes things to watch out for and presents an outline of some SCOL2 commands which may come in useful when writing a program.

5.3.1 Types of Commands

A functional classification of the SCOL2 language was presented in Section 1. Here, we describe the SCOL2 language in terms of internal processing.
SCOL2 commands can be broken down into basic commands, functions and system variables. Basic commands make up the core of the SCOL2 language and are executed in conjunction with parameters following the commands. Functions are provided as a convenience to make SCOL2 easier to use. System variables are used to directly refer to (and change) such items as speed, coordinate systems, etc. They can be handled like any other variable.
These three types of commands are described in more detail below.

(1) Basic commands
Basic commands provided by the SCOL2 language are shown below. Unlike the functions listed below in Para. (2), you may not use any of these inside of a (mathematical) expression.

MOVE	MOVES	MOVEC	MOVEA	MOVEI	MOVEJ
MOVEW	MOVEWI	MOVET	MOVECV	DELAY	
BREAK	PAUSE	RESUME	ON ~ DO	IGNORE	GOTO
RETURN	WAIT	STOP	IF ~ THEN ~	ELSE	FOR ~ NEXT
PRINT	INPUT	ENABLE	DISABLE	PROGRAM	END
RESET	REMARK	DIM ~ AS	TASK	KILL	SWITCH
GLOBAL	MAXTASK	RESTORE	MOVESYNC	SAVEEND	RCYCLE

(2) Functions
As opposed to basic commands, functions have the feature that they can pass arguments back and forth just like subroutine programs (subprograms). Up to ten arguments can be specified.
Functions can be broken down into built-in functions and system library functions (which are so called because they are kept in a file in the system library). In the SCOL2 language, the system library is called SCOL.LIB, and unless SCOL.LIB is in the system ROM drive, you will not be able to use any of the system library functions.

The following built-in functions are provided by SCOL2.

Built-in functions

MOTION	MOTIONT	REMAIN	REMAINT	HERE	DEST
POINT	TRANS	DIN	DOUT	PULOUT	BCDIN
BCDOUT	SIN	COS	TAN	ASIN	ACOS
ATAN	ATAN2	SQRT	ABS	SGN	INT
REAL	LN	LOG10	EXP	MODE	ALARM

Built-in functions may be used in (mathematical) expressions with the exception of the following signal I/O commands: DOUT , PULOUT , BCDOUT and ALARM.

Functions registered to the library

OPEN1	CLOSE1	OPENI1	CLOSEI1	OPEN2	CLOSE2
OPENI2	CLOSEI2	READY	ONGAIN	OFFGAIN	FREELOAD
SETGAIN					

If you make a subprogram and give it the same name as a function contained in the system library, this new subprogram should be executed as a priority.

(3) System variable

System variables are used to change system conditions and can be handled just like any other variable. The following system variables are provided by SCOL2.

CONFIG	ACCUR	ACCEL	DECEL	SPEED	PASS
TORQUE	GAIN	TOOL	BASE	WORK	TIMER
ERROR	PAYLOAD	TID			

5.3.2 Robot Coordinate Systems

This paragraph describes robot coordinate systems handled by the SCOL2 language.

(1) Robot coordinate systems

The robot has five types of coordinate systems, i.e. the world coordinate system, the base coordinate system, the work coordinate system, the mechanical interface coordinate system and the tool coordinate system. A brief description of these systems is presented below:

(a) World coordinate system (absolute coordinate system)

The world coordinate system is a single system used to describe the orientation of the work site surrounding the robot. If this coordinate system can be defined independently of the robot. If this coordinate system is set at the home position of the robot, the world coordinate system and the base coordinate system are the same.

(b) Base coordinate system (mechanical coordinate system)

The base coordinate system is the system used by the robot itself. The location of the base coordinate determine is determined by the robot design, with the origin of the system always being the mechanical origins of the robot.

(c) Work coordinate system

The work coordinate system is defined in terms of the workpiece to be handled by the robot.

(d) Mechanical interface coordinate system

The mechanical interface coordinate system is defined in terms of an end effector attached to the robot. This coordinate system will shift in accordance with robot movement.

(e) Tool coordinate system

The tool coordinate system is defined in terms of the end of a tool attached to the robot. This coordinate system will shift in accordance with robot motion.

These coordinate systems play a role when guiding the robot, when teaching it positions, and when operating it. The world, work, and tool coordinate systems can be specified when guiding the robot. The robot moves along the specified coordinate system. However, there is no particular need to worry about which coordinate system to select when teaching or operating the robot. Under normal usage, the robot will move to the points you taught it to move to.

Coordinate system and additional axes

The 5th axis (T-axis) of the SCARA robot is an additional axis, and it is not affected by the coordinate system. It is the same value regardless of the selected coordinate system.

Fig. 5.1 shows the robot coordinate system.

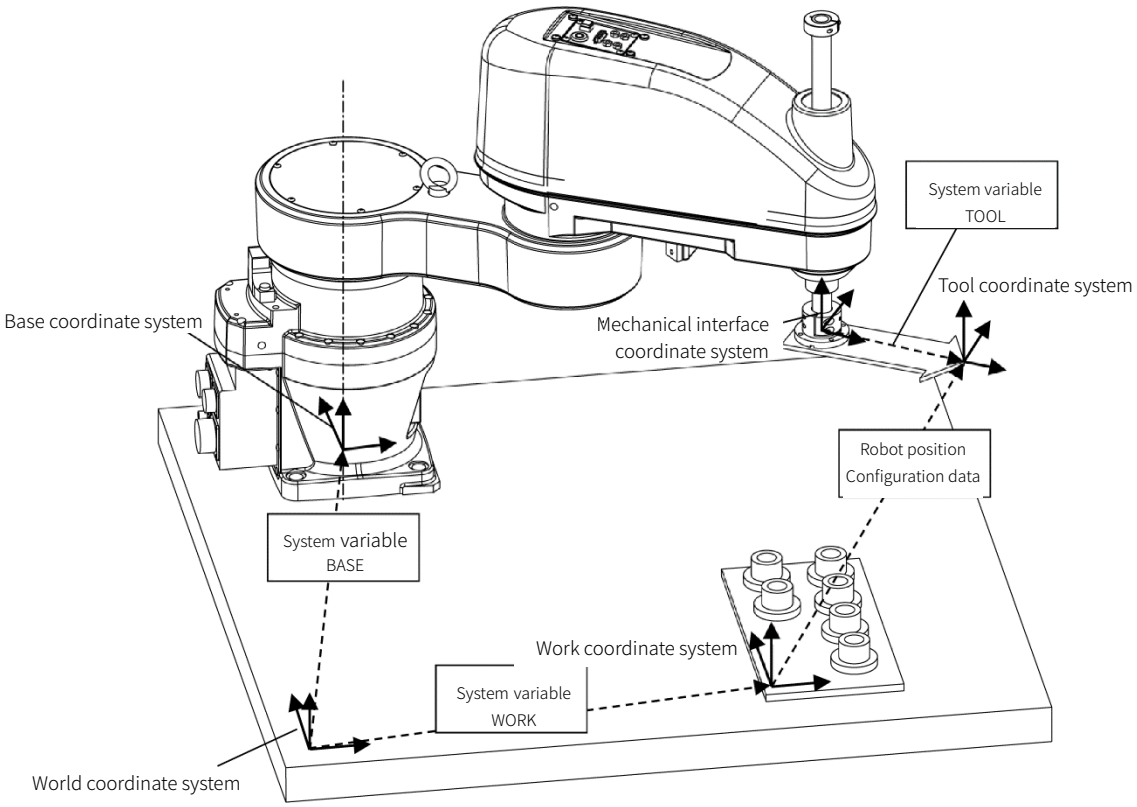


Fig. 5.1 Robot coordinate system

(2) Coordinate systems and system variables

The base, work and tool coordinate systems may each contain multiple coordinate systems, each of which can be selected according to robot tasks. Each coordinate system can be also specified in the SCOL2 language program. These coordinate systems may also be specified (defined) inside of a program by using the system variables BASE, WORK and TOOL. These system variables may be used just like any other TRANS type system variables.

The meaning of each of these system variables is described below. For more information on teaching coordinate systems to the robot or selecting such a coordinate system, refer to the “TP5000 Operation Manual”.

(a) System variable BASE

The system variable BASE defines the origin of the base coordinate system in terms of the world coordinate system. It is used when it is necessary to base robot movement on the coordinate system of the work-site (world coordinate system) or when it is necessary to guide the robot in terms of world coordinates (as opposed to base coordinates).

With such a system, the robot can be reinstalled in a separate location and put back into action after simply redefining the location of the base coordinate system.

Should the value of the BASE system variable be set to all 0, the world coordinate system and the base coordinate system will become the same. (Note that the BASE system variable is of a coordinate data-type and contains four elements, all of which must be set to 0 in order to set the BASE system variable itself to 0.) You should set BASE to 0 if there is no particular need to specify a world coordinate system. Also, be careful not to accidentally change the base coordinate system after it has been set.

(b) System variable WORK

The system variable WORK defines the origin of the work coordinate system in terms of the world coordinate system. Should WORK have a value of 0, the world and work coordinate systems will be the same. (Note that the WORK system variable is of a coordinate data-type and contains four elements.) When handling multiple workpieces, multiple work coordinate systems may be specified to help keep track of where each workpiece is.

Also, the WORK system variable is useful when it is necessary to guide the robot in relation to the workpiece (rather than, for example, the robot base). When there is no particular need to specify a work coordinate system, set the WORK system variable to all 0.

(c) System variable TOOL

The system variable TOOL defines the origin of the tool coordinate system in terms of the mechanical interface coordinate system. Should TOOL have a value of 0, the tool coordinate system and the mechanical interface coordinate system will be the same. When handling multiple tools, multiple tool coordinate systems may be specified to help keep track of where each tool is. Also, the WORK system variable is useful when it is necessary to guide the robot in relation to the tool (rather than, for example, the robot base).

You should be careful when changing any of these system variables in your program since the coordinate system in which the robot moves will also change.

(3) Teaching data and coordinate systems

When you teach a position to the robot, the robot will also record the position of the tool tip relative to a work coordinate system. In addition, the robot will also remember the work coordinate system in effect at the time.

When a program using this data is executed, the robot will move to the position defined by the positional data for that point. However, if the work coordinate system itself was changed (and is therefore not the same as the work coordinate system in effect when taught), the same positional data will define a different point (since the frame of reference is different). Therefore, the robot will not move to the point as taught, but to another point.

(4) Changing coordinate system data in the program

A simple explanation of how to change coordinate system data with the program is presented below. You should not change such data unless you have a good reason for doing so.

(a) Changing the base coordinate system

There is no need to change the base coordinate system from the program.

(b) Changing the work coordinate system

When you teach the robot a position, the robot will also remember the work coordinate system in effect at the time. When a movement command tells the robot to move to that point, the current work coordinate system (in effect at the time the command was encountered in the program) will automatically change over to the previous work coordinate system (in effect when the point was taught to the robot).

Example) You have three different work coordinate systems, i.e. WORK1, WORK2 and WORK3. In each coordinate system, you taught the robot one point, i.e. you have point A1 defined in terms of work coordinate system WORK1, point A2 defined in terms of work coordinate system WORK2, and point A3 defined in terms of work coordinate system WORK3. When you execute the following program, the work coordinate system will change as follows.

```
PROGRAM SAMPLE
  MOVE A1                    The work system in effect during this command
                             will be WORK1.
  MOVE A2                    The work system in effect during this command
                             will be WORK2.
  MOVE A3                    The work system in effect during this command
                             will be WORK3.
END
```

When handling multiple workpieces, you can carry out the same operation over and over on different workpieces by changing the work coordinate system. There are three ways to do this.

1) Changing the work coordinate system itself

The same operation can be carried out on multiple workpieces by changing the values of the work coordinate system itself from the program.

Example) You have three points (A1, A2 and A3) defined in terms of work coordinate system WORK1. You wish to repeat the operation, but this time in terms of work coordinate system WORK2.

```
PROGRAM SAMPLE
  DUMMYWORK = WORK1          Substitutes original WORK1 value to
  WORK1 = WORK2              DUMMYWORK
  MOVE A1                    Substitutes WORK2 value to WORK1
  MOVE A2
  MOVE A3
  WORK1 = DUMMYWORK
END                          Substitutes original WORK1 value that was
                             saved to DUMMYWORK to WORK1
```

Here, the movements to points A1, A2 and A3 were performed in terms of WORK1 just as before. The only difference is that the value of WORK1 itself was changed beforehand to that of WORK2. Therefore, in effect, the robot moved in the frame of reference of WORK2.
The variable DUMMYWORK is used to hold the value of the original WORK1. Otherwise, the value of the original WORK1 would be lost forever when you put the value of WORK2 into WORK1. When the robot is finished moving, the original value of WORK1 will be restored.

2) Changing the work coordinate system with a WITH clause

You can change between different work coordinate systems by using WITH clause.

Example) If an operation is performed in the work coordinate system WORK2 for the position data A1, A2, and A3 that were taught in the work coordinate system WORK1, this appears as follows.

```
PROGRAM SAMPLE
  MOVE A2
  WORK2 = WORK + TRANS( , , 20)
  MOVE A1 WITH WORK = WORK2
  MOVE A2 WITH WORK = WORK2
  MOVE A3 WITH WORK = WORK2
END
```

Using WITH clauses in this way, it is possible to specify work coordinate systems different from that used during teaching.

3) General method

As a general method, we recommend using a combination of the above two methods. Specifically, your program should use temporary variables to define the work coordinate system. Every time the workpiece is changed, a corresponding temporary variable should be use to change over the work coordinate system.

Example) You have three points (A1, A2 and A3) defined in terms of work coordinate system WORK1. You wish to repeat the operation, but this time in terms of work coordinate system WORK2.

```
PROGRAM SAMPLE
  WORK2 = WORK + TRANS( , , 20)
  DUMMYWORK = WORK2
  MOVE A1 WITH WORK = DUMMYWORK
  MOVE A2 WITH WORK = DUMMYWORK
  MOVE A3 WITH WORK = DUMMYWORK
END
```

(c) Changing the Tool Coordinate System

By changing the tool coordinate system, you can handle operations in which tools are changed several times as work progresses.

The robot always uses the current tool coordinate system in order to move the tip of the tool to the position defined by the work coordinate system. Therefore, if you are not careful when specifying the tool coordinate system, the robot may move somewhere unexpected.

Example) You are using two tools with the tool offsets being TOOL1 and TOOL2.

```
PROGRAM SAMPLE
  TOOL1 = TRANS( , , 10)
  TOOL2 = TRANS( , , 30)
  TOOL = TOOL1
  MOVE A1
  TOOL = TOOL2
  MOVE A2
END
```

In the above program, the tool tip moves to the same A1 position for two different tools. (In the above program, the movement for actually replacing the tool is not written.)

5.3.3 Short-Cut Movement

The SCOL2 language considers one movement to start when the robot begins moving and to stop when the robot finishes positioning itself. Normally, one movement command corresponds to one movement.
However, one may also direct the robot to move continuously under multiple movement commands without stopping to position itself before heading for the next destination or passing the nearest point toward the next destination. This is called short-cut movement.
Short-cut movement reduces operating time since the robot can both cut corners and not have to spend time positioning itself.
The short-cut movement cannot be executed between MOVES or MOVEC command and other movement commands. (However, the short-cut movement is available between the MOVES command and the MOVEC command.)

(1) Specifying Short-Cut Movement
Short-cut movement is invoked or discontinued with the system switch PASS.
ENABLE PASS – Invokes short-cut motion.
DISABLE PASS – Discontinues short-cut motion.

With short-cut motion, the robot continuously changes the speeds of the axes while being careful not to exceed any maximum speeds. When the amount of the movement per one movement command has exceeded the specified percentage, the robot will begin to execute the next movement command. This percentage is specified with the system variable PASS (which is not the same thing as the system switch PASS). This percentage which is formally called the short-cut movement parameter may be specified as an integer value between 0 and 100. Note that anything smaller than 50 will be treated as 50%.

Example)

```
PROGRAM SAMPLE
  MOVE A1                    Move to point A1.
  PASS = 80                  Set the short-cut movement parameter to 80%.
  ENABLE PASS                Invoke short-cut movement.
  MOVE A2                    When 80% of the movement to point A2 is completed,
                             begin moving to point A3.
  MOVE A3                    When 80% of the movement to point A3 is completed,
                             begin moving to point A4.
  DISABLE PASS               Discontinue short-cut movement.
  MOVE A4                    Move to point A4.
END
```

The short-cut movement parameter may be changed while short-cut motion is in effect.

Example)

```
PROGRAM SAMPLE
  MOVE A1                    Move to point A1.
  PASS = 80                  Set the short-cut movement parameter to 80%.
  ENABLE PASS                Invoke short-cut movement.
  MOVE A2                    When 80% of the movement to point A2 is completed,
                             begin moving to point A3.
  MOVE A3 WITH PASS = 60     When 60% of the movement to point A3 is completed,
                             begin moving to point A4.
  MOVE A4                    When 80% of the movement to point A4 is completed,
                             begin moving to point A5.
  PASS = 90                  Set the short-cut movement parameter to 90%.
  MOVE A5                    When 90% of the movement to point A5 is completed,
                             begin moving to point A6.
  DISABLE PASS               Discontinue short-cut movement.
  MOVE A6                    Move to point A6.
END
```

(2) Commands which interrupt short-cut movement

The following commands will interrupt short-cut motion should short-cut motion be in effect at the time.

WAIT command
INPUT command
PRINT command
STOP command
BREAK command
PAUSE command

Furthermore, should DISABLE NOWAIT be in effect, the following commands will interrupt short-cut motion.

DOUT command, RESET DOUT command, PULOUT command, DIN command, BCDIN command, BCDOUT command

Moreover, when there are many commands between movement commands or when the amount of motion of an individual motion is small, the short-cut motion may be stopped.

When the short-cut movement is specified between the MOVES or MOVEC command and another movement command, the short-cut movement will be stopped.

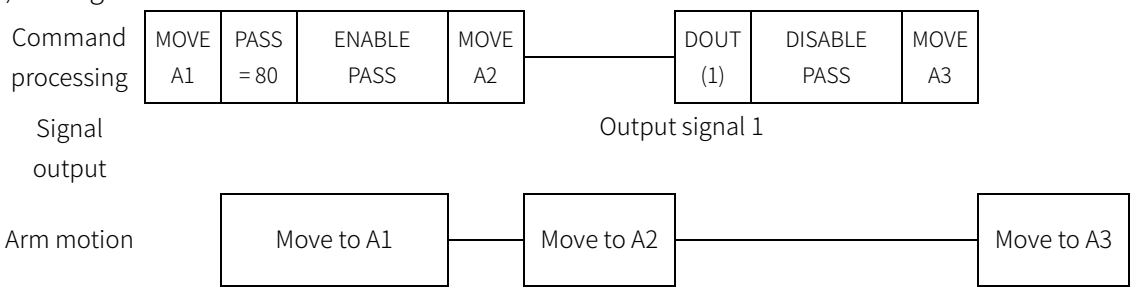
(3) Output signal timing under short-cut movement

Signal output timing under short-cut movement relative to robot arm motion is described below with the following example.

Example) Consider the case when executing the following program.

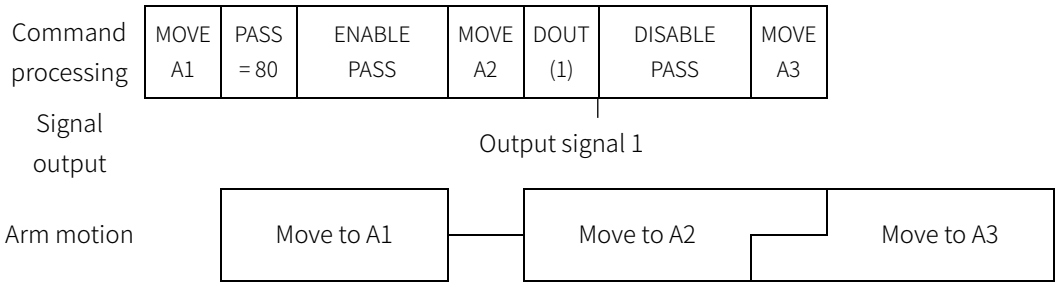
```
PROGRAM SAMPLE
  MOVE A1
  PASS = 80
  ENABLE PASS
  MOVE A2
  DOUT(1)
  DISABLE PASS
  MOVE A3
END
```

(a) Timing under DISABLE NOWAIT



Here, short-cut movement will be interrupted so that the output signal may be processed.

(b) Timing under ENABLE NOWAIT



Here, the output signal is processed even if the arm is in motion.

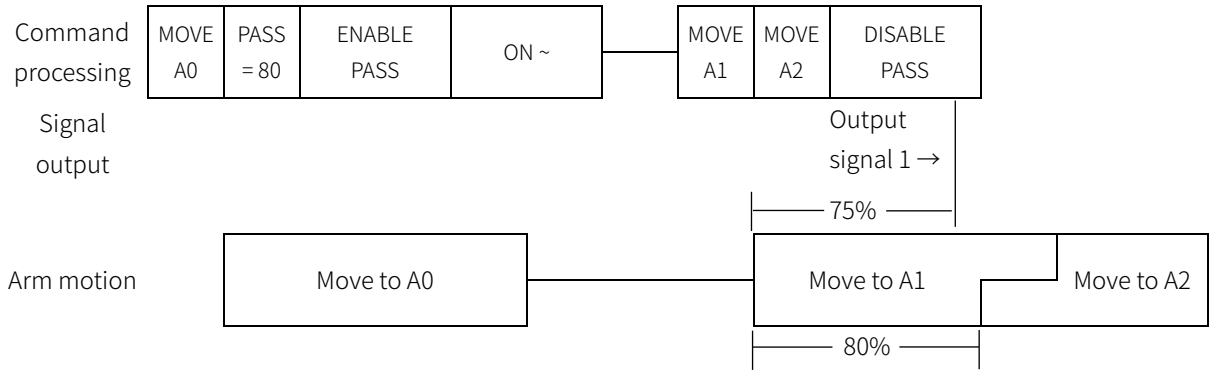
(4) Referring to the robot operating condition under short-cut movement

When you refer to the amount of robot movement with the MOTION, MOTIONT, REMAIN or REMAINT command while under short-cut movement, a value for one movement command will be returned as the result.

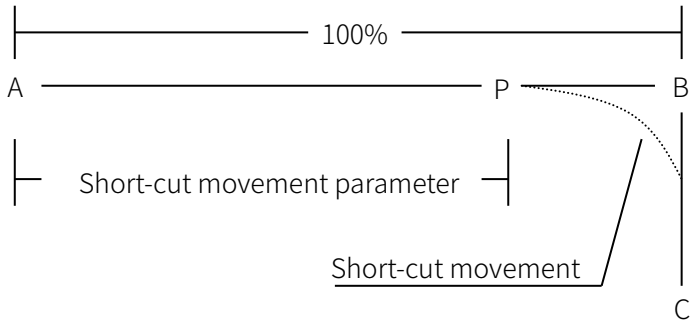
Example)

```
PROGRAM SAMPLE
  MOVE A0
  PASS = 80
  ENABLE PASS
  ON MOTION > = 75 DO DOUT(1)
  MOVE A1
  MOVEA2
  DISABLE PASS
END
```

Here, Signal 1 will be output when the robot has moved 75% of the specified value from A0 to A1.



(5) Control parameters for short-cut movement
The short-cut movement parameter has the following meaning.



In the above diagram, the robot begins at point A, goes through the vicinity of point B, and moves to point C. In short-cut movement, the robot will move from point A towards point B until it reaches point P. When the robot reaches point P, the robot will start moving towards point C. The position of point P is defined as a percentage of the total length between points A and B. In other words, the value of P is the short-cut movement parameter. Sometimes this parameter is also referred to as the pass rate.
In the above example, the short-cut movement parameter (pass rate) is given as follows:

Pass rate = ((Distance from A to P) / (Distance from A to B)) * 100 (%)

The short-cut movement parameter must be in the range of 50 to 100%. Anything less than 50% will be taken as 50%.
At times, short-cut movement may not work exactly as specified for the reasons below.

(a) Restrictions on maximum acceleration
According to the structure of the robot and parts in use, a maximum speed is determined. Furthermore, robot acceleration and deceleration varies depending on the reduction of the movement speed of the robot and the use of the ACCEL and DECEL command written in the robot language. During short-cut movement, the speed of each axis is calculated so that these accelerations are not exceeded.

(b) Restrictions from the subsequent movement
The short-cut movement superimposes the current movement onto the subsequent movement in such a way that the current movement finishes until the robot moves 50% of the subsequent movement.

For the reasons described above, the timing with which short-cut movement begins cannot be speeded up by more than a certain amount even if the short-cut movement parameter (pass rate) is made smaller.

(6) The notice in the case where the direction of the shortcut becomes the direction of the identical vector
When the fellow of the operation to cut short becomes the one-way, it sometimes becomes faster than the speed that the falling of the speed ingredient increased and that the speed specified.

5.3.4 Robot Configuration

With SCRA robots, viewed from the rotation center of the robot to the configuration of which the arm extends straight, when the second arm bends to the left (i.e. the elbow sticks out to the right), it is called the right-handed system, and when the second arm bends to the right (i.e. the elbow sticks out to the left), it is called the left-handed system. Therefore the robot can take two postures: right-handed system configuration and left-handed system configuration to one position (X, Y) specified in the work coordinate system.

(1) Configuration during teaching and configuration during operation
When you teach a position to the robot, the robot will also remember (as positional data) its configuration at the time. With normal operation, the robot will move with the configuration it had when taught.

(a) Specifying the configuration for movement
The configuration the robot is to take while moving is specified by the CONFIG command. The CONFIG command may either be used independently or with a WITH statement.
When the CONFIG command is used with a WITH statement, the CONFIG command only has effect for one single movement command.
When a CONFIG command is used by itself, all subsequent movement commands are executed with that configuration.
When you want the robot to move with the same configuration with which it was taught, you must set the robot configuration to undefined, CONFIG=0 or CONFIG=FREE. The initial setting of the robot is FREE.
CONFIG = 1 or CONFIG = LEFTY will set the robot configuration to left, and CONFIG = 2 or CONFIG = RIGHTY will set the robot configuration to right.
When you set a robot configuration, the robot moves with the specified configuration during the execution of the subsequent movement command.
Should you operate the robot with a configuration different from that which was taught, there may be a discrepancy between where the robot was taught to move and where it does move. Therefore, always try to operate the robot with the configuration the robot was taught.

(b) Commands for which the configuration cannot be changed
Any configuration specifications become invalid when any of the following movement commands are executed.

1) Interpolated movement commands
With command for the linear interpolation and arc interpolation, since the configuration of the robot cannot be changed, an error will occur. There are two interpolated movement commands, i.e. MOVES and MOVEC.

2) Single axis control commands
With commands which direct one single axis of the robot to move, the robot configuration may not be specified. There are two single axis control commands, i.e., MOVEA and MOVEI.

(2) Moving to positions created in the program
When you directly specify coordinate values to which the robot is to move (for example, MOVE POINT (100, 100)), the robot will move with the configuration in effect at the time. This is true even should the configuration be undefined.
Should you create positional data in your program, the configuration of the robot will be the configuration of the positional data in that variable before you substituted in your new values. Should that positional data variable have no specified configuration (i.e., should that variable be substituted into for the first time), the robot configuration will be undefined (free).

5.3.5 Data section

This paragraph describes how positional data is kept in the controller files. Note that there is no particular need for the programmer to worry about this if he or she is programming with the teach pendant. Rather, this paragraph is for use when creating a SCOL2 language program or positional data with any computer other than the robot controller.
All positional data files are in ASCII code.

(1) Data section
The robot position data is stored in the controller in data section units.
Multiple programs and one data section are contained in the controller file. There must always be one data section in one file. The position data in the data section can be shared in the programs of the same file. The data section for different files cannot be referenced. In addition to position data, coordinate data and load data are also stored in the data section.

Declarations are made in the data section using the following format in the file.

```
DATA
(data declarations)
•••
END
```

Declarations are issued from DATA to END in the data section. The data entries are declared one at a time from DATA to END. The data section is declared at the end of the file. The data section cannot be declared in the program.

(2) Data declarations
Positional data, coordinate data and load data are normally taught to the robot with the controller data editor. Data that was taught using the data editor is automatically stored in the data section of the target file. Data defined in the program without using a data editor is not stored in the data section. These data

are created temporarily in a different area from the files in the controller when the program is executed.

Data in a data block is declared with the following format.

<data type> <identifier> = [<element>,] •••

The <data type> designation indicates the type of data you are declaring. POINT, TRANS, and PAYLOAD are specified to designate the data types of POINT type, TRANS type, and PAYLOAD type, respectively. The <identifier> designation indicates the name of the data. The <element> designation indicates the numerical value of each element in real numbers. Any omitted <element> designations will be taken as 0.

(a) Declaring positional data

Positional data is declared with the following format.

POINT <identifier> = X, Y, Z, C, T / <configuration>

X, Y, Z, C and T are coordinate values expressed in real numbers. Units are in millimeters or degrees.

<configuration> is an integer from 0 to 2 which designates the robot configuration.

NONE •••Undefined (free)
LEFTY •••Left handed
RIGHTY •••Right handed

The system constants FREE (undefined), LEFTY (left handed) and RIGHTY (right handed) may also be used to designate the robot configuration.

Example)

```
POINT A = 100, 200, 30, 45, 0
POINT A1 = 444.44, 333.33, , , / RIGHTY
POINT ZERO =
```

(b) Declaring coordinate data

Coordinate data is declared with the following format.

TRANS <identifier> = X, Y, Z, C

X, Y, Z and C are coordinate values expressed in real numbers. Units are in millimeters or degrees.

Example)

```
TRANS WORK1 = 10, 20, 30, 45
TRANS TOOL2 = , , -20,
TRANS ZEROW =
```

(c) Declaring load data

Load data is declared with the following format.

PAYLOAD <identifier> = <mass>,
<center of gravity offset>
<mass> is the mass acting on the tip of the robot hand expressed as a real number. Units are in kilograms.
<center of gravity offset> is the offset for center of gravity acting on the tip of the robot hand expressed as a real number. Units are in millimeters.

Example)

```
PAYLOAD HAND1 = 4.8, 0.48
PAYLOAD HAND2 = 2, 0.004
PAYLOAD HAND0 =
```

(3) Specifying work coordinate systems

Positional data is specified in the SCOL2 language as the position of the tool tip defined in terms of the work coordinate system. Therefore, positional data also contains information as to which work system was used to define that data. To explicitly specify a work coordinate system, use a WORK statement to give the work coordinate system a name. You may then use the work coordinate system in your program. The work coordinates declared in this manner remain in effect until superseded by another WORK statement.

Example)

```
DATA
POINT A00 = 650, 0, 0, 0, 0
POINT A01 = 400, 400, 0, 0 / RIGHTY
TRANS WORK1 = 100, -100, 0, 0
WORK WORK1
POINT A10 = 0, 0, 0, 0, 0, 0
POINT A11 = 200, 0, 0, 0, 0, 0
TRANS WORK2 = -100, -100, 0, 0
WORK WORK2
POINT A20 = 246.8, 69.1, 23.5, 18.3, 0 / RIGHTY
POINT A21 = 0, 0, -30, 0, 0 / LEFTY
END
```

In the above example, positional data A10 and All are defined in terms of WORK1, and positional data A20 and A21 are defined in terms of WORK2. Also, since no particular work coordinate system was in effect when positional data A00 and A01 were specified; the work coordinate system for these points is taken as {0, 0, 0, 0}.

5.3.6 Global section

The variable defined by the SCOL2 language contains the global data and temporary data.
The global data which can be referred to from all parts of the program is described in this paragraph.

Variables defined in the data section are also handled as global data, but the data types that can be defined in the data section are POINT type, TRANS type, and PAYLOAD type only.
However, a global data declaration can be issued to enable global use of integer values, real number values, and array type data.

(1) Global section
Global data is declared in the global section, but in contrast to the data section, it is handled as a part of the program. For this reason, the global section is edited using the program editor. Multiple programs and one data section are contained in the controller file. There must always be one global section in one file.
The global data can be shared in the programs of the same file. The global data of different file cannot be referred to. Integer data, real number data, and array data can be stored in the global section.

Declarations are made in the global section using the following format in the file.

```
GLOBAL
(Declaration of data)
•••
END
```

Declarations are issued from GLOBAL to END in the global section.
Respective data items are declared in the GLOBAL ~ END statements one by one.
The global section is declared at the beginning of the file.
The global section cannot be declared in the program.

(2) Declaration of integer data
The global section cannot be edited by a data editor.
It is edited by the program editor in the same manner as the program.
Data in the global section is declared in the form of a substitution statement for variables in the same way as within the program.

(a) Declaration of integer data
The INT type data is declared as shown below.

<identifier> = <integer constant>

Example)
N1 = 1

If the real number is substituted for the INT type global variable in the program, the decimal places are omitted. Care should be taken.

(b) Declaration of real number
Declarations for real number data appear as shown below.

<identifier> = <real number constant>

Example)
R1 = 1.0

(c) Declaration of array data
The array data is declared as shown below.

DIM <identifier> (i, j•••) AS <variable type>

Example) INT type three dimensional array which has 2 x 3 x 4 elements
DIM IDAT (2, 3, 4) AS INT
REAL type two dimensional array which has 4 x 3 elements
DIM RDAT (4, 3) AS REAL
POINT type one dimensional array which has five elements
DIM PDAT (5) AS POINT

In DIM commands, the number of elements and the type of array type global data are simply specified, so the initial value is not determined.
In the same way as normal global data, set the initial value for integer values and real number values in the global section, and set POINT type, TRANS type, and PAYLOAD type data in the data section.

5.3.7 Robot Movement Speed

(1) Speed in each mode

The range of robot speed utilized in each mode is shown below.

Mode		Speed range
Automatic operation mode	PTP	0 to 100%
	Linear and circular interpolation	0 to 100%
Test operation mode	PTP	0 to 20%
	Linear and circular interpolation	0 to 20%

The maximum speed listed in the specifications is taken as 100%.

(2) Speed for automatic operation

Speeds for automatic and test operation have the following format.

Speed = {Speed setting in program (1 to 100%) x Override speed (1 to 100%); Limit speed}

Here,

Speed setting in program: Setting specified by the SPEED command in the robot program.

Override speed: Changes all speeds by the same fraction.

5.3.8 Robot Acceleration

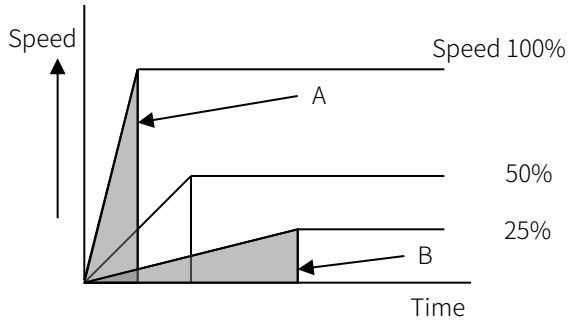
Robot acceleration (and deceleration) will vary depending on the following factors.

① Operating mode: Linear interpolation (MOVES), point-to-point (MOVE)

② Speed: SPEED command, override

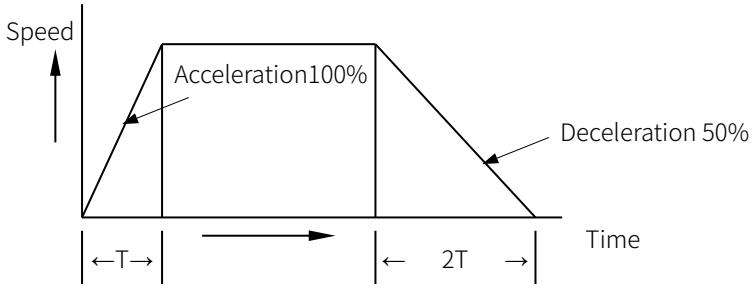
③ Acceleration commands: ACCEL, DECEL

(1) The maximum acceleration in each operating mode is calculated from the robot strength and motor torque and set. Also, the motors will speed up and slow down in such a way that the robot moves smoothly.



As shown in the figure above, the acceleration time is inversely proportional to the speed override setting. The distance of travel during acceleration (areas of triangle A and B in the figure) is constant independent of the speed override setting. Therefore, the path of the robot will stay very nearly the same no matter what the override setting.

(2) Acceleration and deceleration are specified separately in the SCOL2 language. This comes in useful when handling delicate parts in that, for example, you can slow the robot down gradually before stopping in order to keep the hand and workpiece from vibrating.



Appendix

Appendix A List of Commands

Program control commands

DATA ..... END
DIM~AS~
EXIT
FOR~TO~(STEP~)..... NEXT
GLOBAL.....END
GOTO
GOTO ( )
IF~THEN~(ELSE~)
IF~THEN~(ELSEIF~THEN~)(ELSE~)..... ENDIF
IGNORE
KILL( )
LOADLIB
ON~ (BREAK) DO~
PAUSE
PROGRAM ..... END
RCYCLE:
REMARK
RESET
RESUME
RETURN
SELECT~CASE~(CASEELSE~)..... ENDSELECT
STOP
SWITCH
TASK( )
WAIT
WHILE.....ENDWHILE
WITH

Movement control commands

DELAY
MOVE
MOVEA
MOVEC
MOVECV
MOVEI
MOVEJ
MOVES
MOVET
MOVEW
MOVEWI

Calculator commands (Calculation functions)

ABS()
ACOS()
ASIN()
ATAN()
ATAN2()
BITAND()
BITNOT()
BITOR()
BITXOR()
COS()
EXP()
LN()
LOG10()
SGN()
SIN()
SQRT()
TAN()

Calculator commands (operand)

AND
MOD
NOT
OR
XOR

Conversion functions

- INT()
- JOINT()
- PALLET()
- PALLETZ()
- POINT()
- REAL()
- TRANS()

I/O (Input/output) control commands

- ALARM
- BCDIN()
- BCDOUT()
- DIN()
- DOUT()
- HEXIN()
- HEXOUT()
- INPUT
- IPCLOSE()
- IPOPEN()
- PRINT…CR
- PULOUT()

Character string functions

ASC()
CHR()
CINT()
CREAL()
CSTR()
FORMAT()
INSTR()
INSTRREV()
LCASE()
REPLACE()
STR()
STRCAT()
STRCOMP()
STRLEFT()
STRLEN()
STRMID()
STRREAD()
STRRIGHT()
STRSPLIT()
TRIM()
UCASE()

Vector variable calculation functions

CROSS()
INNER()
INV()
JPSN()
REACH()
SHIFT()

System variables (movement condition commands)

ACCEL
ACCUR
CONFIG
CPACCEL
CPDECEL
CPSPEED
DECEL
FGAIN
GAIN
INERTIA
LOCUS
OVERRIDE
PASS
PASSMM
PAYLOAD
SLOWDOWN
SLOWSTART
SLWSPD
SPEED
SPURT
TORQUE

System variables (movement reference commands)

BASE
CUROVRD
DEST
HERE
MINUSLIM
MODE
MOTION
MOTIONT
PLUSLIM
PSNCMD
PSNCMDJ
PSNCMDW
PSNFBK
PSNFBKJ
PSNFBKW
REMAIN
REMAINT
TIMER
TIMER1~4
TOOL
WORK

System variables (Other)

INPSTSTP
INPSTSIP1~8
INPTMOTP
INPTMOIP1~8
IP1~8STATUS
MAXTASK
MEMORYI
MEMORYJ
MEMORYL
MEMORYP
MEMORYR
MEMORYT
PLCDATAR1~8
PLCDATAW1~8
PLCSLR01~08
PLCSLW01~08
PLCSSR01~08
PLCSSW01~08
PULTIME
SAVEF1~8
SAVEI1~8
TID

System constants

COARSE
CONT
CYCLE
FINE
FREE
IP1~8
LEFTY
OFF
ON
PAI
RIGHTY
SEGMENT
TP

System switches

- CTRLCODE
- DISABLE
- ENABLE
- EXTPLT
- MOVESYNC
- NOWAIT
- PASS
- SLOWDOWN
- SLOWSTART
- SMOOTH
- SWITCH

Library functions (SCOL.LIB)

- CLOSE1
- CLOSE2
- CLOSEI1
- CLOSEI2
- FREELOAD
- OFFGAIN()
- ONGAIN()
- OPEN1
- OPEN2
- OPENI1
- OPENI2
- READY
- SETGAIN()

Appendix B List of Reserved Words

ABS	ACCEL	ACCUR	ACOS	ALARM	AND
AS	ASC	ASIN	ATAN	ATAN2	
BASE	BCDIN	BCDOUT	BITAND	BITNOT	BITOR
BITXOR	BREAK				
CASE	CASEELSE	CHR	CINT	CLOSE1,2	CLOSEI1,2
COARSE	CONFIG	CONT	COS	CPACCEL	CPDECEL
CPSPEED	CR	CREAL	CROSS	CSTR	CUROVRD
CYCLE					
DATA	DECEL	DELAY	DEST	DIM	DIN
DISABLE	DO	DOUT			
ELSE	ELSEIF	ENABLE	END	ENDIF	ENDSELECT
ENDWHILE	EXIT	EXP	EXTPLT		
FGAIN	FINE	FOR	FORMAT	FREE	FREELOAD
GAIN	GLOBAL	GOTO	GOTO ( )		
HERE	HEXIN	HEXOUT			
IF	IGNORE	INERTIA	INITPLT	INNER	INPSTSIP1~8
INPSTSTP	INPTMOIP1~8	INPTMOTP	INPUT	INSTR	INSTRREV
INT	INV	IP1~8	IP1~8STATUS	IPCLOSE	IPOPEN
JOINT	JPSN				
KILL					
LATCH	LATCHPSN1~8	LATCHSIG1~8	LATCHTRG1~8	LCASE	LEFTY
LN	LOADLIB	LOCUS	LOG10		
MAXTASK	MEMORYI	MEMORYJ	MEMORYL	MEMORYP	MEMORYR
MEMORYT	MINUSLIM	MOD	MODE	MOTION	MOTIONT
MOVE	MOVEA	MOVEC	MOVEI	MOVEJ	
MOVES	MOVESYNC	MOVET	MOVEW	MOVEWI	MOVECV
NEXT	NOT	NOWAIT			
OFF	OFFGAIN	ON	ONGAIN	OPEN1,2	OPENI1,2
OR	OVERRIDE				
PAI	PALLET	PALLETZ	PASS	PASSMM	PAUSE
PAYLOAD	PLCDATAR1~8	PLCDATAW1~8	PLCDREGDW	PLCDREGF	PLCDREGUW
PLCDREGW	PLCSLR01~08	PLCSLW01~08	PLCSSR01~08	PLCSSW01~08	PLUSLIM
POINT	PRINT	PROGRAM	PSNCMD	PSNCMDJ	PSNCMDW
PSNFBK	PSNFBKJ	PSNFBKW	PULOUT	PULTIME	
RCYCLE	REACH	READY	REAL	REMAIN	REMAINT
REMARK	REPLACE	RESET	RESTORE	RESUME	RETURN
RIGHTY					
SAVEF1~8	SAVEI1~8	SEGMENT	SELECT	SETGAIN	SGN
SHIFT	SIN	SLOWDOWN	SLOWSTART	SLWSPD	SMOOTH

SPEED	SPURT	SQRT	STOP	STR	STRCAT
STRCOMP	STRLEFT	STRLEN	STRMID	STRREAD	STRRIGHT
STRSPLIT	SWITCH	SYNC			
TAN	TASK	THEN	TID	TIMER	TIMER1~4
TO	TOOL	TORQUE	TP	TRANS	TRIM
UCASE					
XOR					

Appendix C Contents of Library File (SCOL.LIB)

The contents of the library file included as standard on the system are shown below. Details may vary slightly by the customer.

After the library file has been changed, be sure to execute the SELECT command again. Otherwise, the change thus made will not be reflected on the currently selected program.

SCOL.LIB

```
PROGRAM READY
 MOVEA 3,0
 MOVEA 4,0
 MOVEA 2,0
 MOVEA 1,0
 MOVEA 5,0
END
PROGRAM OPEN1       'OPEN HAND-1
 WAIT MOTION>=100
 DOUT(203,-204)
END
PROGRAM CLOSE1      'CLOSE HAND-1
 WAIT MOTION>=100
 DOUT(-203,204)
END
PROGRAM OPENI1      'OPEN HAND-1 IMMEDIATE
 DOUT(203,-204)
END
PROGRAM CLOSEI1     'CLOSE HAND-1 IMMEDIATE
 DOUT(-203,204)
END
PROGRAM OPEN2       'OPEN HAND-2
 WAIT MOTION>=100
 DOUT(201,-202)
END
PROGRAM CLOSE2      'CLOSE HAND-2
 WAIT MOTION>=100
 DOUT(-201,202)
END
PROGRAM OPENI2      'OPEN HAND-2 IMMEDIATE
 DOUT(201,-202)
END
PROGRAM CLOSEI2     'CLOSE HAND-2 IMMEDIATE
 DOUT(-201,202)
END
```

```
PROGRAM FREELOAD 'FREE PAYLOAD
  PAYLOAD={0,0}
END
PROGRAM SETGAIN(J_0,J_1,J_2,J_3,J_4) 'SET
GAIN VARIABLE
  WAIT MOTION>=100
  GAIN={J_0,J_1,J_2,J_3,J_4}
  MOVE HERE WITH PASS=100
  MOVE HERE
END
PROGRAM ONGAIN(J_1,J_2,J_3,J_4,J_5)
J_6=0
J_7=0
J_8=0
J_9=0
J_0=0
IF J_1==0 THEN J_6=GAIN.1 ELSE J_6=1
IF J_2==0 THEN J_7=GAIN.2 ELSE J_7=1
IF J_3==0 THEN J_8=GAIN.3 ELSE J_8=1
IF J_4==0 THEN J_9=GAIN.4 ELSE J_9=1
IF J_5==0 THEN J_0=GAIN.5 ELSE J_0=1
WAIT MOTION>=100
GAIN={J_6,J_7,J_8,J_9,J_0}
MOVE HERE WITH PASS=100
MOVE HERE
RETURN
END
PROGRAM OFFGAIN(J_1,J_2,J_3,J_4,J_5)
J_6=0
J_7=0
J_8=0
J_9=0
J_0=0
IF J_1==0 THEN J_6=GAIN.1 ELSE J_6=0
IF J_2==0 THEN J_7=GAIN.2 ELSE J_7=0
IF J_3==0 THEN J_8=GAIN.3 ELSE J_8=0
IF J_4==0 THEN J_9=GAIN.4 ELSE J_9=0
IF J_5==0 THEN J_0=GAIN.5 ELSE J_0=0
WAIT MOTION>=100
GAIN={J_6,J_7,J_8,J_9,J_0}
MOVE HERE WITH PASS=100
MOVE HERE
RETURN
```

```
END
```

Appendix D How to Read Symbols

The meanings of keys and symbols used in the robot are as follows (alphanumeric characters are omitted).

[!] : Exclamation mark
[;] : Semicolon
[:] : Colon
[’] : Apostrophe
[%] : Percent
[^] : Caret, Accent circumflex, Hat
[&] : Ampersand, And
[“] : Quotation marks, double quotation marks
[[ ] : Left bracket
[ ]] : Right bracket
[{] : Left brace
[}] : Right brace
[( ] : Left parenthesis
[ )] : Right parenthesis
[+] : Positive, Plus, Add
[-] : Hyphen, Negative, Minus, Subtract
[/] : Slash, Divide
[*] : Asterisk, Multiply, Star sign
[□] : Space
[<] : Inequality, Less than, Smaller than
[>] : Inequality, More than, Greater than
[,] : Comma, cedilla
[.] : Period, dot, point
[?] : Question mark
[=] : Equal